

Important Information

Thank you for choosing Freenove products!

Getting Started

If you have not yet downloaded the zip file, associated with this kit, please do so now and unzip it.

https://github.com/Freenove/Freenove_ESP32_Display

Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be **used only when there is adult supervision present** as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. **Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.**
- When the product is turned ON, activated or tested, some parts will move or rotate. **To avoid injuries to hands and fingers keep them away from any moving parts!**
- It is possible that an improperly connected or shorted circuit may cause overheating. **Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down!** When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

<http://www.freenove.com>

sale@freenove.com

Copyright

All the files, materials and instructional guides provided are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](https://creativecommons.org/licenses/by-nc-sa/3.0/). A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.



This means you can use these resource in your own derived works, in part or completely but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.



Contents

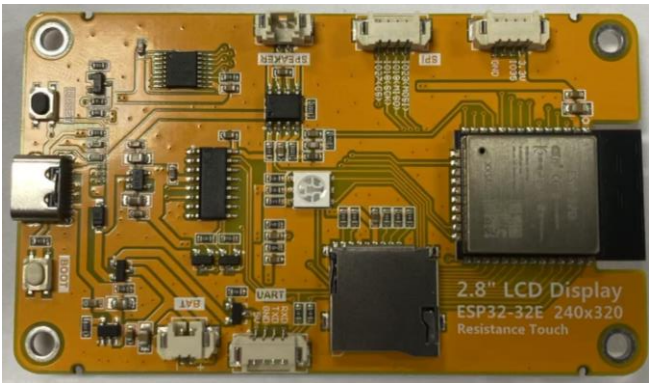
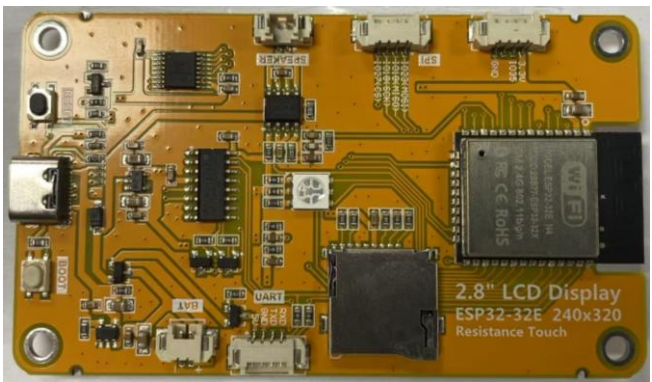
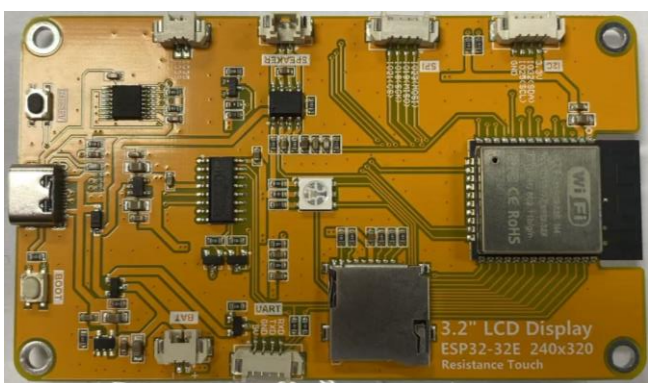
Important Information.....	1
Contents	3
Freenove ESP32 Display.....	5
Freenove ESP32 Display Models.....	5
Related Knowledge.....	6
List	8
Preface	9
ESP32-WROOM	9
Freenove ESP32 Display.....	10
CH340 (Required).....	15
Programming Software.....	24
Environment Configuration	27
Library Installation.....	31
Chapter 1 Serial.....	33
Project 1.1 SerialRW	33
Chapter 2 RGB.....	37
Project 2.1 RGB	37
Project 2.2 PWM RGB	41
Chapter 3 Button.....	47
Project 3.1 Button RGB.....	47
Chapter 4 Button Interrupt	53
Project 4.1 Button Interrupt UART.....	53
Chapter 5 Battery Voltage.....	57
Project 5.1 Battery Voltage	57
Chapter 6 SD Card.....	61
Project 6.1 SD Test.....	61
Chapter 7 Play MP3.....	71
Project 7.1 Play MP3 SD by DAC	71
Chapter 8 BLE	76
Project 8.1 BLE USART	76
Project 8.2 BLE RGB.....	88
Chapter 9 WIFI Web Server	95
Project 9.1 WIFI Web Servers LED	95
Chapter 10 TFT Display	106
Project 10.1 TFT_Rainbow	106
Project 10.2 Flash JPG DMA	116
Project 10.3 SD Jpg	126
Chapter 11 TFT Touch Calibration.....	135
Project 11.1 TFT Touch Calibration.....	135
Chapter 12 TFT Touch Drawing.....	150
Project 12.1 TFT Touch Drawing.....	150

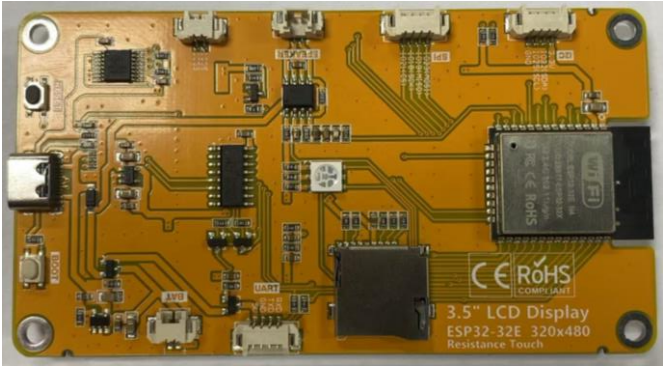
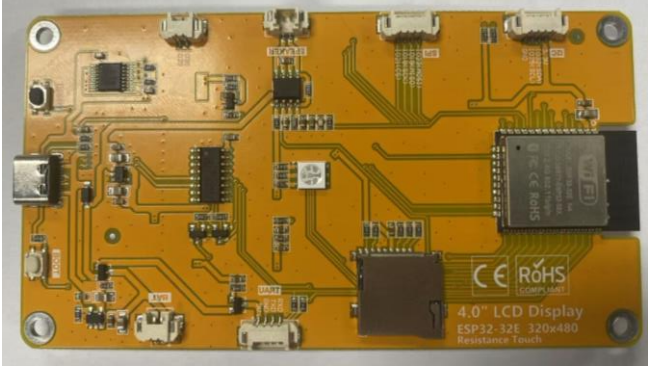
Chapter 13 LVGL.....	168
Project 13.1 LVGL.....	168
Chapter 14 LVGL Picture.....	173
Project 14.1 LVGL Picture	173
Chapter 15 LVGL Timer.....	181
Project 15.1 LVGL Timer	181
Chapter 16 Lvgl RGB.....	184
Project 16.1 LVGL RGB	184
Chapter 17 LVGL Music.....	188
Project 17.1 LVGL Music	188
Chapter 18 LVGL Multifunctionality	193
Project 18.1 LVGL Multifunctionality	193
Chapter 19 LVGL Arduino	198
Project 19.1 LVGL Arduino.....	198

Freenove ESP32 Display

Freenove ESP32 Display Models

At the time this tutorial was written, Freenove ESP32 Display is available in five different models. While they may vary in drivers, resolution, or screen size, this guide applies to all of them. Below is a list of the supported models, along with corresponding images:

Models	Specifications		Images
FNK0103F_2P8	Size	2.8 inch	 2.8" LCD Display ESP32-32E 240x320 Resistance Touch
	Resolution	240x320	
	Driver	ILI9341	
FNK0103B_2P8	Size	2.8inch	 2.8" LCD Display ESP32-32E 240x320 Resistance Touch
	Resolution	240x320	
	Driver	ST7789	
FNK0103L_3P2	Size	3.2inch	 3.2" LCD Display ESP32-32E 240x320 Resistance Touch
	Resolution	240x320	
	Driver	ST7789	

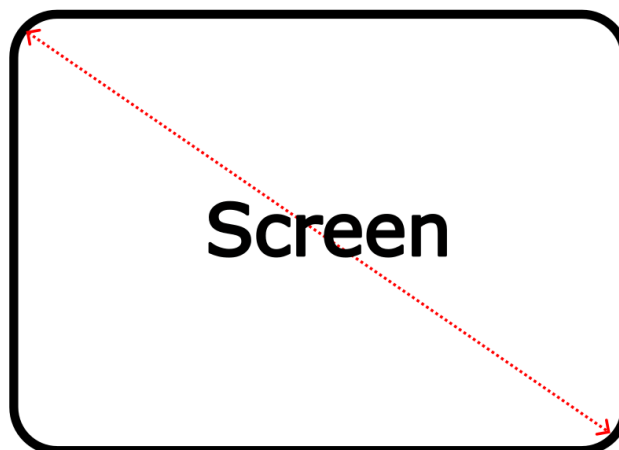
FNK0103N_3P5	Size	3.5inch	
	Resolution	320x480	
	Driver	ST7796	
FNK0103S_4P0	Size	4.0inch	
	Resolution	320x480	
	Driver	ST7796	

Related Knowledge

Screen Size

An internationally recognized unit of length, the **inch** (equivalent to 2.54 centimeters) has been the standard measurement for screen sizes in the display industry with a long history. This convention originated in the early television era when British manufacturers first adopted inches to measure cathode-ray tube (CRT) dimensions. The practice quickly became the global norm and remains the industry standard today.

A critical clarification: screen size always refers to **the diagonal measurement of the display panel**. For instance, a 4-inch display features a 4-inch (10.16 cm) diagonal, ensuring standardized and comparable sizing across all devices.



Resolution

Screen resolution quantifies a display's pixel density along its horizontal and vertical axes, expressed as **width × height** (e.g., 320 × 480). This specification applies universally across displays, such as smartphones, monitors, and television screens.

- **Pixel:** The smallest unit of a display, composed of red (R), green (G), and blue (B) subpixels. Through precise brightness variation, these subpixels generate the complete color spectrum.
- **Resolution vs. Clarity:** Higher resolution means more pixels per unit area, resulting in sharper and more detailed imagery.



Display Driver

The display driver IC serves as the core component of display panels, processing signals from the main controller and controlling pixel illumination. Below is an overview of three widely used display driver ICs.

Display Driver IC	Resolution	Advantages
ILI9341	240x320	widely used with abundant resources available
ST7789	240x320 or 240x240	Suitable for learning and development
ST7796	320x480	High-definition display with richer colors

List

If you have any concerns, please feel free to contact us via support@freenove.com

Freenove ESP32 Display x 1



USB-C Data Cable x 1



Stylus x 1



Jumper Wire x 1



Preface

ESP32-WROOM

The ESP32-WROOM offers two antenna options: the PCB on-board antenna and the IPEX antenna.

- The PCB on-board antenna is an integrated antenna within the chip module itself, making it compact and convenient for both portability and design.
- The IPEX antenna is an external metal antenna connected to the module's integrated antenna, providing enhanced signal performance.

PCB on-board antenna



IPEX antenna

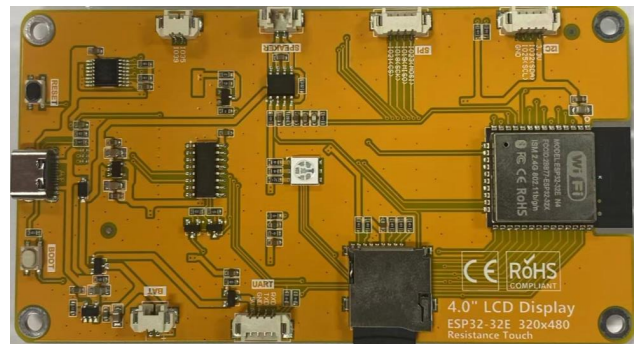


The ESP32-WROOM of this product is based on the ESP32-WROOM-32E module with built-in PCB on-board antenna.

Top



Bottom



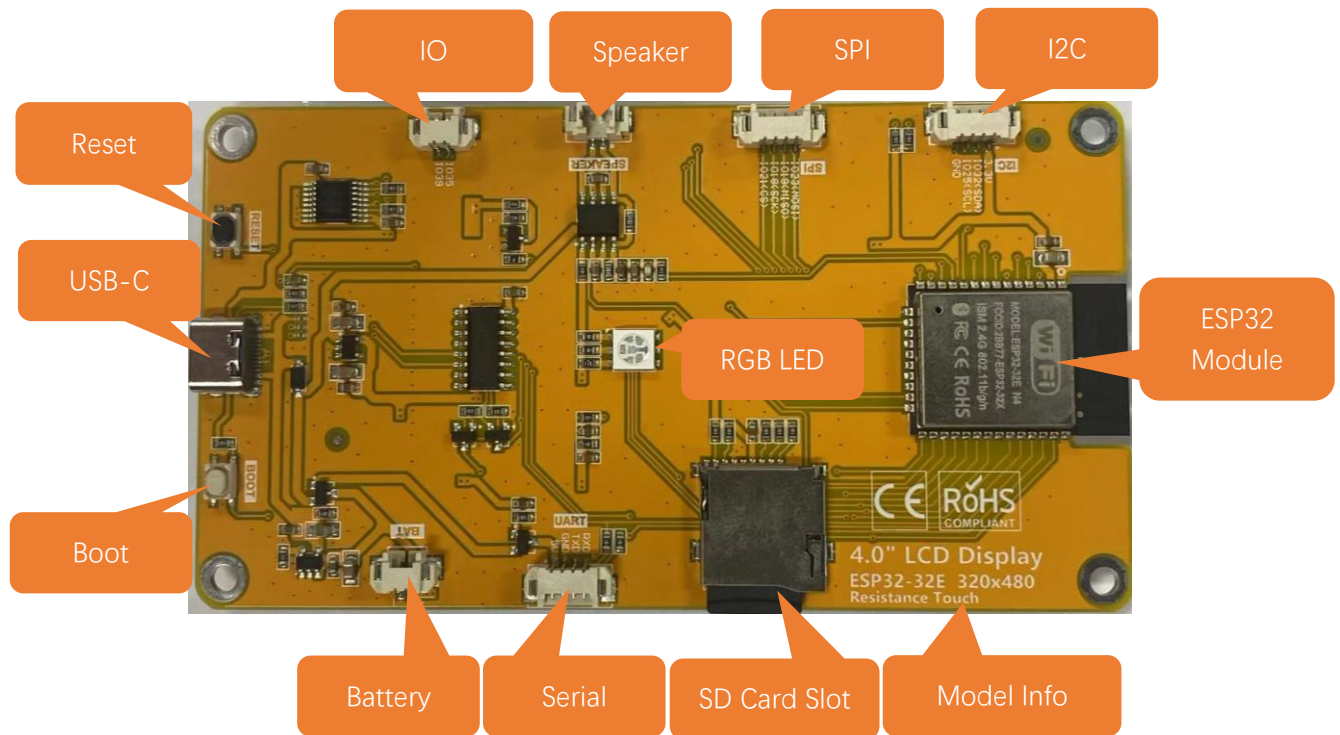
Freenove ESP32 Display is available in five different models. While they may vary in drivers, resolution, or screen size, this guide applies to all of them. For detailed model specifications, please click [here](#).

For datasheet of the ESP32 module, please visit:

https://www.espressif.com.cn/sites/default/files/documentation/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf

Freenove ESP32 Display

Hardware Interfaces



Note: The 2.8-inch version does not provide access to the I2C interface or IO39 pin.

Battery (Optional)

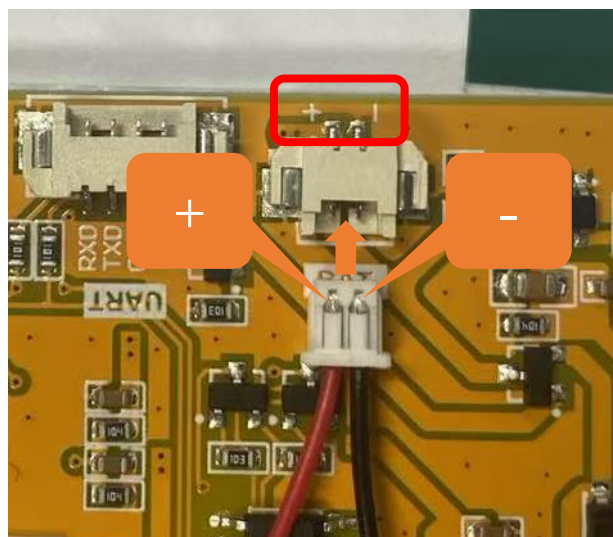
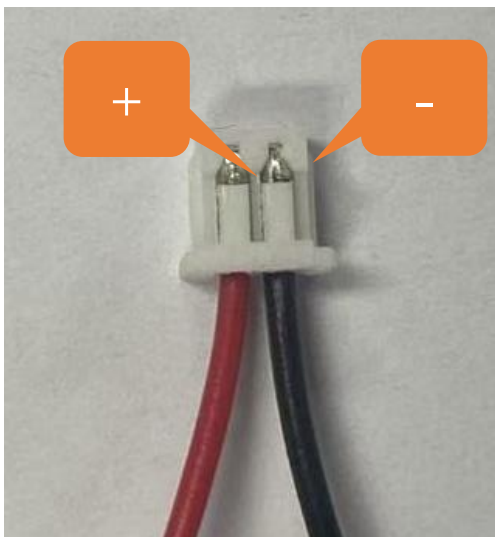
Please note that this product does not come with lithium batteries; please purchase them yourself.

This device supports both **USB-powered and lithium battery-powered operation**. For optimal safety, USB power is recommended. Due to the **hazardous nature of lithium batteries**, we advise against their use unless absolutely necessary.

This device features an **MX1.25mm** connector and supports lithium batteries of various capacities. Note: The input voltage must be maintained within **3.7-4.2V** range.

Market-available batteries may feature **two distinct wiring configurations where the positive (+) and negative (-) terminals are reversed between models**. Please verify the battery's wiring matches the product requirements (refer to the diagram below) to prevent equipment failure or safety risks due to improper connection.

The red cable is the positive terminal while the black one is negative.

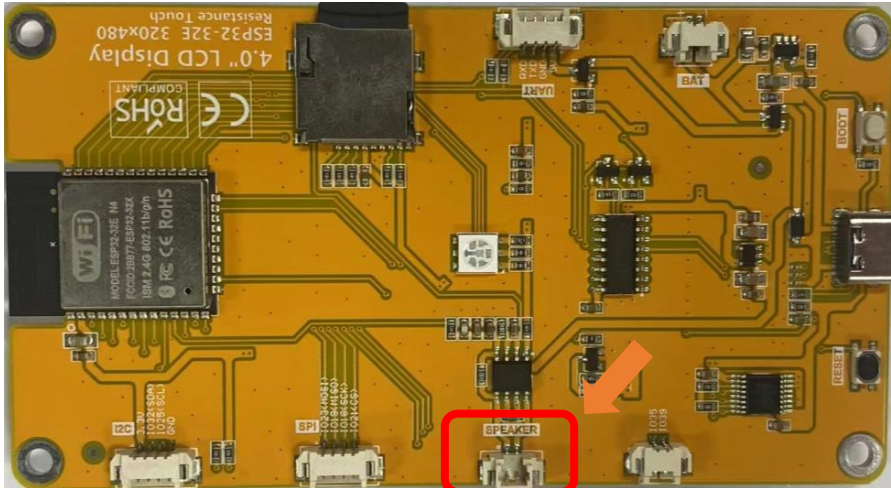


We recommend using a charger specially designed for lithium batteries. Due to various specifications and quality of lithium batteries, using a proper charger helps ensure peak performance, safety, and battery longevity.

While our product also supports USB charging as a backup option, please note that this method does not support fast charging and is limited to standard slow charging.

Speaker

There is a speaker connector (PH1.25mm) on the Freenove ESP32 Display. It is recommended to use an **8Ω 1W speaker**.



Please note that this kit does not include a speaker. Please buy one yourself.

SD Card

The connector circuit uses SPI communication and supports high-speed Micro SD card storage.

Item	Pins	Definition
SD Card	GPIO23	SD_CMD
	GPIO18	SD_CLK
	GPIO19	SD_D0
	GPIO5	SD_CS



Note: This product does not include SD cards or SD card readers. Please buy them yourself.

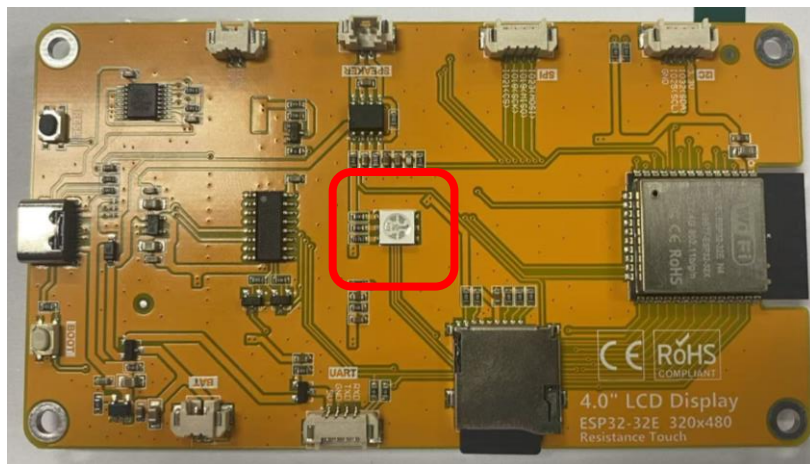
TFT Screen

Freenove ESP32 Display is available in five different models with various TFT screen. This guide applies to all of them. For detailed model specifications, please click [here](#).

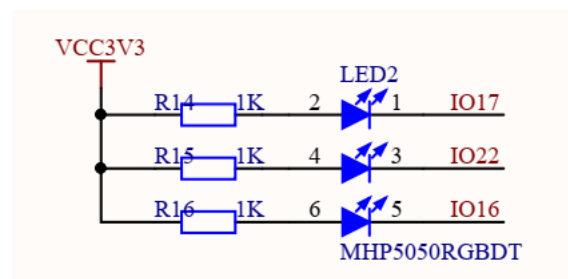
Item	Pins	Definition
TFT Screen	GPIO13	LCD_MOSI
	GPIO12	LCD_MISO
	GPIO14	LCD_SCK
	GPIO2	LCD_RS
	GPIO15	LCD_CS

RGB LED

The Freenove ESP32 Display includes an RGB LED (red, green, blue) that can blend colors to create various lighting effects.



Item	Pins
R	GPIO22
G	GPIO16
B	GPIO17



GPIO Pinout Table

To learn what each GPIO corresponds to, please refer to the following table.

The functions of the pins are allocated as follows:

ESP32	Functions	Description
GPIO22	R	RGB
GPIO16	G	
GPIO17	B	
GPIO13	LCD_MOSI	TFT_LCD
GPIO12	LCD_MISO	
GPIO14	LCD_SCK	
GPIO2	LCD_RS	
GPIO15	LCD_CS	
GPIO23	SD_CMD	SD Card
GPIO18	SD_CLK	
GPIO19	SD_D0	
GPIO5	SD_CS	

For more information, refer to the schematic.

If you have any concerns, please feel free to contact us via support@freenove.com

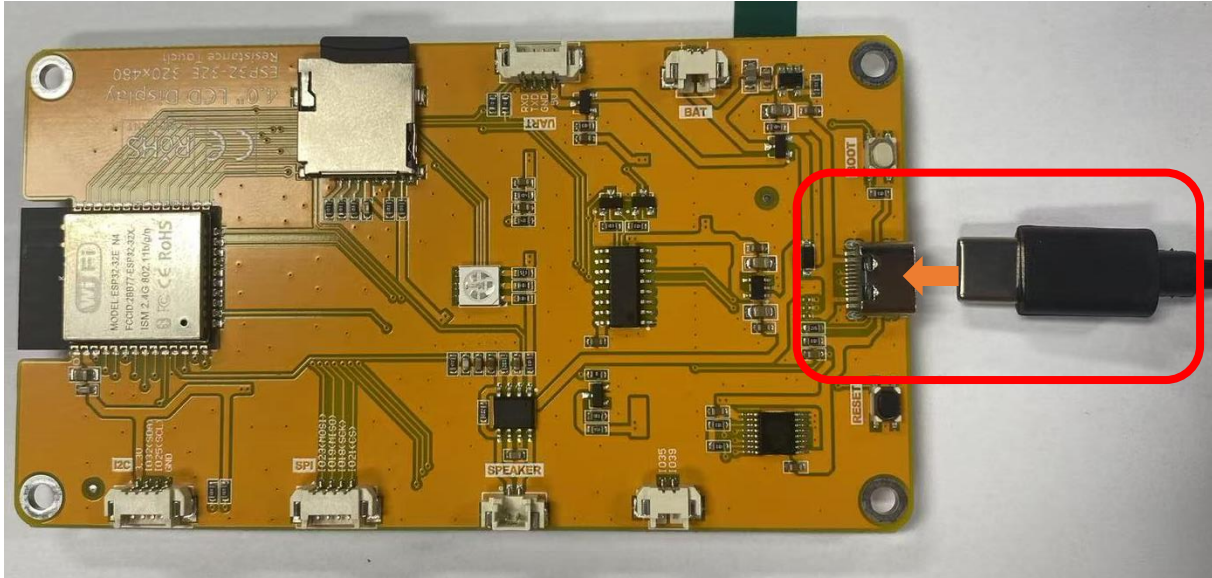
CH340 (Required)

ESP32 uses CH340 to download codes. So before using it, we need to install CH340 driver in our computers.

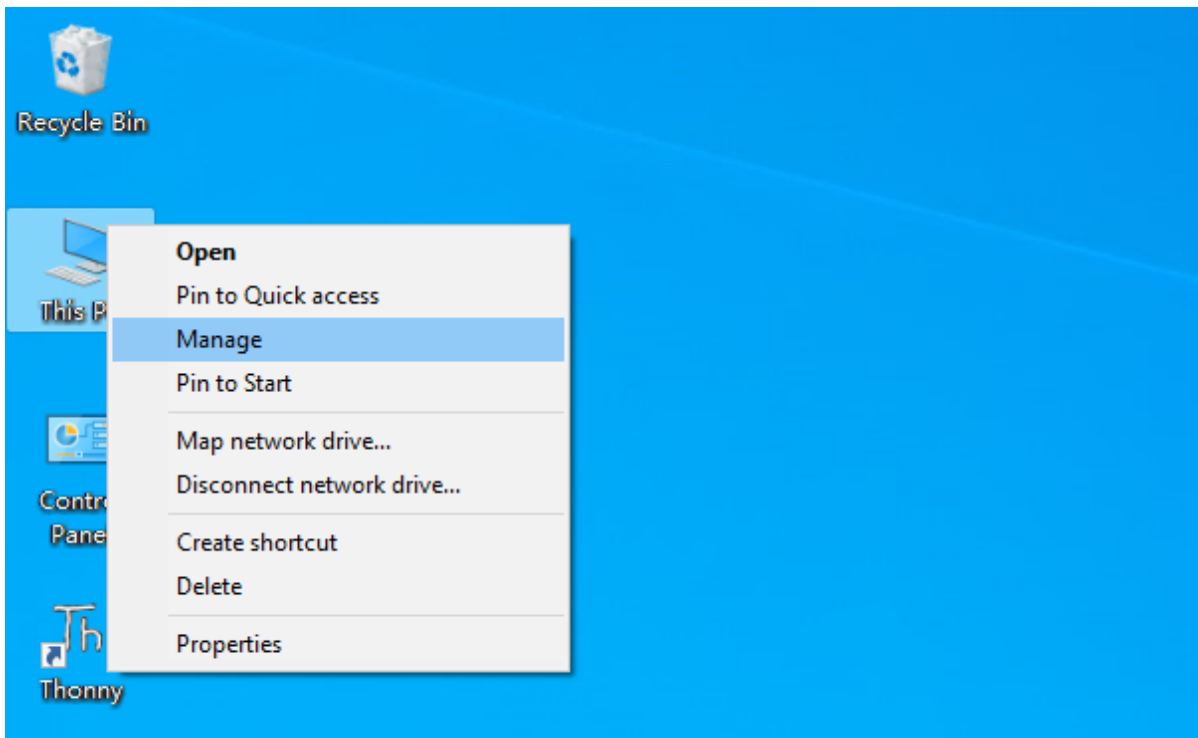
Windows

Check whether CH340 has been installed

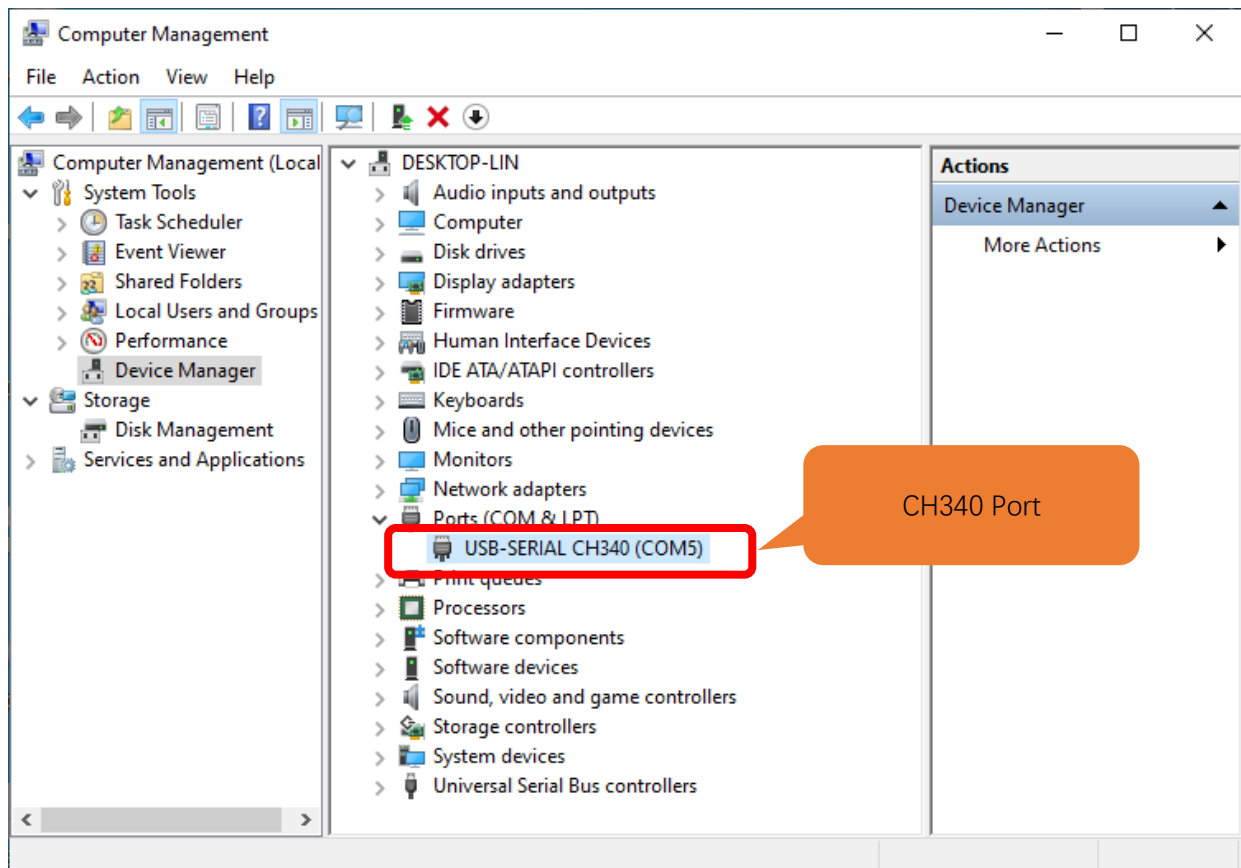
1. Connect your computer and ESP32-WROOM with a USB cable.



2. Turn to the main interface of your computer, select “**This PC**” and right-click to select “**Manage**”.



3. Click “Device Manager”. If your computer has installed CH340, you can see “USB-SERIAL CH340 (COMx)”. And you can click [here](#) to move to the next step.



Installing CH340

- First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

index / search / search CH340

All (14)

Downloads (7)

Products (4)

Application (2)

Video (1)

News (0)

keyword CH340

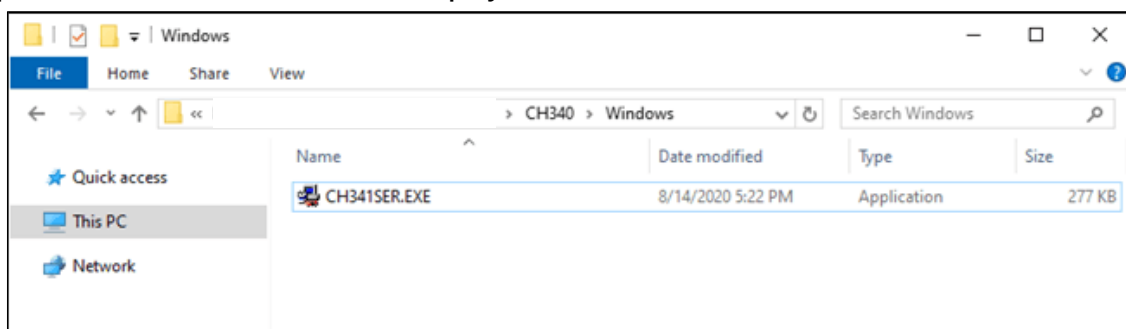
Downloads(7)

file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib (library), App Demo Example (USB to UART)	1.6	2019-04-19
CH341SER_LINUX....	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZI...	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05
Others			
PRODUCT_GUIDE.P...	Electronic selection of product selection manual, please refer to related product technical manual for more technical information.	1.4	2018-12-29
InstallNoteOn64...	Instructions for the driver after 18 years of August cannot be installed under some 64-bit WIN7 (English)	1.0	2019-01-10

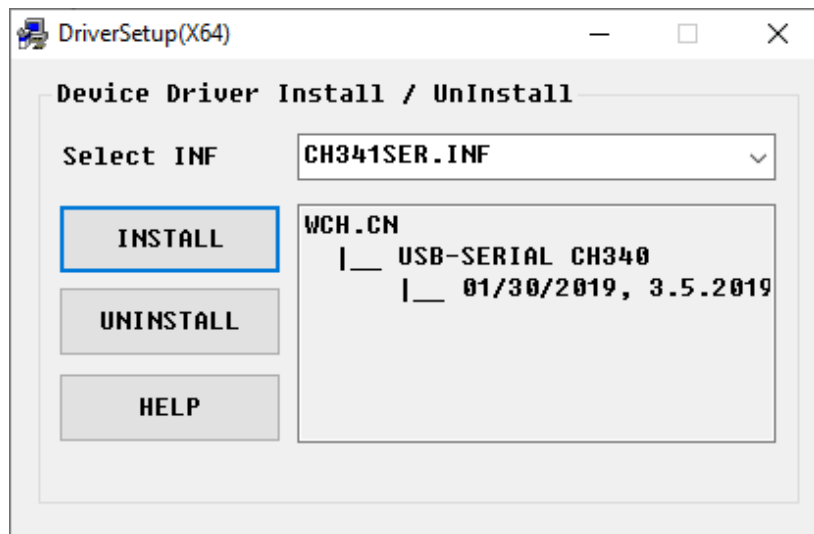
If you would not like to download the installation package, you can open “**FreeNovE_ESP32_Display/CH340**”, we have prepared the installation package.

Linux	10/17/2022 1:30 PM	File folder
MAC	10/17/2022 1:30 PM	File folder
Windows	10/17/2022 1:30 PM	File folder

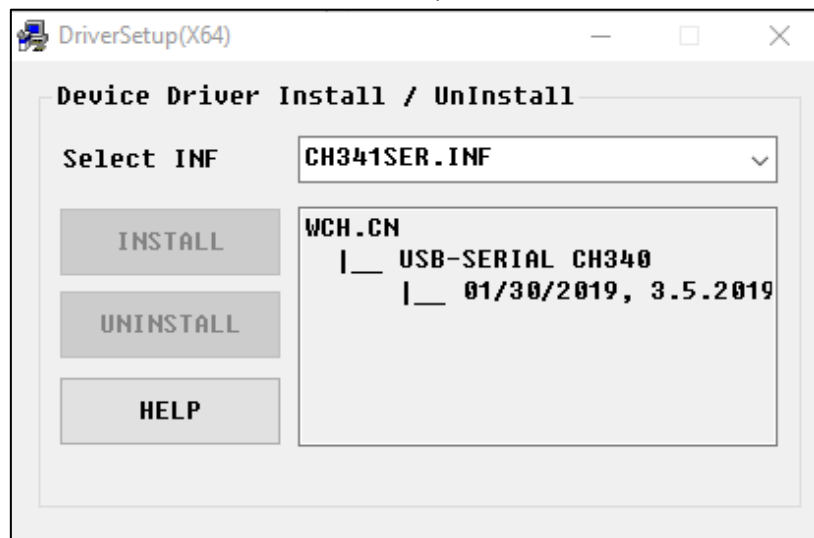
- Open the folder “**FreeNovE_ESP32_Display/CH340/Windows/**”



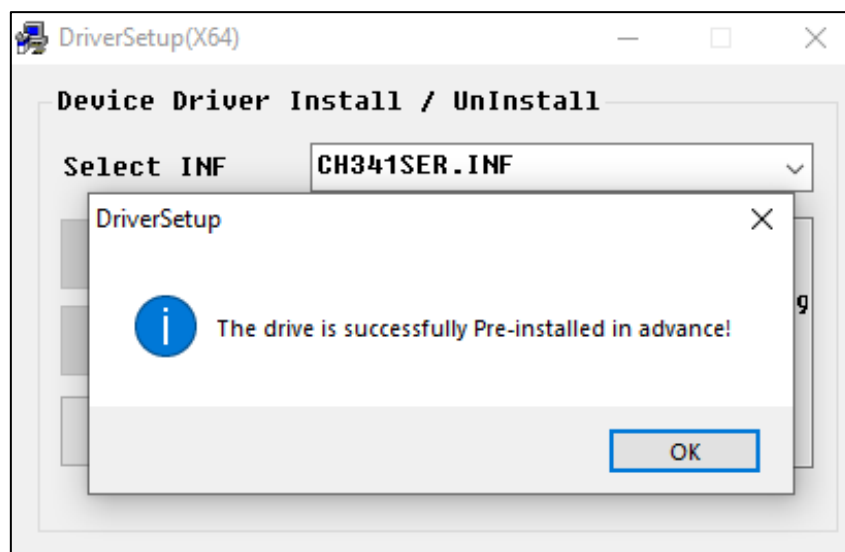
- Double click "CH341SER.EXE".



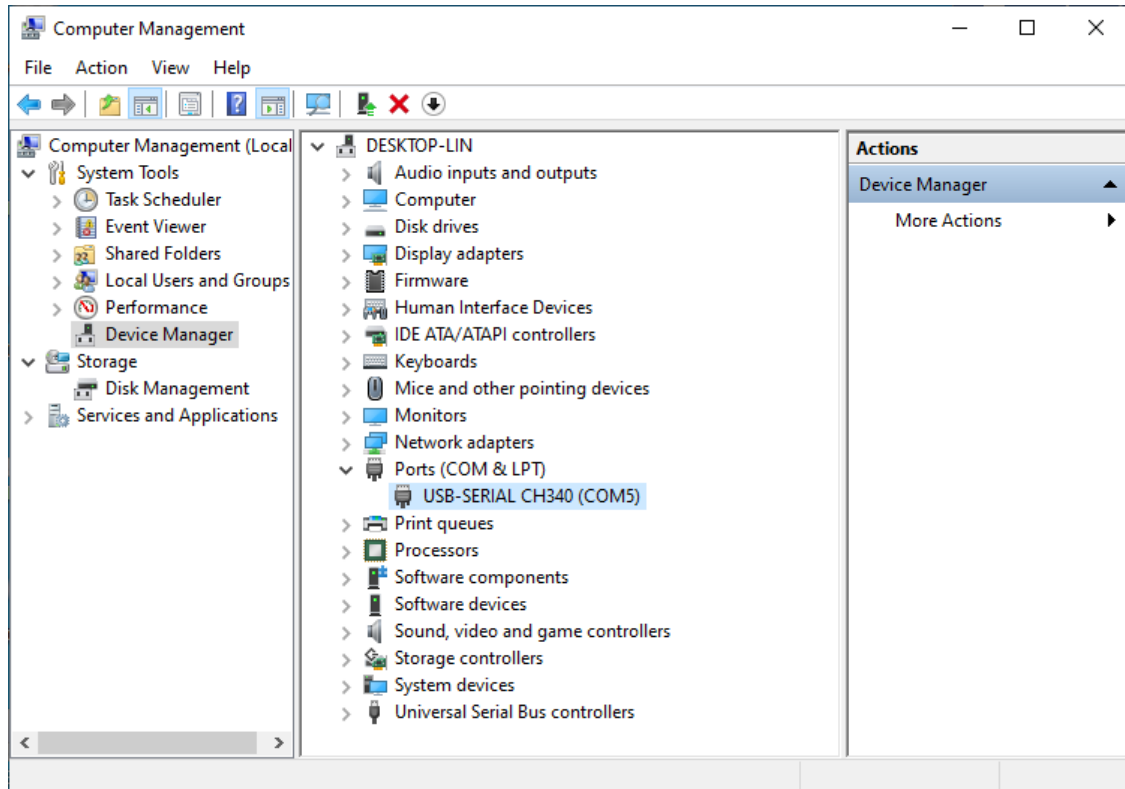
- Click "INSTALL" and wait for the installation to complete.



- Install successfully. Close all interfaces.



6. When ESP32 is connected to computer, select "This PC", right-click to select "Manage" and click "Device Manager" in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH340 has been installed successfully. Close all dialog boxes.

MAC

First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

The screenshot shows the WCH website's search results for 'ch340'. The left sidebar lists categories: All (14), Downloads (7), Products (4), Application (2), Video (1), and News (0). The main content area shows 'keyword ch340' and a list of 'Downloads (7)'. The table below lists the files:

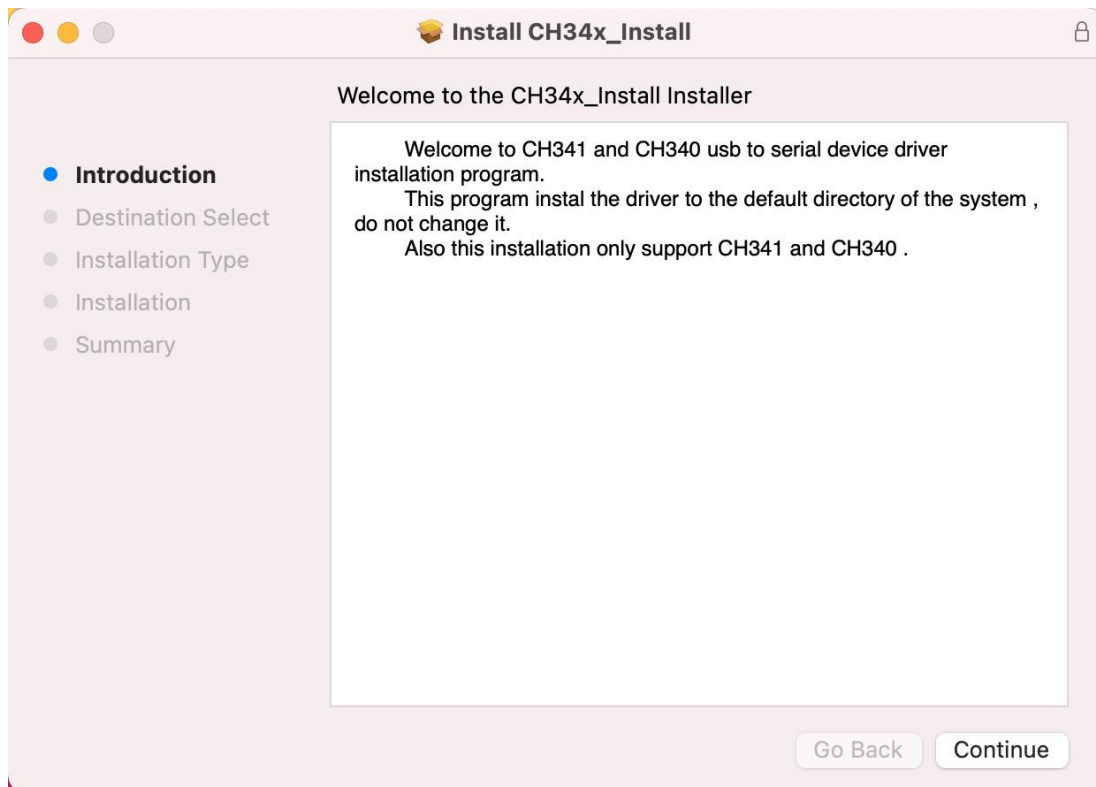
file category	file content	version	upload time
Driver&Tools			
CH341SER.EXE	CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-18
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98	3.5	2019-03-05
CH341SER_ANDROID...	CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library file (Java Driver), App Demo Example (T Demo SDK).	1.6	2019-04-19
CH341SER_LINUX...	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZIP	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05

If you would not like to download the installation package, you can open “Freenove_ESP32_Disply/CH340”, we have prepared the installation package. Second, open the folder “Freenove_ESP32_Disply /CH340/MAC/”

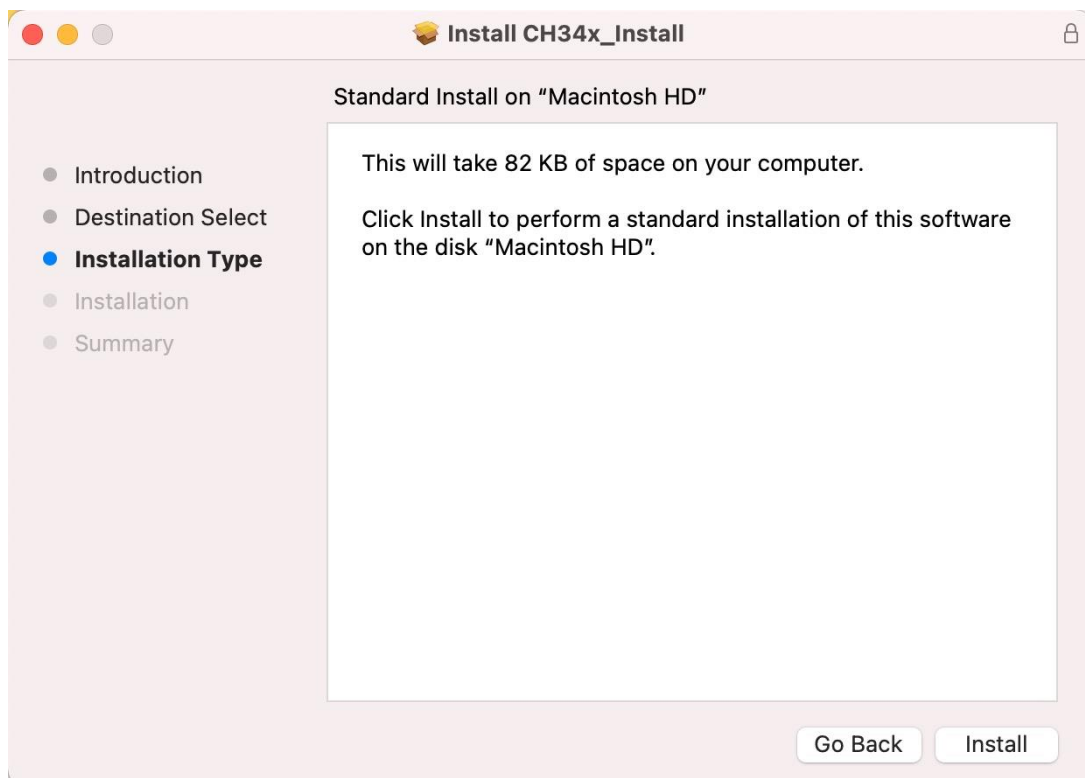
The screenshot shows a Mac file explorer window titled 'MAC'. The left sidebar shows the 'Favorites' list: AirDrop, Recents, Applications, Desktop, Documents, Downloads, iCloud, iCloud Drive, Tags, and Locations. The main area shows the contents of the 'MAC' folder:

Name	Modified	Size	Kind
CH34x_Install_V1.5.pkg	Dec 8, 2018 at 2:00 PM	26 KB	Installer package
ReadMe.pdf	Dec 8, 2016 at 4:09 PM	146 KB	Adobe...ocument

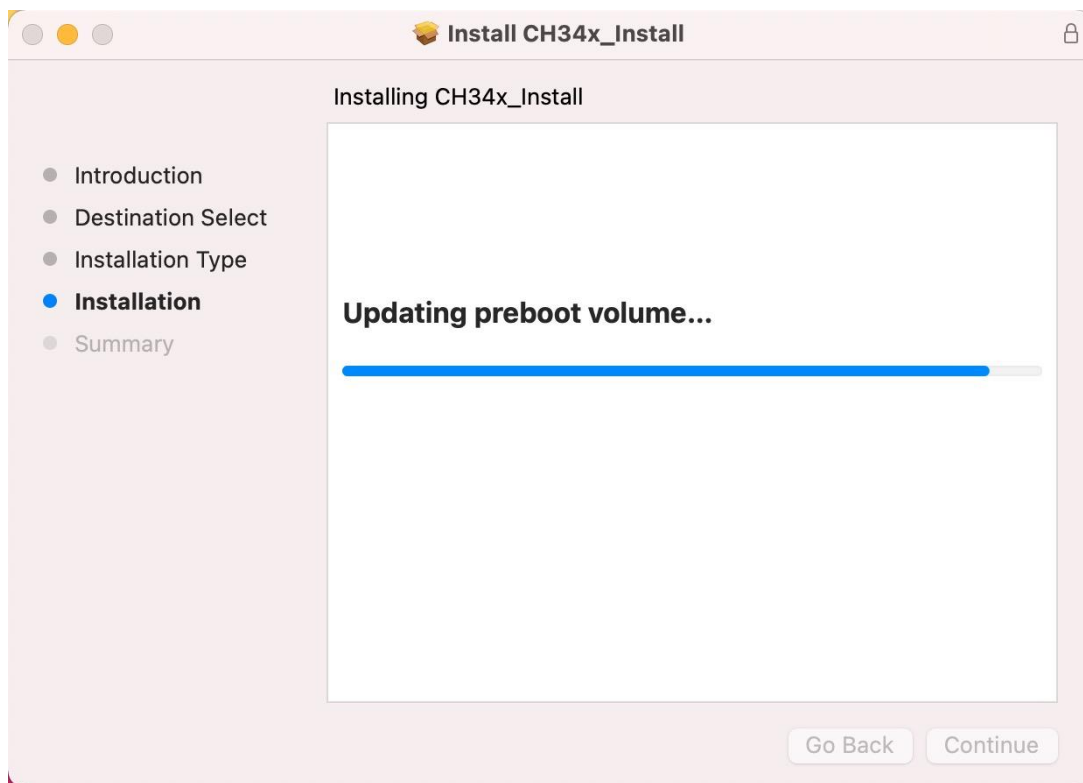
Third, click Continue.



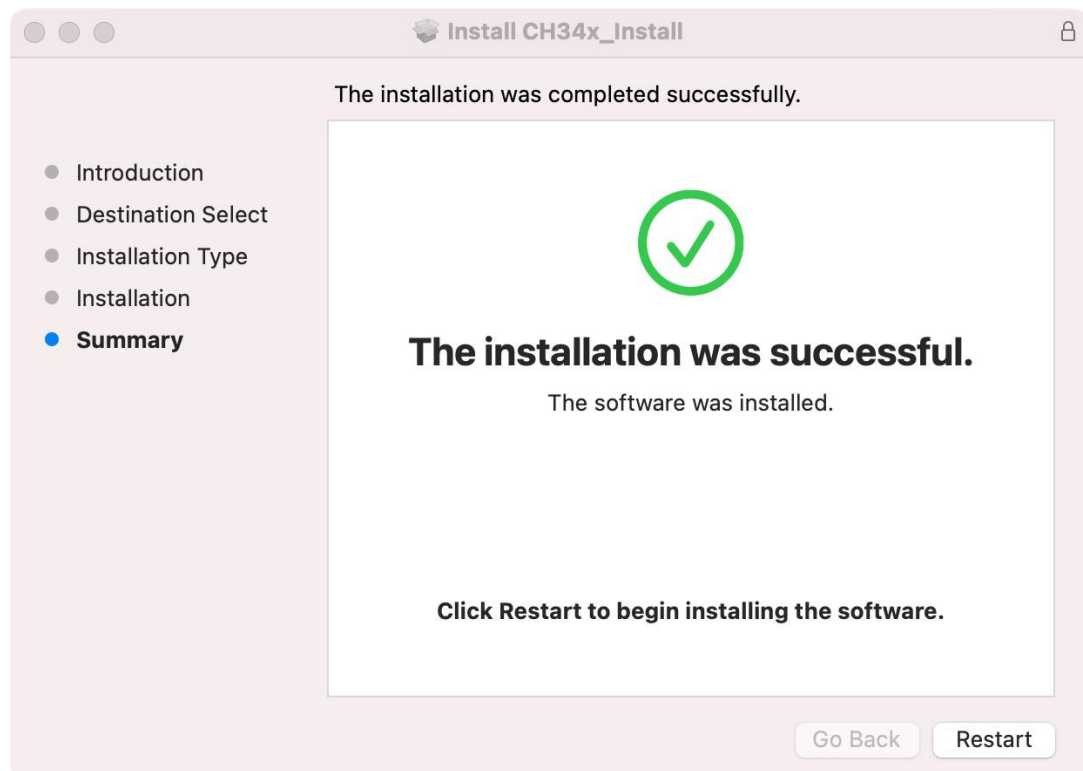
Fourth, click Install.



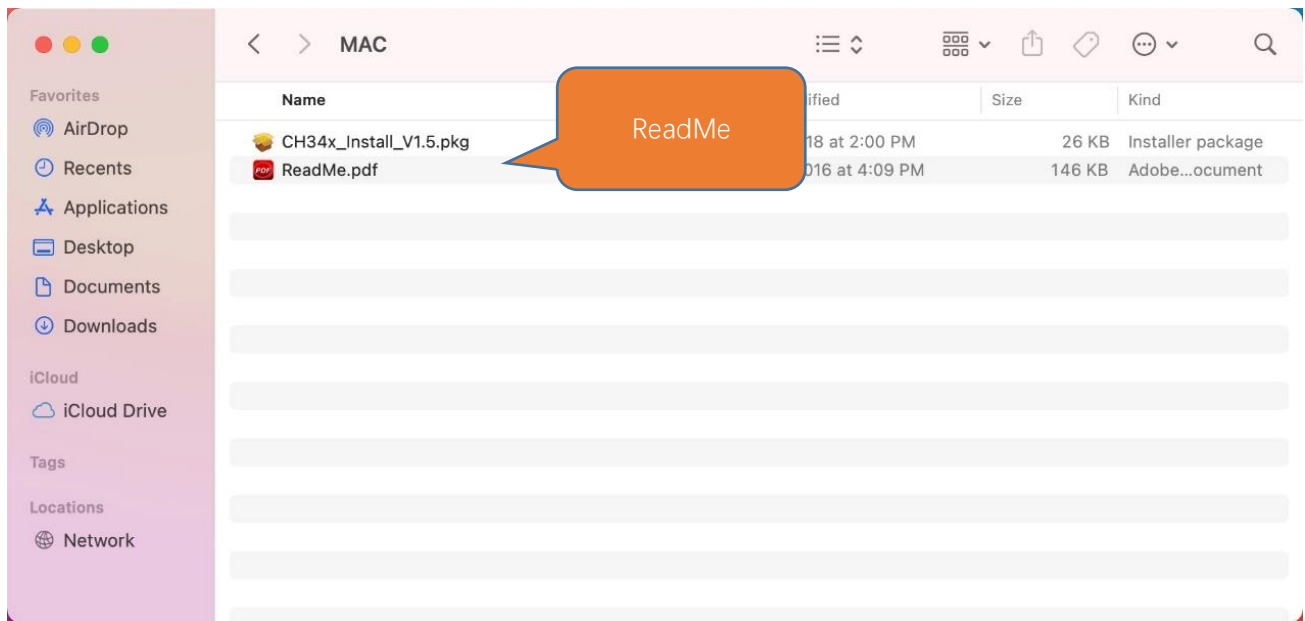
Then, waiting Finish.



Finally, restart your PC.



If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.

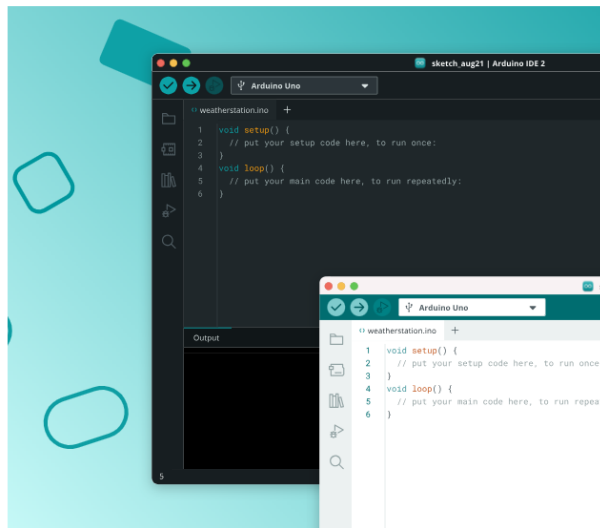


Programming Software

Arduino Software (IDE) is used to write and upload the code for Arduino Board.

First, install Arduino Software (IDE): visit <https://www.arduino.cc/en/software/>

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6

Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD



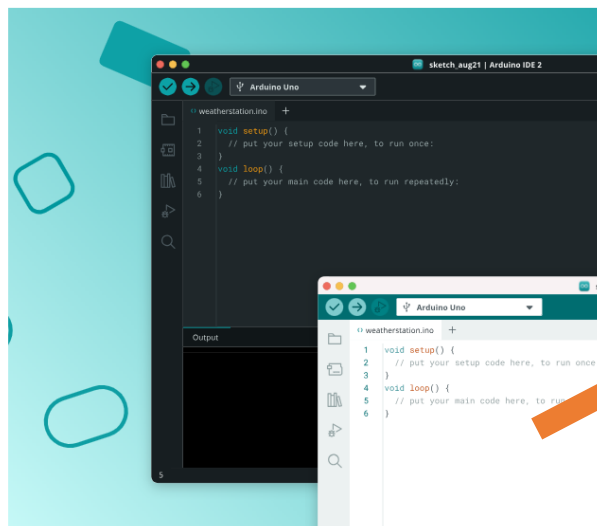
Nightly Builds

Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Select and download corresponding installer based on your operating system. If you are a Windows user, please select the "Windows" to download and install the driver correctly.

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6

Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD

Windows Win 10 or newer (64-bit)

Windows MSI installer

Windows ZIP file

Linux Appliance (64-bit X86-64)

Linux ZIP file (64-bit X86-64)

macOS Intel 10.15 Catalina or newer (64-bit)

macOS Apple Silicon 11 Big Sur or newer (64-bit)



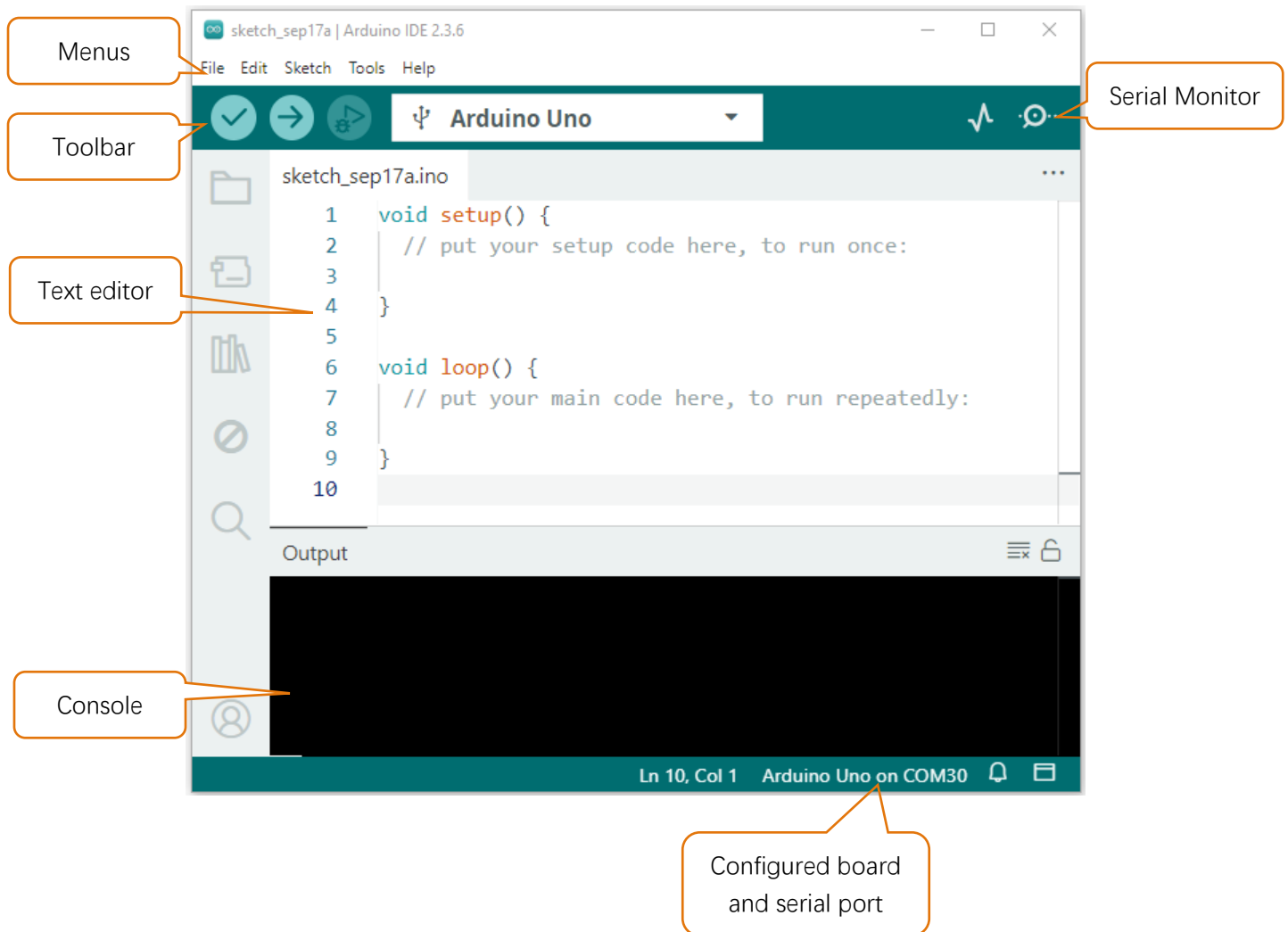
Legacy IDE (1.8.19)

Download a legacy version of the Arduino IDE.

After the downloading completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it is popped up, please allow the installation. After installation is completed, an shortcut will be generated in the desktop.



Run it. The interface of the software is as follows:

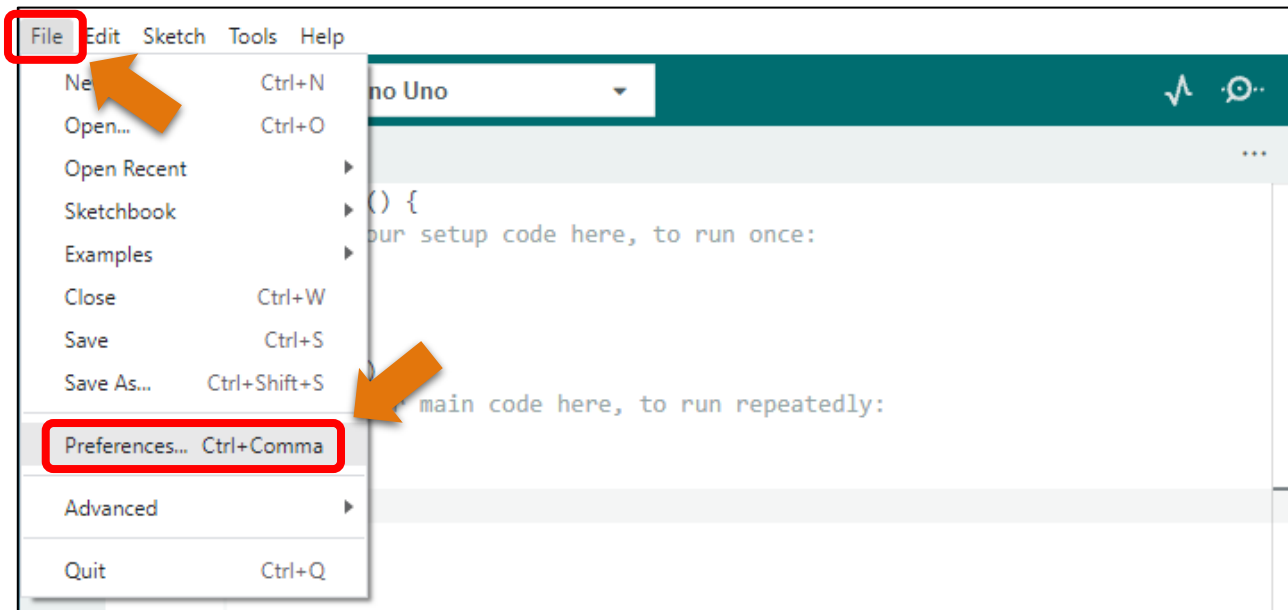


Programs written with Arduino IDE are called sketches. These sketches are written in a text editor and are saved with the file extension.ino. The editor has features for cutting/pasting and for searching/replacing text. The console displays text output by the Arduino IDE, including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, open the serial monitor, and access the serial plotter.

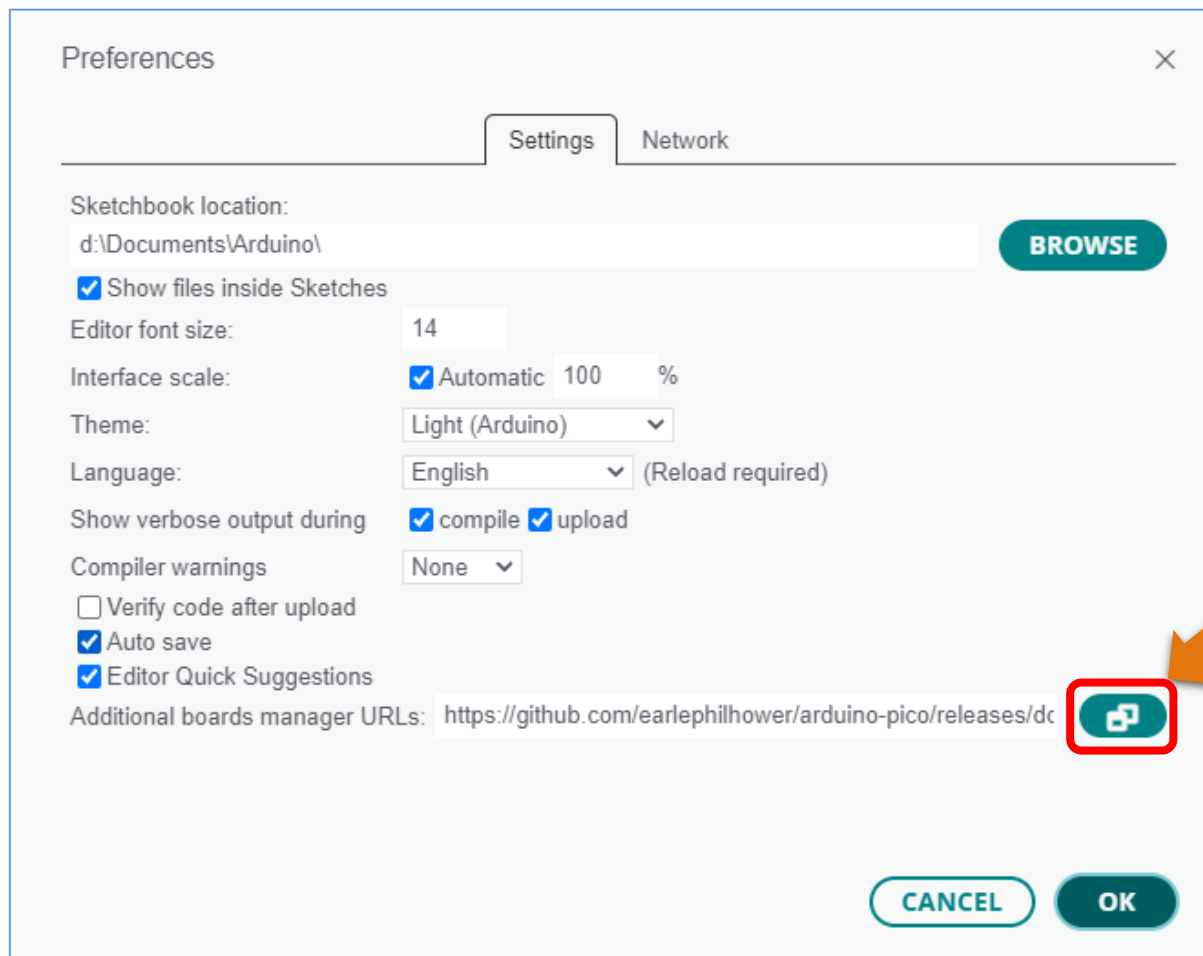
	Verify Checks your code for errors compiling it.
	Upload Compiles your code and uploads it to the configured board.
	Debug Troubleshoot code errors and monitor program running status.
	Serial Plotter Real-time plotting of serial port data charts.
	Serial Monitor Used for debugging and communication between devices and computers.

Environment Configuration

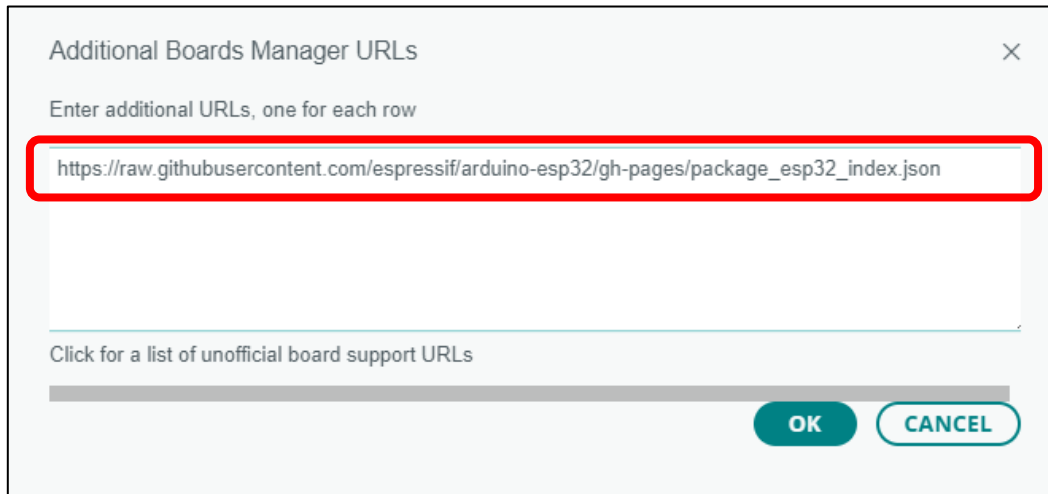
First, open the software platform Arduino, and then click File in Menus and select Preferences.



Second, click on the symbol behind "Additional Boards Manager URLs"

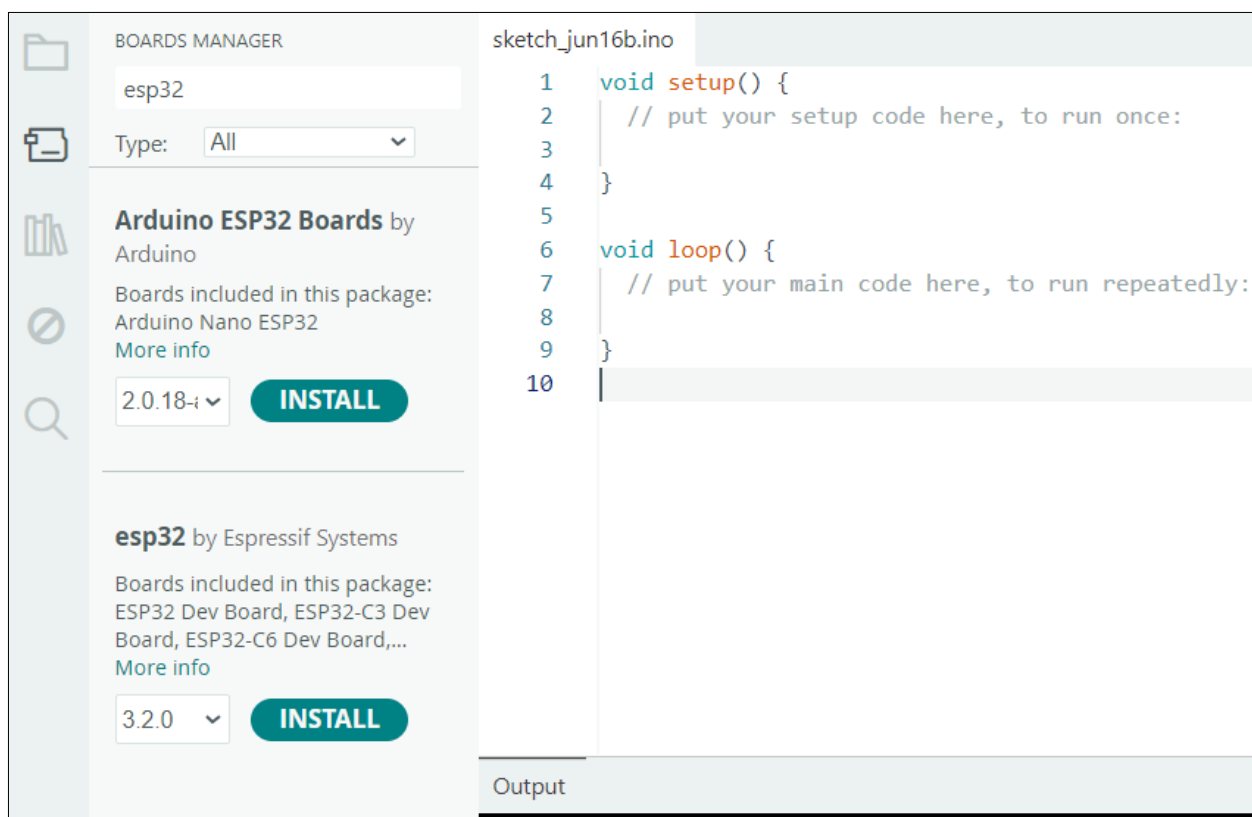


Third, fill in https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json in the new window, click OK, and click OK on the Preferences window again.



Note: if you copy and paste the URL directly, you may lose the "-". Please check carefully to make sure the link is correct.

Fourth, click "Boards Manager". Enter "esp32" in Boards manager, select 3.2.0, and click "INSTALL".



Arduino will download these files automatically. Wait for the installation to complete.

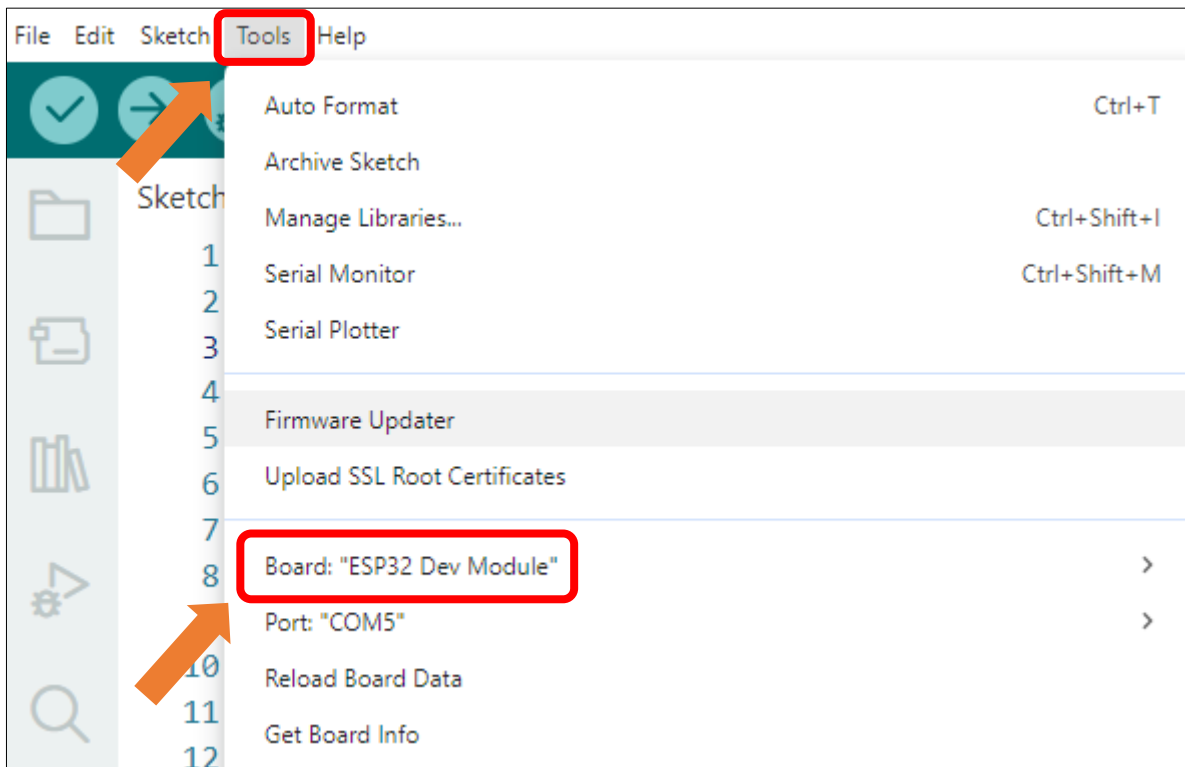
Output

```
Configuring tool.  
esp32:riscv32-esp-elf-gdb@14.2_20240403 installed  
Installing esp32:openocd-esp32@v0.12.0-esp32-20241016  
Configuring tool.  
esp32:openocd-esp32@v0.12.0-esp32-20241016 installed  
Installing esp32:esptool_py@4.9.dev3  
Configuring tool.  
esp32:esptool_py@4.9.dev3 installed  
Installing esp32:mkspiffs@0.2.3  
Configuring tool.  
esp32:mkspiffs@0.2.3 installed  
Installing esp32:mklittlefs@3.0.0-gnu12-dc7f933  
Configuring tool.  
esp32:mklittlefs@3.0.0-gnu12-dc7f933 installed  
Installing platform esp32:esp32@3.2.0  
Configuring platform.  
Platform esp32:esp32@3.2.0 installed
```



 Successfully installed platform esp32:3.2.0

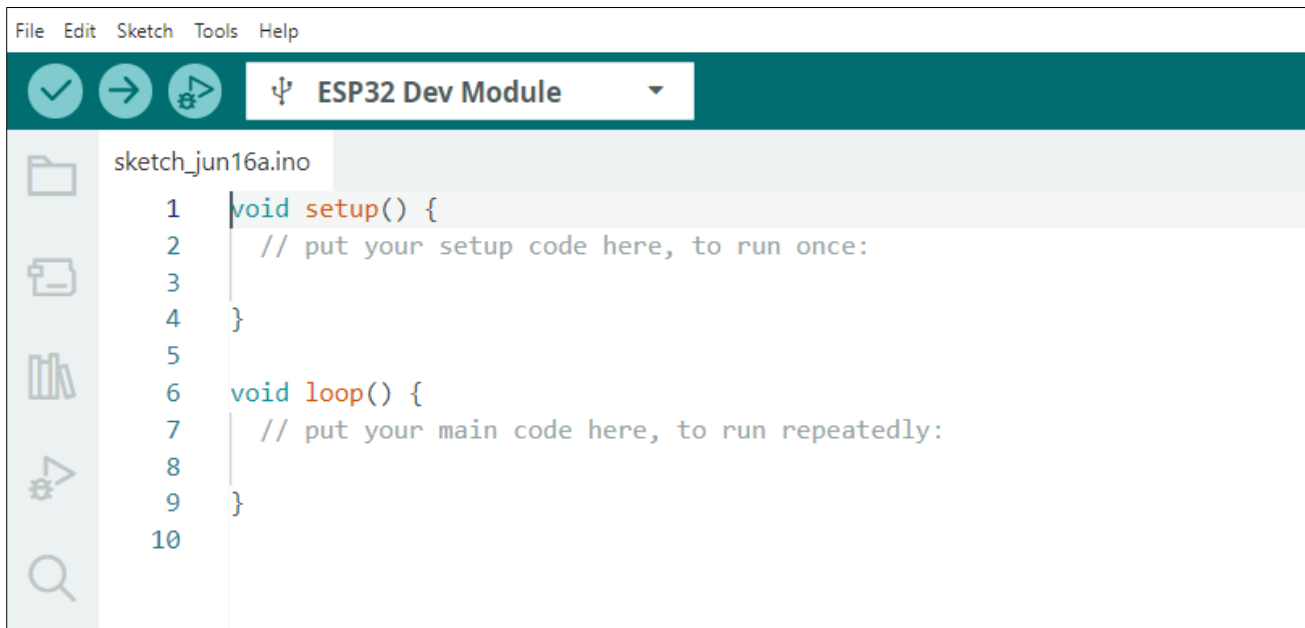
When finishing installation, click Tools in the Menus again and select Board: "ESP32 Dev Module", and then you can see information of ESP32.



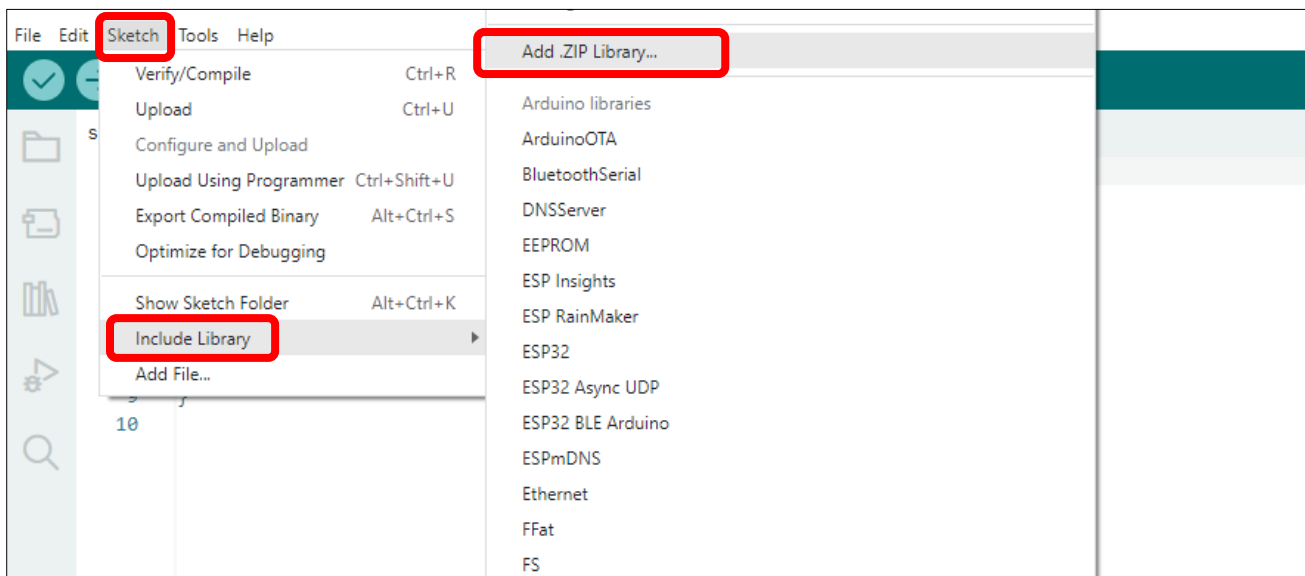
Library Installation

Before starting the learning process, it is necessary to install some libraries in advance to enable the code to be compiled properly. For convenience, we have already packaged these libraries and placed them in the **Freenove_ESP32_Display/Libraries** folder. Please refer to the following steps to install these libraries into the Arduino IDE.

1. Open Arduino IDE.



2. Select Sketch->Include Library->Add .ZIP Library...

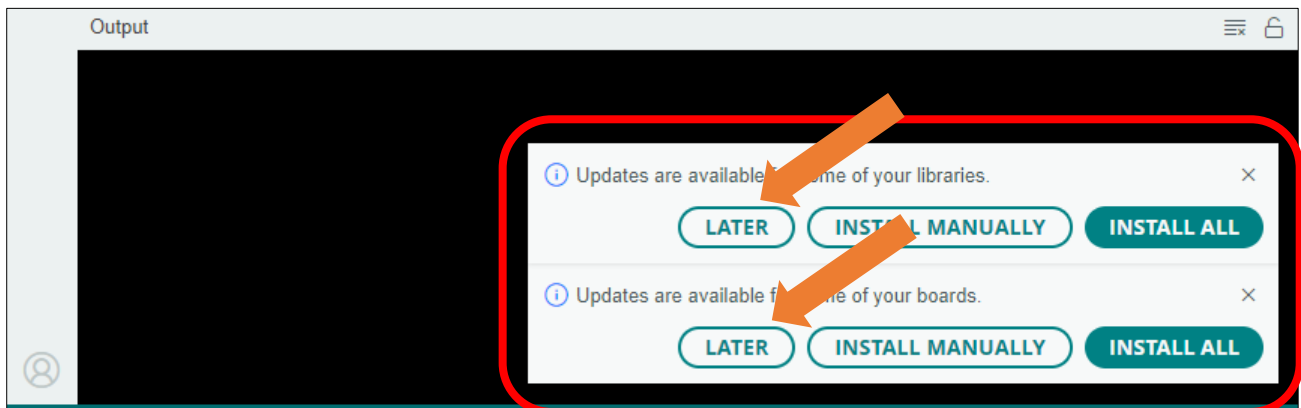


- On the newly pop-up window, select the files from the **Freenove_ESP32_Display /Libraries**. Click Open to install the library.

 ESP8266Audio_v2.0.0.zip	2025/5/30 14:06
 lvgl_v8.4.0.zip	2025/5/30 13:52
 TFT_eSPI_Setups_v1.0.zip	2025/6/11 15:45
 TFT_eSPI_v2.5.43.zip	2025/6/11 15:45
 TFT_Touch_v0.3.zip	2025/5/30 13:51
 Tjpg_Decoder_v1.1.0.zip	2025/5/30 14:08

- Repeat the above steps until all the six libraries are installed to Arduino. So far, all libraries have been installed.

Note: Some libraries are not the latest version. Please do not update them even if it prompts every time you open the IDE. Just click LATER. Otherwise, it may lead to compilation failure.



Chapter 1 Serial

Project 1.1 SerialRW

Related Knowledge

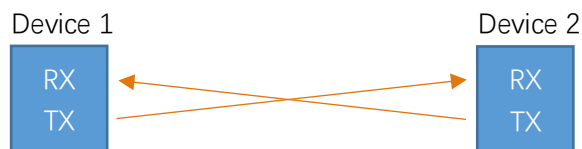
Serial and parallel communication

Serial communication uses one data cable to transfer data one bit by another in turn, while parallel communication means that the data is transmitted simultaneously on multiple cables. Serial communication takes only a few cables to exchange information between systems, which is especially suitable for computers to computer, long distance communication between computers and peripherals. Parallel communication is faster, but it requires more cables and higher cost, so it is not appropriate for long distance communication.



Serial communication

Serial communication generally refers to the Universal Asynchronous Receiver/Transmitter (UART), which is commonly used in electronic circuit communication. It has two communication lines, one is responsible for sending data (TX line) and the other for receiving data (RX line). The serial communication connections of two devices use is as follows:



Before serial communication starts, the baud rate in both sides must be the same. Only use the same baud rate can the communication between devices be normal. The baud rates commonly used are 9600 and 115200.

Component List

Freenove ESP32 Display x 1

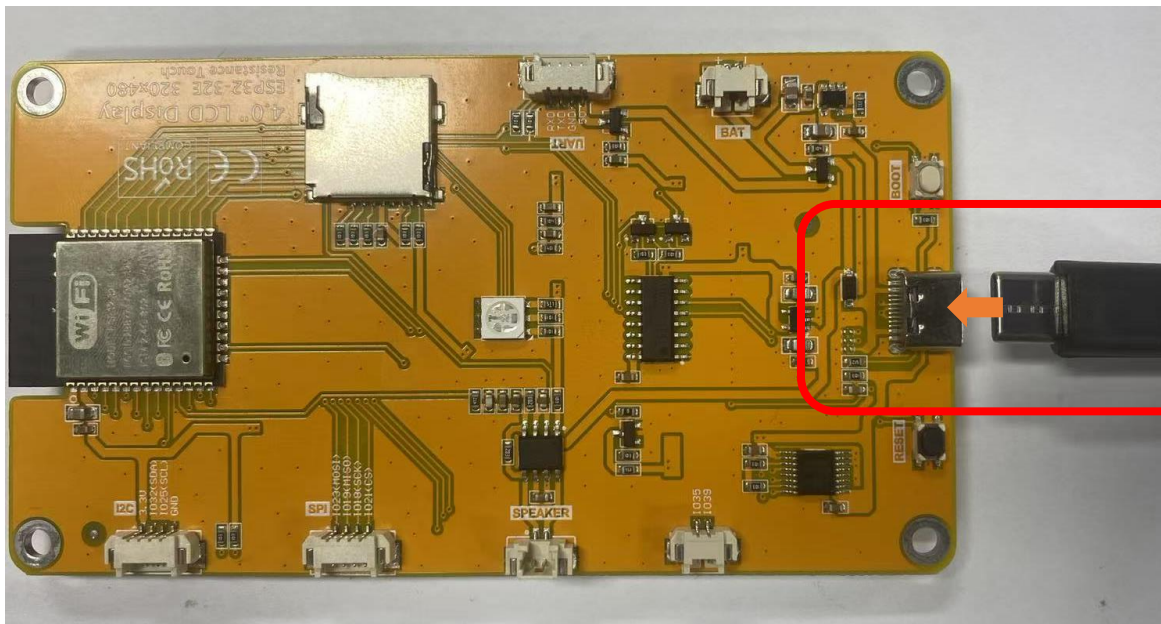


USB cable x1



Circuit

Connect Freenove ESP32 Display to the computer with USB cable.



Sketch

Open “Sketch_01.1_SerialRW” folder under “Freenove_ESP32_Display \Sketch” and double-click “Sketch_01.1_SerialRW.ino”.

Sketch_01.1_SerialRW

```
1 String inputString = "";    //a String to hold incoming data
2 bool stringComplete = false; // whether the string is complete
3
4 void setup() {
5     Serial.begin(115200);
6     Serial.println(String("\nESP32 initialization completed!\n"))
```

```

7         + String("Please input some characters,\n")
8         + String("select \"\\Newline\\\" below and click send button. \\n"));
9     }
10
11 void loop() {
12     if (Serial.available()) {          // judge whether data has been received
13         char inChar = Serial.read();    // read one character
14         inputString += inChar;
15         if (inChar == '\\n') {
16             stringComplete = true;
17         }
18     }
19     if (stringComplete) {
20         Serial.printf("InputString: %s", inputString);
21         inputString = "";
22         stringComplete = false;
23     }
24 }

```

Code Explanation

Set the baud rate to 115200.

```
5 Serial.begin(115200);
```

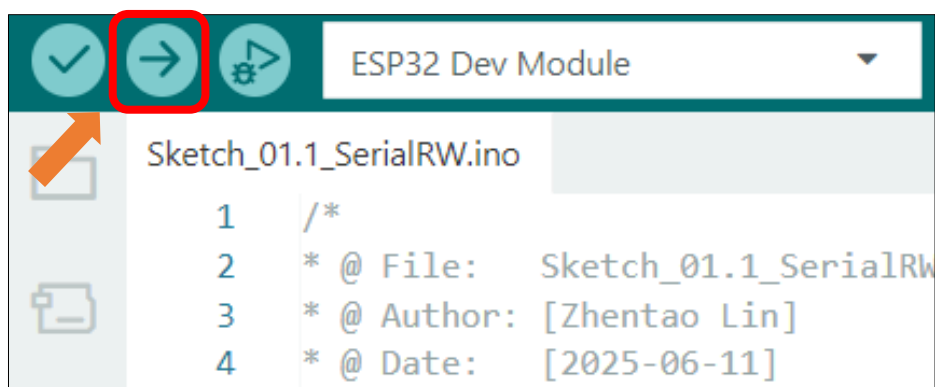
Determine whether there is data in the serial port buffer.

```
8 if (Serial.available())
```

Receive serial port data and save it in the inputString string.

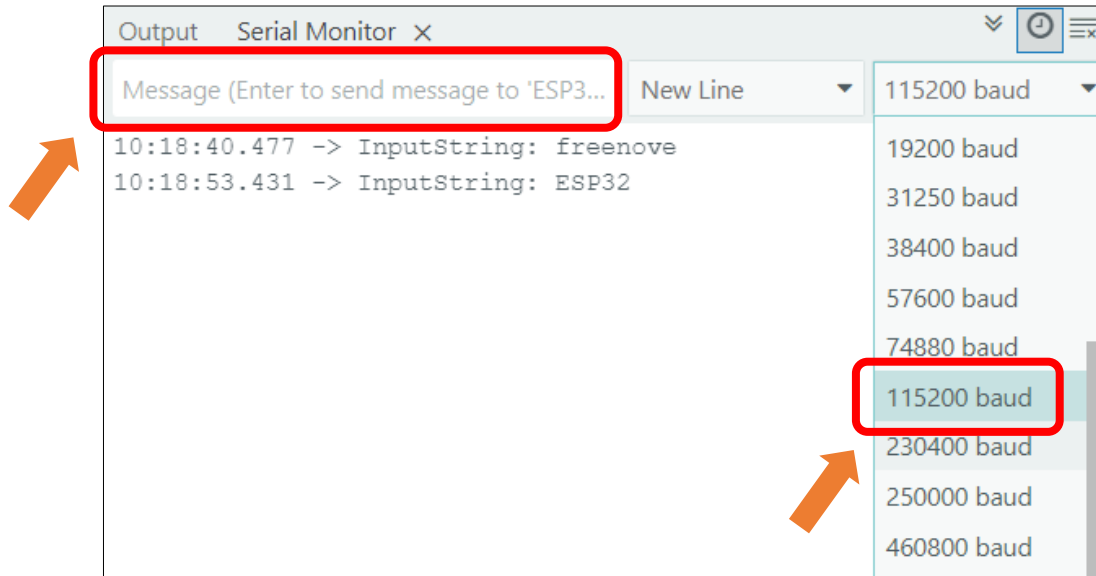
```
13 char inChar = Serial.read();          // read one character
14 inputString += inChar;
```

The purpose of this code is to display data on the serial monitor. Click **"Upload"** to upload the code to Freenove ESP32 Display.



After downloading the code, open the serial port monitor, and set the baud rate to **115200**, input any data in the messages bard and press Enter key, Freenove ESP32 Display will print the received data.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



Reference

`int available()`

`Serial.available()` checks the number of bytes currently available to read in the Serial receive buffer. It returns the number of bytes available (int type), or 0 if the buffer is empty.

`int read ()`

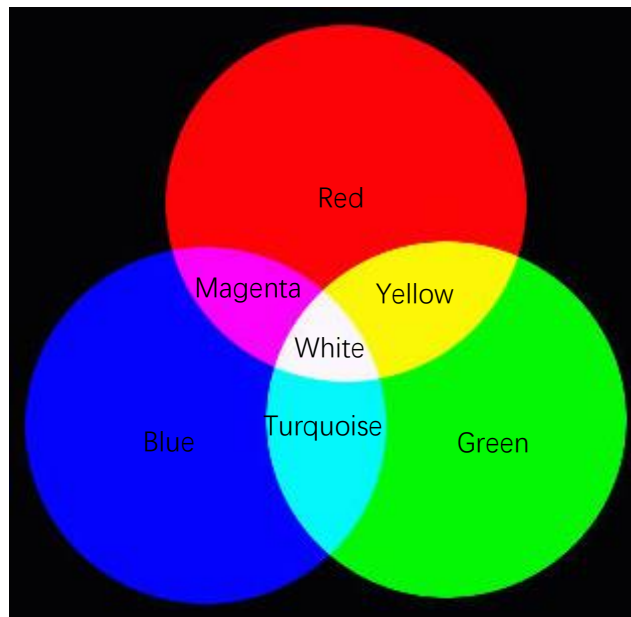
`Serial.read()` reads one byte of data from the Serial receive buffer and returns it as an int. If no data is available to read, it returns -1.

Chapter 2 RGB

Project 2.1 RGB

Related Knowledge

Red, green, and blue are called the three primary colors. When you combine these three primary colors of different brightness, it can produce almost all kinds of visible light.



RGB

The onboard RGB LED can emit three basic colors of red, green and blue, and supports 256-level brightness adjustment, which means that it can emit $2^{24}=16,777,216$ different colors.

Component List

Freenove ESP32 Display x 1

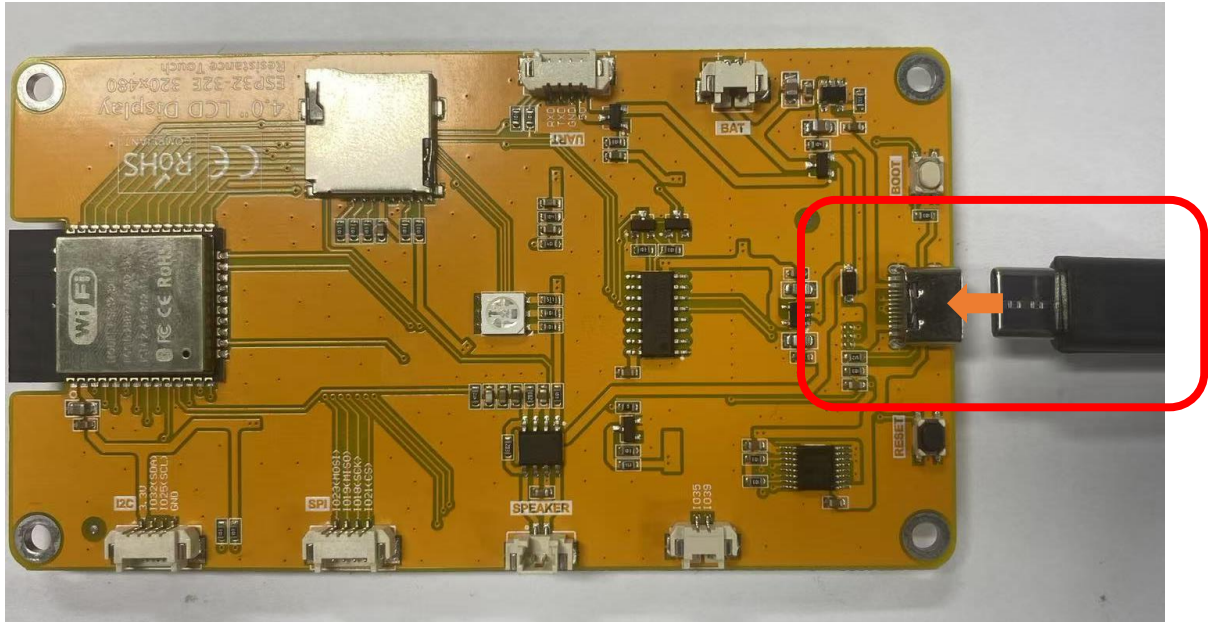


USB cable x1



Circuit

Connect Freenove ESP32 Display to the computer with USB cable.



Sketch

Open “**Sketch_02.1_RGB**” folder under “**Freenove_ESP32_Display\Sketches**” and double-click “**Sketch_02.1_RGB.ino**”.

Sketch_02.1_RGB

The following is the program code:

```

1  #include <Arduino.h>
2  #define RED_PIN 22
3  #define GREEN_PIN 16
4  #define BLUE_PIN 17
5
6  void rgbInit(void) {
7      pinMode(RED_PIN, OUTPUT);
8      pinMode(GREEN_PIN, OUTPUT);
9      pinMode(BLUE_PIN, OUTPUT);
10 }
11
12 void setRGB(bool redLevel, bool greenLevel, bool blueLevel) {
13     digitalWrite(RED_PIN, !redLevel);
14     digitalWrite(GREEN_PIN, !greenLevel);
15     digitalWrite(BLUE_PIN, !blueLevel);
16 }
17

```

```
18 void setup() {
19     rgbInit();
20 }
21
22 void loop() {
23     setRGB(1, 0, 0);
24     delay(500);
25     setRGB(0, 1, 0);
26     delay(500);
27     setRGB(0, 0, 1);
28     delay(500);
29     setRGB(0, 0, 0);
30     delay(500);
31 }
```

Code Explanation

Define the pins for the RGB LED.

```
2 #define RED_PIN 22
3 #define GREEN_PIN 16
4 #define BLUE_PIN 17
```

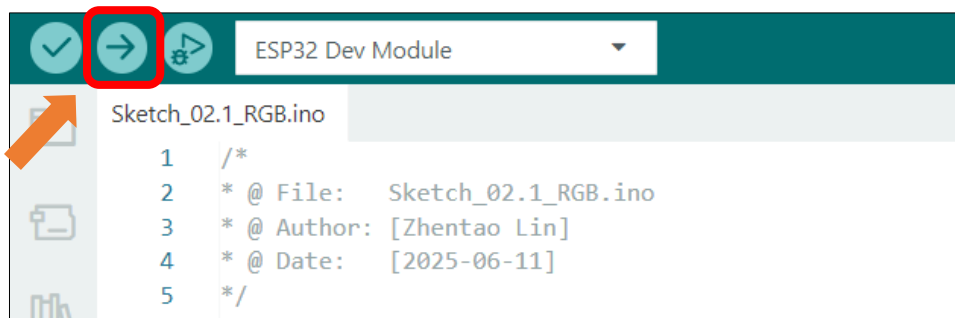
Initialize the LED.

```
19 rgbInit();
```

Change the color of the RGB LED continuously with a cycle period of 500ms.

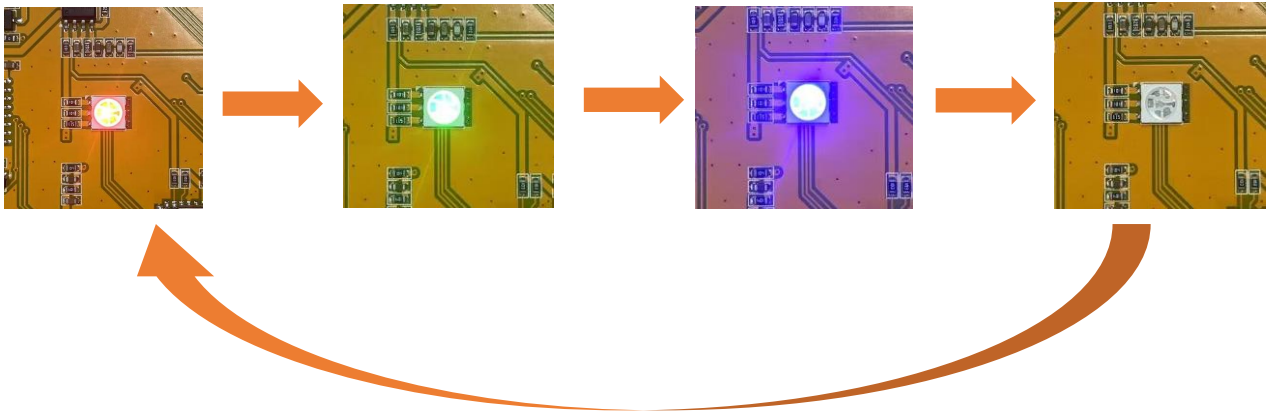
```
23 setRGB(1, 0, 0);
24 delay(500);
25 setRGB(0, 1, 0);
26 delay(500);
27 setRGB(0, 0, 1);
28 delay(500);
29 setRGB(0, 0, 0);
30 delay(500);
```

Click “**Upload**” to upload the code to Freenove_ESP32_Display.



Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to Chapter 10 for testing.

The RGB LEDs will cycle through Red -> Green -> Blue -> OFF every 500ms after code upload.



Reference

```
void setRGB(bool redLevel, bool greenLevel, bool blueLevel)
```

This function controls the RGB light to display different colors.

Parameters:

redLevel: When set to 1, the red light turns on; when 0, it turns off.

greenLevel: When set to 1, the green light turns on; when 0, it turns off.

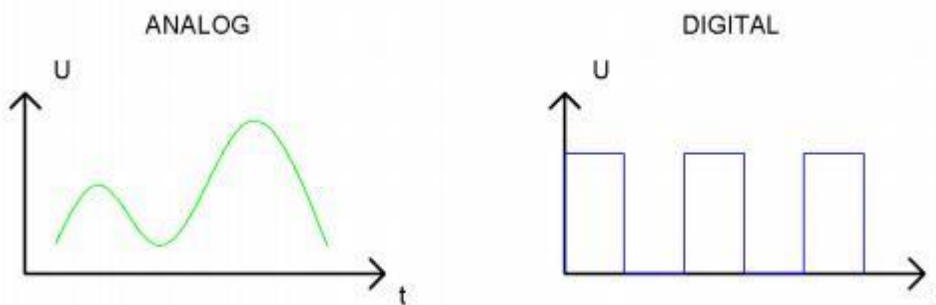
blueLevel: When set to 1, the blue light turns on; when 0, it turns off.

Project 2.2 PWM RGB

Related Knowledge

Analog & Digital

An Analog Signal is a continuous signal in both time and value. On the contrary, a Digital Signal or discrete - time signal is a time series consisting of a sequence of quantities. Most signals in life are analog signals. A familiar example of an Analog Signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0°C to 10°C. However, Digital Signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.

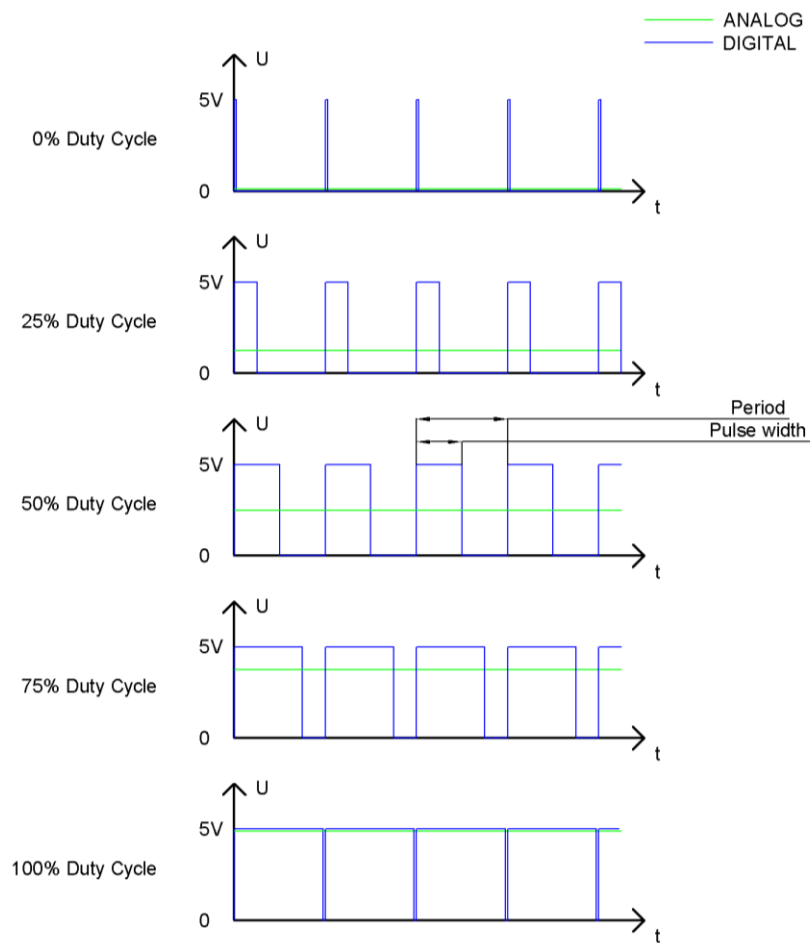


In practical application, we often use binary as the digital signal, which is a series of 0's and 1's. Since a binary signal only has two values (0 or 1), it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse Width Modulation, uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (the reciprocal of the period is frequency). The time of high level outputs are generally called "pulse width", and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform.

The longer the output of high levels last, the larger the duty cycle and the higher the corresponding voltage in analog signal will be. The following figures show how the analog signal voltage vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



Component List

Freenove ESP32 Display x 1

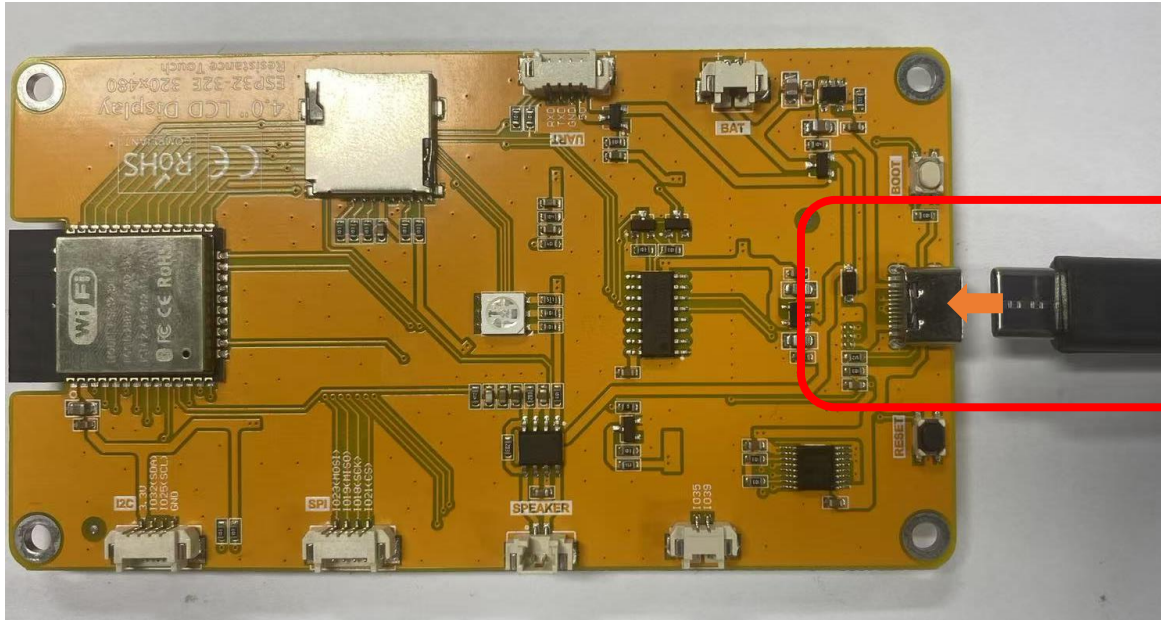


USB cable x1



Circuit

Connect Freenove ESP32 Display to the computer with USB cable.



Sketch

Open “**Sketch_02.2_PWM_RGB**” folder under “**Freenove_ESP32_Display\Sketches**” and double-click “**Sketch_02.2_PWM_RGB.ino**”.

Sketch_02.2_PWM_RGB

The following is the program code:

```
1  #include <Arduino.h>
2
3  #define RED_PIN 22
4  #define GREEN_PIN 16
5  #define BLUE_PIN 17
6  #define LEDC_RESOLUTION 8
7  #define LEDC_FREQ 2000
8  #define LEDC_RED_CHANNEL 0
9  #define LEDC_GREEN_CHANNEL 1
10 #define LEDC_BLUE_CHANNEL 2
11
12 #define PWM_BRIGHTNESS 255
13 #define DELAY_TIME 2
14
15 void pwmInit(void) {
16     ledcAttachChannel(RED_PIN, LEDC_FREQ, LEDC_RESOLUTION, LEDC_RED_CHANNEL);
17     ledcAttachChannel(GREEN_PIN, LEDC_FREQ, LEDC_RESOLUTION, LEDC_GREEN_CHANNEL);
```



```
18     ledcAttachChannel(BLUE_PIN, LEDC_FREQ, LEDC_RESOLUTION, LEDC_BLUE_CHANNEL);
19 }
20
21 void setPwm(int redValue, int greenValue, int blueValue) {
22     ledcWrite(RED_PIN, 255 - redValue);
23     ledcWrite(GREEN_PIN, 255 - greenValue);
24     ledcWrite(BLUE_PIN, 255 - blueValue);
25 }
26
27 void setup() {
28     pwmInit();
29 }
30
31 void loop() {
32     for (int i = 0; i <= PWM_BRIGHTNESS; i++) {
33         setPwm(i, 0, 0);
34         delay(DELAY_TIME);
35     }
36     for (int i = PWM_BRIGHTNESS; i >= 0; i--) {
37         setPwm(i, 0, 0);
38         delay(DELAY_TIME);
39     }
40
41     for (int i = 0; i <= PWM_BRIGHTNESS; i++) {
42         setPwm(0, i, 0);
43         delay(DELAY_TIME);
44     }
45     for (int i = PWM_BRIGHTNESS; i >= 0; i--) {
46         setPwm(0, i, 0);
47         delay(DELAY_TIME);
48     }
49
50     for (int i = 0; i <= PWM_BRIGHTNESS; i++) {
51         setPwm(0, 0, i);
52         delay(DELAY_TIME);
53     }
54     for (int i = PWM_BRIGHTNESS; i >= 0; i--) {
55         setPwm(0, 0, i);
56         delay(DELAY_TIME);
57     }
58 }
```

Code Explanation

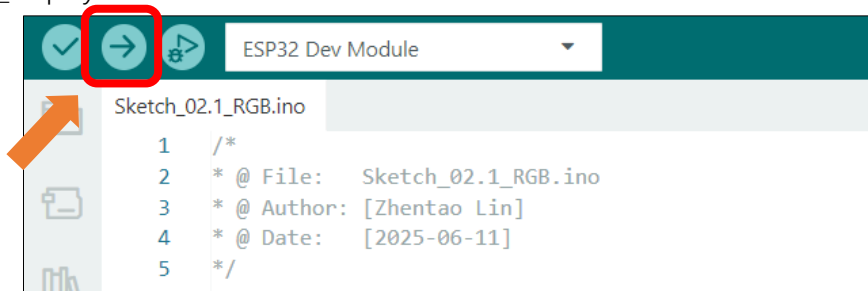
Write a PWM initialization function to configure the pins controlling the RGB lights as PWM output mode.

```
28     pwmInit();
```


Use a “for” loop to create a smooth breathing light effect on the RGB LED.

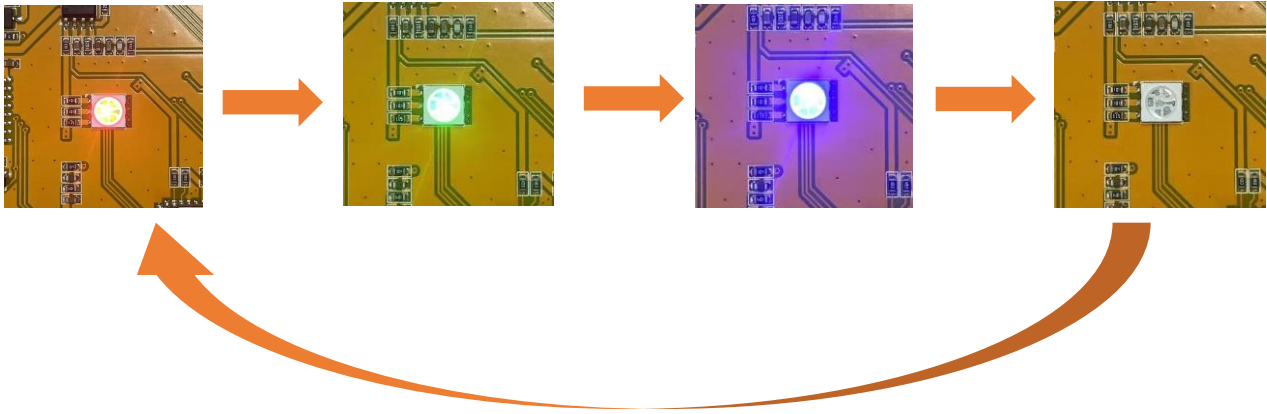
```
32 for (int i = 0; i <= PWM_BRIGHTNESS; i++) {
33     setPwm(i, 0, 0);
34     delay(DELAY_TIME);
35 }
36 for (int i = PWM_BRIGHTNESS; i >= 0; i--) {
37     setPwm(i, 0, 0);
38     delay(DELAY_TIME);
39 }
40
41 for (int i = 0; i <= PWM_BRIGHTNESS; i++) {
42     setPwm(0, i, 0);
43     delay(DELAY_TIME);
44 }
45 for (int i = PWM_BRIGHTNESS; i >= 0; i--) {
46     setPwm(0, i, 0);
47     delay(DELAY_TIME);
48 }
49
50 for (int i = 0; i <= PWM_BRIGHTNESS; i++) {
51     setPwm(0, 0, i);
52     delay(DELAY_TIME);
53 }
54 for (int i = PWM_BRIGHTNESS; i >= 0; i--) {
55     setPwm(0, 0, i);
56     delay(DELAY_TIME);
57 }
```

This code achieves the function of printing data on serial monitor. Click “**Upload**” to upload the code to Freenove_ESP32_Display.



Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.

After uploading the code, the RGB LED produces a smooth breathing effect, gradually increasing in brightness while cycling through colors in the sequence: red → green → blue → off (repeat).



Reference

```
bool ledcAttachChannel(uint8_t pin, uint32_t freq, uint8_t resolution, uint8_t channel);
```

This function binds the specified PWM channel to a GPIO pin and configures the frequency and resolution of the PWM signal.

Parameters:

pin: GPIO pin number to bind

freq: PWM signal frequency in Hz

resolution: Bit depth for PWM duty cycle resolution (range: 1-16 bits).

For example, 8 represents 2^8 (0~255) levels.

channel: PWM channel number to assign (integer)

```
void ledcWrite(uint8_t channel, uint32_t duty);
```

This function sets the duty cycle for a specified PWM channel to control output signal intensity.

Parameters:

channel: PWM channel number

duty: Duty cycle value (range determined by resolution bit depth)

Chapter 3 Button

The Boot button on the Freenove ESP32 Display can be configured as a regular input button after the program starts.

Project 3.1 Button RGB

In the previous chapter, we learned about RGB LEDs. In this section, we will further explore how to integrate RGB LEDs with buttons.

Related Knowledge

Button

In embedded systems, buttons are one of the most common human-machine input devices. When a button is not pressed, its opposing contacts are connected while adjacent contacts remain disconnected. Conversely, when the button is pressed, adjacent contacts connect while opposing contacts disconnect.

Usage of the Boot Button on the Freenove ESP32 Display:

Manual Entry into Download Mode:

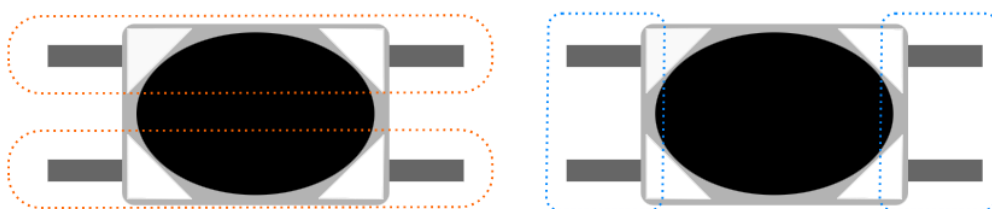
When the board is powered on or reset, pressing and holding the Boot button forces the board into download mode.

In this mode, users can upload programs to the board via the serial port.

Use as a Regular Input Button:

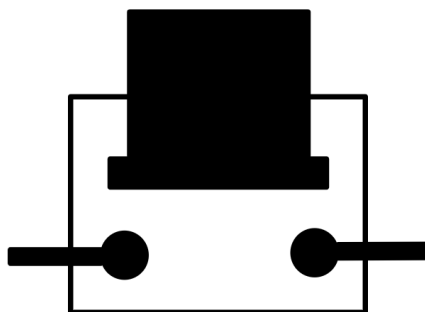
If the Boot button is not pressed during power-on or reset, the board enters normal operation mode.

Once in operation mode, the Boot button can function as GPIO0, typically configured as a standard input button for user interaction.



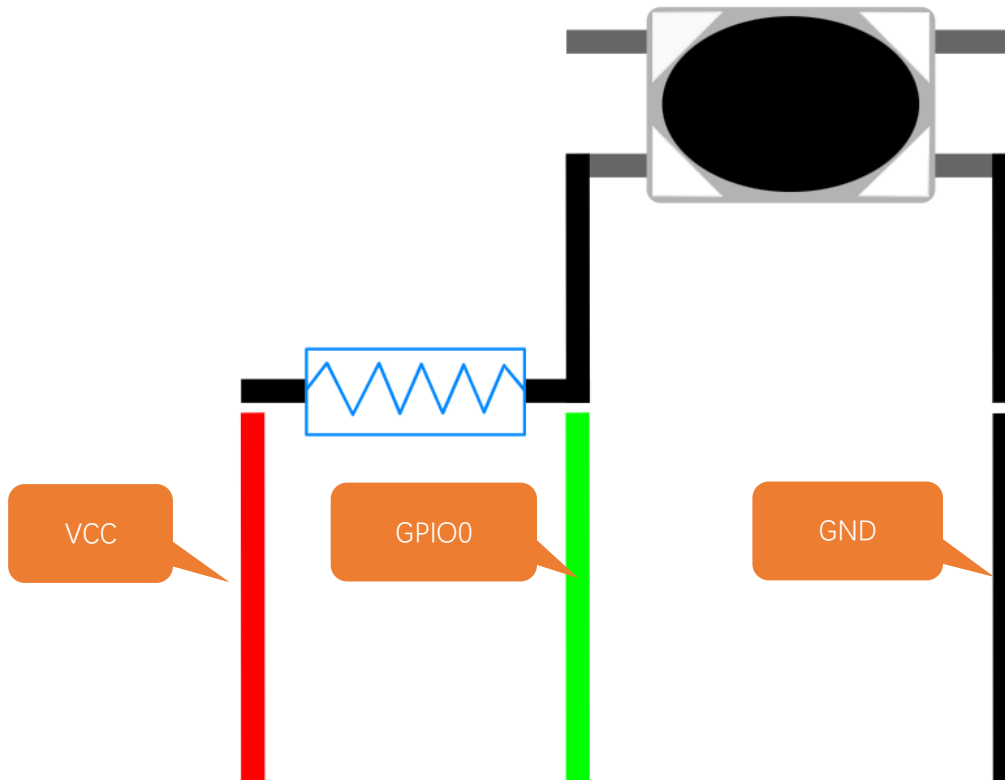
Before pressing

After pressing



Pull-up Resistor

The primary function of a pull-up resistor is to ensure that the button maintains a **stable high-level state** when not pressed, preventing false triggering caused by floating pins or signal noise. The pull-up resistor is connected between the GPIO pin and VCC.



When the button is not pressed, Vcc is connected to GPIO0 through the resistor, resulting in a high-level signal at this pin.

When the button is pressed, GPIO0 is connected to GND, resulting in a low-level signal.

Therefore, by reading the GPIO0 pin state, we can determine whether the button is pressed or not.

Component List

Freenove ESP32 Display x 1

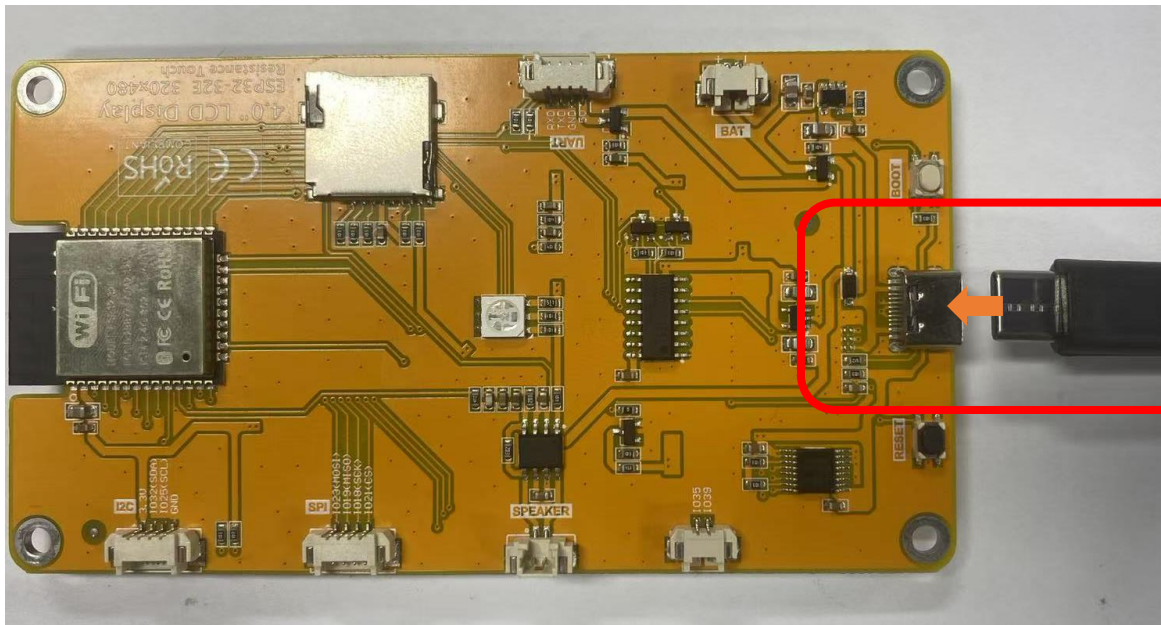


USB cable x1



Circuit

Connect Freenove ESP32 Display to the computer with USB cable.



Sketch

Open “Sketch_03.1_Button_RGB” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_03.1_Button_RGB.ino”.

Sketch_03.1_Button_RGB

The following is the program code:

```
1  #include <Arduino.h>
2  #define RED_PIN 22
3  #define GREEN_PIN 16
4  #define BLUE_PIN 17
5  #define KEY_PIN 0
6
7  static unsigned int keyCount = 0;
8  static unsigned int lastKeyCount = 1;
9
10 void buttonInit(void) {
11     pinMode(KEY_PIN, INPUT_PULLUP);
12 }
13
14 bool readButton(void) {
15     return digitalRead(KEY_PIN);
16 }
17
18 void rgbInit(void) {
19     pinMode(RED_PIN, OUTPUT);
20     pinMode(GREEN_PIN, OUTPUT);
21     pinMode(BLUE_PIN, OUTPUT);
22 }
23
24 void setRGB(bool redLevel, bool greenLevel, bool blueLevel) {
25     digitalWrite(RED_PIN, !redLevel);
26     digitalWrite(GREEN_PIN, !greenLevel);
27     digitalWrite(BLUE_PIN, !blueLevel);
28 }
29
30 void switchRGB(int value) {
31     int colorIndex = value % 4;
32     switch (colorIndex) {
33         case 1:
34             setRGB(1, 0, 0);
35             break;
36         case 2:
37             setRGB(0, 1, 0);
```

```
38     break;
39     case 3:
40         setRGB(0, 0, 1);
41         break;
42     default:
43         setRGB(0, 0, 0);
44         break;
45     }
46 }
47
48 void setup() {
49     rgbInit();
50     buttonInit();
51     Serial.println("ESP32 initialization completed!");
52 }
53
54 void loop() {
55     if (readButton() == 0) {
56         delay(20);
57         if (readButton() == 0) {
58             keyCount++;
59             while (!readButton());
60         }
61     }
62     if (keyCount != lastKeyCount) {
63         switchRGB(keyCount);
64         lastKeyCount = keyCount;
65     }
66 }
```

Code Explanation

Define the pins for the button and the RGB LED.

```
2  #define RED_PIN 22
3  #define GREEN_PIN 16
4  #define BLUE_PIN 17
5  #define KEY_PIN 0
```

Initialize the RGB LED and the button.

```
49  rgbInit();
50  buttonInit();
```

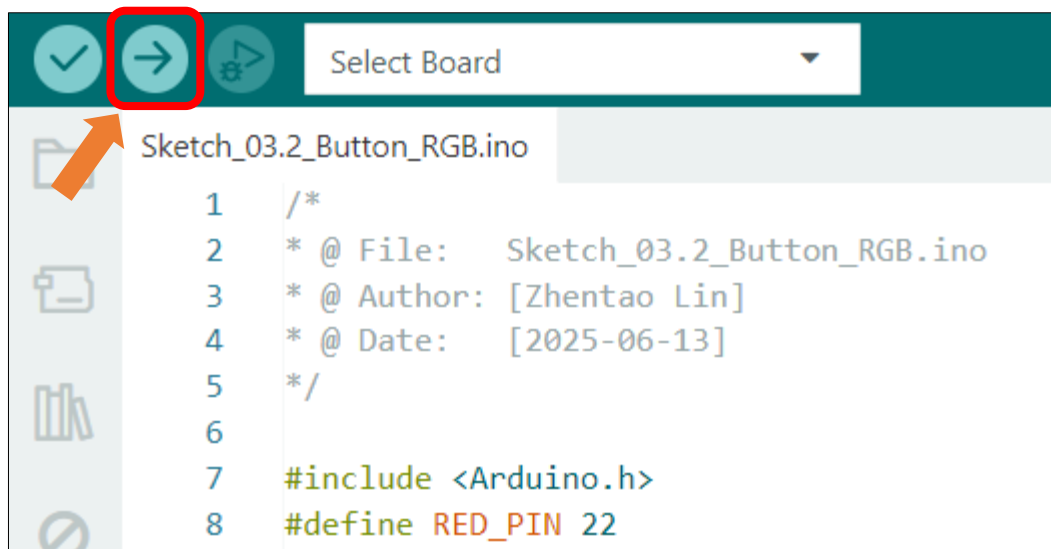
Control the color of the RGB LED through the button.

```
55  if (readButton() == 0) {
56      delay(20);
57      if (readButton() == 0) {
58          keyCount++;
59          while (!readButton());
```

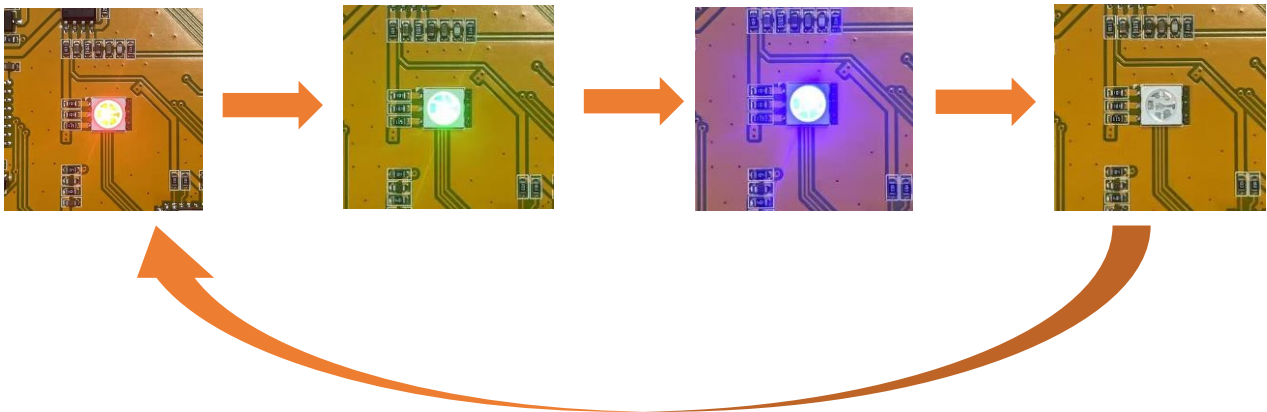
```
60     }
61   }
62   if (keyCount != lastKeyCount) {
63     switchRGB(keyCount);
64     lastKeyCount = keyCount;
65   }
```

Click “**Upload**” to upload the code to Freenove_ESP32_Display.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



Press the button and the RGB LED will change color in the sequence of red -> blue -> green -> OFF.



Chapter 4 Button Interrupt

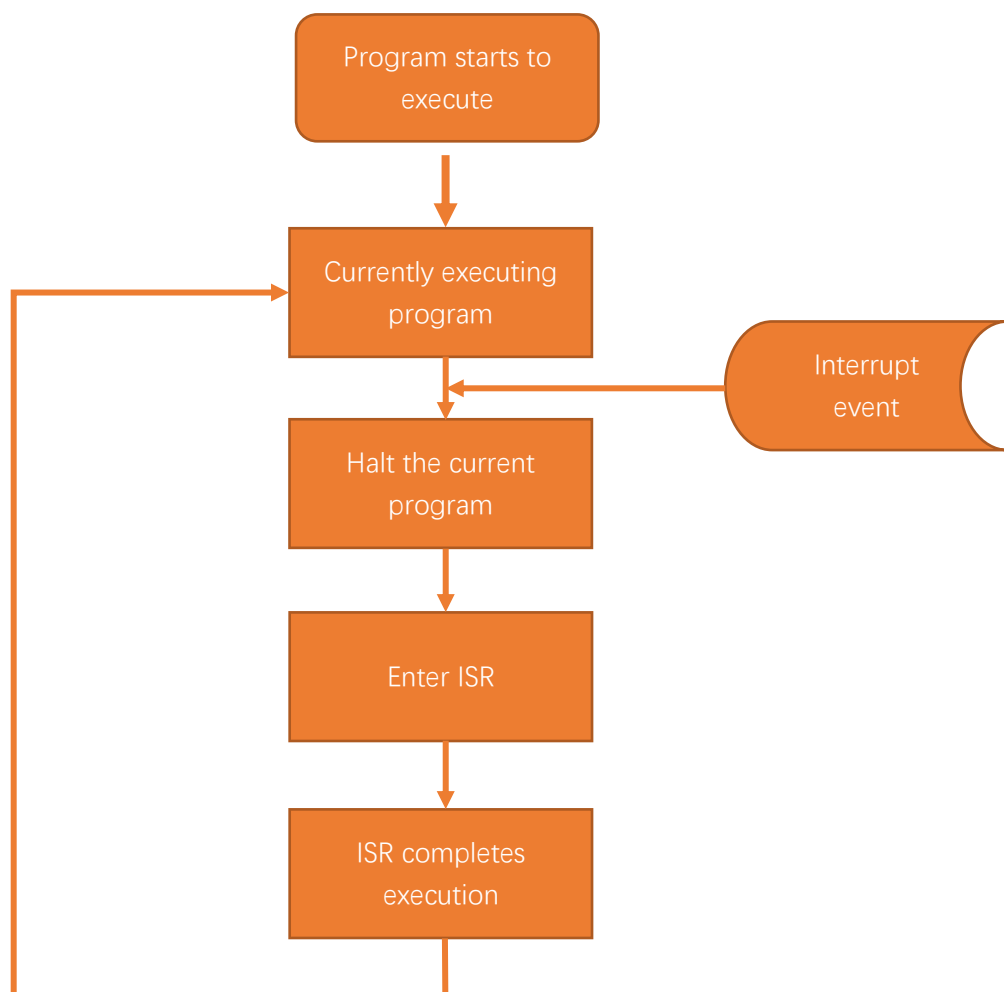
In this chapter will learn to use interrupt to detect the button state.

Project 4.1 Button Interrupt UART

Related Knowledge

How Interrupts Work

When an interrupt event is triggered, the Freenove ESP32 Display receives an interrupt signal. At this moment, the Freenove ESP32 Display pauses the currently executing program and instead executes the Interrupt Service Routine (ISR). The ISR contains the code that needs to be processed after the interrupt event occurs. Once the ISR execution is complete, the Freenove ESP32 Display will return to the state it was in before the interrupt occurred and continue executing the paused program, as shown in the diagram below.



Component List

Freenove ESP32 Display x 1

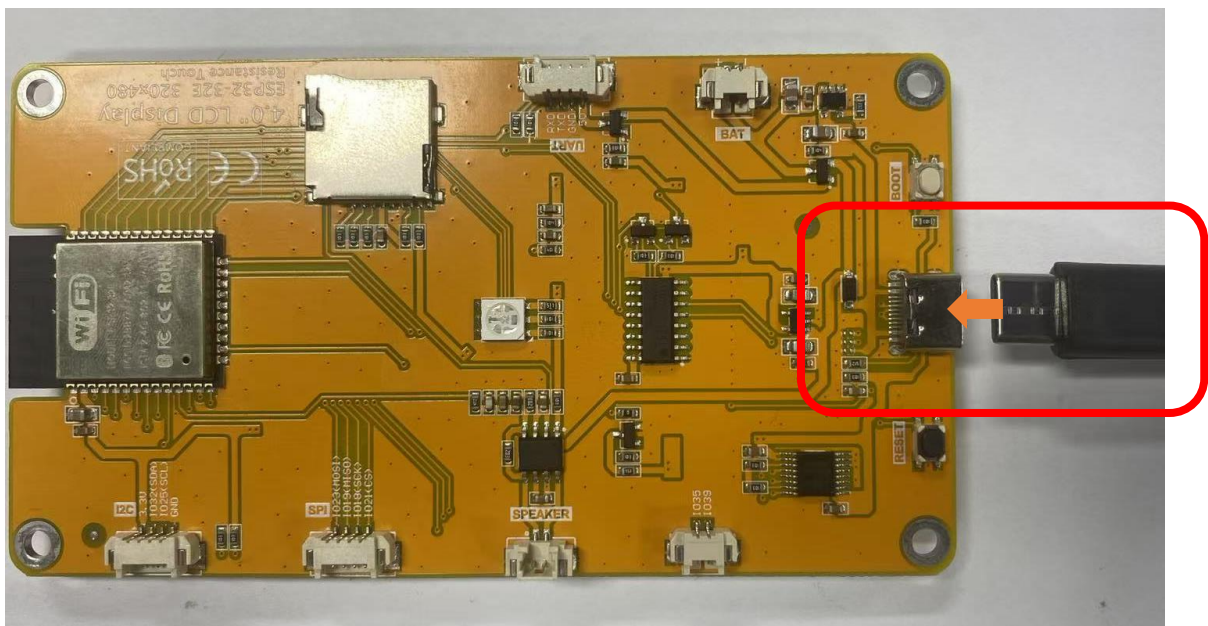


USB cable x1



Circuit

Connect Freenove ESP32 Display to the computer with USB cable.



Sketch

Open “Sketch_04.1_Button_Interrupt_UART” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_04.1_Button_Interrupt_UART.ino”.

Sketch_04.1_Button_Interrupt_UART

The following is the program code:

```
1 #include <Arduino.h>
2
3 #define KEY_PIN 0
```

```

4  volatile int interruptCounter = 0;
5  int lastValue = 0;
6  portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
7
8  void buttonInit(void) {
9      pinMode(KEY_PIN, INPUT_PULLUP);
10     attachInterrupt(digitalPinToInterrupt(KEY_PIN), handleInterrupt, FALLING);
11 }
12
13 void delayDebounce(unsigned int delayTime) {
14     unsigned int i = delayTime;
15     while (--i > 0) {
16         if (digitalRead(KEY_PIN) == HIGH) {
17             return;
18         }
19     }
20 }
21
22 void handleInterrupt(void) {
23     portENTER_CRITICAL_ISR(&mux);
24     interruptCounter++;
25     delayDebounce(20);
26     portEXIT_CRITICAL_ISR(&mux);
27 }
28
29 void setup() {
30     Serial.begin(115200);
31     buttonInit();
32     Serial.println("ESP32 initialization completed!");
33 }
34
35 void loop() {
36     if (interruptCounter != lastValue) {
37         Serial.printf("interruptCounter: %d\r\n", interruptCounter);
38         lastValue = interruptCounter;
39     }
40 }

```

Code Explanation

Define the pin of the button.

```
3  #define KEY_PIN 0
```

Configure the button pin hardware interrupt to trigger on the falling edge signal.

```
10 attachInterrupt(digitalPinToInterrupt(KEY_PIN), handleInterrupt, FALLING);
```

The interrupt processing function, counting how many times the button is pressed.

```
22 void handleInterrupt(void) {
```

```

23     portENTER_CRITICAL_ISR(&mux);
24     interruptCounter++;
25     delayDebounce(20);
26     portEXIT_CRITICAL_ISR(&mux);
27 }

```

When the number of button presses changes, print new data in the serial monitor.

```

35 void loop() {
36     if (interruptCounter != lastValue) {
37         Serial.printf("interruptCounter: %d\r\n", interruptCounter);
38         lastValue = interruptCounter;
39     }
40 }

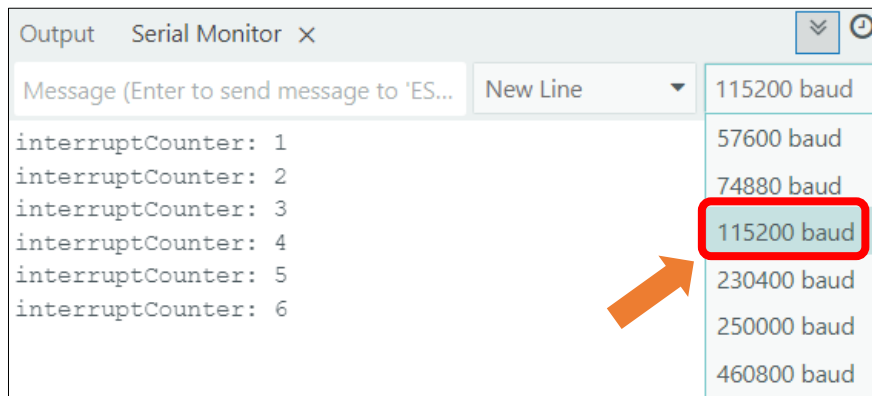
```

Click **Upload** to upload the code to Freenove_ESP32_Display.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



When the code finishes uploading, open the serial monitor and set the baud rate to 115200. The serial monitor will print the number of button presses.



Reference

```
void attachInterrupt(uint8_t pin, void (*)(void), int mode);
```

external interrupts

Parameters:

pin: The digital pin number for the interrupt

void: The name of the interrupt function

mode:

LOW (trigger on low level)

CHANGE (trigger on change)

RISING (trigger on rising edge)

FALLING (trigger on falling edge)

Chapter 5 Battery Voltage

Project 5.1 Battery Voltage

Related Knowledge

ADC

ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC on Freenove ESP32 Display is 12 bits, which means the resolution is $2^{12}=4096$, and it represents a range (at 3.3V) will be divided equally to 4096 parts. The range of analog values corresponds to ADC values. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.

Subsection 1: the analog in range of 0V---3.3/4095 V corresponds to digital 0;

Subsection 2: the analog in range of 3.3/4095 V---2*3.3 /4095 V corresponds to digital 1;

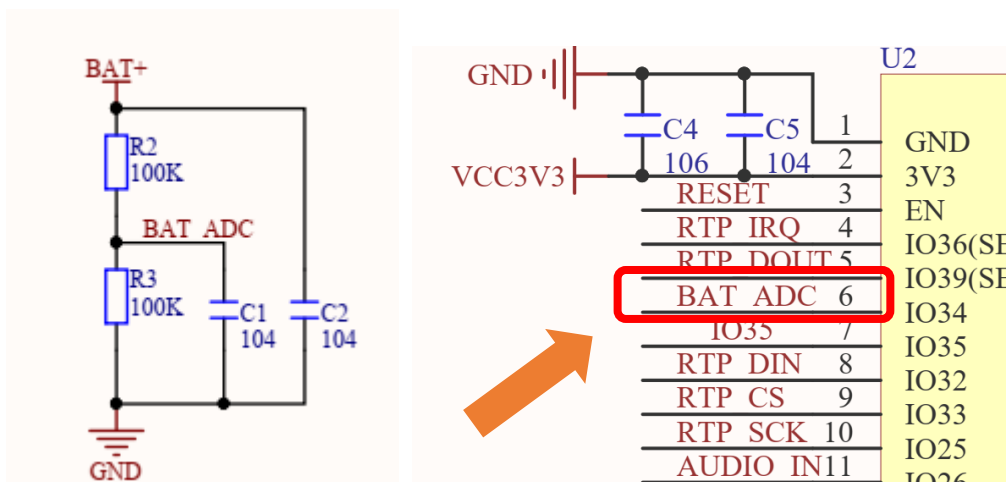
The following analog will be divided accordingly.

The conversion formula is as follows:

$$ADC\ Value = \frac{Analog\ Voltage}{3.3} * 4095$$

Battery Voltage

From the schematic diagram, we can know that Freenove_ESP32_Display reads the battery voltage through the ADC pin (GPIO34).



The ADC pin on this board has a voltage sampling range of 0~3.3V. However, when powered by a 3.7V lithium battery, the output voltage in a fully charged state can reach 4.2V, exceeding the ADC's rated input range.

To ensure accurate voltage measurement and ADC pin protection, the circuit employs a resistive voltage divider that scales the battery voltage down by a factor of 0.5. At a full charge (4.2V), the divided voltage at

the ADC input is 2.1V, well within the safe operating range. This design enables reliable battery monitoring while preventing overvoltage damage to the ADC.

Please note that this kit does not include a lithium battery, please buy one yourself. For more information about battery, please refer to [Battery](#).

Component List

Freenove ESP32 Display x 1

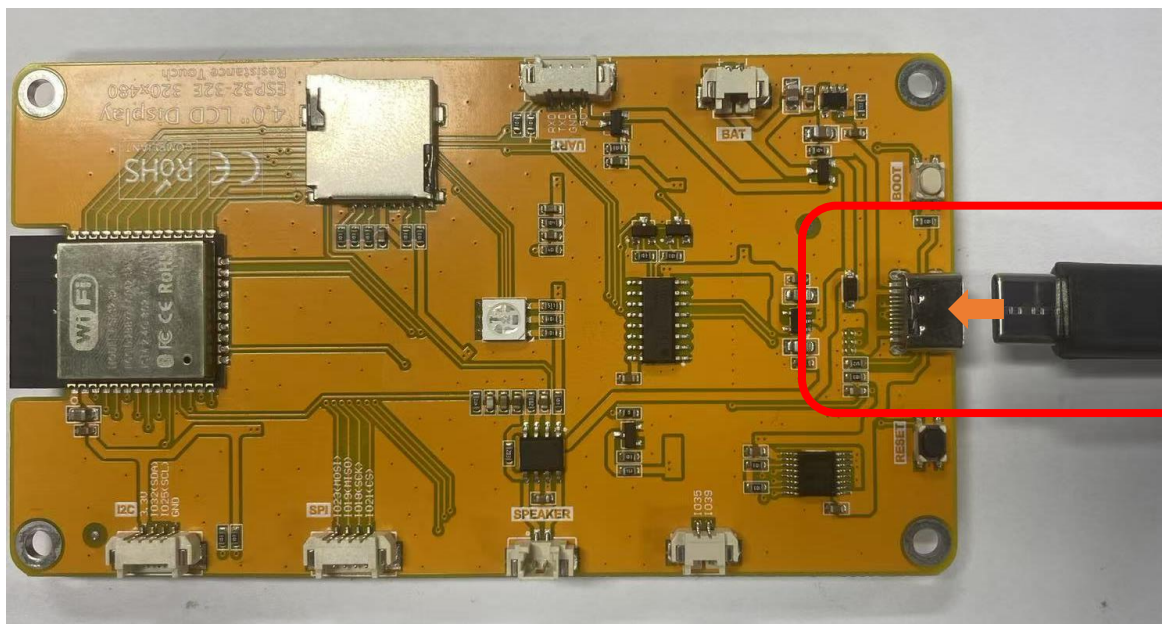


USB cable x1



Circuit

Connect Freenove ESP32 Display to the computer with USB cable.



Sketch

Next, we download the code to Freenove_ESP32_Display to test Serial. Open

“Sketch_05.1_Battery_Voltage.ino” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_05.1_Battery_Voltage.ino”.

Sketch_05.1_Battery_Voltage

The following is the program code:

```
1  #define BAT_ADC_PIN  34
2
3  void setup() {
4      Serial.begin(115200);
5      pinMode(BAT_ADC_PIN, INPUT);
6  }
7
8  void loop() {
9      int adcValue= analogRead(BAT_ADC_PIN);
10     float batteryVoltage = analogReadMilliVolts(BAT_ADC_PIN) * 2.0 / 1000;
11     Serial.printf("Reading on Pin(%d), adcValue=%d, batteryVoltage2=%fV\n", BAT_ADC_PIN,
12 adcValue, batteryVoltage);
13     delay(300);
14 }
```

Code Explanation

Define the pins for the button and RGB LED.

```
1  #define BAT_ADC_PIN  34
```

Set the baud rate to 115200.

```
4  Serial.begin(115200);
```

Set the battery voltage detecting pin to input mode.

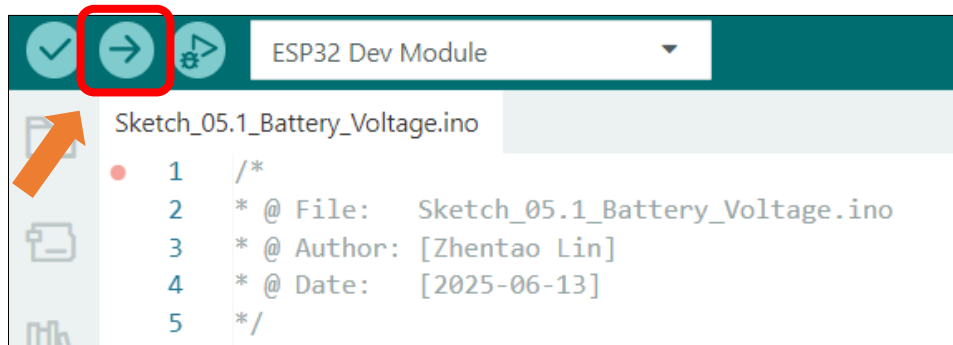
```
11 pinMode(BAT_ADC_PIN, INPUT);
```

Measure the battery voltage every 300ms and outputs the reading to the serial monitor.

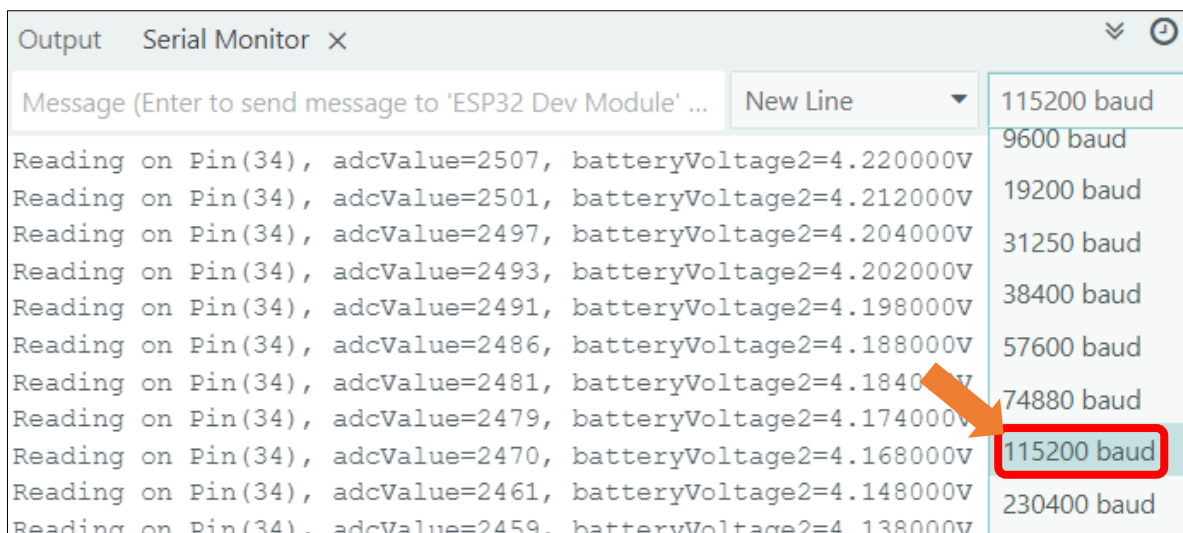
```
14 void loop() {
15     int adcValue= analogRead(BAT_ADC_PIN);
16     float batteryVoltage = analogReadMilliVolts(BAT_ADC_PIN) * 2.0 / 1000;
17     Serial.printf("Reading on Pin(%d), adcValue=%d, batteryVoltage2=%fV\n", BAT_ADC_PIN,
18 adcValue, batteryVoltage);
19     delay(300);
20 }
```


Click “**Upload**” to upload the code to Freenove_ESP32_Display.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



Once the program starts, open the serial monitor to view the battery voltage readings, updated every 300ms.



Reference

uint32_t analogReadMilliVolts(uint8_t pin)

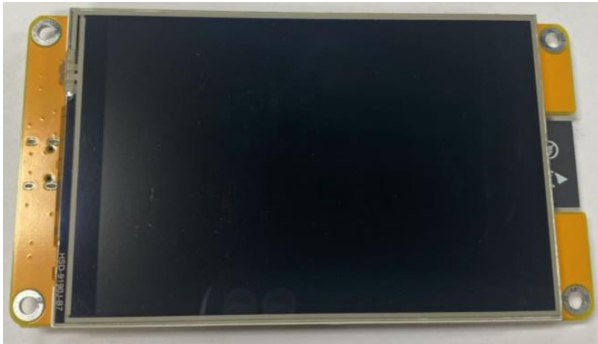
This function reads the voltage directly from the analog pin and returns the value in millivolts (mV), with a maximum return value of 3300mV (3.3V)

Chapter 6 SD Card

Project 6.1 SD Test

Component List

Freenove ESP32 Display x 1



USB cable x1



Please note that this kit does not include SD card and card reader, please buy them by yourself.

Component Knowledge

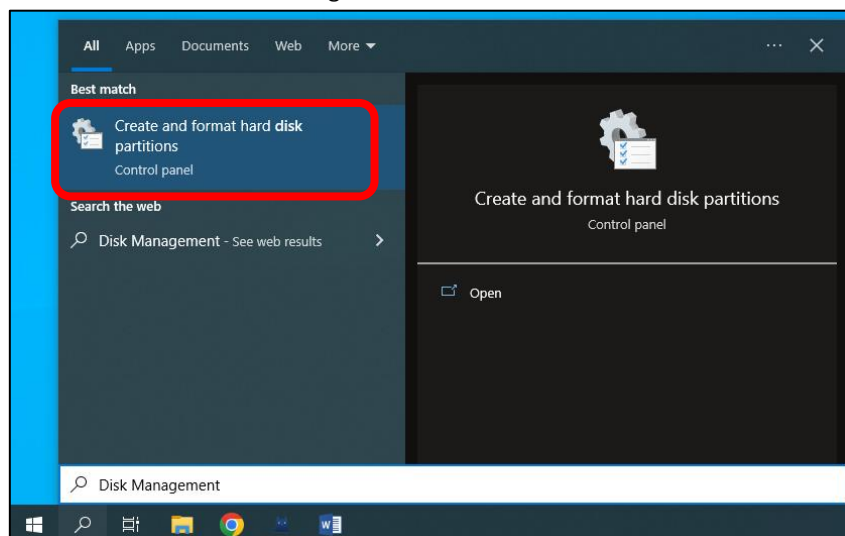
Format SD card

To initiate the project, you'll first need to prepare a blank SD card and a compatible card reader. The initial setup involves assigning a drive letter to the SD card and formatting it properly. Below you'll find system-specific instructions to complete these preparatory steps. Please follow the guide corresponding to your operating system.

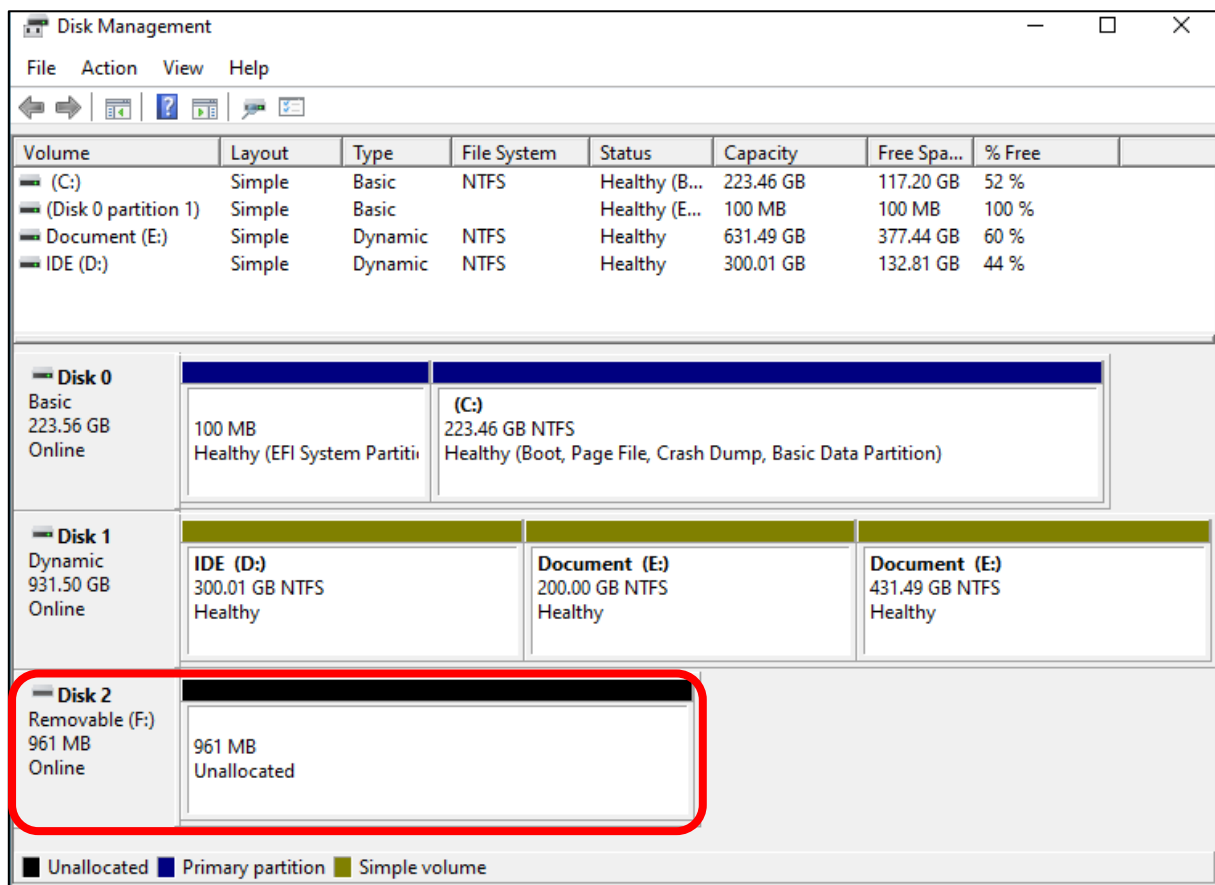
Windows

Insert the SD card into the card reader, and then insert the card reader into the computer.

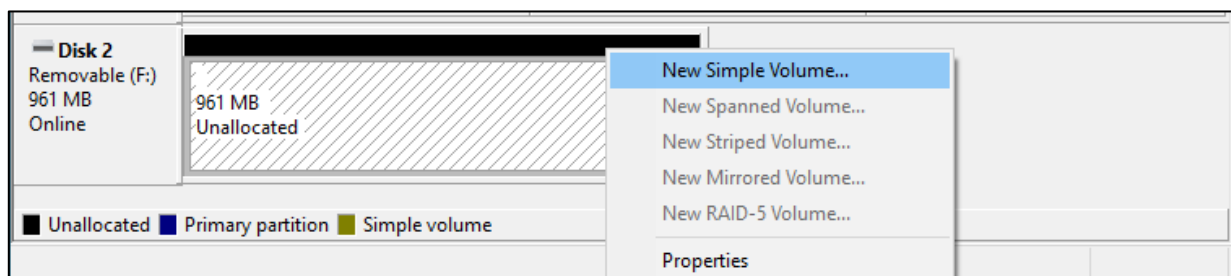
In the Windows search box, enter "Disk Management" and select "Create and format hard disk partitions".



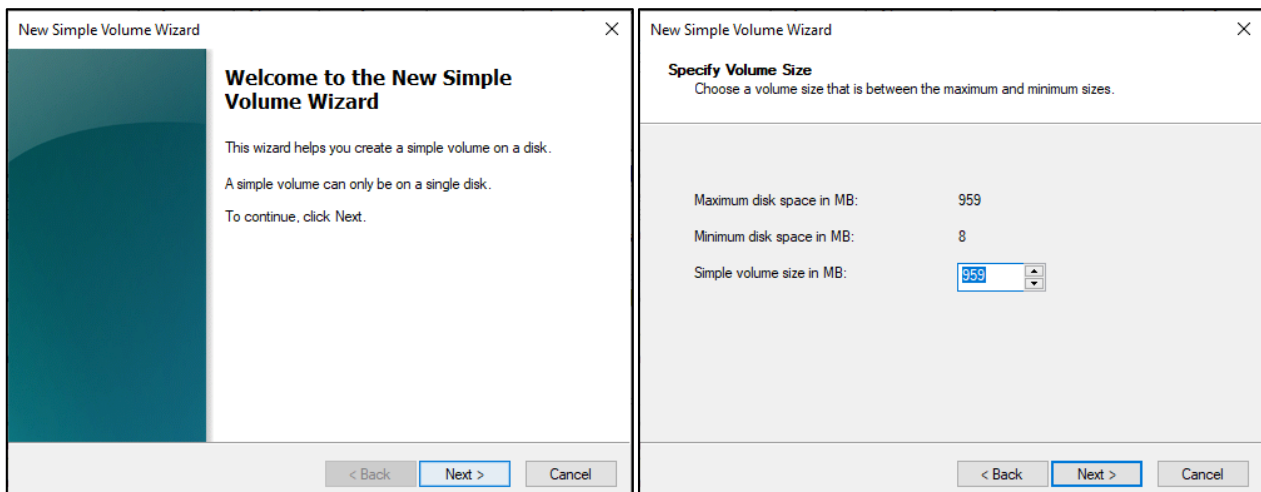
In the newly opened window, locate an unallocated volume with a size close to 1GB.



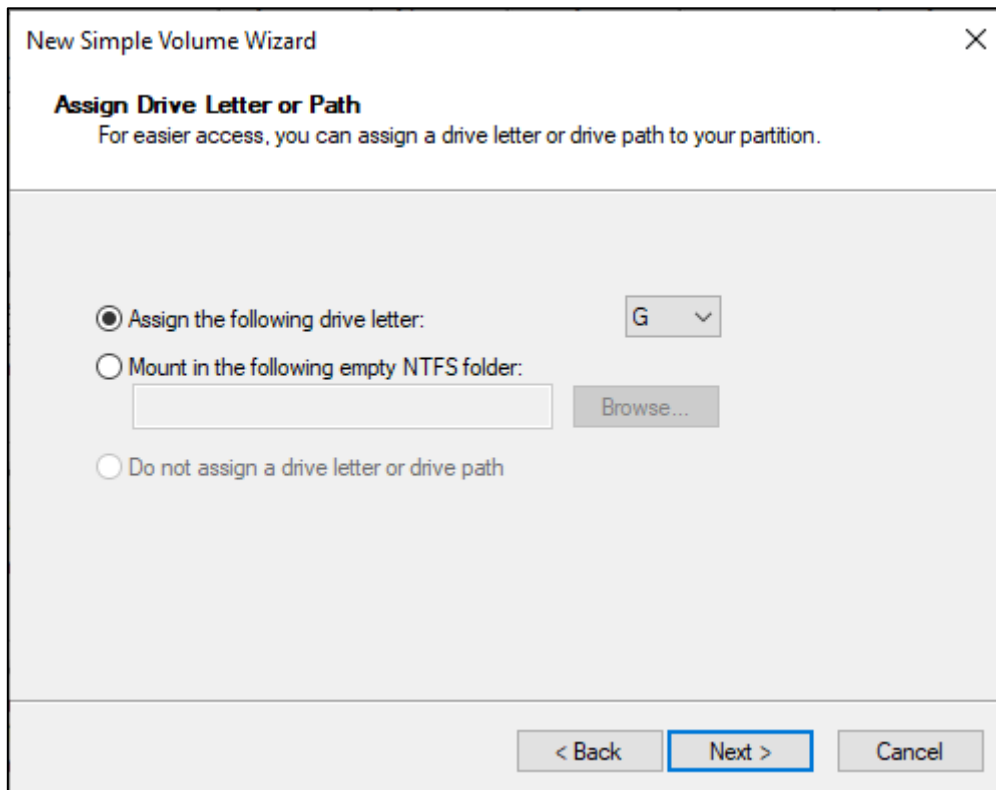
Click to select the volume, right-click and select "New Simple Volume".



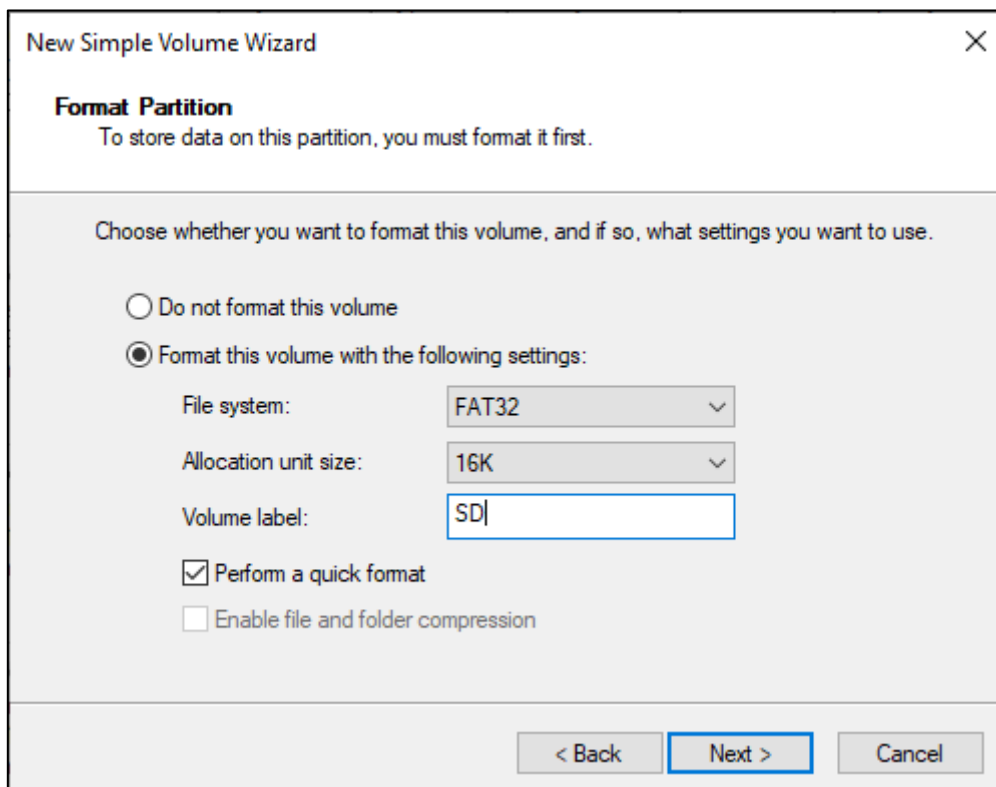
Click Next.



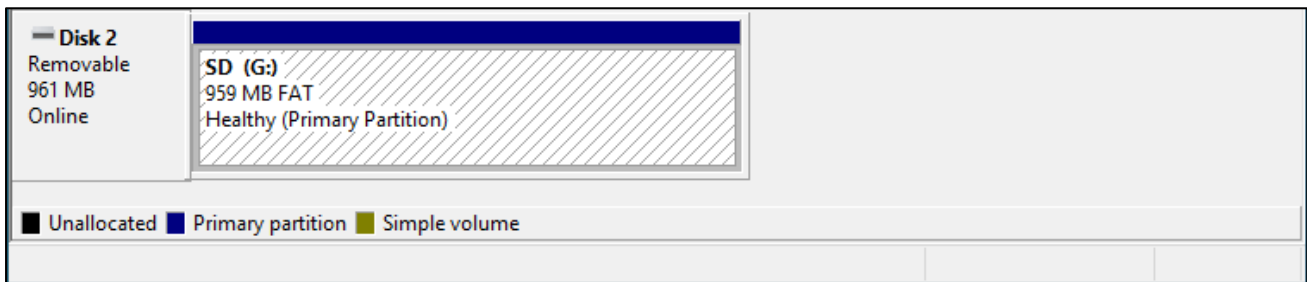
On the right-hand side, you can select a preferred drive letter for the SD card or simply proceed with the default setting by clicking on the "Next" button.



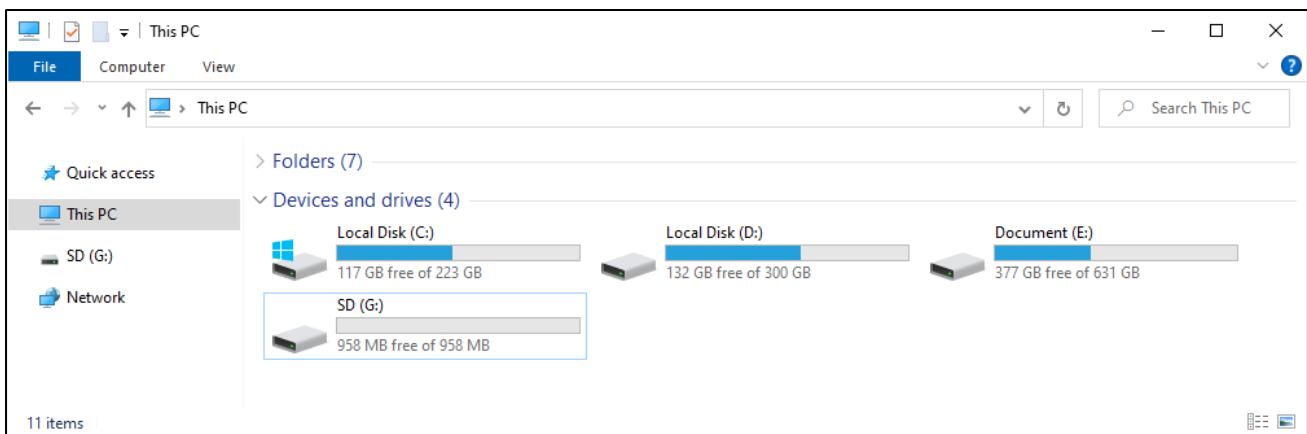
When formatting the SD card, select the file system as FAT (or FAT32) and set the allocation unit size to 16K. You can set the volume label to any name of your choice. Once you have made these selections, click on the "Next" button to proceed.



Click Finish. Wait for the SD card initialization to complete.



After completing the formatting process, you should be able to see the SD card in the "This PC" section of your computer.

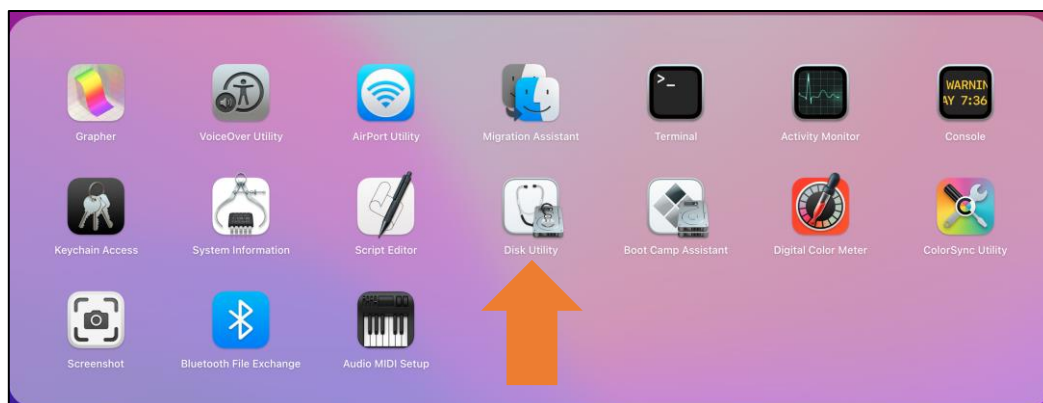


MAC

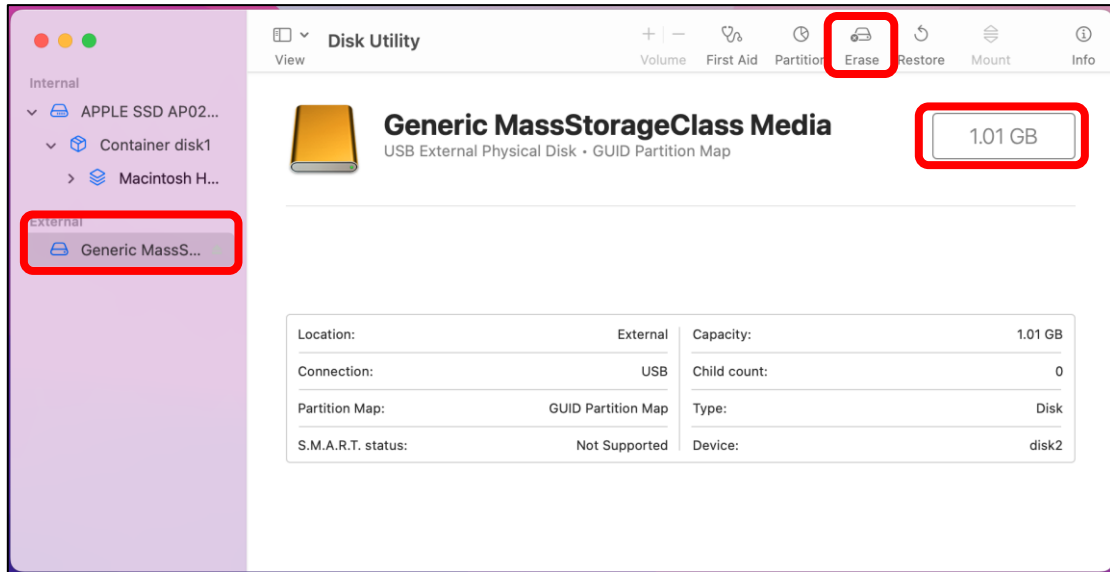
Insert the SD card into the card reader and then insert the card reader into your computer. Some computers may display a prompt with the following message. In this case, please click on the "Ignore" option to proceed.



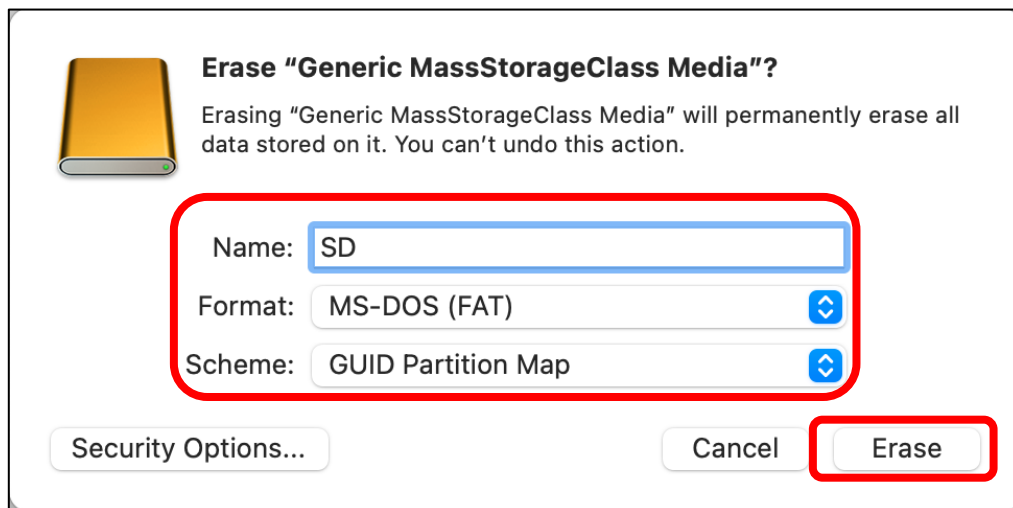
Find "Disk Utility" in the MAC system and click to open it.



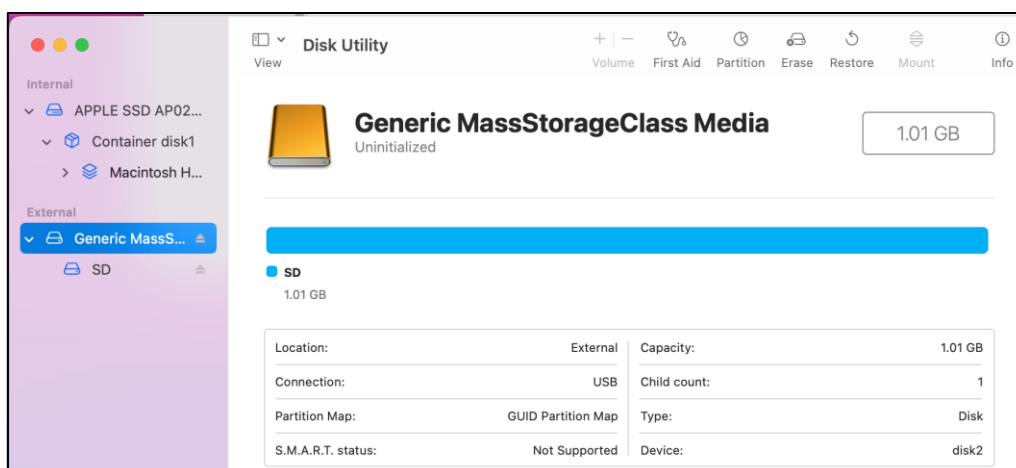
Select "Generic MassStorageClass Media", note that its size is about 1G. It is important to select the correct item to avoid accidentally erasing other device. Once you have verified that you have selected the correct device, click on the "Erase" button to proceed with erasing the SD card.



Select the configuration as shown in the figure below, and then click "Erase".



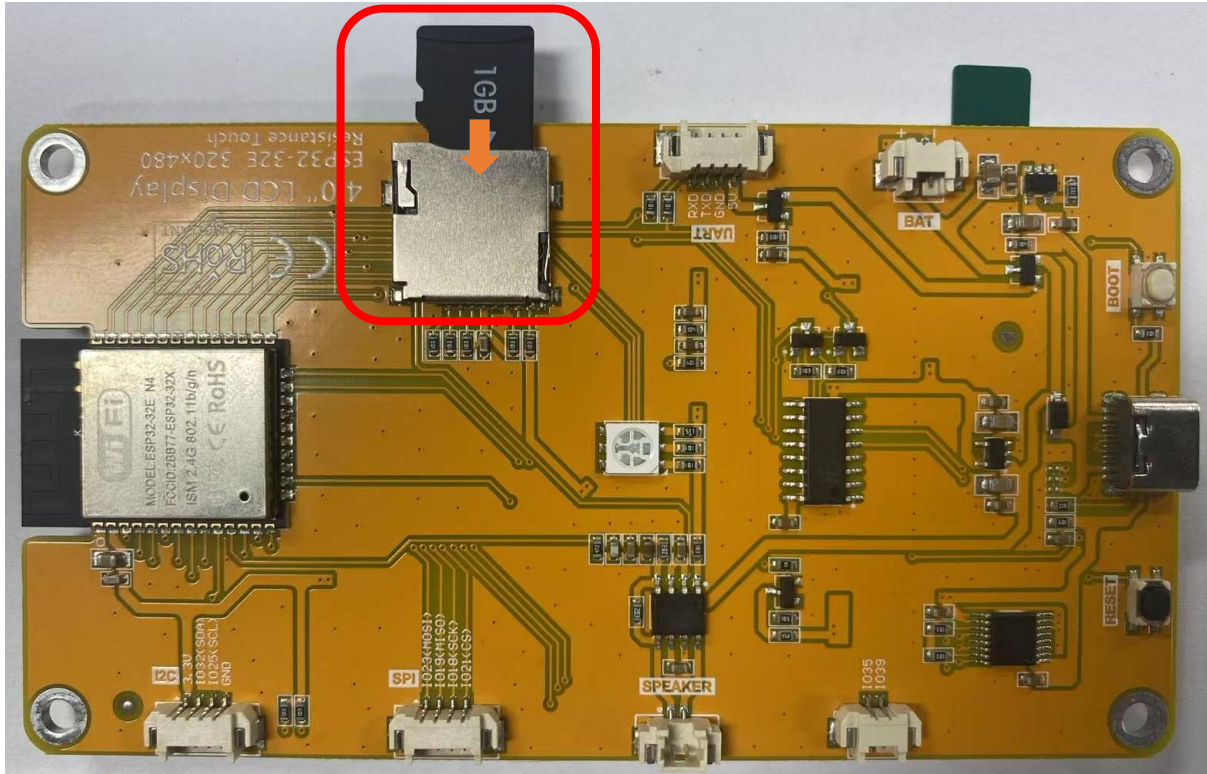
Please wait for the formatting process to complete. Once it is finished, the interface should resemble the image below. You should now be able to see a new disk named "SD" on your desktop.



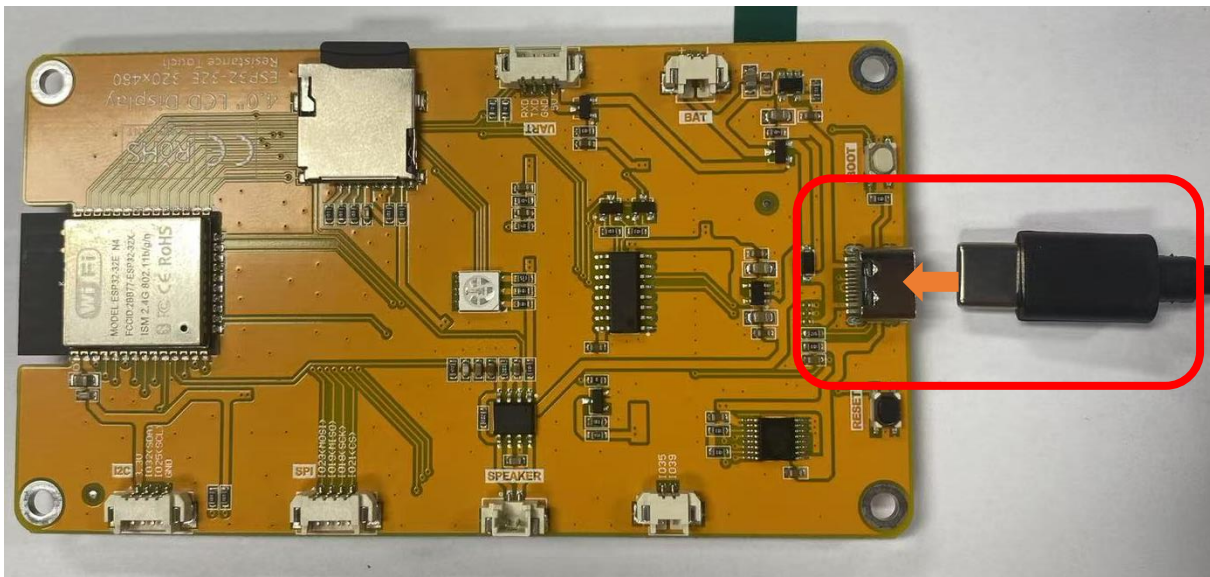
Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32.

Please note that this kit does not include SD card and card reader; please buy them yourself.



Connect Freenove ESP32 Display to the computer using the USB cable.



If you have any concerns, please feel free to contact us via support@freenove.com

Sketch

Next, we download the code to Freenove_ESP32_Display to test Serial. Open “**Sketch_06.1_SD_Test.ino**” folder under “**Freenove_ESP32_Display\Sketches**” and double-click “**Sketch_06.1_SD_Test.ino**”.

The following is the program code:

```
1  #include "sd_read_write.h"
2  #include "SPI.h"
3
4  #define SD_VSPI_SS 5
5  #define SD_VSPI_SCK 18
6  #define SD_VSPI_MISO 19
7  #define SD_VSPI_MOSI 23
8
9  SPIClass spi(VSPI);
10
11 void setup() {
12     Serial.begin(115200);
13     spi.begin(SD_VSPI_SCK, SD_VSPI_MISO, SD_VSPI_MOSI, SD_VSPI_SS);
14     if (!SD.begin(SD_VSPI_SS, spi)) {
15         Serial.println(F("SD.begin failed!"));
16         while (1) delay(1);
17     }
18     Serial.println("\r\nInitialisation done.");
19     uint8_t cardType = SD.cardType();
20     if (cardType == CARD_NONE) {
21         Serial.println("No SD card attached");
22         return;
23     }
24
25     Serial.print("SD Card Type: ");
26     if (cardType == CARD_MMC) {
27         Serial.println("MMC");
28     } else if (cardType == CARD_SD) {
29         Serial.println("SDSC");
30     } else if (cardType == CARD_SDHC) {
31         Serial.println("SDHC");
32     } else {
33         Serial.println("UNKNOWN");
34     }
35
36     uint64_t cardSize = SD.cardSize() / (1024 * 1024);
37     Serial.printf("SD Card Size: %lluMB\n", cardSize);
```

```

38
39     listDir(SD, "/", 0);
40
41     createDir(SD, "/mydir");
42     listDir(SD, "/", 0);
43
44     removeDir(SD, "/mydir");
45     listDir(SD, "/", 2);
46
47     writeFile(SD, "/hello.txt", "Hello ");
48     appendFile(SD, "/hello.txt", "World!\n");
49     readFile(SD, "/hello.txt");
50
51     deleteFile(SD, "/foo.txt");
52     renameFile(SD, "/hello.txt", "/foo.txt");
53     readFile(SD, "/foo.txt");
54
55     testFileIO(SD, "/test.txt");
56
57     Serial.printf("Total space: %lluMB\r\n", SD.totalBytes() / (1024 * 1024));
58     Serial.printf("Used space: %lluMB\r\n", SD.usedBytes() / (1024 * 1024));
59 }
60
61 void loop() {
62     delay(10000);
63 }

```

Code Explanation

Include necessary header files.

```

1  #include "sd_read_write.h"
2  #include "SPI.h"

```

Define SD card pins. In this example, we read and write the SD card via SPI.

```

4  #define SD_VSPI_SS 5
5  #define SD_VSPI_SCK 18
6  #define SD_VSPI_MISO 19
7  #define SD_VSPI_MOSI 23

```

Initialize SPI bus.

```

9  SPIClass spi(VSPI);

```

Set the baud rate to 115200.

```

12 Serial.begin(115200);

```

Start SPI communication and try to mount the SD card.

```

13 spi.begin(SD_VSPI_SCK, SD_VSPI_MISO, SD_VSPI_MOSI, SD_VSPI_SS);
14 if (!SD.begin(SD_VSPI_SS, spi)) {
15     Serial.println(F("SD.begin failed!"));
16     while (1) delay(1);

```



```
17 }
```

List all files under the root directory.

```
39 listDir(SD, "/", 0);
```

Create a directory named "mydir" and list all files under the root directory.

```
41 createDir(SD, "/mydir");
```

```
42 listDir(SD, "/", 0);
```

Delete the "mydir" directory and list all files under the root directory.

```
44 removeDir(SD, "/mydir");
```

```
45 listDir(SD, "/", 0);
```

Write "Hello" in the hello.txt file, append "World!" to it, then read and print the file contents.

```
47 writeFile(SD, "/hello.txt", "Hello ");
```

```
48 appendFile(SD, "/hello.txt", "World!\n");
```

```
49 readFile(SD, "/hello.txt");
```

Delete the file "foo.txt". Rename hello.txt to foo.txt and read the file.

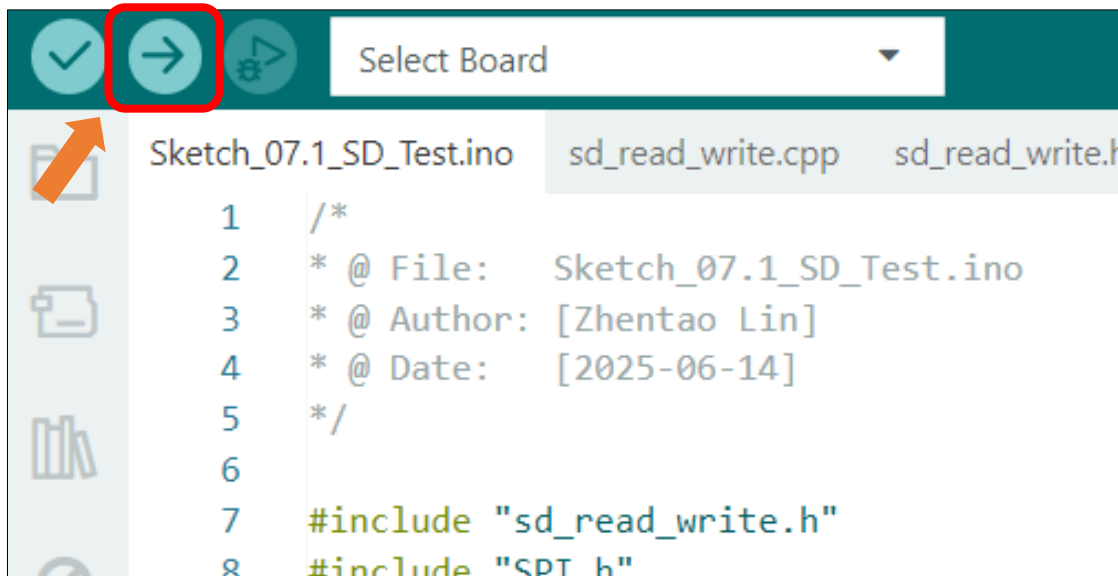
```
51 deleteFile(SD, "/foo.txt");
```

```
52 renameFile(SD, "/hello.txt", "/foo.txt");
```

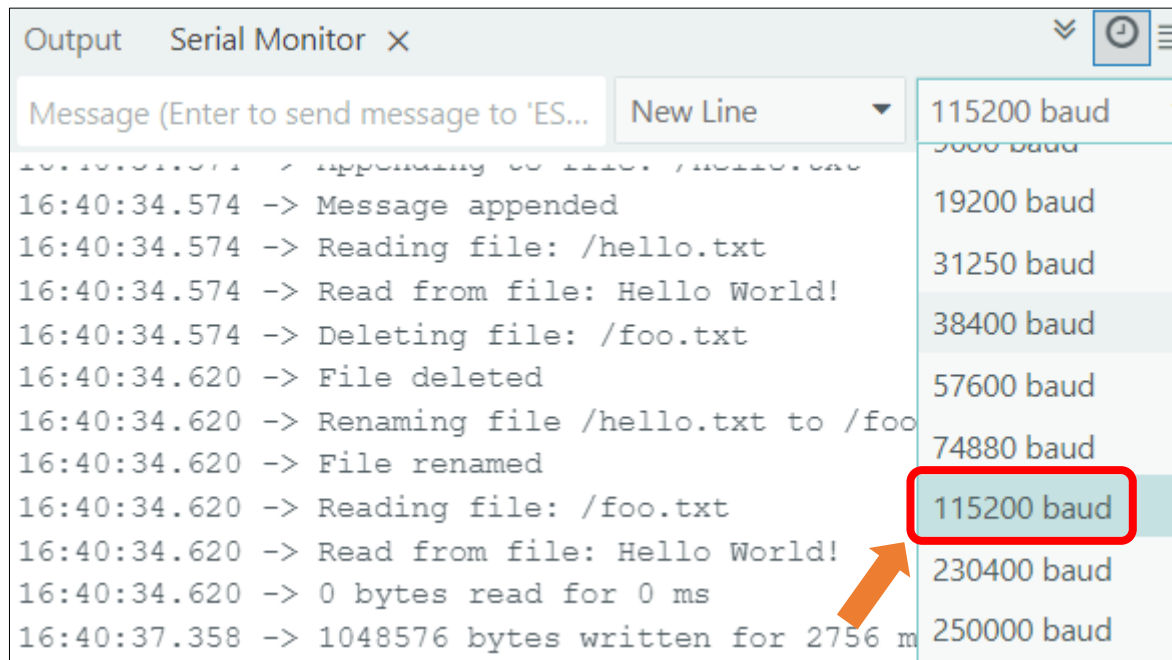
```
53 readFile(SD, "/foo.txt");
```

Click "Upload" to upload the code to Freenove_ESP32_Display.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



Upon the code runs, open the serial and set the baud rate to 115200.



Chapter 7 Play MP3

Project 7.1 Play MP3 SD by DAC

Component List

Freenove ESP32 Display x 1



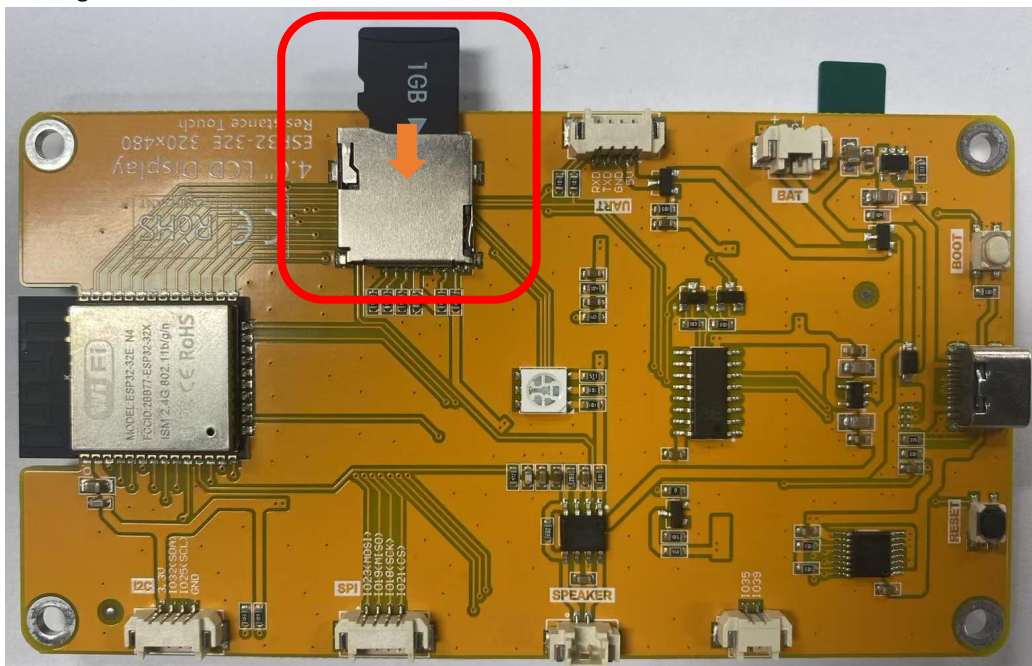
USB cable x1



This product does not include speaker, SD card, and SD card reader, please buy them by yourself. For more information, please refer to [Speaker](#), [SD card](#) sections.

Circuit

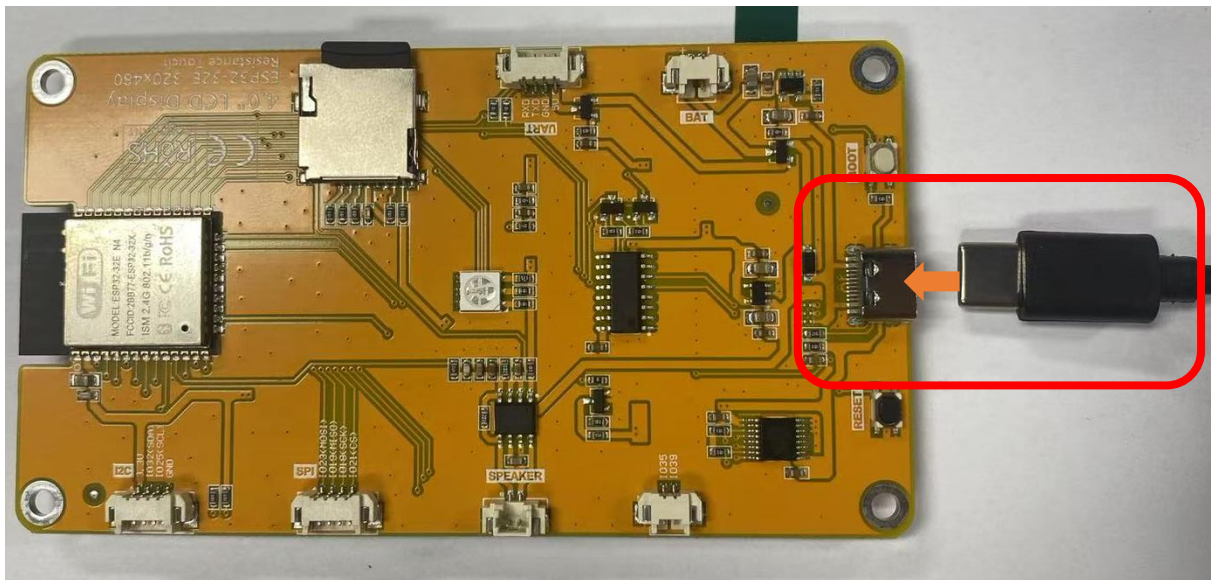
Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32 Display.



Connect the speaker.



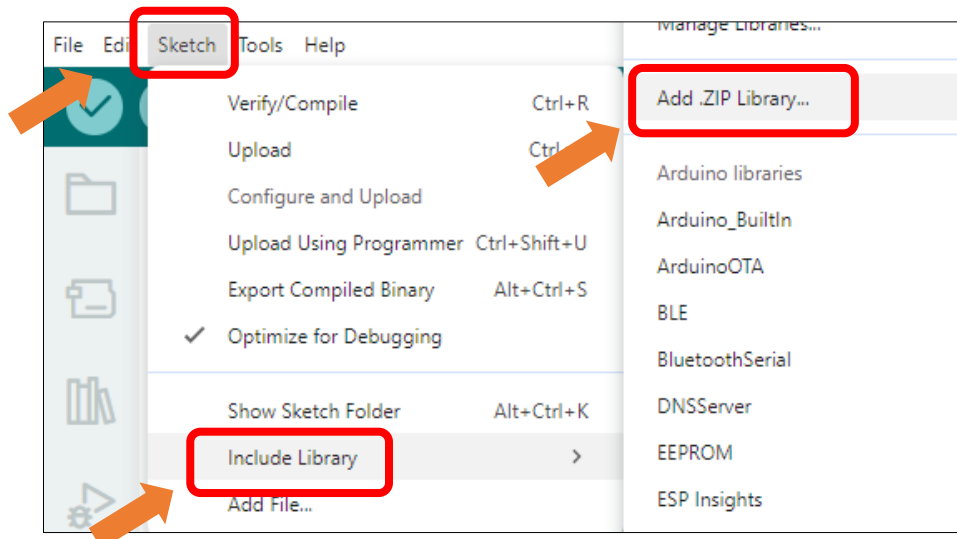
Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Install the needed libraries.

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Select **ESP8266Audio_v2.0.0.zip**



Name	Date modified	Type	Size
 ESP8266Audio_v2.0.0.zip	2025/5/30 14:06	WinRAR ZIP...	7,821 KB
 lvgl_v8.4.0.zip	2025/5/30 13:52	WinRAR ZIP...	26,083 KB
 TFT_eSPI_Setups_v1.0.zip	2025/6/11 15:45	WinRAR ZIP...	45 KB
 TFT_eSPI_v2.5.43.zip	2025/6/11 15:45	WinRAR ZIP...	5,975 KB
 TFT_Touch_v0.3.zip	2025/5/30 13:51	WinRAR ZIP...	14 KB
 Tjpg_Decoder_v1.1.0.zip	2025/5/30 14:08	WinRAR ZIP...	313 KB

Next, we download the code to Freenove_ESP32_Display to test. Open

“**Sketch_07.1_Play_MP3_SD_by_DAC**” folder under “**Freenove_ESP32_Display\Sketches**” and double-click “**Sketch_07.1_Play_MP3_SD_by_DAC.ino**”.

[Sketch_07.1_Play_MP3_SD_by_DAC](#)

The following is the program code:

```

1  #include <Arduino.h>
2  #include <WiFi.h>
3  #include "AudioFileSourceSD.h"
4  #include "AudioFileSourceID3.h"
5  #include "AudioGeneratorMP3.h"
6  #include "AudioOutputI2S.h"
7
8  #define AUDIO_EN 4
9  #define SD_VSPI_SS 5
10 #define SD_VSPI_MOSI 23
11 #define SD_VSPI_MISO 19
12 #define SD_VSPI_SCK 18
13
14 AudioFileSourceSD *file;
15 AudioFileSourceID3 *id3;
16 AudioOutputI2S *out;
17 AudioGeneratorMP3 *mp3;
18
19 SPIClass *vspi = NULL;
20
21 void playMP3(char *filename) {
22     file = new AudioFileSourceSD(filename);
23     id3 = new AudioFileSourceID3(file);
24     out = new AudioOutputI2S(0, 1);
25     out->SetGain(1);
26     mp3 = new AudioGeneratorMP3();
27     mp3->begin(id3, out);
28 }
29
30 void setup() {

```

```

31 Serial.begin(115200);
32 Serial.println();
33 WiFi.mode(WIFI_OFF);
34 pinMode(AUDIO_EN, OUTPUT);
35 digitalWrite(AUDIO_EN, LOW);
36 delay(500);
37 vspi = new SPIClass(VSPI);
38 vspi->begin(SD_VSPI_SCK, SD_VSPI_MISO, SD_VSPI_MOSI, SD_VSPI_SS);
39 if (!SD.begin(SD_VSPI_SS, *vspi)) {
40     Serial.println(F("SD.begin failed!"));
41     while (1) delay(1);
42 }
43 playMP3("/Olsen-Banden.mp3");
44 }
45
46 void loop() {
47     while (mp3->isRunning()) {
48         if (!mp3->loop()) mp3->stop();
49     }
50 }

```

Code Explanation

Include necessary header files.

```

1 #include <Arduino.h>
2 #include <WiFi.h>
3 #include "AudioFileSourceSD.h"
4 #include "AudioFileSourceID3.h"
5 #include "AudioGeneratorMP3.h"
6 #include "AudioOutputI2S.h"

```

Define the pins.

```

8 #define AUDIO_EN 4
9 #define SD_VSPI_SS 5
10 #define SD_VSPI_MOSI 23
11 #define SD_VSPI_MISO 19
12 #define SD_VSPI_SCK 18

```

Declare audio objects.

```

14 AudioFileSourceSD *file;
15 AudioFileSourceID3 *id3;
16 AudioOutputI2S *out;
17 AudioGeneratorMP3 *mp3;

```

Set the baud rate to 115200

```

31 Serial.begin(115200);

```

Initialize MP3 decoder and play the music. To play other music, please change the name in the code.

```

43 playMP3("/Olsen-Banden.mp3");

```


Read audio data and play it.

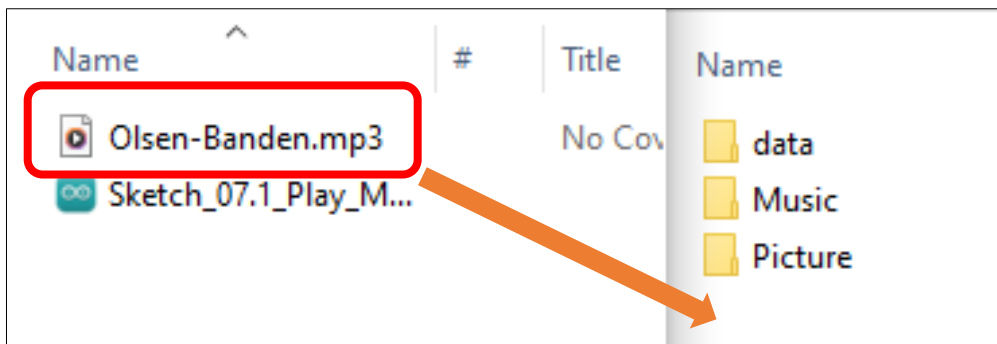
```

46 void loop() {
47     while (mp3->isRunning()) {
48         if (!mp3->loop()) mp3->stop();
49     }
50 }

```

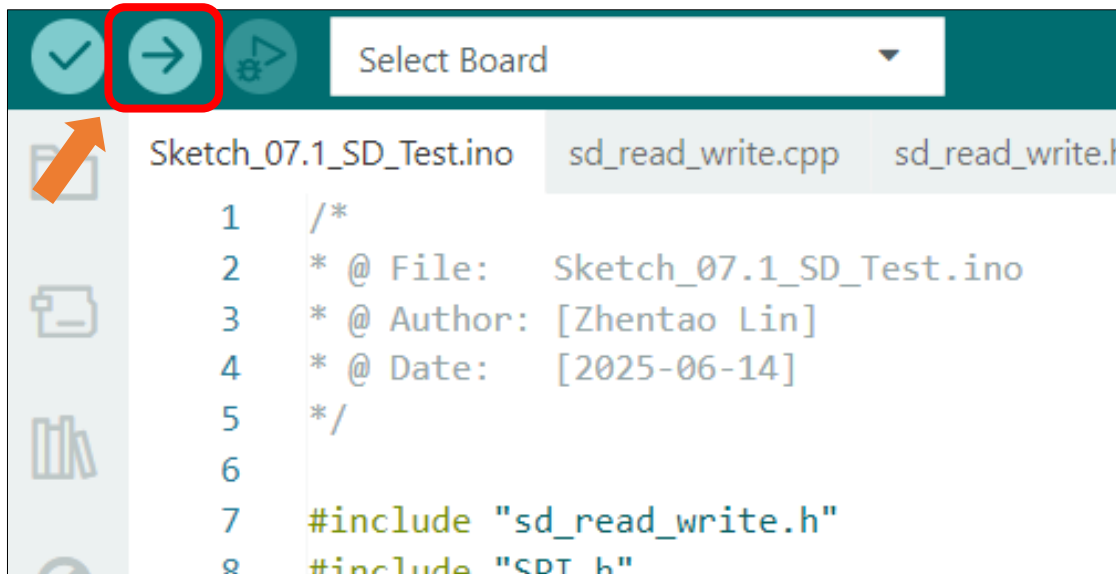
This product does not include speaker, SD card, and SD card reader, please buy them by yourself. For more information, please refer to [Speaker](#), [SD card](#) sections.

Before uploading the code, copy the music to the root directory of the SD card with the SD card reader.



Click "Upload" to upload the code to Freenove ESP32 Display.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



The speaker plays the music in the SD card.

Chapter 8 BLE

Project 8.1 BLE USART

Component List

Freenove ESP32 Display x 1  A photograph of a Freenove ESP32 Display module. It features a black rectangular screen mounted on a yellow PCB. The PCB has several pins and components visible along its edges.	USB cable x1  A photograph of a black USB cable. It has a standard USB-A connector on one end and a micro-USB connector on the other. The cable is coiled and lies on a light-colored surface.
---	---

Component Knowledge

BLE (Bluetooth Low Energy)

Low Energy Bluetooth (BLE) is a new feature introduced in the Bluetooth 4.0 specification, specifically designed for low-power devices and suitable for applications involving intermittent transmission of small amounts of data.

The BLE architecture follows a client-server model and consists of the following key components:

GATT (Generic Attribute Protocol): The foundational protocol for BLE communication, defining a hierarchical data structure of services and characteristics.

Service: A collection of data that performs a specific function or feature, containing one or more characteristics.

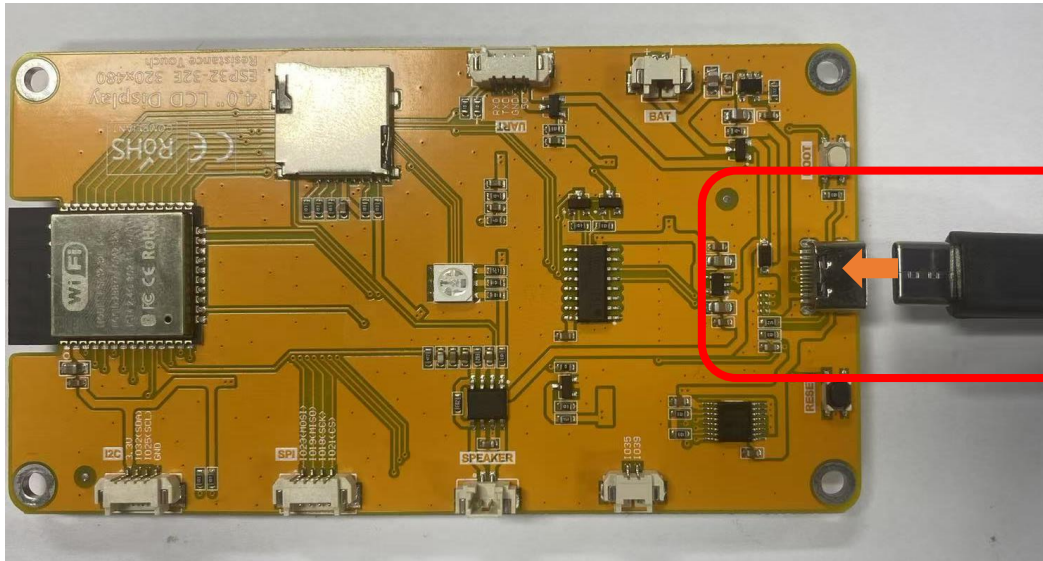
Characteristic: A specific data point within a service, consisting of a value and descriptors.

UUID: A 128-bit universally unique identifier used to distinguish services and characteristics.

BLE device can function simultaneously as a Peripheral (providing services) and a Central (connecting to peripherals). In this section, we will learn how to configure the Freenove_ESP32_Display as a peripheral device to provide services.

Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_Display to test. Open “**Sketch_08.1_BLE_USART**” folder under “**Freenove_ESP32_Display\Sketches**” and double-click “**Sketch_08.1_BLE_USART.ino**”.

Sketch_08.1_BLE_USART

The following is the program code:

```

1  #include "BLEDevice.h"
2  #include "BLEServer.h"
3  #include "BLEUtils.h"
4  #include "BLE2902.h"
5  #include "String.h"
6
7  BLECharacteristic *pCharacteristic;
8  bool deviceConnected = false;
9  uint8_t txValue = 0;
10 long lastMsg = 0;
11 String rxload="Test\n";
12
13 #define SERVICE_UUID          "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
14 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
15 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
16
17 class MyServerCallbacks: public BLEServerCallbacks {
18     void onConnect(BLEServer* pServer) {
19         deviceConnected = true;
20     };
21     void onDisconnect(BLEServer* pServer) {
22         deviceConnected = false;
23     }

```

```
24 };
25
26 class MyCallbacks: public BLECharacteristicCallbacks {
27     void onWrite(BLECharacteristic *pCharacteristic) {
28         String rxValue = pCharacteristic->getValue();
29         if (rxValue.length() > 0) {
30             rxload="";
31             for (int i = 0; i < rxValue.length(); i++){
32                 rxload +=(char)rxValue[i];
33             }
34         }
35     }
36 };
37
38 void setupBLE(String BLEName){
39     const char *ble_name=BLEName.c_str();
40     BLEDevice::init(ble_name);
41     BLEServer *pServer = BLEDevice::createServer();
42     pServer->setCallbacks(new MyServerCallbacks());
43     BLEService *pService = pServer->createService(SERVICE_UUID);
44     pCharacteristic=
45     pService->createCharacteristic(CHARACTERISTIC_UUID_TX, BLECharacteristic::PROPERTY_NOTIFY);
46     pCharacteristic->addDescriptor(new BLE2902());
47     BLECharacteristic *pCharacteristic =
48     pService->createCharacteristic(CHARACTERISTIC_UUID_RX, BLECharacteristic::PROPERTY_WRITE);
49     pCharacteristic->setCallbacks(new MyCallbacks());
50     pService->start();
51     pServer->getAdvertising()->start();
52     Serial.println("Waiting a client connection to notify...");
53 }
54
55 void setup() {
56     Serial.begin(115200);
57     setupBLE("ESP32_BLE");
58 }
59
60 void loop() {
61     long now = millis();
62     if (now - lastMsg > 100) {
63         if (deviceConnected&&rxload.length()>0) {
64             Serial.println(rxload);
65             rxload="";
66         }
67         if(Serial.available()>0){
```

```

68     String str=Serial.readString();
69     const char *newValue=str.c_str();
70     pCharacteristic->setValue(newValue);
71     pCharacteristic->notify();
72 }
73     lastMsg = now;
74 }
75 }

```

Code Explanation

Include the necessary header libraries.

```

1  #include "BLEDevice.h"
2  #include "BLEServer.h"
3  #include "BLEUtils.h"
4  #include "BLE2902.h"
5  #include "String.h"

```

Define service UUID and characteristic UUID.

```

13 #define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
14 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
15 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

```

The MyServerCallbacks class handles device connection and disconnection events and updates the deviceConnected status.

```

17 class MyServerCallbacks: public BLEServerCallbacks {
18     void onConnect(BLEServer* pServer) {
19         deviceConnected = true;
20     };
21     void onDisconnect(BLEServer* pServer) {
22         deviceConnected = false;
23     }
24 };

```

The MyServerCallbacks class handles the received data and save them to the rxload string.

```

26 class MyCallbacks: public BLECharacteristicCallbacks {
27     void onWrite(BLECharacteristic *pCharacteristic) {
28         String rxValue = pCharacteristic->getValue();
29         if (rxValue.length() > 0) {
30             rxload="";
31             for (int i = 0; i < rxValue.length(); i++){
32                 rxload +=(char)rxValue[i];
33             }
34         }
35     }
36 };

```

Set the baud rate to 115200.

```

56 Serial.begin(115200);

```

BLE Device Initialization, Service Creation, and Characteristic Setup

```
57 setupBLE("ESP32_BLE");
```

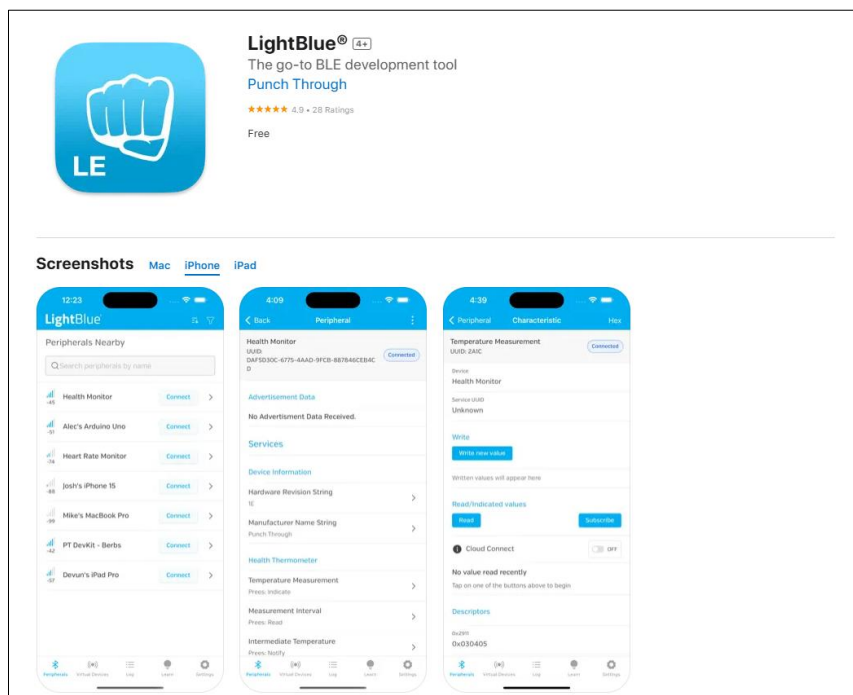
The Loop function check the connection status and serial data every 100 milliseconds.

```
60 void loop() {
61     long now = millis();
62     if (now - lastMsg > 100) {
63         if (deviceConnected && rxload.length() > 0) {
64             Serial.println(rxload);
65             rxload="";
66         }
67         if (Serial.available() > 0) {
68             String str = Serial.readString();
69             const char *newValue = str.c_str();
70             pCharacteristic->setValue(newValue);
71             pCharacteristic->notify();
72         }
73         lastMsg = now;
74     }
75 }
```

LightBlue

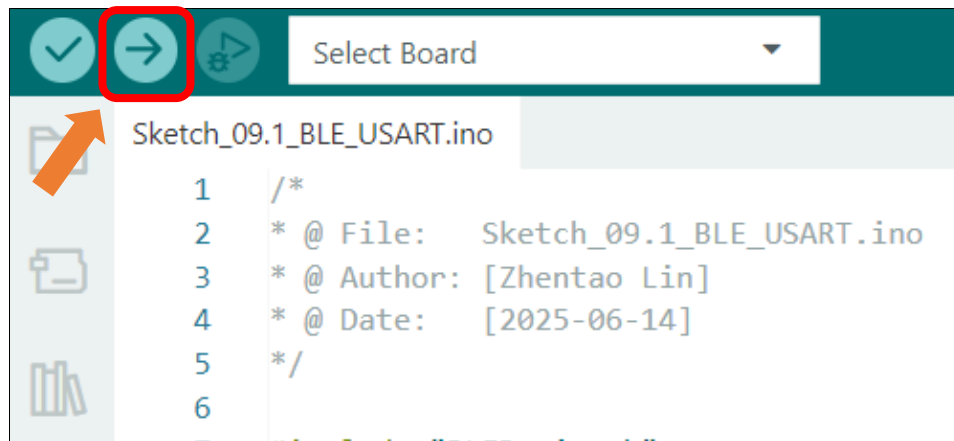
If you do not have this software installed on your phone, you can refer to this link:

<https://apps.apple.com/us/app/lightblue/id557428110>

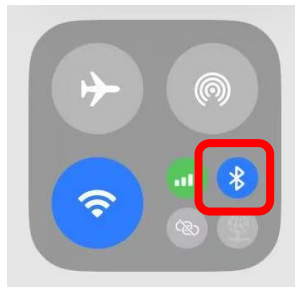


Click “**Upload**” to upload the code to Freenove ESP32 Display.

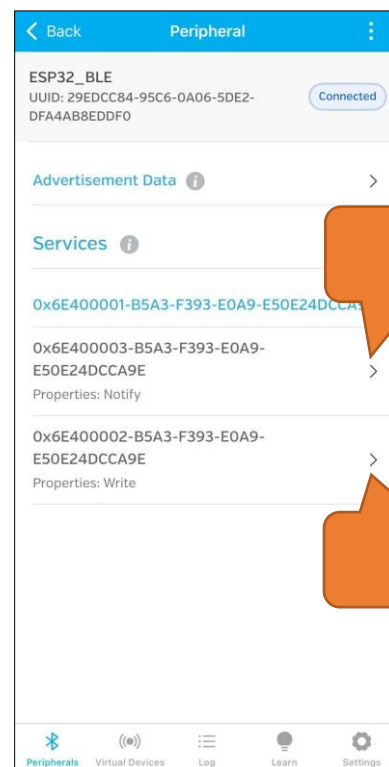
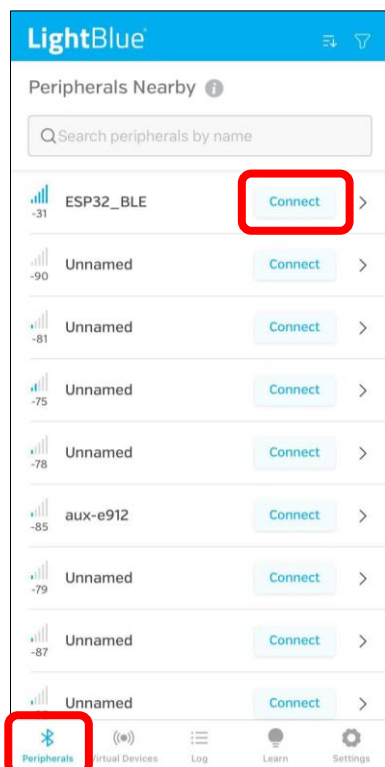
Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



Turn ON Bluetooth on your phone, and open the LightBlue APP.

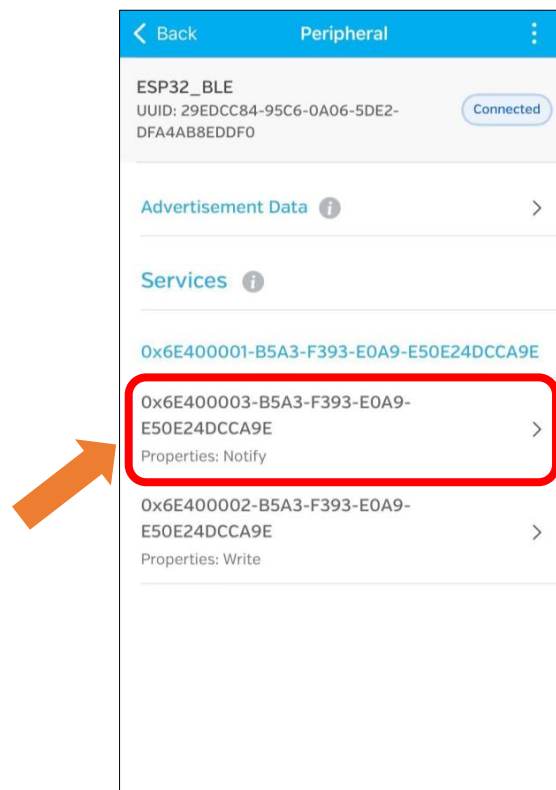


In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click the Connection button of ESP32_BLE.

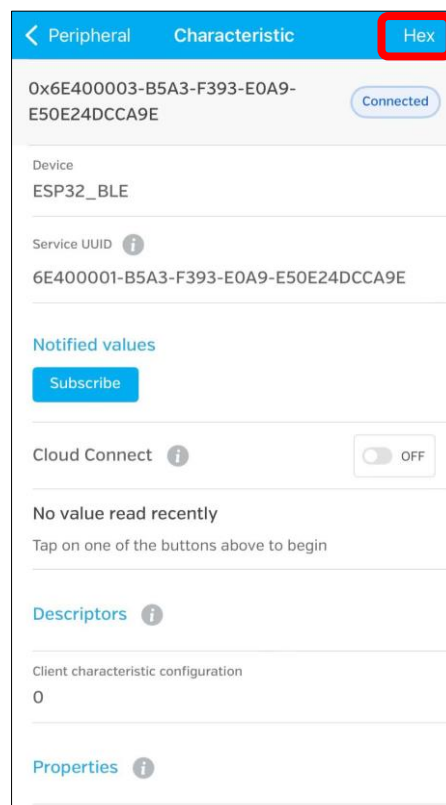


Receiving Data

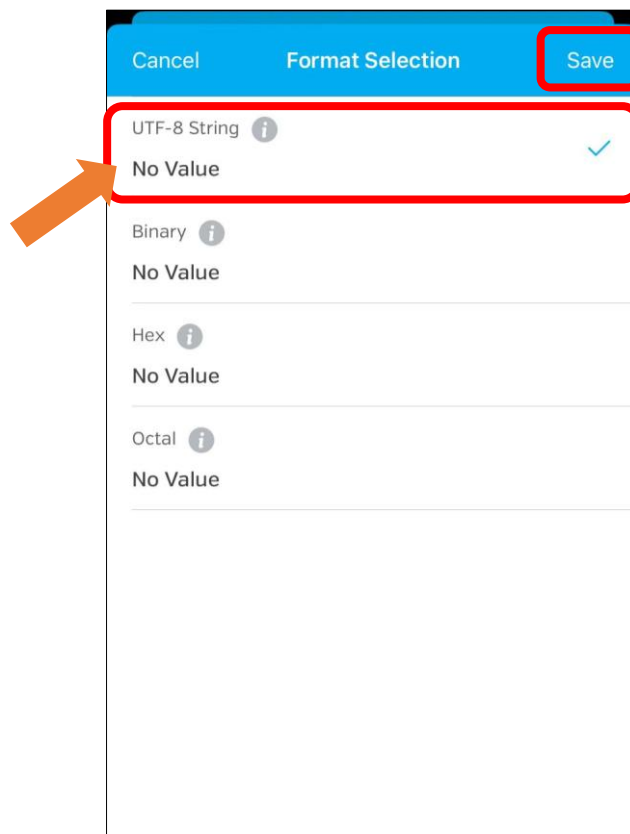
Click Notify



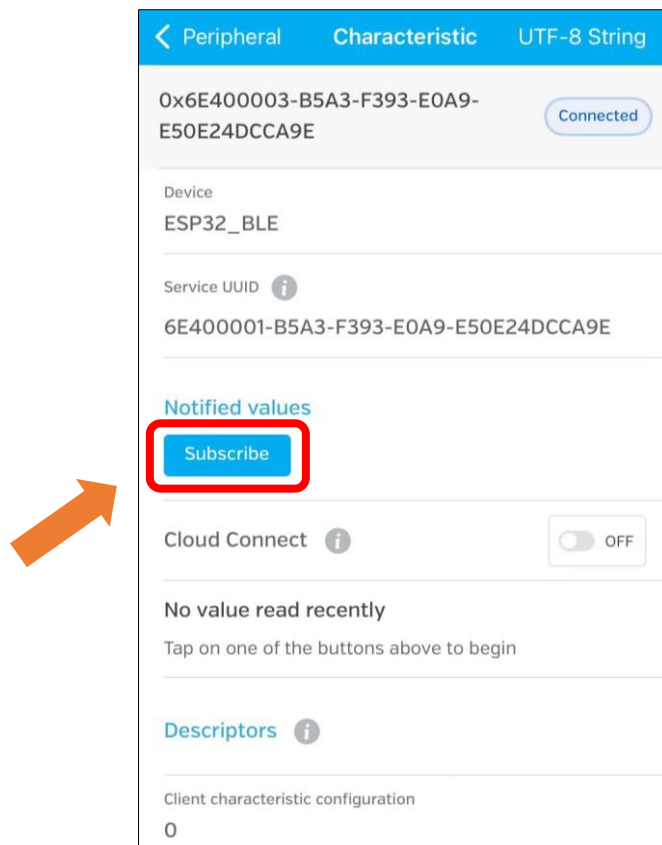
Click the top right corner to change the string type.



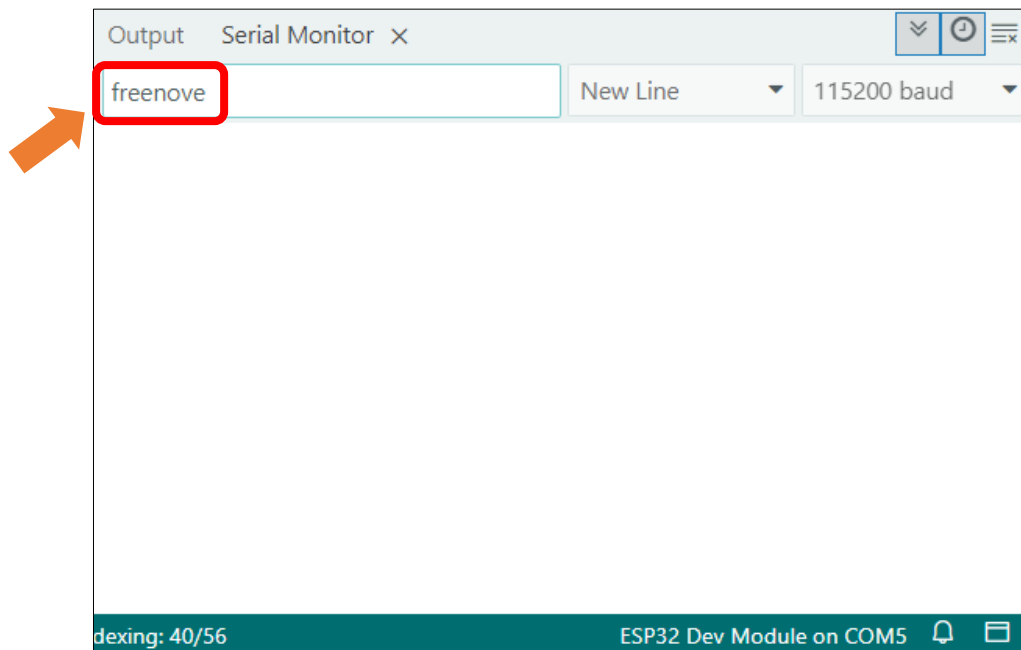
Select UTF-8 String, and click “Save”.



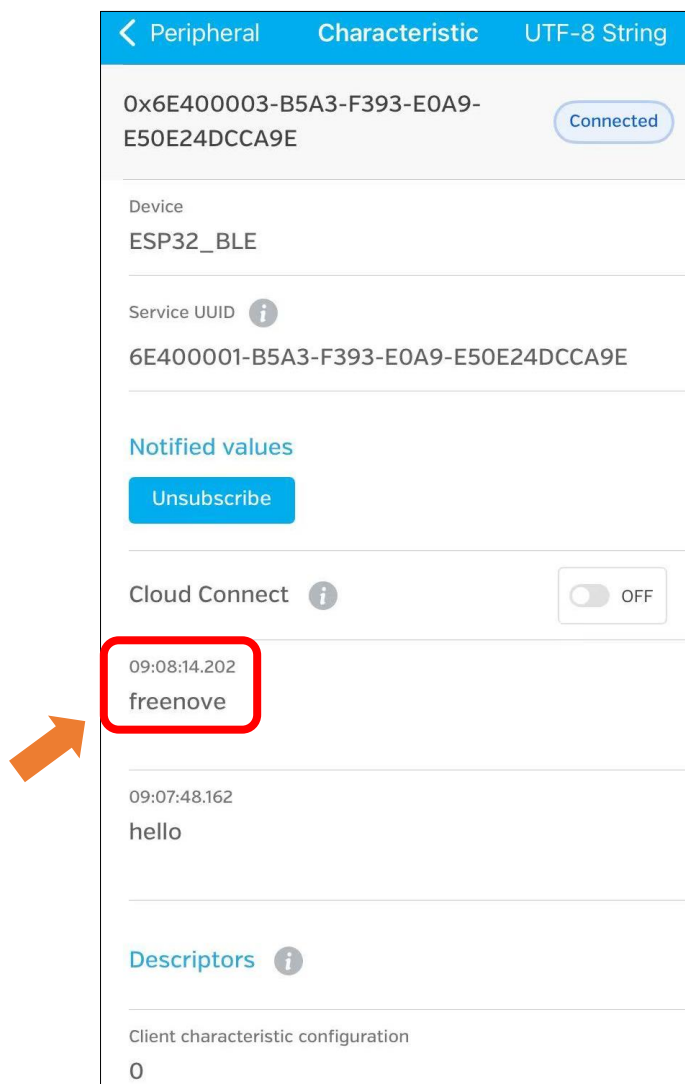
Click Subscribe



Send data on the serial monitor.

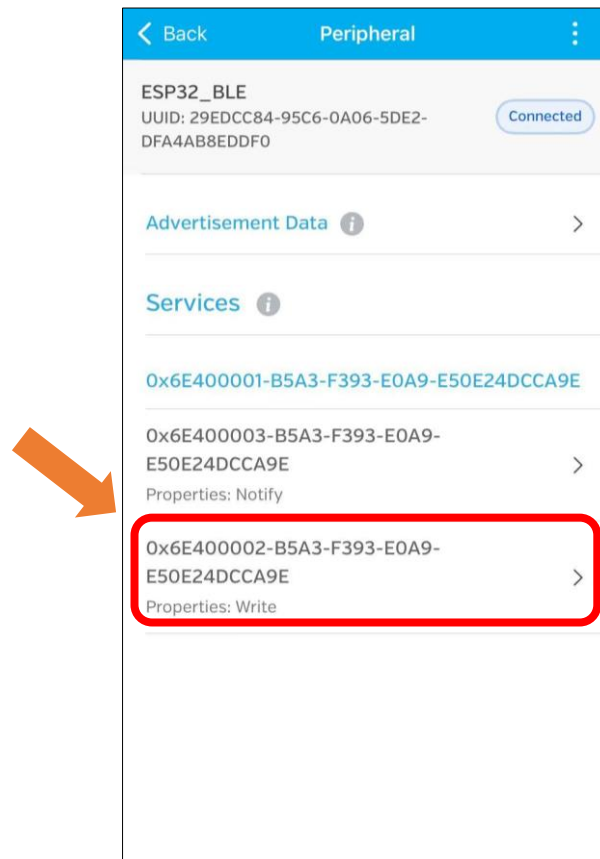


Data will be received on the LightBlue app.

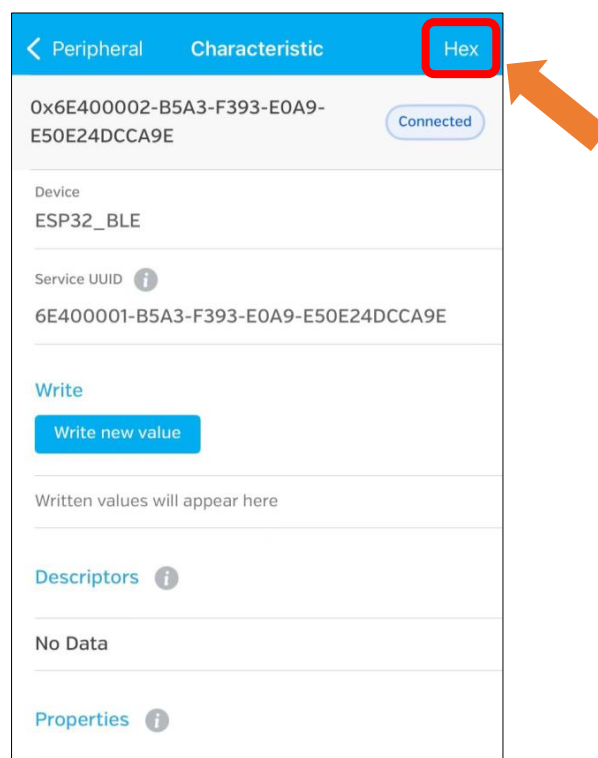


Sending Data

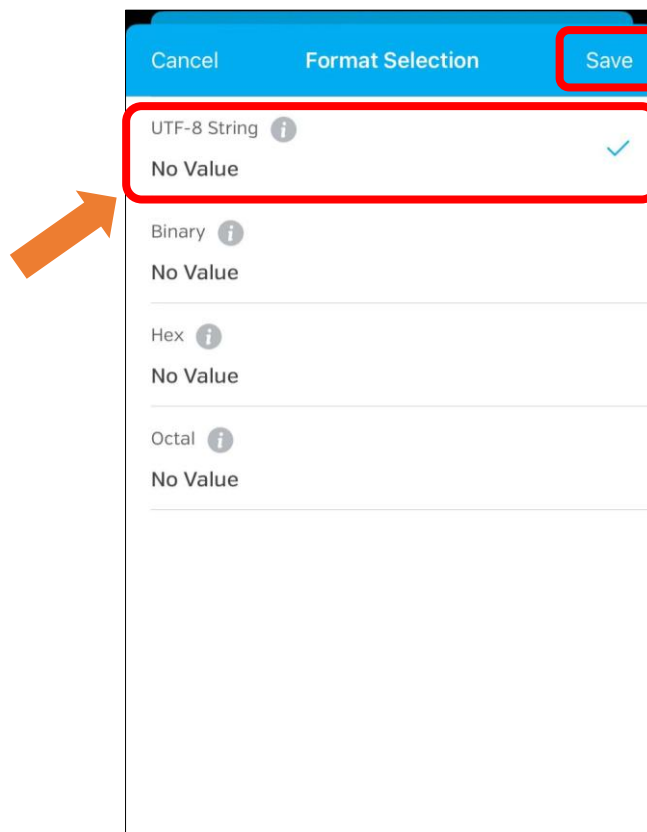
Click Write



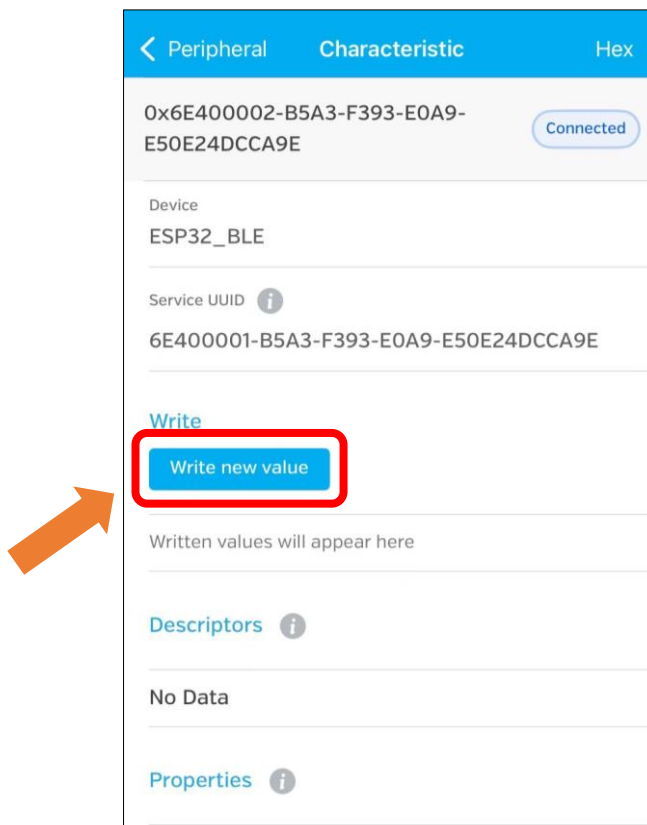
Click the top right corner to change the string type.



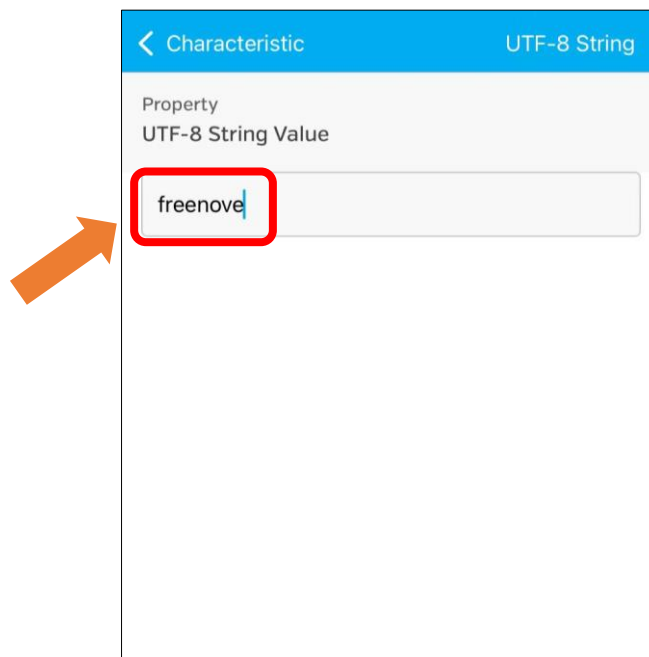
Select UTF-8 String and click “Save”.



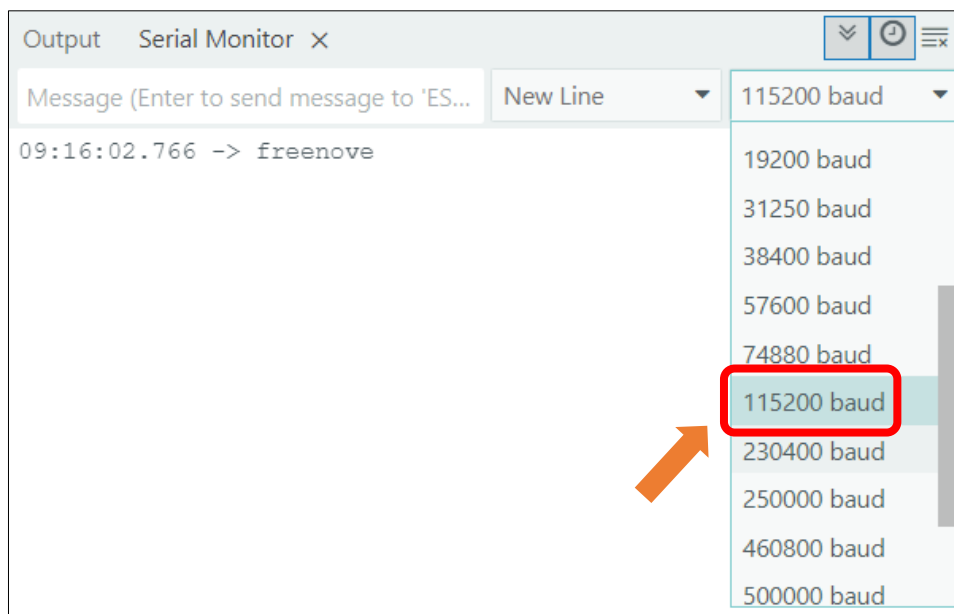
Click “Write new value”



Enter the messages to send.



Set the baud rate to 115200, and the serial monitor will print the data received.



Project 8.2 BLE RGB

In this section we will control the RGB LED via BLE.

Component List

Freenove ESP32 Display x 1

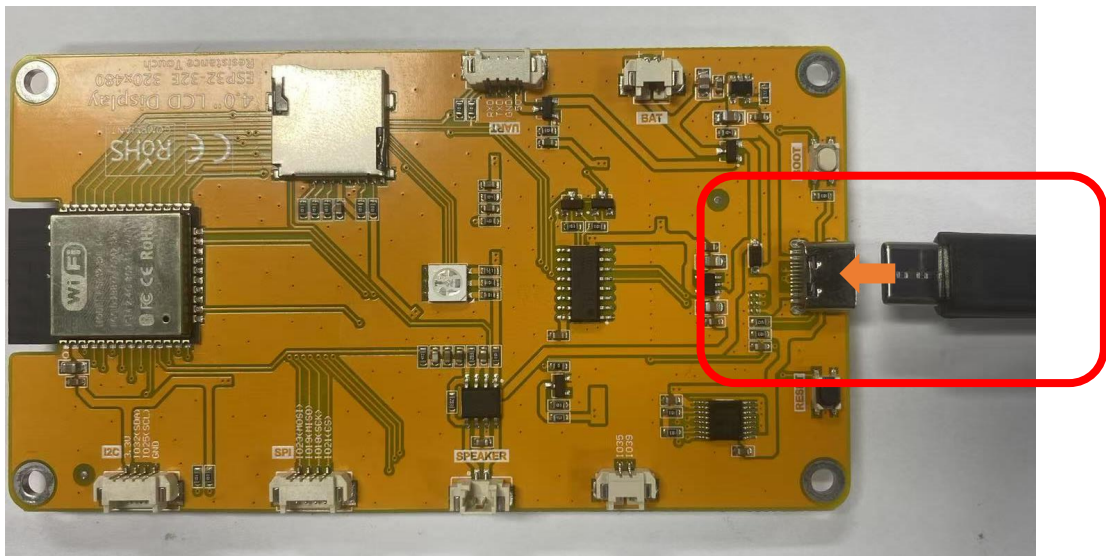


USB cable x1



Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Next, we download the code to Freenove_ESP32_Display to test. Open **"Sketch_08.2_BLE_RGB"** folder under **"Freenove_ESP32_Display\Sketches"** and double-click **"Sketch_08.2_BLE_RGB.ino"**.

[Sketch_08.2_BLE_RGB](#)

The following is the program code:

```
1  /*
2  * @ File:   Sketch_09.2_BLE_RGB.ino
```

```
3  * @ Author: [Zhentao Lin]
4  * @ Date:   [2025-06-14]
5  */
6
7  #include "BLEDevice.h"
8  #include "BLEServer.h"
9  #include "BLEUtils.h"
10 #include "BLE2902.h"
11 #include "String.h"
12
13 BLECharacteristic *pCharacteristic;
14 bool deviceConnected = false;
15 uint8_t txValue = 0;
16 long lastMsg = 0;
17 String rxload = "Test\n";
18
19 #define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
20 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
21 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
22
23 #define RED_PIN 22
24 #define GREEN_PIN 16
25 #define BLUE_PIN 17
26
27 void rgbInit(void) {
28     pinMode(RED_PIN, OUTPUT);
29     pinMode(GREEN_PIN, OUTPUT);
30     pinMode(BLUE_PIN, OUTPUT);
31 }
32
33 void setRGB(bool redLevel, bool greenLevel, bool blueLevel) {
34     digitalWrite(RED_PIN, !redLevel);
35     digitalWrite(GREEN_PIN, !greenLevel);
36     digitalWrite(BLUE_PIN, !blueLevel);
37 }
38
39 class MyServerCallbacks : public BLEServerCallbacks {
40     void onConnect(BLEServer *pServer) {
41         deviceConnected = true;
42     };
43     void onDisconnect(BLEServer *pServer) {
44         deviceConnected = false;
45     }
46 };
```

```

47
48 class MyCallbacks : public BLECharacteristicCallbacks {
49     void onWrite(BLECharacteristic *pCharacteristic) {
50         String rxValue = pCharacteristic->getValue();
51         if (rxValue.length() > 0) {
52             rxload = "";
53             for (int i = 0; i < rxValue.length(); i++) {
54                 rxload += (char)rxValue[i];
55             }
56         }
57     }
58 };
59
60 void setupBLE(String BLEName) {
61     const char *ble_name = BLEName.c_str();
62     BLEDevice::init(ble_name);
63     BLEServer *pServer = BLEDevice::createServer();
64     pServer->setCallbacks(new MyServerCallbacks());
65     BLEService *pService = pServer->createService(SERVICE_UUID);
66     pCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_TX,
67     BLECharacteristic::PROPERTY_NOTIFY);
68     pCharacteristic->addDescriptor(new BLE2902());
69     BLECharacteristic *pCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_RX,
70     BLECharacteristic::PROPERTY_WRITE);
71     pCharacteristic->setCallbacks(new MyCallbacks());
72     pService->start();
73     pServer->getAdvertising()->start();
74     Serial.println("Waiting a client connection to notify...");
75 }
76
77 void setup() {
78     Serial.begin(115200);
79     rgbInit();
80     setRGB(0, 0, 0);
81     setupBLE("ESP32_BLE");
82 }
83
84 void loop() {
85     long now = millis();
86     if (now - lastMsg > 100) {
87         if (deviceConnected && rxload.length() > 0) {
88             Serial.println(rxload);
89             if (strcmp(rxload.c_str(), "red_on", 6) == 0) {
90                 setRGB(1, 0, 0);

```

```

91     } else if (strcmp(rxload.c_str(), "red_off", 7) == 0) {
92         setRGB(0, 0, 0);
93     } else if (strcmp(rxload.c_str(), "green_on", 8) == 0) {
94         setRGB(0, 1, 0);
95     } else if (strcmp(rxload.c_str(), "green_off", 9) == 0) {
96         setRGB(0, 0, 0);
97     } else if (strcmp(rxload.c_str(), "blue_on", 7) == 0) {
98         setRGB(0, 0, 1);
99     } else if (strcmp(rxload.c_str(), "blue_off", 8) == 0) {
100         setRGB(0, 0, 0);
101     }
102     rxload = "";
103 }
104 if (Serial.available() > 0) {
105     String str = Serial.readString();
106     const char *newValue = str.c_str();
107     pCharacteristic->setValue(newValue);
108     pCharacteristic->notify();
109 }
110 lastMsg = now;
111 }
112 }

```

Code Explanation

Include the necessary header files.

```

7  #include "BLEDevice.h"
8  #include "BLEServer.h"
9  #include "BLEUtils.h"
10 #include "BLE2902.h"
11 #include "String.h"

```

Define Service UUID and Characteristic UUID.

```

19 #define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
20 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
21 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

```

Define pins for the RGB LED.

```

23 #define RED_PIN 22
24 #define GREEN_PIN 16
25 #define BLUE_PIN 17

```

The MyServerCallbacks class handles device connection and disconnection events and updates the deviceConnected status.

```

23 class MyServerCallbacks: public BLEServerCallbacks {
24     void onConnect(BLEServer* pServer) {
25         deviceConnected = true;
26     };
27     void onDisconnect(BLEServer* pServer) {

```

```

28     deviceConnected = false;
29 }
30 };

```

The MyCallbacks class handles the receiving data and save them to the rxload string.

```

32 class MyCallbacks: public BLECharacteristicCallbacks {
33     void onWrite(BLECharacteristic *pCharacteristic) {
34         String rxValue = pCharacteristic->getValue();
35         if (rxValue.length() > 0) {
36             rxload="";
37             for (int i = 0; i < rxValue.length(); i++){
38                 rxload +=(char)rxValue[i];
39             }
40         }
41     }
42 };

```

Set the baud rate to 115200

```

62 Serial.begin(115200);

```

Initialize RGB LED setting, initialize the BLE device, create services, and set up characteristics.

```

79 rgbInit();
80 setRGB(0, 0, 0);
81 setupBLE("ESP32_BLE");

```

The Loop function checks the sending command every 100 milliseconds.

```

84 void loop() {
85     long now = millis();
86     if (now - lastMsg > 100) {
87         if (deviceConnected && rxload.length() > 0) {
88             Serial.println(rxload);
89             if (strcmp(rxload.c_str(), "red_on", 6) == 0) {
90                 setRGB(1, 0, 0);
91             } else if (strcmp(rxload.c_str(), "red_off", 7) == 0) {
92                 setRGB(0, 0, 0);
93             } else if (strcmp(rxload.c_str(), "green_on", 8) == 0) {
94                 setRGB(0, 1, 0);
95             } else if (strcmp(rxload.c_str(), "green_off", 9) == 0) {
96                 setRGB(0, 0, 0);
97             } else if (strcmp(rxload.c_str(), "blue_on", 7) == 0) {
98                 setRGB(0, 0, 1);
99             } else if (strcmp(rxload.c_str(), "blue_off", 8) == 0) {
100                 setRGB(0, 0, 0);
101             }
102             rxload = "";
103         }
104         if (Serial.available() > 0) {
105             String str = Serial.readString();

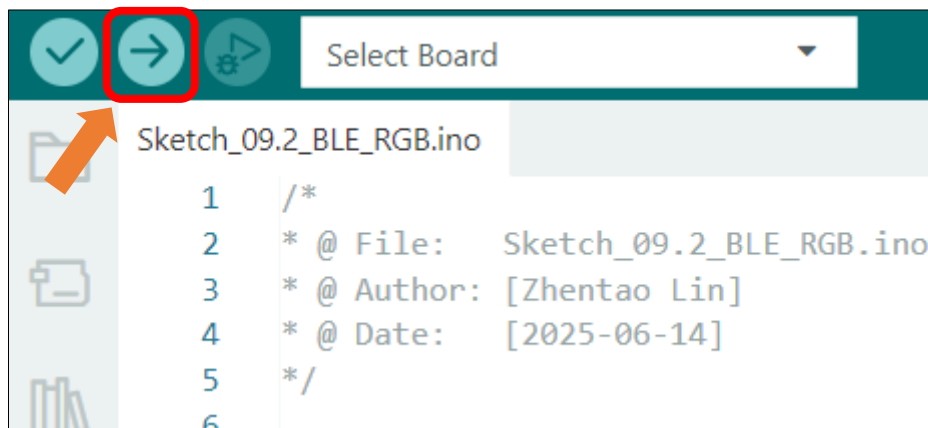
```



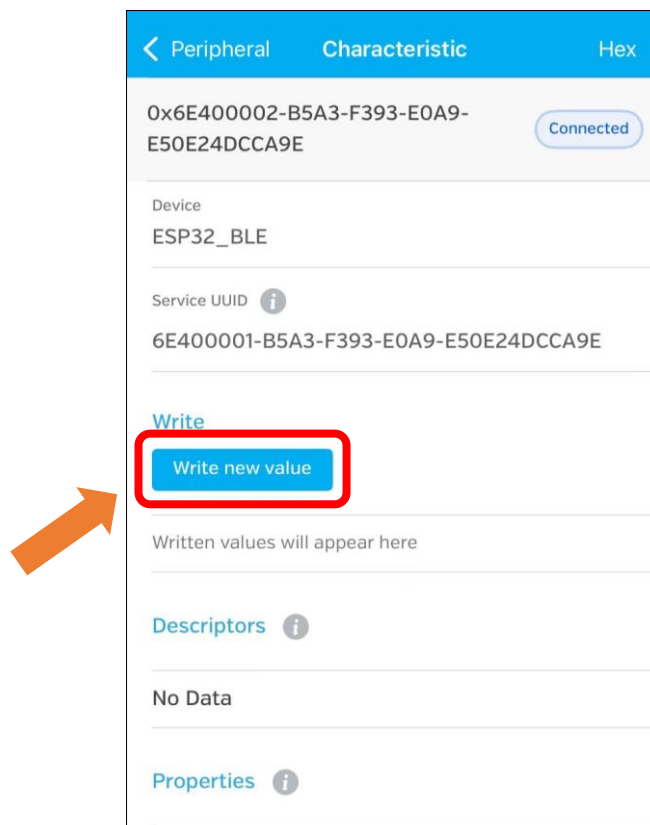
```
106     const char *newValue = str.c_str();
107     pCharacteristic->setValue(newValue);
108     pCharacteristic->notify();
109 }
110 lastMsg = now;
111 }
112 }
```

Click “**Upload**” to upload the code to Freenove ESP32 Display.

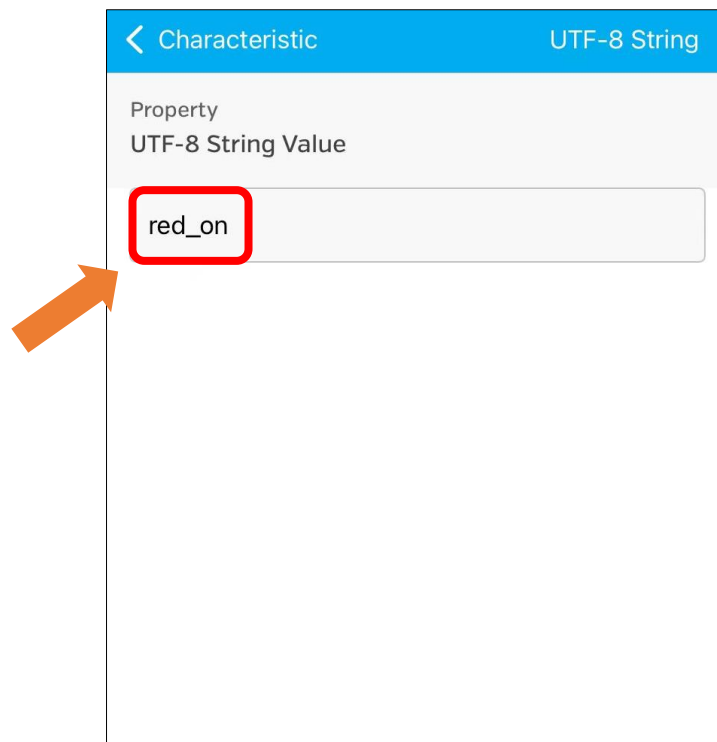
Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



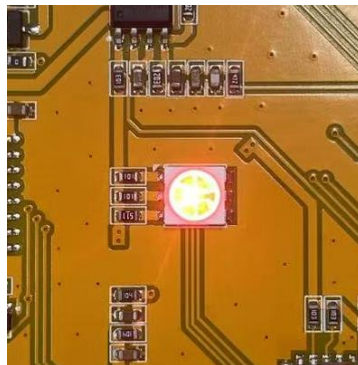
Click “Write new value”



Enter the messages to send. Here we take “red_on” as an example.



The RGB LED on the Freenove ESP32 Display emits red light.



You can also use the following instructions to control the RGB LED:

red_off: red light off
green_on: green light on
green_off: green light off
blue_on: blue light on
blue_off: blue light off

Chapter 9 WIFI Web Server

Project 9.1 WIFI Web Servers LED

Component List

Freenove ESP32 Display x 1  A photograph of a Freenove ESP32 Display module. It features a black rectangular screen mounted on a yellow PCB. The module has four gold-colored pins on the left side and two on the right side.	USB cable x1  A photograph of a black USB cable. It has a standard USB-A connector on one end and a micro-USB connector on the other.
--	--

Component Knowledge

Wi-Fi

Wi-Fi is a wireless LAN (WLAN) technology compliant with the IEEE 802.11 standard, enabling devices to transmit data or connect to the internet via radio waves within short-range coverage (typically up to tens of meters). In embedded systems, Wi-Fi modules provide wireless communication capabilities, allowing devices to connect to routers, access cloud services, or interact with other devices.

Its core features include:

- Wireless connectivity (eliminating the need for physical cabling)
- Moderate to high-speed data transfer (depending on protocol versions such as 802.11n/ac)
- Secure communication (ensured by encryption protocols like WPA2/WPA3)

In embedded development, Wi-Fi is one of the key technologies for enabling IoT (Internet of Things) device connectivity.

Web

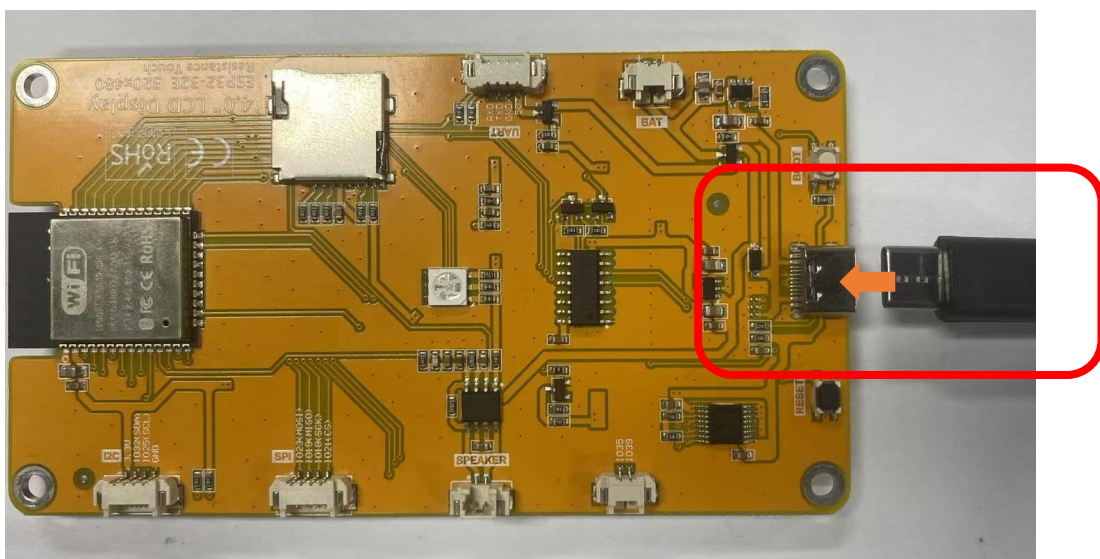
The Web (World Wide Web) is an internet-based information system that integrates global resources through hypertext links. In the embedded field, Web technology refers to devices equipped with lightweight built-in web servers, allowing users to remotely access the device interface via browsers (such as Chrome or Edge). By entering the device's IP address, users can open an interactive page to perform functions like status monitoring, parameter configuration, or firmware upgrades. Its core value lies in cross-platform compatibility (no need for dedicated software installation) and standardized protocols (HTTP/HTTPS), making it a mainstream solution for remote management of embedded devices.

HTML & CSS & JavaScript

- **HTML (HyperText Markup Language)** is the standard language for structuring web pages. It uses tags to define page elements, forming the foundational framework. In embedded web interfaces, HTML describes the layout of components such as device status displays and configuration forms.
- **CSS (Cascading Style Sheets)** controls the visual presentation of web pages, including colors, fonts, spacing, and responsive layouts. Through CSS rules, developers style HTML elements into intuitive interactive interfaces. The combination of HTML and CSS enables the creation of lightweight device control pages without complex graphics libraries, reducing resource overhead in embedded systems.
- **JavaScript (a scripting language)** adds dynamic behavior and interaction logic to web pages. In embedded web interfaces, JavaScript plays a crucial role: it responds to user actions and enables asynchronous communication with the device backend through technologies like **AJAX** or **WebSocket**. This allows the webpage to fetch real-time device status data, update displays without refreshing, and send control commands or configuration parameters—delivering a smooth and efficient remote management experience.

Circuit

Connect Freenove ESP32 Display to the computer with USB cable.



Sketch

Next, we download the code to Freenove_ESP32_Display to test. Open

“Sketch_09.1_WiFi_Web_Servers_LED” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_09.1_WiFi_Web_Servers_LED.ino”.

Important Note: Before upload the code, please ensure that the Wi-Fi SSID and Password are correctly configured in the code, and that the Wi-Fi network to connect is 2.4GHz.

```
10  const char* ssid    = "*****";
11  const char* password = "*****";
```

Sketch_09.1_WiFi_Web_Servers_LED

The following is the program code:

```
1  #include <WiFi.h>
2
3  // WiFi credentials
4  const char* ssid      = "*****";
5  const char* password = "*****";
6
7  // Web server on port 80
8  WiFiServer server(80);
9
10 // HTTP request buffer
11 String request;
12
13 // LED pin definitions
14 const int redLedPin = 22;
15 const int greenLedPin = 16;
16 const int blueLedPin = 17;
17
18 // LED state tracking
19 String greenLedState = "OFF";
20 String blueLedState = "OFF";
21 String redLedState = "OFF";
22
23 // Timeout settings
24 unsigned long currentTime = millis();
25 unsigned long previousTime = 0;
26 const long timeoutTime = 2000; // 2 seconds
27
28 void setup() {
29     Serial.begin(115200);
30
31     // Initialize LED pins
32     pinMode(redLedPin, OUTPUT);
33     pinMode(greenLedPin, OUTPUT);
34     pinMode(blueLedPin, OUTPUT);
35
36     digitalWrite(redLedPin, HIGH);
37     digitalWrite(greenLedPin, HIGH);
38     digitalWrite(blueLedPin, HIGH);
39
40     // Connect to WiFi
41     Serial.print("Connecting to ");
42     Serial.println(ssid);
```

[illegible]

```

87         greenLedState = "ON";
88         digitalWrite(greenLedPin, LOW);
89     } else if (request.indexOf("GET /16/OFF") >= 0) {
90         Serial.println("Green LED OFF");
91         greenLedState = "OFF";
92         digitalWrite(greenLedPin, HIGH);
93     } else if (request.indexOf("GET /17/ON") >= 0) {
94         Serial.println("Blue LED ON");
95         blueLedState = "ON";
96         digitalWrite(blueLedPin, LOW);
97     } else if (request.indexOf("GET /17/OFF") >= 0) {
98         Serial.println("Blue LED OFF");
99         blueLedState = "OFF";
100        digitalWrite(blueLedPin, HIGH);
101    } else {
102        // Send the Complete HTML page
103        client.println("HTTP/1.1 200 OK");
104        client.println("Content-type:text/html");
105        client.println("Connection: close");
106        client.println();
107
108        // Generate HTML response
109        client.println("<!DOCTYPE html><html>");
110        client.println("<head><meta charset=\"UTF-8\">");
111        client.println("<meta name=\"viewport\" content=\"width=device-width, initial-
112scale=1.0\">");
113        client.println("<title>ESP32 Web Server LED</title>");
114        client.println("<style>");
115        client.println("html{font-family:'Segoe UI',Roboto,sans-
116serif;background:#f5f5f5;height:100vh;display:flex;justify-content:center;align-
117items:center;}");
118        client.println("body{margin:0;padding:20px;background:white;border-
119radius:15px;box-shadow:0 10px 30px rgba(0,0,0,0.1);max-width:800px;width:90vw;}");
120        client.println(".container{display:grid;grid-template-columns:repeat(3,
1211fr);gap:20px;margin-top:20px;}");
122        client.println(".card{border-radius:15px;padding:25px;text-align:center;min-
123width:180px;transition:all 0.3s;cursor:pointer;border:2px
124solid;position:relative;overflow:hidden;}");
125        client.println(".card:hover{transform:translateY(-5px);box-shadow:0 15px 35px
126rgba(0,0,0,0.15);}");
127        client.println(".card:active{transform:translateY(0) scale(0.98);}");
128        client.println("#red-card{background:linear-
129gradient(145deg,#ffcd22,#ef9a9a);border-color:#f44336;}");
130

```

```

131         client.println("#green-card{background:linear-
132 gradient(145deg,#c8e6c9,#a5d6a7);border-color:#4caf50;}");
133         client.println("#blue-card{background:linear-
134 gradient(145deg,#bbdefb,#90caf9);border-color:#2196f3;}");
135         client.println(".card.off-state{filter:grayscale(70%) brightness(0.85);}");
136         client.println("h1{color:#333;text-align:center;font-size:2.4rem;margin-
137 bottom:10px;border-bottom:2px solid #eee;padding-bottom:15px;}");
138         client.println(".status{font-size:1.4rem;font-weight:600;margin:25px
139 0;color:#333;position:relative;z-index:2;}");
140         client.println(".device-info{text-align:center;margin-top:25px;color:#777;font-
141 size:0.9rem;padding-top:15px;border-top:1px solid #eee;}");
142         client.println("@media (max-width:768px){.container{grid-template-columns:1fr;}
143 body{padding:15px;} h1{font-size:2rem;}}");
144         client.println("</style></head>");
145         client.println("<body>");
146         client.println("<h1>ESP32 Web Server LED</h1>");
147         client.println("<div class=\"container\">");
148
149         // Red LED control
150         client.println("<div class=\"card \" + String(redLedState == \"OFF\" ? \"off-
151 state\" : \"\") + \"\" id=\"red-card\" onclick=\"toggleLED('22', this)\">");
152         client.println("<h2>Red LED</h2>");
153         client.println("<p class=\"status\" id=\"status-22\">\" + redLedState + "</p>");
154         client.println("</div>");
155
156         // Green LED control
157         client.println("<div class=\"card \" + String(greenLedState == \"OFF\" ? \"off-
158 state\" : \"\") + \"\" id=\"green-card\" onclick=\"toggleLED('16', this)\">");
159         client.println("<h2>Green LED</h2>");
160         client.println("<p class=\"status\" id=\"status-16\">\" + greenLedState +
161 "</p>");
162         client.println("</div>");
163
164         // Blue LED control
165         client.println("<div class=\"card \" + String(blueLedState == \"OFF\" ? \"off-
166 state\" : \"\") + \"\" id=\"blue-card\" onclick=\"toggleLED('17', this)\">");
167         client.println("<h2>Blue LED</h2>");
168         client.println("<p class=\"status\" id=\"status-17\">\" + blueLedState +
169 "</p>");
170         client.println("</div>");
171
172         client.println("</div>");
173         client.println("<div class=\"device-info\">Device IP: \" +
174 WiFi.localIP().toString() + \" | ESP32 Web Server LED</div>");

```



```

175         client.println("<script>");
176         client.println("function toggleLED(pin, element) {");
177         client.println("    const statusElement = document.getElementById('status-' +
178 pin);");
179         client.println("    const currentState = statusElement.innerText;");
180         client.println("    const newState = currentState === 'ON' ? 'OFF' : 'ON';");
181         client.println("    fetch('/' + pin + '/' + newState);");
182         client.println("    statusElement.innerText = newState;");
183         client.println("    if (newState === 'ON') { element.classList.remove('off-
184 state'); } else { element.classList.add('off-state'); }");
185         client.println("}");
186         client.println("</script>");
187         client.println("</body></html>");
188     }
189     break;
190 } else {
191     currentLine = "";
192 }
193 } else if (c != '\r') {
194     currentLine += c;
195 }
196 }
197 }
198
199 request = "";
200 client.stop();
201 Serial.println("Client disconnected.");
202 Serial.println("");
203 }
204 }

```

Code Explanation

Include necessary header files.

```
7 #include <WiFi.h>
```

Configure WiFi SSID and password.

```

10 const char* ssid    = "*****";
11 const char* password = "*****";

```

Define pins for the RGB LED.

```

19 // LED pin definitions
20 const int redLedPin = 22;
21 const int greenLedPin = 16;
22 const int blueLedPin = 17;

```

Initialize the configuration related to RGB LED.

```

37 // Initialize LED pins
38 pinMode(redLedPin, OUTPUT);

```

```

39 pinMode(greenLedPin, OUTPUT);
40 pinMode(blueLedPin, OUTPUT);
41
42 digitalWrite(redLedPin, HIGH);
43 digitalWrite(greenLedPin, HIGH);
44 digitalWrite(blueLedPin, HIGH);

```

Connect to WIFI.

```

46 // Connect to WiFi
47 Serial.print("Connecting to ");
48 Serial.println(ssid);
49 WiFi.begin(ssid, password);
50 while (WiFi.status() != WL_CONNECTED) {
51     delay(500);
52     Serial.print(".");
53 }
54
55 Serial.println("");
56 Serial.println("WiFi connected.");
57 Serial.println("IP address: ");
58 Serial.println(WiFi.localIP());
59
60 server.begin();

```

Check if there are any new clients connected to the server.

```

64 WiFiClient client = server.available();

```

Parse the URL command and control the corresponding LED on and off.

```

87 // Handle incoming commands
88 if (request.indexOf("GET /22/on") >= 0) {
89     Serial.println("Red LED ON");
90     redLedState = "on";
91     digitalWrite(redLedPin, LOW);
92 } else if (request.indexOf("GET /22/off") >= 0) {
93     Serial.println("Red LED OFF");
94     redLedState = "off";
95     digitalWrite(redLedPin, HIGH);
96 } else if (request.indexOf("GET /16/on") >= 0) {
97     Serial.println("Green LED ON");
98     greenLedState = "on";
99     digitalWrite(greenLedPin, LOW);
100 } else if (request.indexOf("GET /16/off") >= 0) {
101     Serial.println("Green LED OFF");
102     greenLedState = "off";
103     digitalWrite(greenLedPin, HIGH);
104 } else if (request.indexOf("GET /17/on") >= 0) {
105     Serial.println("Blue LED ON");

```

```

106 blueLedState = "on";
107 digitalWrite(blueLedPin, LOW);
108 } else if (request.indexOf("GET /17/off") >= 0) {
109     Serial.println("Blue LED OFF");
110     blueLedState = "off";
111     digitalWrite(blueLedPin, HIGH);
112 }

```

Use CSS styles to enhance the visual appeal of web interfaces.

```

121 client.println("html{font-family:' Segoe UI', Roboto, sans-
122 serif;background:#f5f5f5;height:100vh;display:flex;justify-content:center;align-
123 items:center;}");
124 client.println("body{margin:0;padding:20px;background:white;border-radius:15px;box-shadow:0
125 10px 30px rgba(0,0,0,0.1);max-width:800px;width:90vw;}");
126 client.println(".container{display:grid;grid-template-columns:repeat(3, 1fr);gap:20px;margin-
127 top:20px;}");
128 client.println(".card{border-radius:15px;padding:25px;text-align:center;min-
129 width:180px;transition:all 0.3s;cursor:pointer;border:2px
130 solid;position:relative;overflow:hidden;}");
131 client.println(".card:hover{transform:translateY(-5px);box-shadow:0 15px 35px
132 rgba(0,0,0,0.15);}");
133 client.println(".card:active{transform:translateY(0) scale(0.98);}");
134 client.println("#red-card{background:linear-gradient(145deg,#ffcdd2,#ef9a9a);border-
135 color:#f44336;}");
136 client.println("#green-card{background:linear-gradient(145deg,#c8e6c9,#a5d6a7);border-
137 color:#4caf50;}");
138 client.println("#blue-card{background:linear-gradient(145deg,#bbdefb,#90caf9);border-
139 color:#2196f3;}");
140 client.println(".card.off-state{filter:grayscale(70%) brightness(0.85);}");
141 client.println("h1{color:#333;text-align:center;font-size:2.4rem;margin-bottom:10px;border-
142 bottom:2px solid #eee;padding-bottom:15px;}");
143 client.println(".status{font-size:1.4rem;font-weight:600;margin:25px
144 0;color:#333;position:relative;z-index:2;}");
145 client.println(".device-info{text-align:center;margin-top:25px;color:#777;font-
146 size:0.9rem;padding-top:15px;border-top:1px solid #eee;}");
147 client.println("@media (max-width:768px){.container{grid-template-columns:1fr;}
148 body{padding:15px;} h1{font-size:2rem;}}");

```

Generate the HTML contents to control the LED.

```

154 // Red LED control
155 client.println("<div class=\"card \" + String(redLedState == \"OFF\" ? \"off-state\" : \"\") + \"\"
156 id=\"red-card\" onclick=\"toggleLED(' 22', this)\">");
157 client.println("<h2>Red LED</h2>");
158 client.println("<p class=\"status\" id=\"status-22\">\" + redLedState + "</p>");
159 client.println("</div>");
160

```

```

161 // Green LED control
162 client.println("<div class=\"card \" + String(greenLedState == \"OFF\" ? \"off-state\" : \"\") + \"\
163 id=\"green-card\" onclick=\"toggleLED(' 16', this)\">>");
164 client.println("<h2>Green LED</h2>");
165 client.println("<p class=\"status\" id=\"status-16\">\" + greenLedState + "</p>");
166 client.println("</div>");
167
168 // Blue LED control
169 client.println("<div class=\"card \" + String(blueLedState == \"OFF\" ? \"off-state\" : \"\") + \"\
170 id=\"blue-card\" onclick=\"toggleLED(' 17', this)\">>");
171 client.println("<h2>Blue LED</h2>");
172 client.println("<p class=\"status\" id=\"status-17\">\" + blueLedState + "</p>");
173 client.println("</div>");

```

Disconnect from the client and print the disconnection information in the serial monitor

```

172 client.stop();
173 Serial.println("Client disconnected.");
174 Serial.println("");

```

Input correct WiFi(2.4GHz) SSID and password.

```

9 // WiFi credentials
10 const char* ssid = "*****";
11 const char* password = "*****";

```

Click **Upload** to upload the code to Freenove ESP32 Display.

Please note: This chapter does not involve the use of the screen. After the code for this chapter, the screen may not light up, which is normal and not a hardware malfunction. If you need to verify whether the screen is functioning properly, please refer to [Chapter 10](#) for testing.



When the serial monitor prints the following information, it means the network connection is successful. Use a mobile phone or computer browser to open the IP address printed by the serial monitor.

Please note: If it keeps printing dots, please confirm whether the WiFi is in the 2.4G band.

```

14:31:24.553 -> Connecting to FYI_2.4G
14:31:25.115 -> .....
14:31:27.088 -> WiFi connected.
14:31:27.088 -> IP address:
14:31:27.122 -> 192.168.1.29

```

The following screen will be displayed in the computer's browser, where you can control the color of the RGB light by clicking on the three cards.



On the phone web browser, you'll see the following contents. The color of the RGB light can be controlled by clicking on the three cards.



Reference

`server.begin()`

This function starts the server to monitor the port.

`server.available()`

This function is used to detect and return the connected client object.

Chapter 10 TFT Display

Project 10.1 TFT_Rainbow

Freenove ESP32 Display is available in five different models, each featuring a unique TFT display. This guide applies to all of them. For detailed specifications of each model, please click [here](#).

Component List

Freenove ESP32 Display x 1



USB cable x1



Component Knowledge

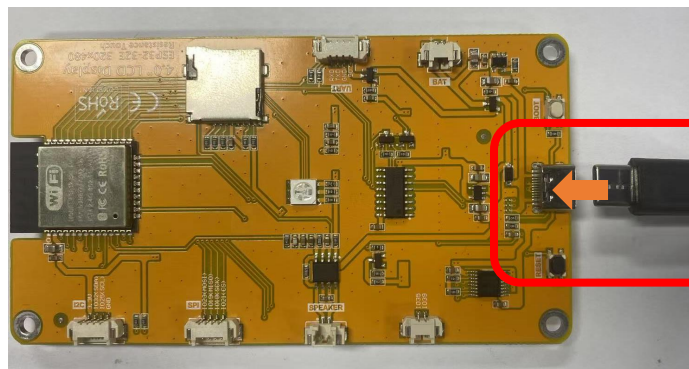
TFT Display

TFT (Thin Film Transistor) is an electronic component that serves as the foundation for TFT displays, the mainstream display technology in modern laptops and desktop computers. In these displays, each individual liquid crystal pixel is controlled by its own dedicated thin-film transistor embedded directly behind it. This architecture classifies TFT screens as a form of active-matrix LCD (AMLCD) technology.

As one of the finest LCD color displays available, TFT screens offer superior performance characteristics including rapid response times, exceptional brightness levels, and outstanding contrast ratios.

Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.

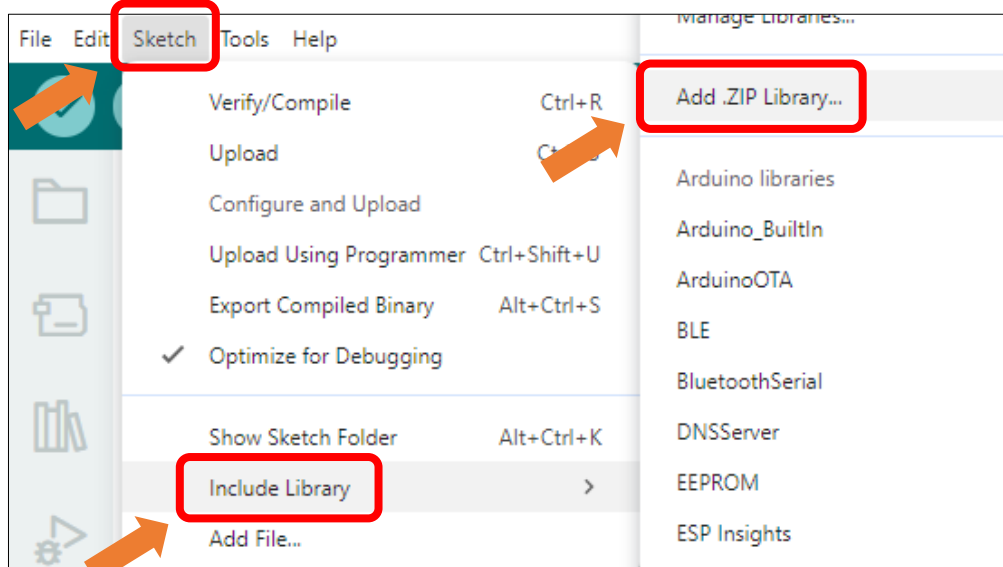


Sketch

Open “Sketch_10.1_TFT_Rainbow.ino” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_10.1_TFT_Rainbow.ino”.

Install Libraries

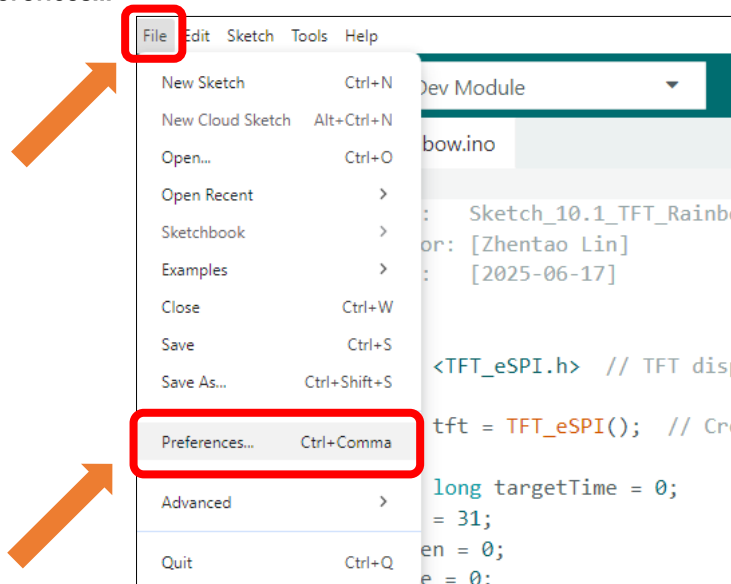
Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



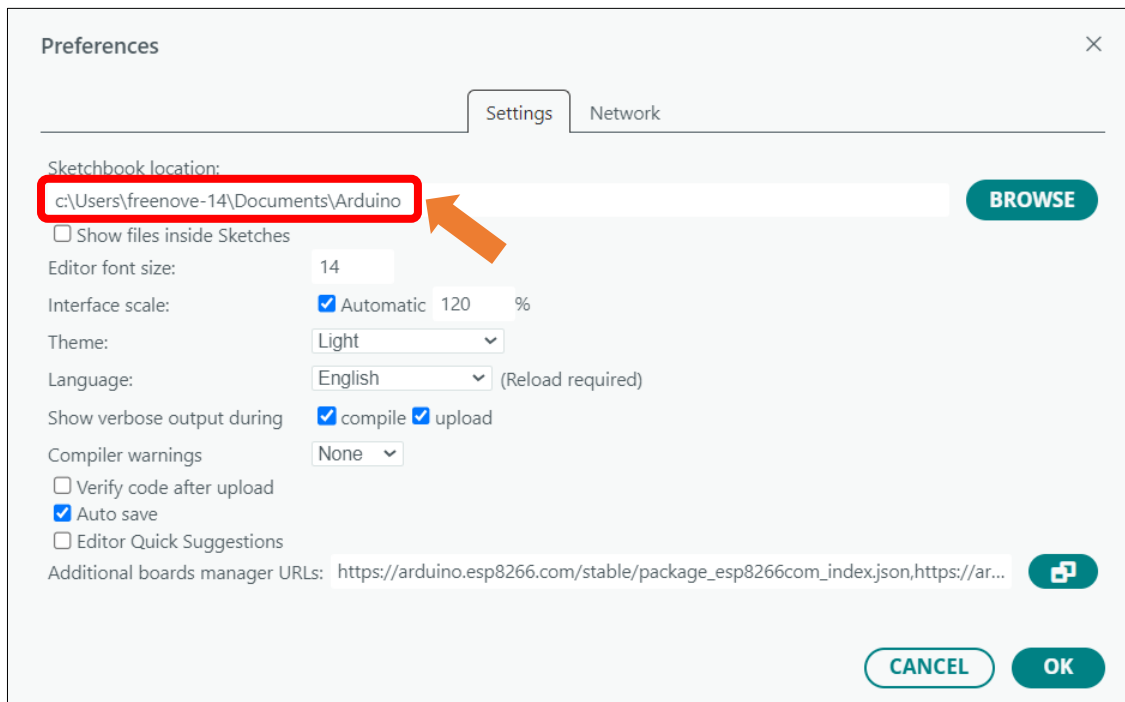
Install **TFT_eSPI_v2.5.43.zip** and **TFT_eSPI_Setups_v1.0.zip**

Name	Date modified	Type	Size
ESP8266Audio_v2.0.0.zip	2025/5/30 14:06	WinRAR ZIP...	7,821 KB
lvgl_v8.4.0.zip	2025/5/30 13:52	WinRAR ZIP...	26,083 KB
TFT_eSPI_Setups_v1.0.zip	2025/6/11 15:45	WinRAR ZIP...	45 KB
TFT_eSPI_v2.5.43.zip	2025/6/11 15:45	WinRAR ZIP...	5,975 KB
TFT_Touch_v0.3.zip	2025/5/30 13:51	WinRAR ZIP...	14 KB
Tpg_Decoder_v1.1.0.zip	2025/5/30 14:08	WinRAR ZIP...	313 KB

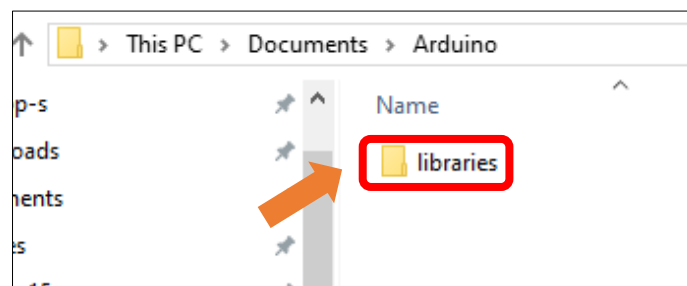
Click **File** -> **Preferences...**



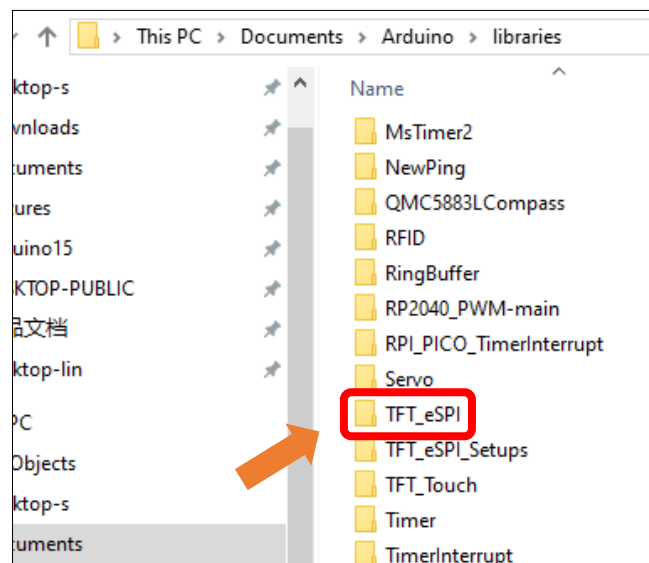
Linux and Mac users can use the **cd** command on Terminal to enter the sketchbook location, and Windows users can copy and paste it in the file explorer.



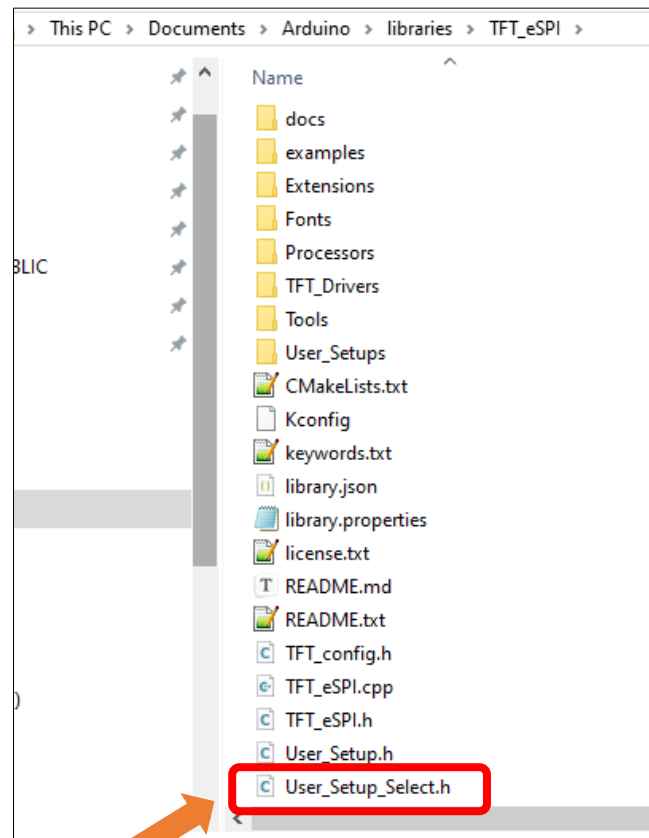
Double click **libraries**



Double click **TFT_eSPI**



Open the **User_Setup_Select.h** file



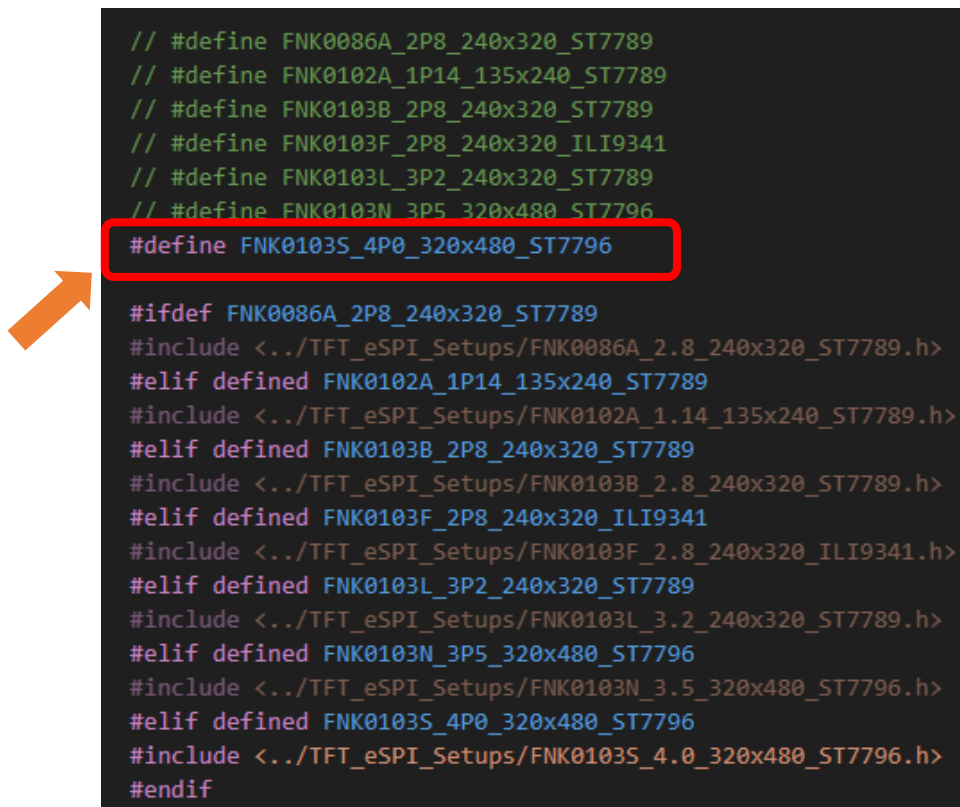
Remove the "//" comment markers from the macro definition line **corresponding to your Freenove ESP32 Display model**.

```
// #define FNK0086A_2P8_240x320_ST7789
// #define FNK0102A_1P14_135x240_ST7789
// #define FNK0103B_2P8_240x320_ST7789
// #define FNK0103F_2P8_240x320_ILI9341
// #define FNK0103L_3P2_240x320_ST7789
// #define FNK0103N_3P5_320x480_ST7796
// #define FNK0103S_4P0_320x480_ST7796

#ifdef FNK0086A_2P8_240x320_ST7789
#include <../TFT_eSPI_Setups/FNK0086A_2.8_240x320_ST7789.h>
#elif defined FNK0102A_1P14_135x240_ST7789
#include <../TFT_eSPI_Setups/FNK0102A_1.14_135x240_ST7789.h>
#elif defined FNK0103B_2P8_240x320_ST7789
#include <../TFT_eSPI_Setups/FNK0103B_2.8_240x320_ST7789.h>
#elif defined FNK0103F_2P8_240x320_ILI9341
#include <../TFT_eSPI_Setups/FNK0103F_2.8_240x320_ILI9341.h>
#elif defined FNK0103L_3P2_240x320_ST7789
#include <../TFT_eSPI_Setups/FNK0103L_3.2_240x320_ST7789.h>
#elif defined FNK0103N_3P5_320x480_ST7796
#include <../TFT_eSPI_Setups/FNK0103N_3.5_320x480_ST7796.h>
#elif defined FNK0103S_4P0_320x480_ST7796
#include <../TFT_eSPI_Setups/FNK0103S_4.0_320x480_ST7796.h>
#endif
```

The screenshot shows a code editor with the contents of the `User_Setup_Select.h` file. A red rectangular box highlights the macro definitions for Freenove ESP32 models. An orange arrow points from the left towards the first line of this block: `// #define FNK0103B_2P8_240x320_ST7789`.

Here we take the 4-in display as an example, modifying it as shown below:



```
// #define FNK0086A_2P8_240x320_ST7789
// #define FNK0102A_1P14_135x240_ST7789
// #define FNK0103B_2P8_240x320_ST7789
// #define FNK0103F_2P8_240x320_ILI9341
// #define FNK0103L_3P2_240x320_ST7789
// #define FNK0103N_3P5_320x480_ST7796
#define FNK0103S_4P0_320x480_ST7796

#ifdef FNK0086A_2P8_240x320_ST7789
#include <../TFT_eSPI_Setups/FNK0086A_2.8_240x320_ST7789.h>
#elif defined FNK0102A_1P14_135x240_ST7789
#include <../TFT_eSPI_Setups/FNK0102A_1.14_135x240_ST7789.h>
#elif defined FNK0103B_2P8_240x320_ST7789
#include <../TFT_eSPI_Setups/FNK0103B_2.8_240x320_ST7789.h>
#elif defined FNK0103F_2P8_240x320_ILI9341
#include <../TFT_eSPI_Setups/FNK0103F_2.8_240x320_ILI9341.h>
#elif defined FNK0103L_3P2_240x320_ST7789
#include <../TFT_eSPI_Setups/FNK0103L_3.2_240x320_ST7789.h>
#elif defined FNK0103N_3P5_320x480_ST7796
#include <../TFT_eSPI_Setups/FNK0103N_3.5_320x480_ST7796.h>
#elif defined FNK0103S_4P0_320x480_ST7796
#include <../TFT_eSPI_Setups/FNK0103S_4.0_320x480_ST7796.h>
#endif
```

Important Note: Only one macro definition should be uncommented.

Sketch_10.1_TFT_Rainbow

The following is the program code:

```
1  #include <TFT_eSPI.h> // TFT display library
2
3  TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance
4
5  unsigned long targetTime = 0; // Timing variable to control animation intervals
6  byte red = 31; // Start with full red
7  byte green = 0; // No green
8  byte blue = 0; // No blue
9  byte state = 0; // State machine variable
10 unsigned int colour = red << 11; // Initial color value: Red only
11
12 void setup(void) {
13     tft.init(); // Initialize the TFT screen
14     tft.setRotation(1); // Set screen rotation (landscape mode)
15     targetTime = millis() + 100; // Set initial target time for rainbow effect
16     tft.fillScreen(TFT_RED); // Fill screen with solid red
17     delay(1000);
18     tft.fillScreen(TFT_GREEN); // Fill screen with solid green
19     delay(1000);
```

```
20  tft.fillScreen(TFT_BLUE); // Fill screen with solid blue
21  delay(1000);
22  tft.fillScreen(TFT_BLACK); // Fill screen with solid black
23  delay(1000);
24  tft.fillScreen(TFT_WHITE); // Fill screen with solid white
25  delay(1000);
26  }
27
28  void loop() {
29      if (targetTime < millis()) { // Check if it's time to start the rainbow animation
30          targetTime = millis() + 10000; // Set next trigger after 10 seconds
31
32          for (int i = 0; i < 320; i++) { // Generate horizontal rainbow using
33  vertical lines
34              tft.drawFastVLine(i, 0, tft.height(), colour); // Draw one vertical line
35              switch (state) {
36                  case 0:
37                      green += 2; // Transition from red to yellow (increase green)
38                      if (green == 64) {
39                          green = 63; // Cap at max green value
40                          state = 1;
41                      }
42                      break;
43                  case 1:
44                      red--; // Transition from yellow to green (decrease red)
45                      if (red == 255) {
46                          red = 0;
47                          state = 2;
48                      }
49                      break;
50                  case 2:
51                      blue++; // Transition from green to cyan (increase blue)
52                      if (blue == 32) {
53                          blue = 31; // Cap at max blue value
54                          state = 3;
55                      }
56                      break;
57                  case 3:
58                      green -= 2; // Transition from cyan to blue (decrease green)
59                      if (green == 255) {
60                          green = 0;
61                          state = 4;
62                      }
63                      break;
```

```

64     case 4:
65         red++; // Transition from blue to magenta (increase red)
66         if (red == 32) {
67             red = 31; // Cap at max red value
68             state = 5;
69         }
70         break;
71     case 5:
72         blue--; // Transition from magenta to red (decrease blue)
73         if (blue == 255) {
74             blue = 0;
75             state = 0;
76         }
77         break;
78     }
79     colour = red << 11 | green << 5 | blue; // Combine RGB values into 16-bit color
80 }
81 tft.setTextColor(TFT_BLACK); // Set text color to black
82 tft.setCursor(12, 5); // Set cursor position
83 tft.print("Original Adafruit font!"); // Print simple string
84 tft.setTextColor(TFT_BLACK, TFT_BLACK); // Transparent background
85 tft.drawCentreString("Font size 2", 80, 14, 2); // Font 2
86 tft.drawCentreString("Font size 4", 80, 30, 4); // Font 4
87 tft.drawCentreString("12.34", 80, 54, 6); // Font 6
88 tft.drawCentreString("12.34 is in font size 6", 80, 92, 2); // Font 2
89 float pi = 3.14159; // Value to print
90 int precision = 3; // Number of decimal digits
91 int xpos = 50; // X position
92 int ypos = 110; // Y position
93 int font = 2; // Font type
94 xpos += tft.drawFloat(pi, precision, xpos, ypos, font); // Draw float and update x-
95 position
96 tft.drawString(" is pi", xpos, ypos, font); // Continue drawing string from
97 updated x position
98 delay(6000); // Wait before next cycle
99 }
100 }

```

Code Explanation

Include the necessary header file.

```
7 #include <TFT_eSPI.h> // TFT display library
```

Define TFT display object.

```
9 TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance
```

Initialize the TFT display.

```
19 tft.init(); // Initialize the TFT screen
20 tft.setRotation(1); // Set screen rotation (landscape mode)
```

Change the color of the screen in the sequence of red -> green -> blue -> black ->white.

```
22 tft.fillRect(TFT_RED); // Fill screen with solid red
23 delay(1000);
24 tft.fillRect(TFT_GREEN); // Fill screen with solid green
25 delay(1000);
26 tft.fillRect(TFT_BLUE); // Fill screen with solid blue
27 delay(1000);
28 tft.fillRect(TFT_BLACK); // Fill screen with solid black
29 delay(1000);
30 tft.fillRect(TFT_WHITE); // Fill screen with solid white
31 delay(1000);
```

Implement the rainbow animation effect.

```
38 for (int i = 0; i < 320; i++) { // Generate horizontal rainbow using vertical lines
39     tft.drawFastVLine(i, 0, tft.height(), colour); // Draw one vertical line
40     switch (state) {
41         case 0:
42             green += 2; // Transition from red to yellow (increase green)
43             if (green == 64) {
44                 green = 63; // Cap at max green value
45                 state = 1;
46             }
47             break;
48         case 1:
49             red--; // Transition from yellow to green (decrease red)
50             if (red == 255) {
51                 red = 0;
52                 state = 2;
53             }
54             break;
55         case 2:
56             blue++; // Transition from green to cyan (increase blue)
57             if (blue == 32) {
58                 blue = 31; // Cap at max blue value
59                 state = 3;
60             }
61             break;
```

```

62     case 3:
63         green -= 2; // Transition from cyan to blue (decrease green)
64         if (green == 255) {
65             green = 0;
66             state = 4;
67         }
68         break;
69     case 4:
70         red++; // Transition from blue to magenta (increase red)
71         if (red == 32) {
72             red = 31; // Cap at max red value
73             state = 5;
74         }
75         break;
76     case 5:
77         blue--; // Transition from magenta to red (decrease blue)
78         if (blue == 255) {
79             blue = 0;
80             state = 0;
81         }
82         break;
83 }
84 colour = red << 11 | green << 5 | blue; // Combine RGB values into 16-bit color
85 }

```

Text display.

```

87 tft.setTextColor(TFT_BLACK); // Set text color to black
88 tft.setCursor(12, 5); // Set cursor position
89 tft.print("Original Adafruit font!"); // Print simple string
90 tft.setTextColor(TFT_BLACK, TFT_BLACK); // Transparent background
91 tft.drawCentreString("Font size 2", 80, 14, 2); // Font 2
92 tft.drawCentreString("Font size 4", 80, 30, 4); // Font 4
93 tft.drawCentreString("12.34", 80, 54, 6); // Font 6
94 tft.drawCentreString("12.34 is in font size 6", 80, 92, 2); // Font 2

```

Click **Upload** to upload the code to Freenove_ESP32_Display.



The TFT screen will change colors in the order of red -> green -> blue -> black -> white before displaying text and rainbow effects.



Project 10.2 Flash JPG DMA

Component List

Freenove ESP32 Display x 1



USB cable x1



Component Knowledge

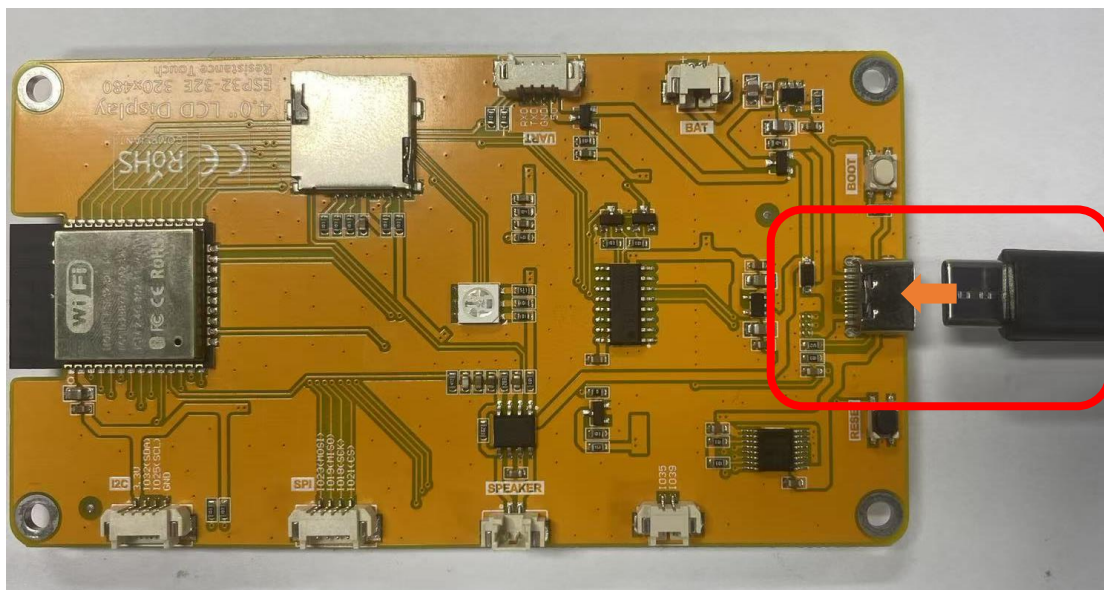
DMA

DMA, or Direct Memory Access, is a hardware feature that allows peripherals to transfer data to and from memory without needing the CPU to be directly involved, dramatically improving overall system efficiency while minimizing processor workload.

The core mechanism of DMA relies on a dedicated DMA controller taking over data transfer tasks. The CPU only needs to initialize the transfer parameters before offloading the operation, allowing computation and I/O operations to proceed in parallel.

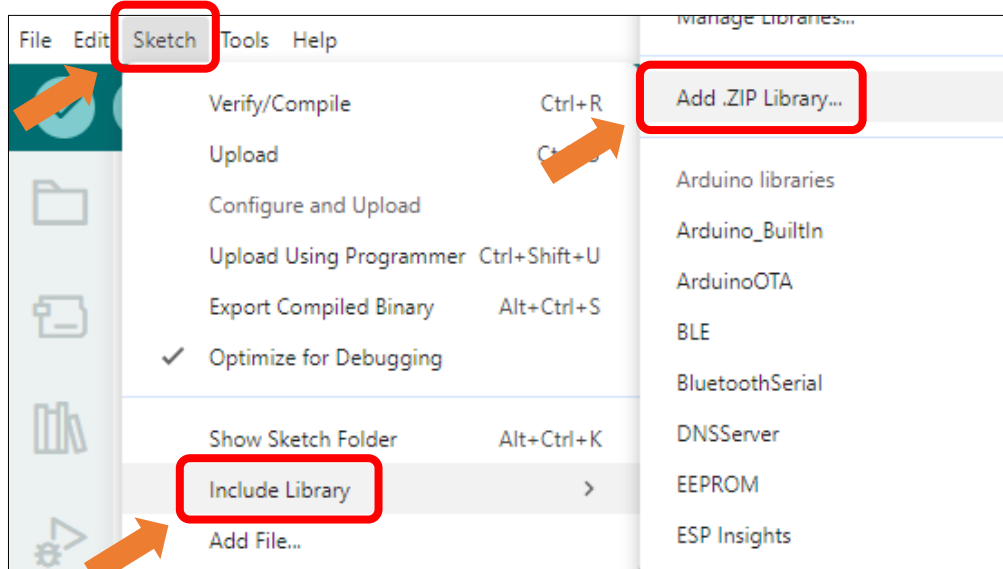
Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **TJpg_Decoder_v1.1.0.zip**

Name	Date modified	Type	Size
ESP8266Audio_v2.0.0.zip	2025/5/30 14:06	WinRAR ZIP...	7,821 KB
lvgl_v8.4.0.zip	2025/5/30 13:52	WinRAR ZIP...	26,083 KB
TFT_eSPI_Setups_v1.0.zip	2025/6/11 15:45	WinRAR ZIP...	45 KB
TFT_eSPI_v2.5.43.zip	2025/6/11 15:45	WinRAR ZIP...	5,975 KB
TFT_Touch_v0.3.zip	2025/5/30 13:51	WinRAR ZIP...	14 KB
TJpg_Decoder_v1.1.0.zip	2025/5/30 14:08	WinRAR ZIP...	313 KB

Open "Sketch_10.2_Flash_Jpg_DMA" folder under "Freenove_ESP32_Display\Sketches" and double-click "Sketch_10.2_Flash_Jpg_DMA.ino".

Sketch_10.2_Flash_Jpg_DMA

The following is the program code:

```

1 // Example for library: https://github.com/Bodmer/TJpg_Decoder
2
3 // This example renders a JPEG file stored in Flash memory (see panda.h)
4
5 #include <TFT_eSPI.h> // TFT display library
6 #define USE_DMA
7 #include "panda.h" // Include raw JPEG image data array
8 #include <TJpg_Decoder.h> // Include JPEG decoder library
9
10 #ifdef USE_DMA
11 uint16_t dmaBuffer1[16 * 16]; // DMA buffer for 16x16 pixel block
12 uint16_t dmaBuffer2[16 * 16]; // Second DMA buffer for toggling
13 uint16_t* dmaBufferPtr = dmaBuffer1;
14 bool dmaBufferSel = 0;

```

```

15  #endif
16
17  TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance
18
19  // Callback function to draw decoded JPEG blocks on screen
20  bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
21      if (y >= tft.height()) return 0; // Stop decoding if off-screen
22
23      #ifdef USE_DMA
24          if (dmaBufferSel) dmaBufferPtr = dmaBuffer2;
25          else dmaBufferPtr = dmaBuffer1;
26          dmaBufferSel = !dmaBufferSel;
27          tft.pushImageDMA(x, y, w, h, bitmap, dmaBufferPtr); // Start DMA transfer
28      #else
29          tft.pushImage(x, y, w, h, bitmap); // Draw without DMA
30      #endif
31
32      return 1; // Continue decoding next block
33  }
34
35  void setup() {
36      Serial.begin(115200);
37      Serial.println("\n\n Testing TJpg_Decoder library");
38
39      tft.begin(); // Initialize TFT display
40      tft.setTextColor(TFT_WHITE, TFT_BLACK);
41      tft.fillScreen(TFT_BLACK); // Clear screen with black background
42
43      #ifdef USE_DMA
44          tft.initDMA(); // Setup SPI DMA engine
45      #endif
46
47      TJpgDec.setJpgScale(1); // Set scale factor (1=full size)
48      tft.setSwapBytes(true); // Match color byte order
49      TJpgDec.setCallback(tft_output); // Register drawing callback
50  }
51
52  void loop() {
53      uint16_t w = 0, h = 0;
54      TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset)); // Get image dimensions
55      Serial.print("Width = ");
56      Serial.print(w);
57      Serial.print(", height = ");
58      Serial.print(h);

```

```

59
60     uint32_t dt = millis();
61     tft.startWrite();
62     TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset)); // Draw the JPEG image
63     tft.endWrite();
64
65     dt = millis() - dt;
66     Serial.print(", dt = ");
67     Serial.print(dt);
68     Serial.println(" ms");
69
70     delay(2000); // Wait before redraw
71 }

```

Code Explanation

Include necessary header files.

```

1  #include <TFT_eSPI.h> // TFT display library
   ...
3  #include "panda.h"    // Include raw JPEG image data array
4  #include <TJpg_Decoder.h> // Include JPEG decoder library

```

Configure DMA buffer

```

7  uint16_t dmaBuffer1[16 * 16]; // DMA buffer for 16x16 pixel block
8  uint16_t dmaBuffer2[16 * 16]; // Second DMA buffer for toggling
9  uint16_t* dmaBufferPtr = dmaBuffer1;
10 bool dmaBufferSel = 0;

```

Create TFT object instance.

```

19 TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance

```

JPEG decoding callback function

```

15 // Callback function to draw decoded JPEG blocks on screen
16 bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
17     if (y >= tft.height()) return 0; // Stop decoding if off-screen
18
19     #ifndef USE_DMA
20         if (dmaBufferSel) dmaBufferPtr = dmaBuffer2;
21         else dmaBufferPtr = dmaBuffer1;
22         dmaBufferSel = !dmaBufferSel;
23         tft.pushImageDMA(x, y, w, h, bitmap, dmaBufferPtr); // Start DMA transfer
24     #else
25         tft.pushImage(x, y, w, h, bitmap); // Draw without DMA
26     #endif
27
28     return 1; // Continue decoding next block
29 }

```

Get JPG size.

```

46 TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset)); // Get image dimensions

```

Draw images on the TFT screen.

```
57 tft.startWrite();  
58 TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset)); // Draw the JPEG image  
59 tft.endWrite();
```

Click “**Upload**” to upload the code to Freenove ESP32 Display



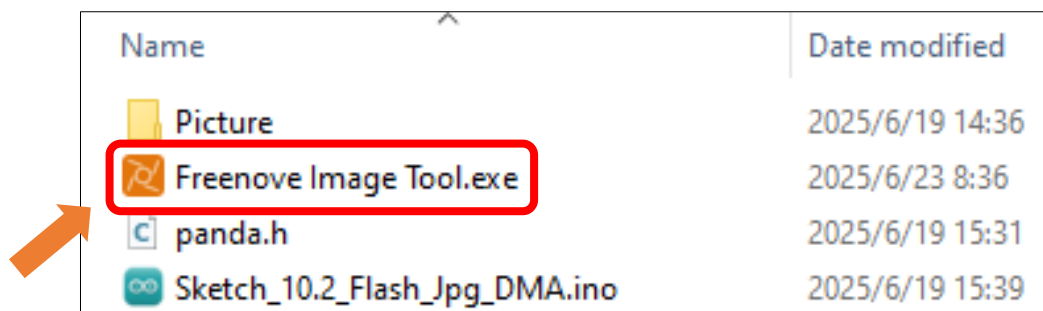
After the code is uploaded, an image will be displayed on the TFT screen. The image is from https://github.com/Bodmer/TJpg_Decoder



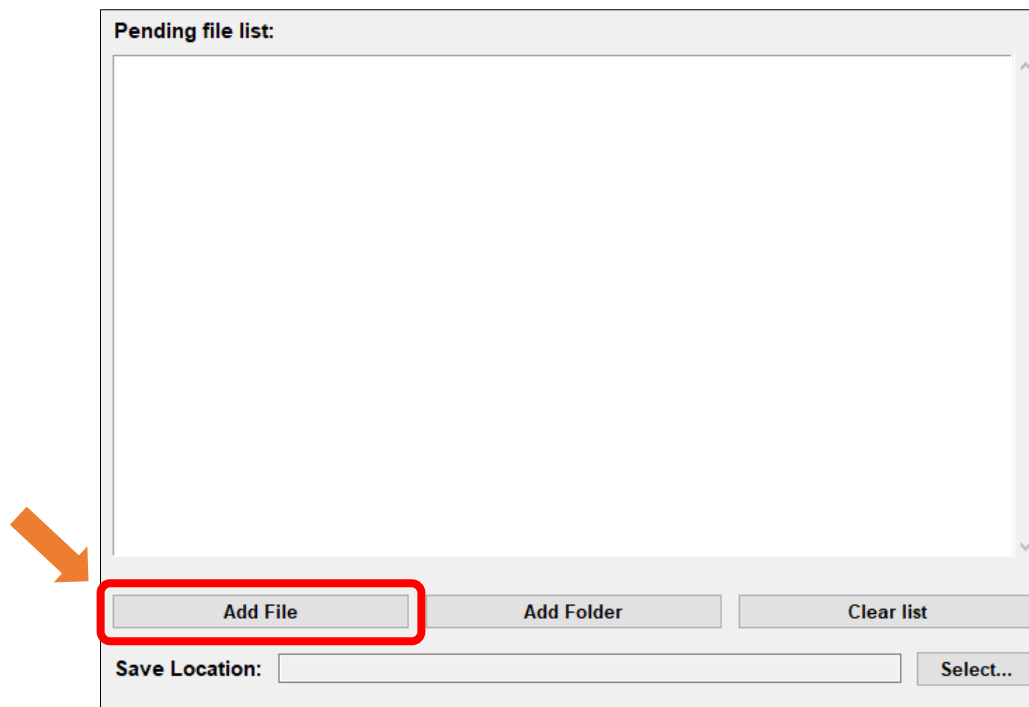
Custom image display

You can customize the image displayed on the display according to your personal preferences.

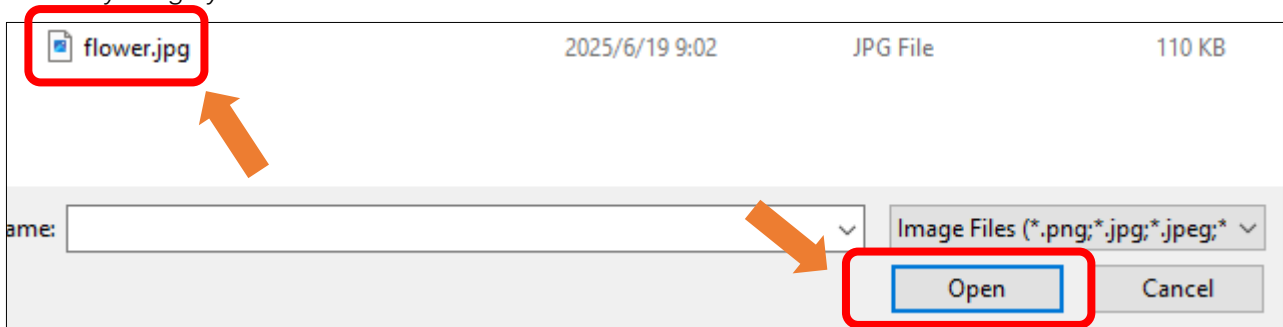
First, open **Freenove_ESP32_Display\Sketches\Sketch_10.2_Flash_Jpg_DMA\Freenove Image Tool.exe**



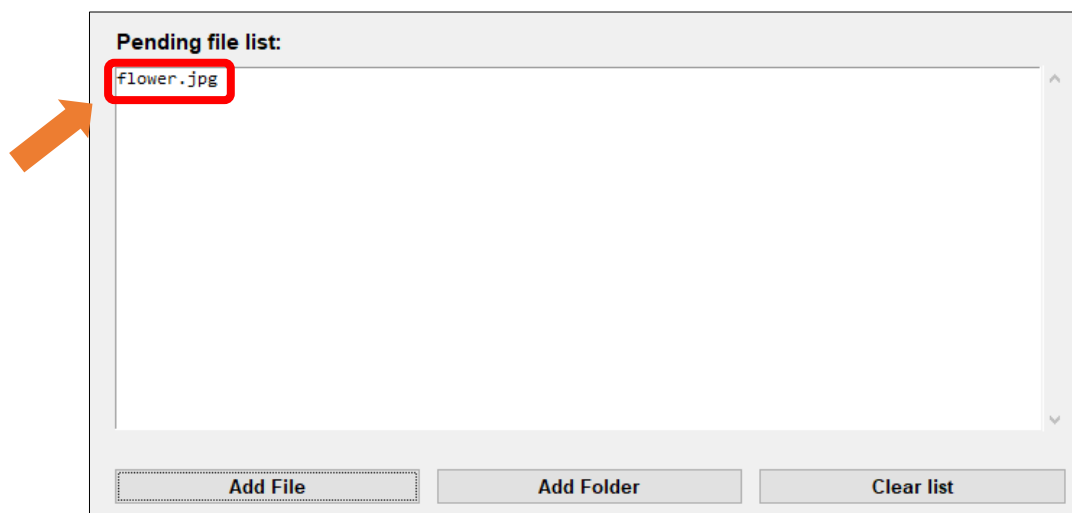
Click “Add File”



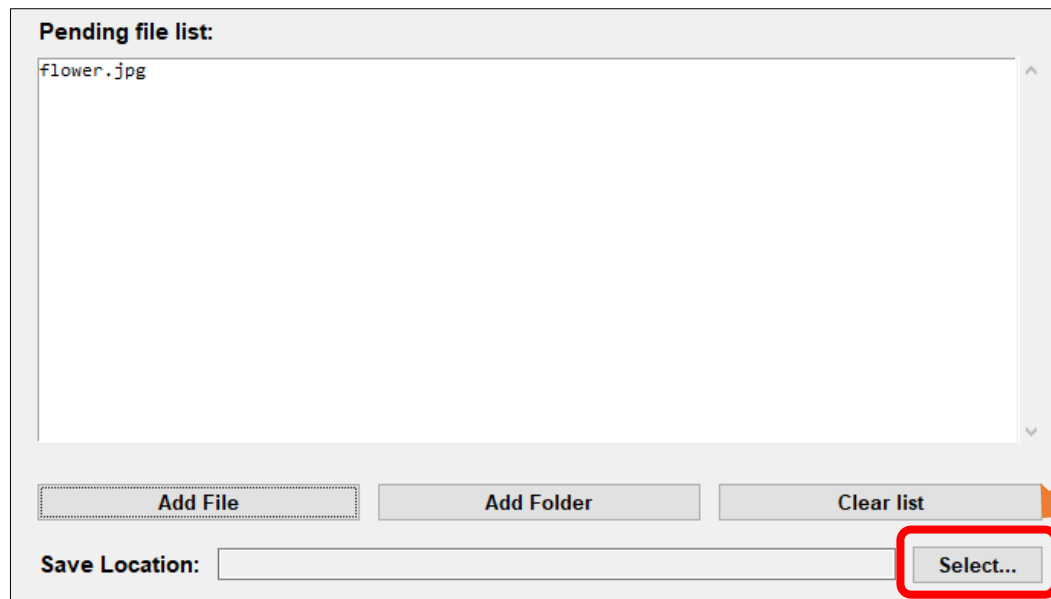
Select any image you like



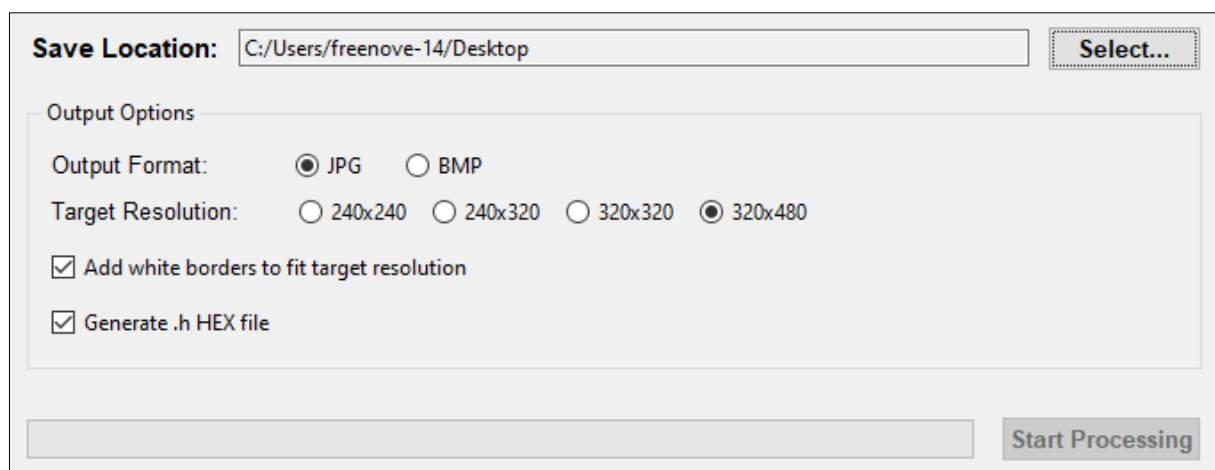
The image files from your folder will now appear in the **Pending File List**.



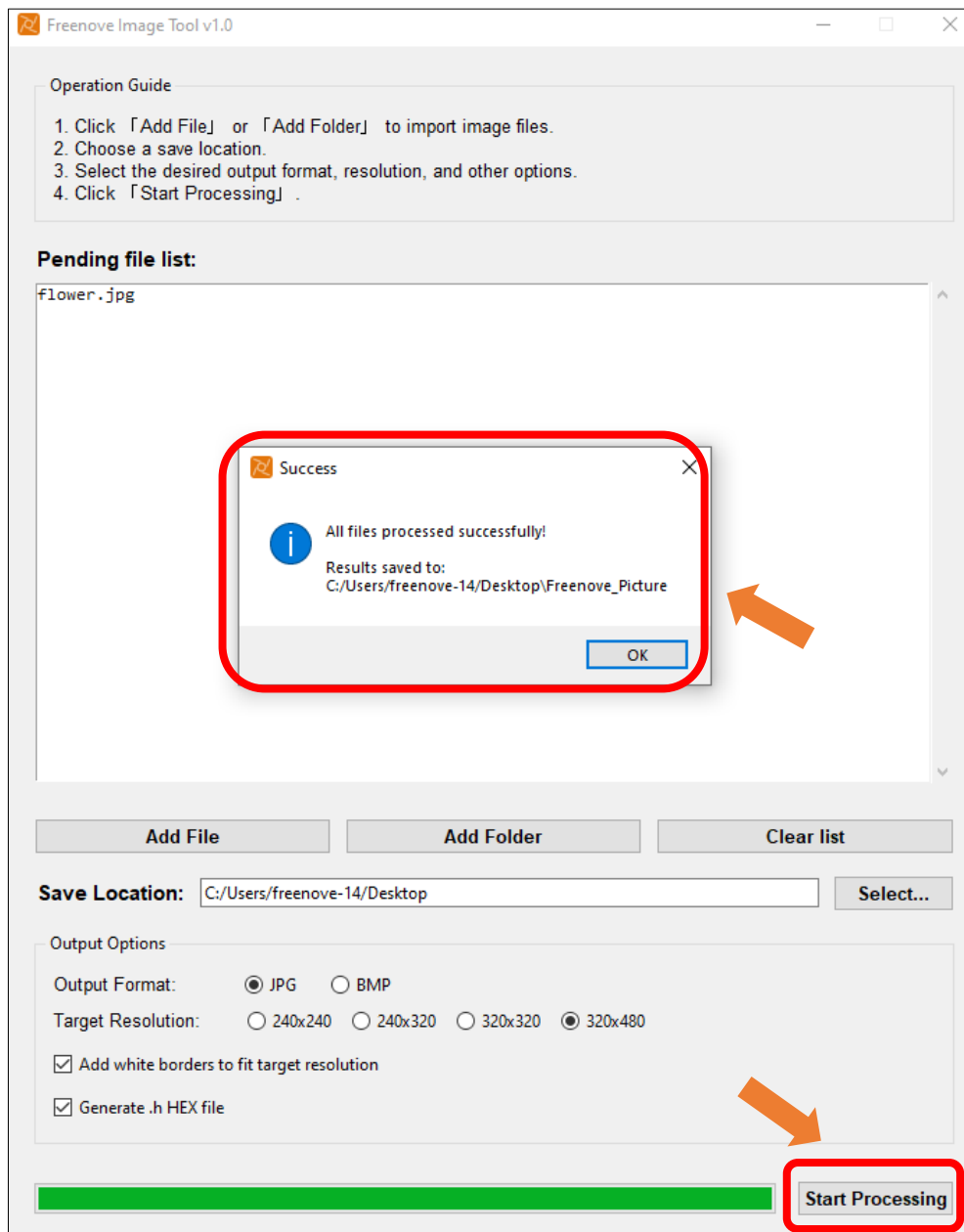
Click “Select...” to change the save location.



The resolution size is selected according to the [screen resolution](#). Check the "Add white borders to fit target resolution" and "Generate .h HEX file" options.



Click **Start Processing**. Wait for the progress bar to complete and the target folder will be generated.



There will be three folders in the generated folder

original_images: backup of images before processing

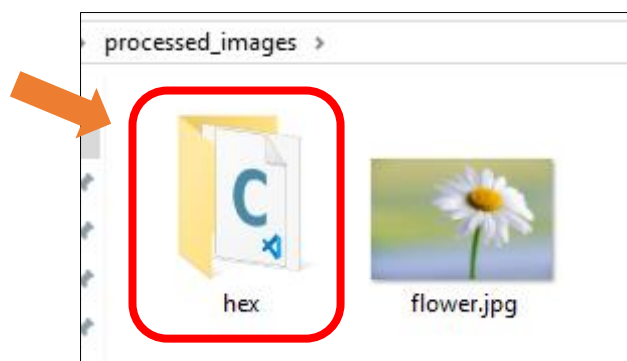
processed_images: processed images

processed_images\hex: generated .h files corresponding to the images

Double click to open **processed_images**



Double click to open **hex**



Replace the entire content of panda.h with the copied data from the .h file, then upload the sketch again.

```

Sketch_10.2_Flash_Jpg_DMA.ino  panda.h
1  /*
2      If you need to customize the displayed image, first use the Freenove Image Tool
3      to crop it to the appropriate width and height for your display.
4
5      The tool will automatically adjust the image quality to ensure proper display.
6
7      Paste the generated array into this file, ensuring the correct array format:
8
9      const uint8_t name[] PROGMEM = {
10
11      to start and end with:
12
13      };
14  */
15
16  const unsigned char img_asset[] PROGMEM = {
17      0xFF, 0xD8, 0xFF, 0xE0, 0x00, 0x10, 0x4A, 0x46, 0x49, 0x46, 0x00, 0x01,
18      0x01, 0x00, 0x00, 0x01, 0x00, 0x01, 0x00, 0x00, 0xFF, 0xDB, 0x00, 0x43,
19      0x00, 0x05, 0x03, 0x04, 0x04, 0x04, 0x03, 0x05, 0x04, 0x04, 0x04, 0x05,
20      0x05, 0x05, 0x06, 0x07, 0x0C, 0x08, 0x07, 0x07, 0x07, 0x07, 0x0F, 0x0B,
21      0x0B, 0x09, 0x0C, 0x11, 0x0F, 0x12, 0x11, 0x0F, 0x11, 0x11, 0x13,
22      0x16, 0x1C, 0x17, 0x13, 0x14, 0x1A, 0x15, 0x11, 0x11, 0x18, 0x21, 0x18,
23      0x1A, 0x1D, 0x1D, 0x1F, 0x1F, 0x1F, 0x13, 0x17, 0x22, 0x24, 0x22, 0x1E,
24      0x24, 0x1C, 0x1E, 0x1F, 0x1E, 0xFF, 0xDB, 0x00, 0x43, 0x01, 0x05, 0x05,
25      0x05, 0x07, 0x06, 0x07, 0x0E, 0x08, 0x08, 0x0E, 0x1E, 0x14, 0x11, 0x14,
26      0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
27      0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
28      0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
29      0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E, 0x1E,
30      0x1E, 0x1E, 0xFF, 0xC0, 0x00, 0x11, 0x08, 0x01, 0x40, 0x01, 0xE0, 0x03,
31      0x01, 0x22, 0x00, 0x02, 0x11, 0x01, 0x03, 0x11, 0x01, 0xFF, 0xC4, 0x00,
32      0x1C, 0x00, 0x00, 0x02, 0x02, 0x03, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00,
33      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x01, 0x01, 0x01, 0x01,

```


The image will display on the screen.



Note:

To adjust the image display orientation, modify the following code in Sketch_10.2_Flash_Jpg_DMA.ino:

```
42  tft.setRotation(uint8_t rotation);
```

Value	Rotation Angle	Description
0	0°	Default orientation (no rotation)
1	90°	Clockwise 90-degree rotation
2	180°	Clockwise 180-degree rotation
3	270°	Clockwise 270-degree rotation

Project 10.3 SD JPG

Having learned SD card operations in earlier chapters, this section will demonstrate how to render images stored on the SD card to the display.

Component List

Freenove ESP32 Display x 1



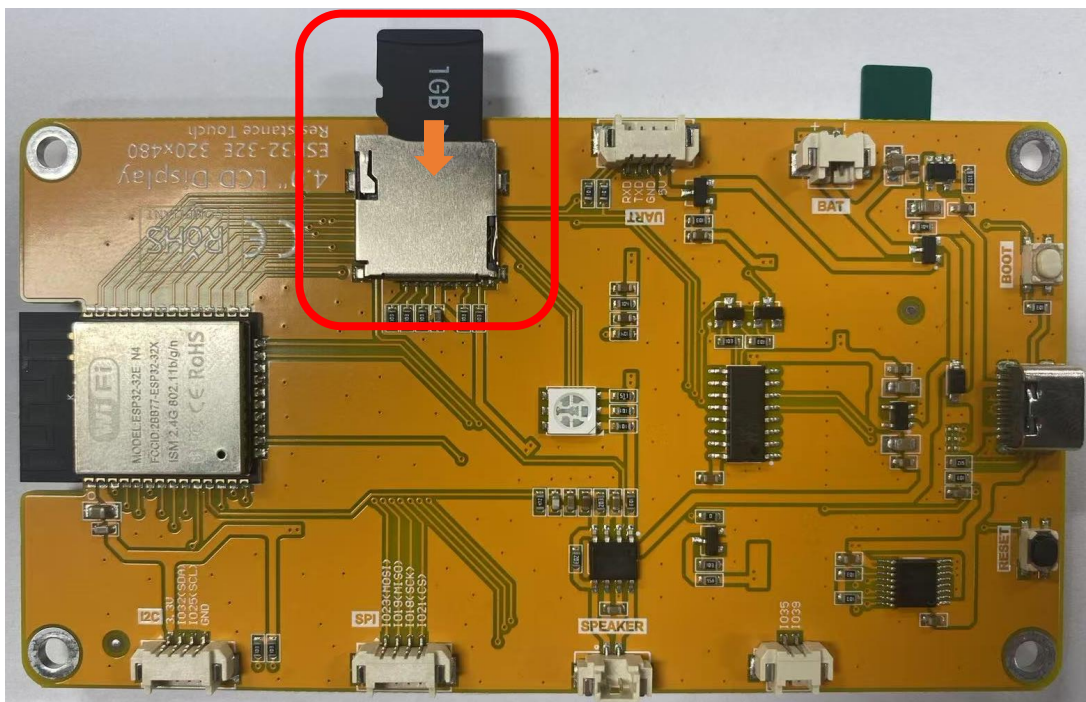
USB cable x1



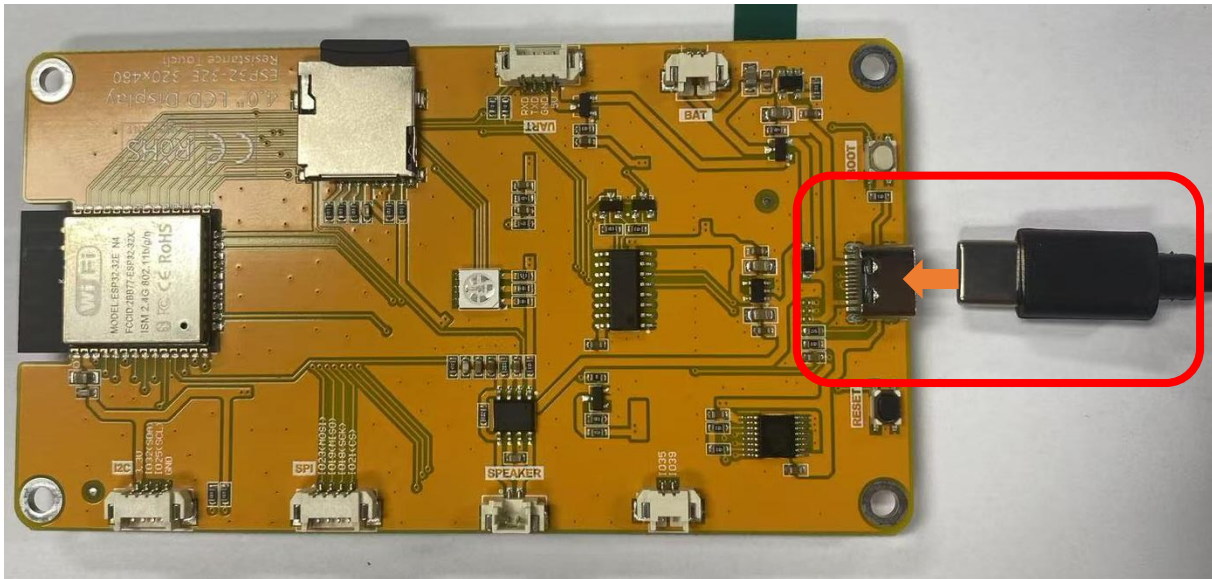
This kit does not include an SD card and the card reader, please buy them yourself. For more details, please refer to [SD card](#).

Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32 Display.



Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open “Sketch_10.3_SD_Jpg” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_10.3_SD_Jpg.ino”.

Sketch_10.3_SD_Jpg

The following is the program code:

```

1  #include <TJpg_Decoder.h> // JPEG decoder library
2  #include <TFT_eSPI.h> // TFT display library
3  TFT_eSPI tft = TFT_eSPI();
4
5  #define SD_VSPI_SS 5 // SD card chip select pin
6  #define SD_VSPI_MOSI 23 // SD card SPI MOSI pin
7  #define SD_VSPI_MISO 19 // SD card SPI MISO pin
8  #define SD_VSPI_SCK 18 // SD card SPI SCK pin
9  SPIClass* vspi = NULL; // VSPi
10
11 bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
12     if (y >= tft.height()) return 0;
13     tft.pushImage(x, y, w, h, bitmap);
14     return 1;
15 }
16
17 void setup() {
18     Serial.begin(115200);
19     Serial.println("\n\n Testing TJpg_Decoder library");
20     tft.begin();
21     // tft.setRotation(1);
22     vspi = new SPIClass(VSPi);

```

```

23  vspi->begin(SD_VSPI_SCK, SD_VSPI_MISO, SD_VSPI_MOSI, SD_VSPI_SS);
24  if (!SD.begin(SD_VSPI_SS, *vspi, 1000000)) {
25      Serial.println(F("SD.begin failed!"));
26      while (1) delay(0);
27  }
28  Serial.println("\r\nInitialisation done.");
29  tft.setTextColor(0xFFFF, 0x0000);
30  tft.fillScreen(TFT_BLACK);
31  TJpgDec.setJpgScale(1);
32  tft.setSwapBytes(true);
33  TJpgDec.setCallback(tft_output);
34  }
35
36  void loop() {
37      uint32_t t = millis();
38      uint16_t w = 0, h = 0;
39      TJpgDec.getSdJpgSize(&w, &h, "/Board.jpg");
40      Serial.print("Width = ");
41      Serial.print(w);
42      Serial.print(", height = ");
43      Serial.println(h);
44      TJpgDec.drawSdJpg(0, 0, "/Board.jpg");
45      t = millis() - t;
46      Serial.print(t);
47      Serial.println(" ms");
48      delay(2000);
49  }
50
51
52
53
54
55

```

Code Explanation

Included the necessary header files.

```

7  #include <TFT_eSPI.h>      // TFT display library
8  #include <TJpg_Decoder.h>  // Include JPEG decoder library

```

Create TFT object instance.

```

19  TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance

```

Define pins for the SD card.

```

11  #define SD_VSPI_SS 5      // SD card chip select pin
12  #define SD_VSPI_MOSI 23  // SD card SPI MOSI pin
13  #define SD_VSPI_MISO 19  // SD card SPI MISO pin
14  #define SD_VSPI_SCK 18   // SD card SPI SCK pin

```

JPEG decoding callback function

```

15 // Callback function to draw decoded JPEG blocks on screen
16 bool tft_output(int16_t x, int16_t y, uint16_t w, uint16_t h, uint16_t* bitmap) {
17     if (y >= tft.height()) return 0; // Stop decoding if off-screen
18
19     #ifdef USE_DMA
20         if (dmaBufferSel) dmaBufferPtr = dmaBuffer2;
21         else dmaBufferPtr = dmaBuffer1;
22         dmaBufferSel = !dmaBufferSel;
23         tft.pushImageDMA(x, y, w, h, bitmap, dmaBufferPtr); // Start DMA transfer
24     #else
25         tft.pushImage(x, y, w, h, bitmap); // Draw without DMA
26     #endif
27
28     return 1; // Continue decoding next block
29 }

```

Obtain image dimensions.

```

46 TJpgDec.getJpgSize(&w, &h, img_asset, sizeof(img_asset)); // Get image dimensions

```

Draw an image on the TFT screen.

```

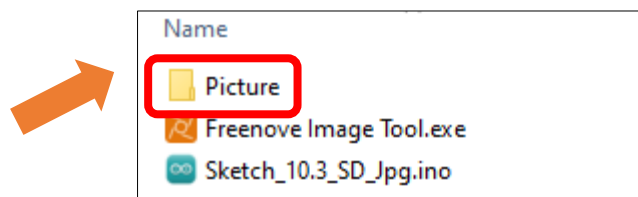
57 tft.startWrite();
58 TJpgDec.drawJpg(0, 0, img_asset, sizeof(img_asset)); // Draw the JPEG image
59 tft.endWrite();

```

"Before uploading, remove the SD card from the Freenove ESP32 Display. Use a card reader to copy your image files to the root directory of the SD card.

WARNING: To ensure stable operation and prevent SD card damage, never insert or remove the SD card while the Freenove ESP32 Display is powered on. Sudden removal may cause data corruption or permanent hardware failure.

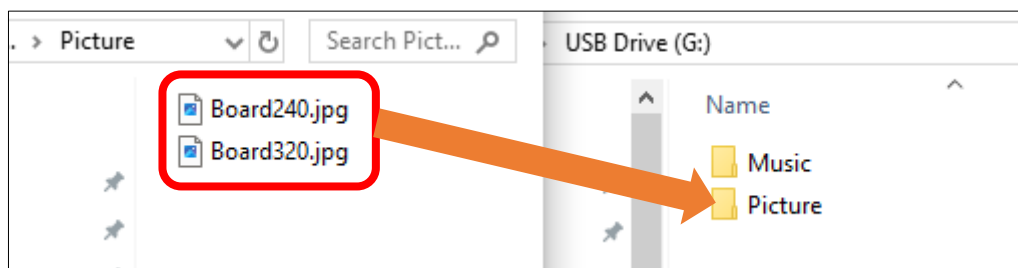
Double click to open **Sketch_10.3_SD_Jpg\Picture**



Copy the image files from the Picture folder to your SD card. Please select the appropriate files based on your Freenove ESP32 Display's [screen resolution](#):

Files containing '240' in their names are for 240×320 resolution displays

Files containing '320' in their names are for 320×480 resolution displays"



Modify the corresponding filenames in the code. Here we take 'Board320.jpg' as an example:

```

42 void loop() {
43     uint32_t t = millis();
44     uint16_t w = 0, h = 0;
45     TJpgDec.getSdJpgSize(&w, &h, "/Board320.jpg");
46     Serial.print("Width = ");
47     Serial.print(w);
48     Serial.print(", height = ");
49     Serial.println(h);
50     TJpgDec.drawSdJpg(0, 0, "/Board320.jpg");
51     t = millis() - t;
52     Serial.print(t);

```

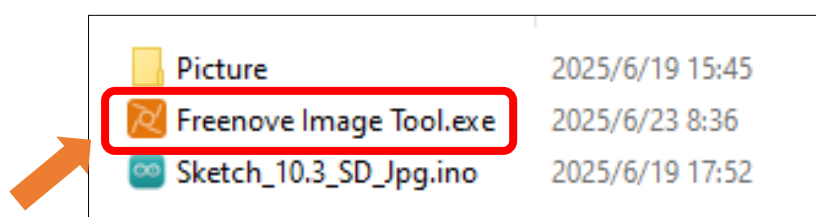
Click "Upload" to upload the code to Freenove ESP32 Display



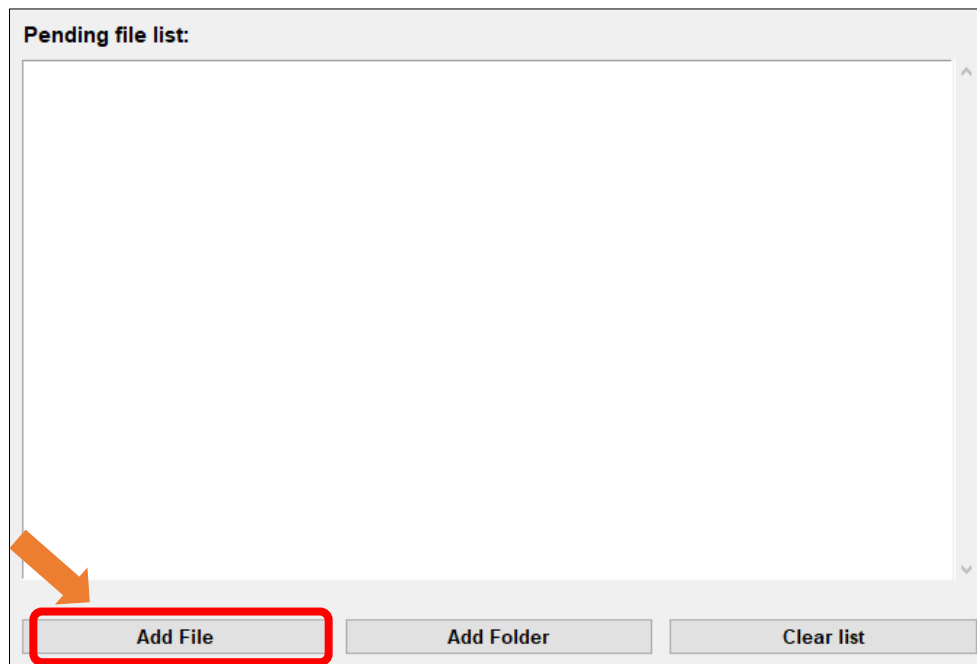
Custom Image Display

You can customize display images according to your preferences. **Ensure all custom images use JPG format.**

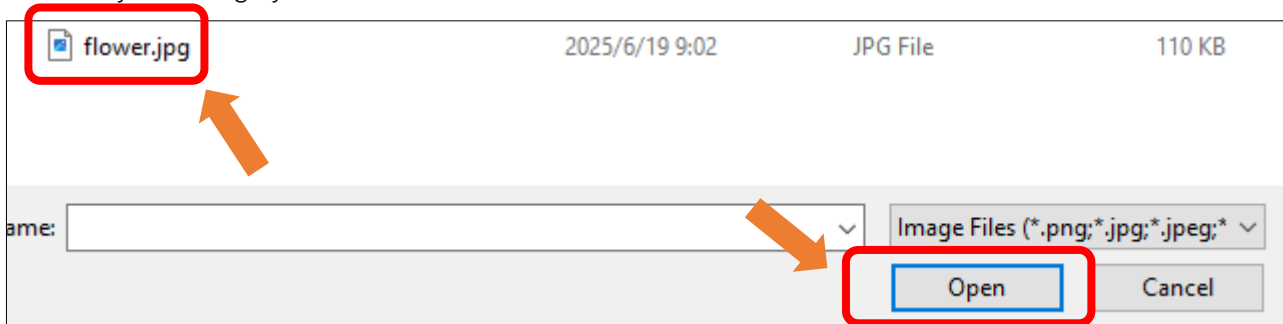
First, open **Freenove_ESP32_Display\Sketches\Sketch_10.3_SD_Jpg\Freenove Image Tool.exe**



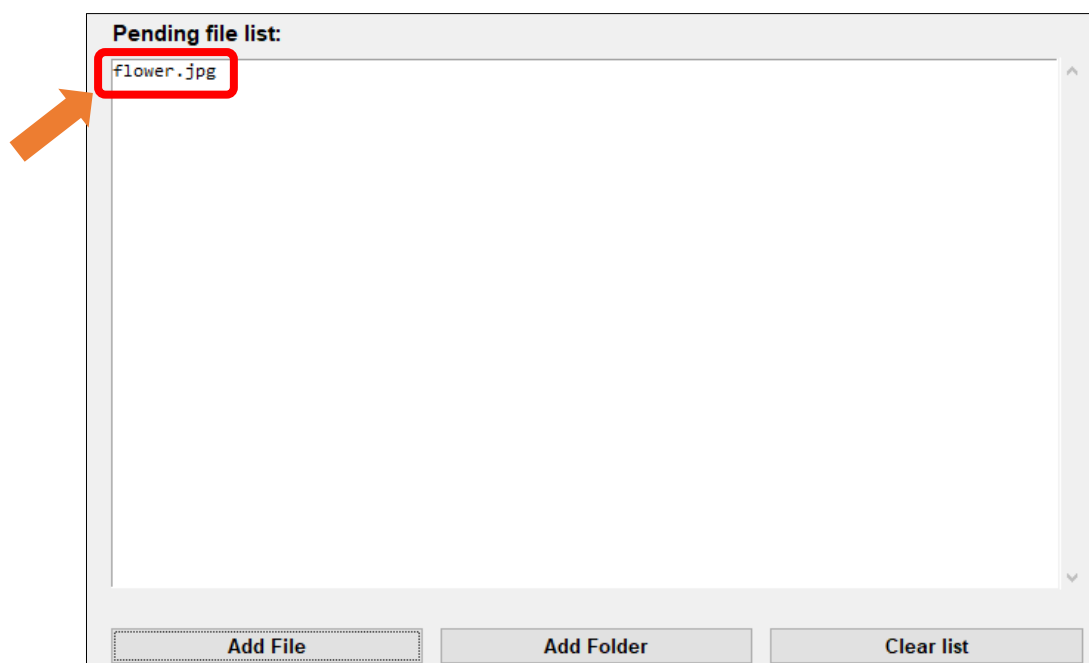
Click “Add File”



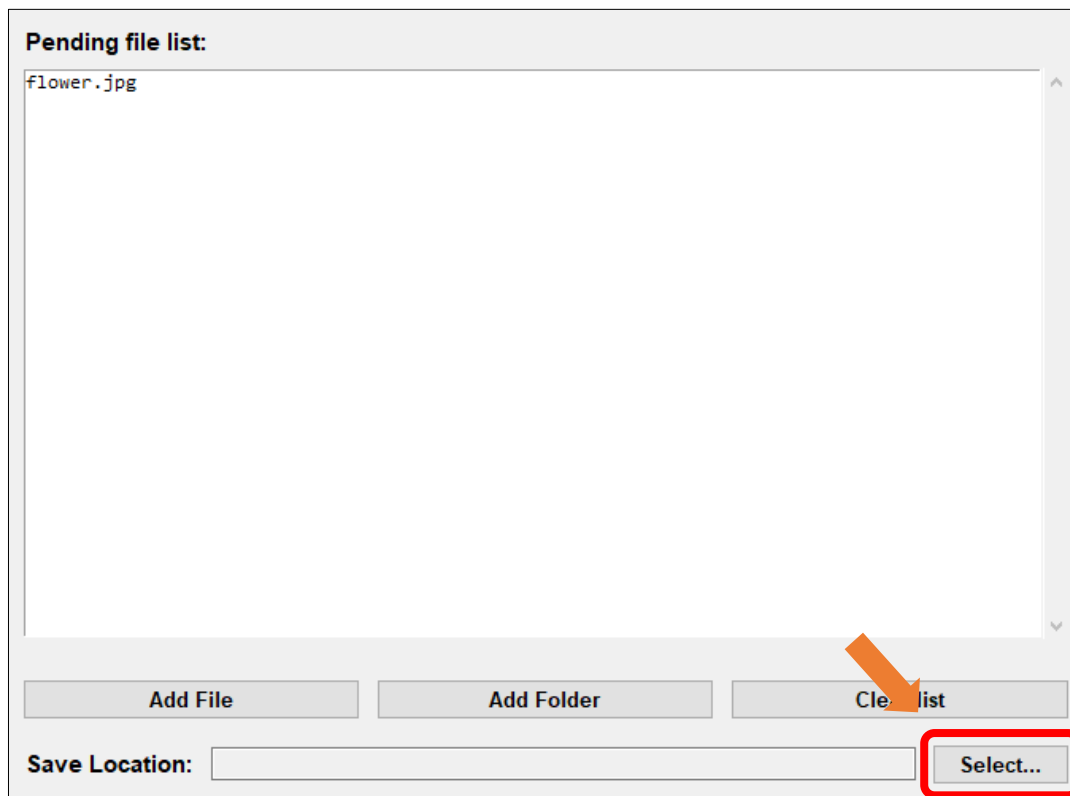
Select any JPG image you like.



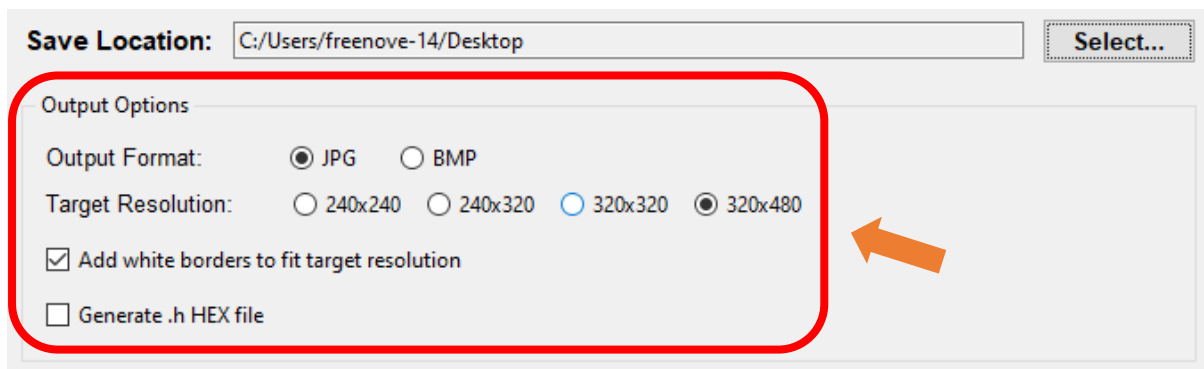
The image files from your folder will now appear in the **Pending File List**.



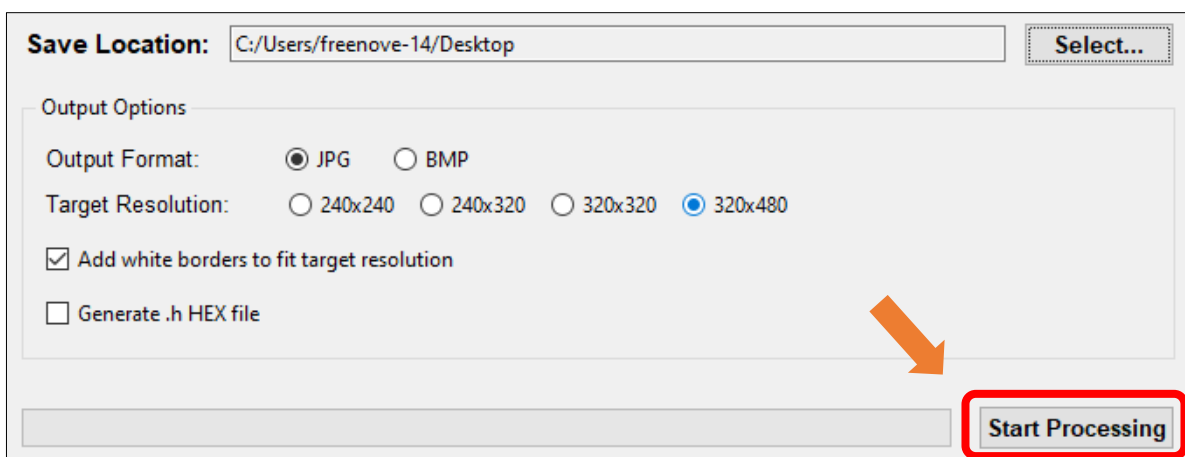
Click “Select...” to select the image save location.

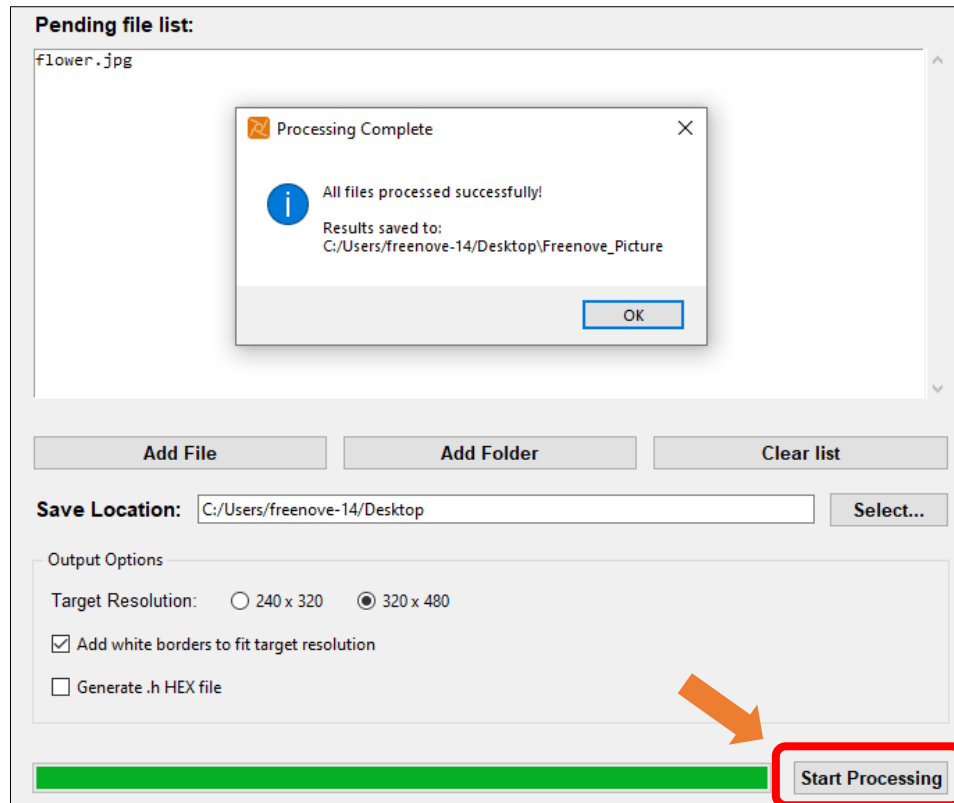


The resolution size is selected according to the [screen resolution](#). Check the "Add white borders to fit target resolution"



Click **Start Processing**. Wait for the progress bar to complete and the target folder will be generated.



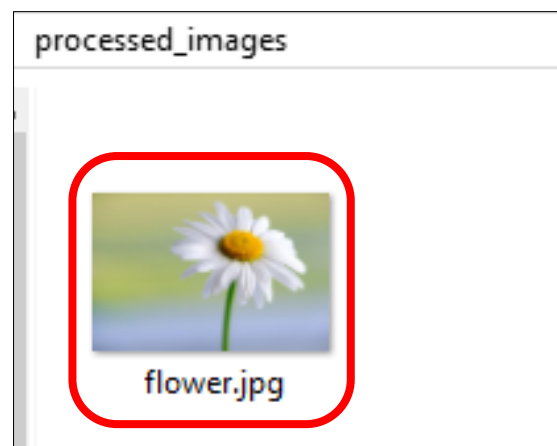


There will be two folders in the generated folder
original_images: backup of images before processing
processed_images: processed images

Double click to open **processed_images**



Upload the generated image file to the root directory of the SD card.



Change the file name in the code.

```
42 void loop() {  
43     uint32_t t = millis();  
44     uint16_t w = 0, h = 0;  
45     TJpgDec.getSdJpgSize(&w, &h, "/flower.jpg");  
46     Serial.print("Width = ");  
47     Serial.print(w);  
48     Serial.print(", height = ");  
49     Serial.println(h);  
50     TJpgDec.drawSdJpg(0, 0, "/flower.jpg");  
51     t = millis() - t;  
52     Serial.print(t);
```

Upload the sketch and the image will display on the screen.



Chapter 11 TFT Touch Calibration

Project 11.1 TFT Touch Calibration

Component List

Freenove ESP32 Display x 1



USB cable x1

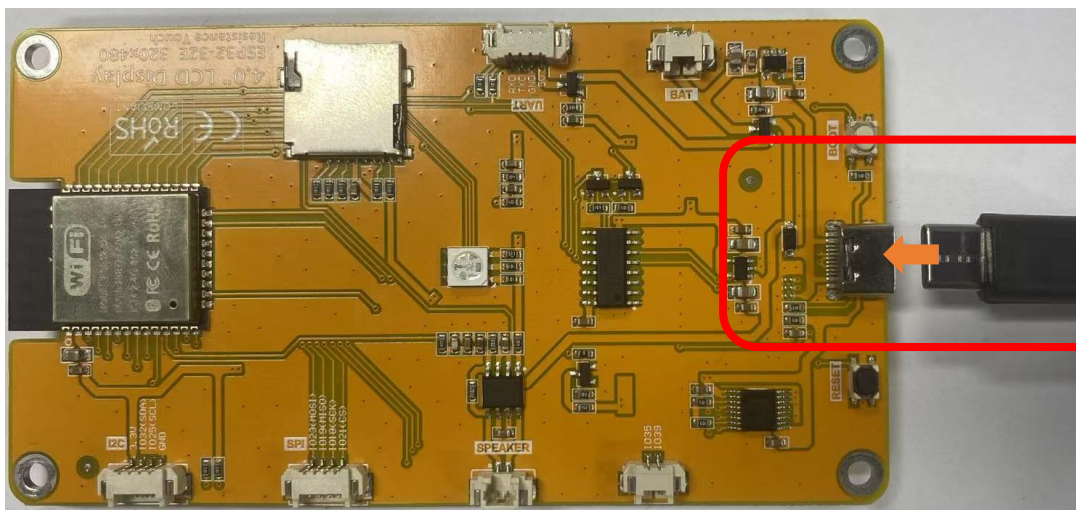


Stylus x 1

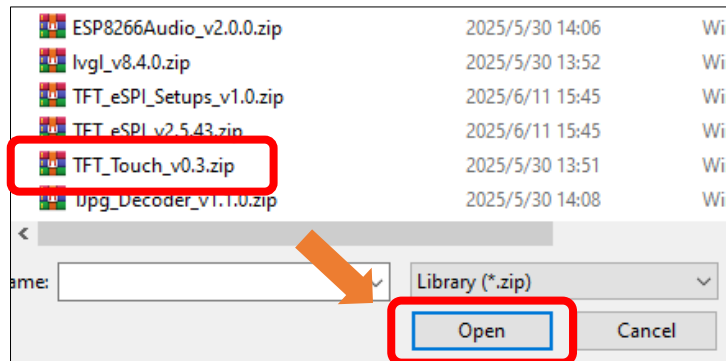


Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.







The following message indicates the successful installation of the library.



The following is the program code:

```

1  #include "TFT_eSPI.h" // TFT display library
2  #include <TFT_Touch.h>
3
4  // TFT Screen pixel resolution in landscape orientation, change these to suit your display
5  // Defined in landscape orientation !
6  #define HRES 320
7  #define VRES 240
8
9  TFT_eSPI tft = TFT_eSPI(); // Invoke custom library
10
11 #define DOUT 39 /* Data out pin (T_D0) of touch screen */
12 #define DIN 32 /* Data in pin (T_DIN) of touch screen */
13 #define DCS 33 /* Chip select pin (T_CS) of touch screen */
14 #define DCLK 25 /* Clock pin (T_CLK) of touch screen */
15
16 /* Create an instance of the touch screen library */
17 TFT_Touch touch = TFT_Touch(DCS, DCLK, DIN, DOUT);
18
19 int X_Raw = 0, Y_Raw = 0;
20
21 void setup()

```

```
22 {
23     // Messages are sent to the serial port
24     Serial.begin(115200);
25
26     // Initialise the TFT
27     tft.init();
28
29     // Set the TFT screen to landscape orientation
30     tft.setRotation(1);
31
32     tft.setTextDatum(TC_DATUM); // Set text plotting reference datum to Top Centre (TC)
33     tft.setTextColor(TFT_WHITE, TFT_BLACK); // Set text to white on black
34 }
35
36 /* Main program */
37 void loop()
38 {
39     int x1, y1;
40     int x2, y2;
41     int x3, y3;
42     bool xyswap = 0, xflip = 0, yflip = 0;
43
44     Serial.println("TFT_Touch Calibration, follow TFT screen prompts..");
45
46     // Reset the calibration values
47     touch.setCal(0, 4095, 0, 4095, 320, 240, 0); //, 0, 0);
48
49     // Set TFT the screen to landscape orientation
50     tft.setRotation(1);
51
52     // Set Touch the screen to the same landscape orientation
53     touch.setRotation(1);
54
55     // Clear the screen
56     tft.fillRect(TFT_BLACK);
57
58     // Show the screen prompt
59     drawPrompt();
60
61     drawCross(30, 30, TFT_RED);
62     while (!touch.Pressed());
63     delay(100);
64
65     getCoord(); // This function assigns values to X_Raw and Y_Raw
```

```
66
67     drawCross(30, 30, TFT_BLACK);
68     Serial.print("First point : Raw x,y = ");
69     Serial.print(X_Raw);
70     Serial.print(",");
71     Serial.println(Y_Raw);
72
73     x1 = X_Raw;
74     y1 = Y_Raw;
75
76     drawCross(HRES/2, VRES/2, TFT_RED);
77     delay(10);
78
79     while (getCoord()); // This waits for the centre area to be touched
80
81     drawCross(HRES/2, VRES/2, TFT_BLACK);
82     Serial.print("Second point : Raw x,y = ");
83     Serial.print(X_Raw);
84     Serial.print(",");
85     Serial.println(Y_Raw);
86
87     drawCross(HRES-30, VRES-30, TFT_RED);
88
89     while (!getCoord()); // This waits until the centre area is no longer pressed
90     delay(10);          // Wait a little for touch bounces to stop after release
91
92     getCoord();
93     drawCross(HRES-30, VRES-30, TFT_BLACK);
94     Serial.print("Third point : Raw x,y = ");
95     Serial.print(X_Raw);
96     Serial.print(",");
97     Serial.println(Y_Raw);
98
99     x2 = X_Raw;
100    y2 = Y_Raw;
101
102    drawCross(HRES/2, VRES/2, TFT_RED);
103    delay(10);
104
105    while (getCoord()); // This waits for the centre area to be touched
106
107    drawCross(HRES/2, VRES/2, TFT_BLACK);
108    Serial.print("Fourth point : Raw x,y = ");
109    Serial.print(X_Raw);
```

```
110 Serial.print(",");
111 Serial.println(Y_Raw);
112
113 drawCross(30, VRES-30, TFT_RED);
114
115 while (!getCoord()); // This waits until the centre area is no longer pressed
116 delay(10);           // Wait a little for touch bounces to stop after release
117
118 getCoord();
119 drawCross(30, VRES-30, TFT_BLACK);
120 Serial.print("Fifth point : Raw x,y = ");
121 Serial.print(X_Raw);
122 Serial.print(",");
123 Serial.println(Y_Raw);
124
125 x3 = X_Raw;
126 y3 = Y_Raw;
127
128 int temp;
129 if (abs(x1 - x3) > 1000) {
130     xyswap = 1;
131     temp = x1; x1 = y1; y1 = temp;
132     temp = x2; x2 = y2; y2 = temp;
133     temp = x3; x3 = y3; y3 = temp;
134 }
135 else xyswap = 0;
136
137
138 //if (x2 < x1) {
139 //    temp = x2; x2 = x1; x1 = temp;
140 //    xflip = 1;
141 //}
142
143 //if (y2 < y1) {
144 //    temp = y2; y2 = y1; y1 = temp;
145 //    yflip = 1;
146 //}
147
148 int hmin = x1; // - (x2 - x1) * 3 / (HRES/10 - 6);
149 int hmax = x2; // + (x2 - x1) * 3 / (HRES/10 - 6);
150
151 int vmin = y1; // - (y2 - y1) * 3 / (VRES/10 - 6);
152 int vmax = y2; // + (y2 - y1) * 3 / (VRES/10 - 6);
153
```



```
154 Serial.println();
155 Serial.println(" //This is the calibration line to use in your sketch");
156 Serial.println(" //you can copy and paste into your sketch setup()");
157 Serial.print(" touch.setCal(");
158 Serial.print(hmin); Serial.print(", ");
159 Serial.print(hmax); Serial.print(", ");
160 Serial.print(vmin); Serial.print(", ");
161 Serial.print(vmax); Serial.print(", ");
162 Serial.print(HRES); Serial.print(", ");
163 Serial.print(VRES); Serial.print(", ");
164 Serial.print(xyswap);
165
166 Serial.println(");");
167
168 Serial.println();
169 Serial.println("Test the touch screen, green crosses appear at the touch coordinates!");
170 Serial.println("Send any character from the serial monitor window to restart calibration");
171
172 // These are the calibration settings the sketch has calculated to try out!
173 touch.setCal(hmin, hmax, vmin, vmax, HRES, VRES, xyswap);//, xflip, yflip); // Raw xmin,
174 xmax, ymin, ymax, 320, 240, XYswap, xflip, yflip
175
176 // Keep TFT and Touch rotations the same, try values 0 to 3
177 // Start with the current orientation
178 // Receiving any serial character moves on to next orientation test
179 tft.setRotation(1);
180 touch.setRotation(1);
181 test();
182
183 tft.setRotation(2);
184 touch.setRotation(2);
185 test();
186
187 tft.setRotation(3);
188 touch.setRotation(3);
189 test();
190
191 tft.setRotation(0);
192 touch.setRotation(0);
193 test();
194
195 Serial.println();
196 }
197
```

```
198 void test(void)
199 {
200     tft.fillScreen(TFT_BLACK);
201
202     drawCross(30, 30, TFT_WHITE);
203
204     drawCross(tft.width() - 30, tft.height() - 30, TFT_WHITE);
205
206     int centre = tft.width()/2; // Get and work out x coord of screen centre
207
208     String text;
209     text+= "Screen rotation = ";
210     text+= tft.getRotation();
211     char buffer[30];
212     text.toCharArray(buffer, 30);
213
214     tft.drawString(buffer, centre, 50, 2);
215
216     tft.drawString("Touch anywhere on screen", centre, 70, 2);
217     tft.drawString("to test settings", centre, 90, 2);
218
219     tft.drawString("Send a character from the", centre, 120, 2);
220     tft.drawString("IDE Serial Monitor to", centre, 140, 2);
221     tft.drawString("continue!", centre, 160, 2);
222
223     while (Serial.available()) Serial.read(); // Empty the serial buffer before we start
224
225     while (!Serial.available()) {
226         if (touch.Pressed()) // Note this function updates coordinates stored within library
227         variables
228         {
229             /* Read the current X and Y axis as co-ordinates at the last touch time*/
230             // The values returned were captured when Pressed() was called!
231             int X_Coord = touch.X();
232             int Y_Coord = touch.Y();
233
234             drawCross(X_Coord, Y_Coord, TFT_GREEN);
235
236             //delay(20);
237             tft.setCursor(centre, 0, 2);
238             tft.print("X = ");tft.print(X_Coord);tft.print(" ");
239             tft.setCursor(centre, 20, 2);
240             tft.print("Y = ");tft.print(Y_Coord);tft.print(" ");
241
```

```
242     }
243 }
244 }
245
246 void drawPrompt(void)
247 {
248     tft.setTextColor(TFT_WHITE, TFT_BLACK);
249
250     int centre = tft.width()/2; // Get and work out x coord of screen centre
251
252     tft.drawString("CALIBRATION", centre, 20, 2);
253
254     tft.drawString("Touch the red cross accurately", centre, 61, 2);
255     tft.drawString("( using a cocktail stick works well! )", centre, 81, 2);
256 }
257
258 void drawCross(int x, int y, unsigned int color)
259 {
260     tft.drawLine(x - 5, y, x + 5, y, color);
261     tft.drawLine(x, y - 5, x, y + 5, color);
262 }
263
264 bool getCoord()
265 {
266     bool Xwait = 1, Ywait = 1;
267     int X_Temp1 = 9999, Y_Temp1 = 9999;
268     int X_Temp2 = -1, Y_Temp2 = -1;
269     X_Raw = -1;
270     Y_Raw = -1;
271
272     while (Xwait || Ywait) {
273         if (touch.Pressed()) // Note this function updates coordinates stored within library
274         variables
275         {
276             /* Read the current X and Y axis as co-ordinates at the last touch time*/
277             // The values returned were captured when Pressed() was called!
278             X_Temp1 = touch.RawX();
279             Y_Temp1 = touch.RawY();
280         }
281         delay(5);
282         if (touch.Pressed()) // Note this function updates coordinates stored within library
283         variables
284         {
285             /* Read the current X and Y axis as co-ordinates at the last touch time*/
```

```

286      // The values returned were captured when Pressed() was called!
287      X_Temp2 = touch.RawX();
288      Y_Temp2 = touch.RawY();
289  }
290
291  #define RAW_ERROR 10
292
293  if ( (abs(X_Temp1 - X_Temp2) < RAW_ERROR) && Xwait ) {
294      X_Raw = (X_Temp1 + X_Temp2) / 2;
295      Xwait = 0;
296  }
297  if ( (abs(Y_Temp1 - Y_Temp2) < RAW_ERROR) && Ywait ) {
298      Y_Raw = (Y_Temp1 + Y_Temp2) / 2;
299      Ywait = 0;
300  }
301  }
302
303  // Check if press is near middle third of screen
304  if ((X_Raw > 1365) && (X_Raw < 2731) && (Y_Raw > 1365) && (Y_Raw < 2371)) return 0;
305
306  // otherwise it is near edge for calibration points
307  else return 1;
308  }
309
310
311
312
313
314

```

Code Explanation

Include the necessary header files.

```

7  #include <SPI.h>
8  #include <TFT_eSPI.h>

```

Create TFT object instance.

```

19  TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance

```

Adjust the display direction of the screen.

```

20  tft.setRotation(1);

```

The touch calibration function.

```

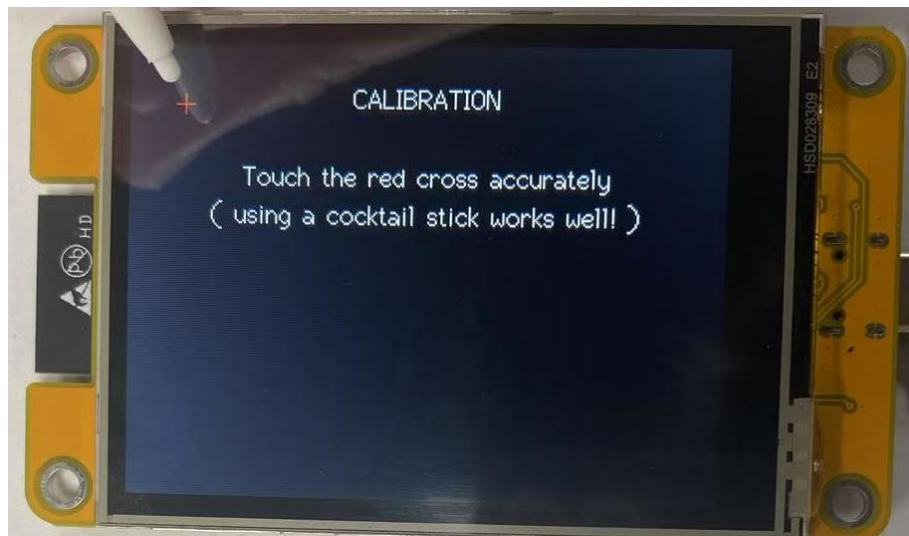
56  tft.calibrateTouch(calData, TFT_MAGENTA, TFT_BLACK, 15);

```

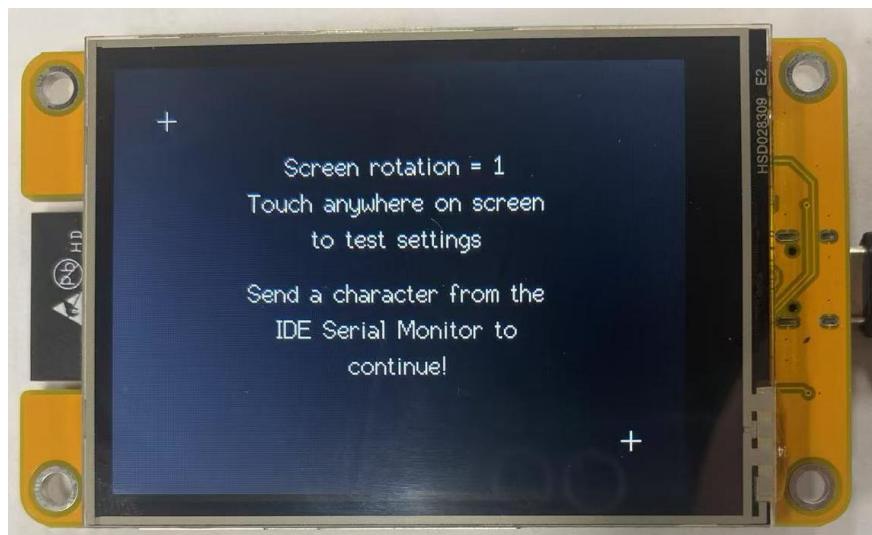
Click **“Upload”** to upload the code to Freenove ESP32 Display, set the baud rate to 115200.



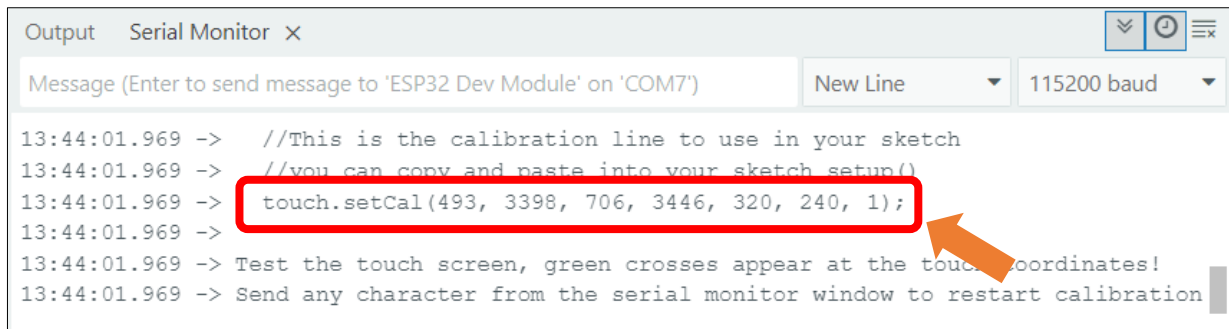
Click the positions of the red cross cursor on the screen in sequence according to the prompts on the display



When the following message appears on the screen, the calibration is complete.



Save the calibration results printed in the serial monitor.



```
Output  Serial Monitor  X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM7')  New Line  115200 baud
13:44:01.969 -> //This is the calibration line to use in your sketch
13:44:01.969 -> //you can copy and paste into your sketch setup()
13:44:01.969 -> touch.setCal(493, 3398, 706, 3446, 320, 240, 1);
13:44:01.969 ->
13:44:01.969 -> Test the touch screen, green crosses appear at the touch coordinates!
13:44:01.969 -> Send any character from the serial monitor window to restart calibration
```

Note: This calibration code only needs to run once during initial setup to obtain screen calibration parameters. After successful calibration, please save the calibration values for future use – no need to repeat this calibration procedure afterward.

Sketch_11.1_TFT_Touch_Calibrate_3.2_3.5_4.0_Inch

The following is the program code:

```
1  #include <SPI.h>
2  #include <TFT_eSPI.h>
3  TFT_eSPI tft = TFT_eSPI();
4
5  void setup() {
6      // Use serial port
7      Serial.begin(115200);
8
9      // Initialise the TFT screen
10     tft.init();
11
12     // Set the rotation to the orientation you wish to use in your project before calibration
13     // (the touch coordinates returned then correspond to that rotation only)
14     tft.setRotation(1);
15
16     // Calibrate the touch screen and retrieve the scaling factors
17     touch_calibrate();
18
19     /*
20     // Replace above line with the code sent to Serial Monitor
21     // once calibration is complete, e.g.:
22     uint16_t calData[5] = { 286, 3534, 283, 3600, 6 };
23     tft.setTouch(calData);
24     */
25
26     // Clear the screen
27     tft.fillScreen(TFT_BLACK);
28     tft.drawCentreString("Touch screen to test!", tft.width()/2, tft.height()/2, 2);
29 }
30
31 void loop(void) {}
32
33 // Code to run a screen calibration, not needed when calibration values set in setup()
34 void touch_calibrate()
35 {
36     uint16_t calData[5];
37
38     // Calibrate
39     tft.fillScreen(TFT_BLACK);
40     tft.setCursor(20, 0);
41     tft.setTextFont(2);
42     tft.setTextSize(1);
```

```
43  tft.setTextColor(TFT_WHITE, TFT_BLACK);
44
45  tft.println("Touch corners as indicated");
46
47  tft.setTextFont(1);
48  tft.println();
49
50  tft.calibrateTouch(calData, TFT_MAGENTA, TFT_BLACK, 15);
51
52  Serial.println(); Serial.println();
53  Serial.println("// Use this calibration code in setup():");
54  Serial.print("  uint16_t calData[5] = ");
55  Serial.print("{ ");
56
57  for (uint8_t i = 0; i < 5; i++)
58  {
59      Serial.print(calData[i]);
60      if (i < 4) Serial.print(", ");
61  }
62
63  Serial.println(" };");
64  Serial.print("  tft.setTouch(calData);");
65  Serial.println(); Serial.println();
66
67  tft.fillScreen(TFT_BLACK);
68
69  tft.setTextColor(TFT_GREEN, TFT_BLACK);
70  tft.println("Calibration complete!");
71  tft.println("Calibration code sent to Serial port.");
72
73  delay(4000);
74 }
```

Code Explanation

Include necessary header files.

```
7  #include <SPI.h>
8  #include <TFT_eSPI.h>
```


Create TFT object instance.

```
19 TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance
```

Adjust the display direction.

```
20 tft.setRotation(1);
```

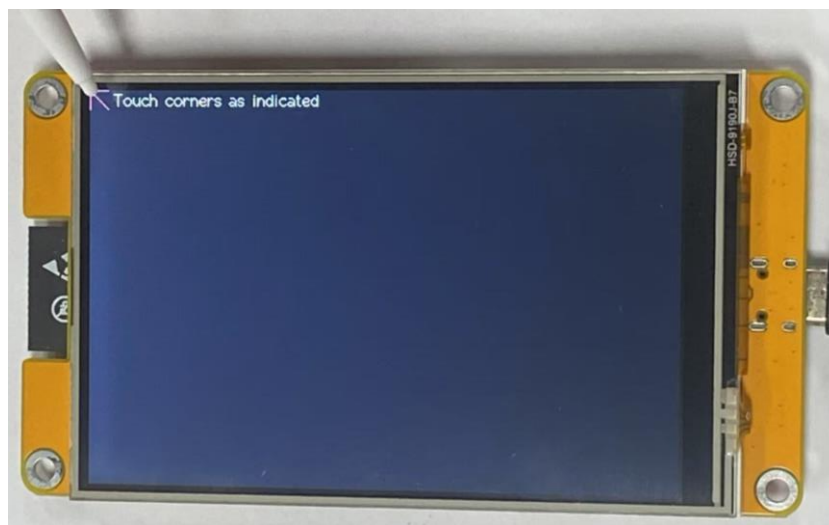
Touch calibrate function.

```
56 tft.calibrateTouch(calData, TFT_MAGENTA, TFT_BLACK, 15);
```

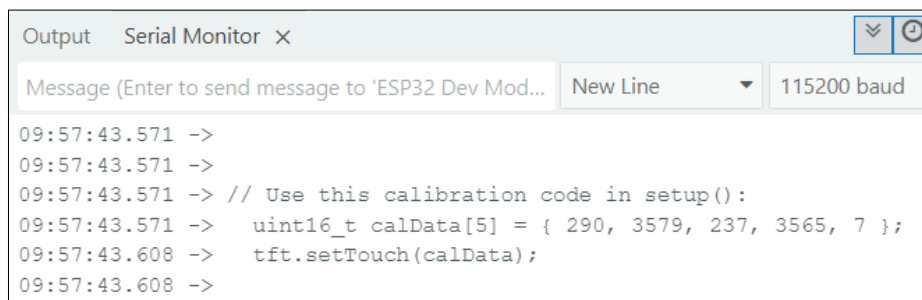
Click “Upload” to upload the code to Freenove ESP32 Display and set the baud rate to 115200.



Follow the on-screen instructions and sequentially tap the positions indicated by the purple arrows on the display.



The calibration results will be printed in the Serial Monitor. After calibration, you must **save** these values, as they will be used in the code implementation in the next section.



This calibration routine should be executed only once during initial device setup to acquire screen calibration parameters.

Chapter 12 TFT Touch Drawing

After learning this chapter, you will be able to draw freely on the screen.

Project 12.1 TFT Touch Drawing

Component List

Freenove ESP32 Display x 1



USB cable x1

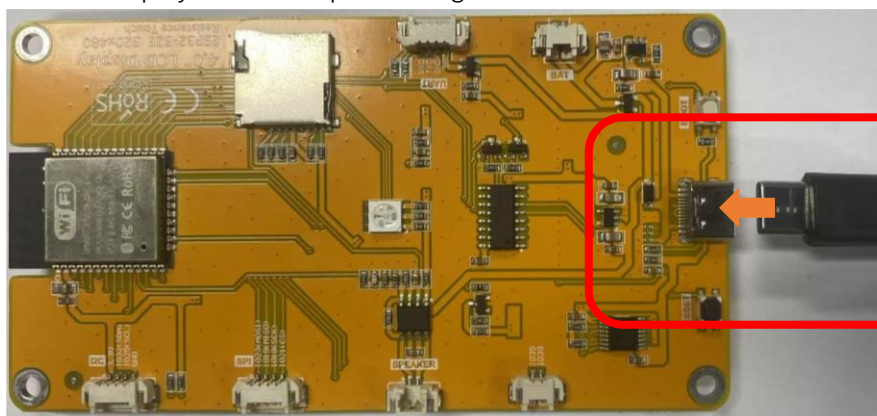


Stylus x 1



Circuit

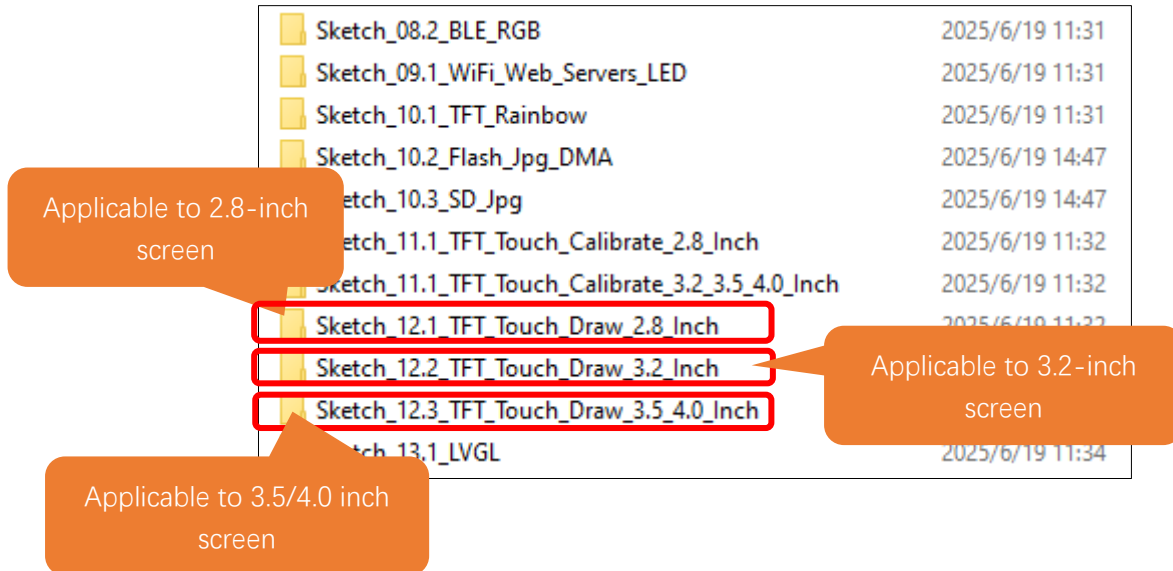
Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open “Sketch_12.1_TFT_Touch_Calibrate” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_12.1_TFT_Touch_Calibrate.ino”.

The sample code provides two codes for different models of [Freenove ESP32 Display](#). You can upload the corresponding sample code according to the actual model.



Sketch_12.1_TFT_Touch_Draw_2.8_Inch

The following is the program code:

```

1  #include <TFT_eSPI.h> // TFT display library
2
3  // Call up touch screen library
4  #include <TFT_Touch.h>
5
6  // Invoke custom TFT driver library
7  TFT_eSPI tft = TFT_eSPI(); // Invoke custom library
8
9  #define SCREEN_WIDTH 240
10 #define SCREEN_HEIGHT 320
11
12 #define DOUT 39 /* Data out pin (T_DO) of touch screen */
13 #define DIN 32 /* Data in pin (T_DIN) of touch screen */
14 #define DCS 33 /* Chip select pin (T_CS) of touch screen */
15 #define DCLK 25 /* Clock pin (T_CLK) of touch screen */
16
17 /* Create an instance of the touch screen library */
18 TFT_Touch touch = TFT_Touch(DCS, DCLK, DIN, DOUT);
19
20 // Define coordinate variables
21 uint16_t x, y;

```

```
22  uint16_t last_x, last_y;
23
24  const uint8_t ColorPalette = 21;      // Height of the color palette area
25  const uint8_t BrushSize = 2;         // Brush thickness (radius)
26  const uint8_t LineSize = 5;          // Line thickness (radius)
27  const uint8_t color_num = 12;
28  int color = TFT_WHITE;
29
30  unsigned int colors[color_num] = { TFT_RED, TFT_PINK, TFT_GREEN, TFT_BLUE, TFT_OLIVE ,
31  TFT_CYAN, TFT_YELLOW, TFT_WHITE, TFT_MAGENTA, TFT_ORANGE, TFT_BLACK, TFT_SILVER};
32
33  void drawUI()
34  {
35      // Draw color palette
36      for (int i = 0; i < color_num; i++) {
37          tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
38      }
39
40      // Draw clear screen button
41      tft.setCursor(260 , 3, 2);
42      tft.setTextColor(TFT_WHITE);
43      tft.print("Clear");
44
45      // Display current color
46      tft.fillRect(300, 5, 12, 12, color);
47
48      // Add palette border/outline
49      tft.drawRect(0, 0, SCREEN_HEIGHT, ColorPalette, TFT_WHITE);
50  }
51
52  void setup()
53  {
54      Serial.begin(115200);
55
56      tft.init();
57
58      //This is the calibration line produced by the TFT_Touch_Calibrate_v2 sketch
59      // touch.setCal(493, 3398, 706, 3446, 320, 240, 1);
60
61      // Set the TFT and touch screen to landscape orientation
62      tft.setRotation(1);
63      touch.setRotation(1);
64
65      tft.fillScreen(TFT_BLACK);
```

```
66
67     drawUI();
68 }
69
70 /* Main program */
71 void loop()
72 {
73     // Check if the touch screen is currently pressed
74     // Raw and coordinate values are stored within library at this instant
75     // for later retrieval by GetRaw and GetCoord functions.
76
77     if (touch.Pressed()) // Note this function updates coordinates stored within library
78 variables
79     {
80         // Read the current X and Y axis as co-ordinates at the last touch time
81         // The values were captured when Pressed() was called!
82         x = touch.X();
83         y = touch.Y();
84
85         Serial.print(x); Serial.print(","); Serial.println(y);
86
87         /* Color palette area logic */
88         if(y < ColorPalette + 3)
89         {
90             // Enter color selection area, stroke interrupted
91             last_x = 0;
92             last_y = 0;
93
94             /* Determine stylus click area */
95             // Clicked Clear button
96             if(x >= ColorPalette * color_num)
97             {
98                 tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
99             }
100             // Clicked color palette
101             else{
102                 // Calculate current color based on x-coordinate
103                 color = colors[x / ColorPalette];
104
105                 // Update display of current color
106                 tft.fillRect(300, 5, 12, 12, color);
107             }
108             delay(5);
109             return;
```

```

110     }
111
112     /* Drawing logic */
113     // Start of a new stroke
114     if(last_x == 0 && last_y == 0)
115     {
116         // Record current position
117         last_x = x;
118         last_y = y;
119     }
120
121     // Draw a line between the previous position and current position
122     tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
123
124     // Update coordinates
125     last_x = x;
126     last_y = y;
127 }
128 else{
129     // Reset coordinates when stylus leaves the screen
130     last_x = 0;
131     last_y = 0;
132 }
133 }

```

Code Explanation

Include the necessary header files.

```

7  #include <TFT_eSPI.h> // TFT display library
...
10 #include <TFT_Touch.h>

```

Create TFT object instance.

```

12 // Invoke custom TFT driver library
13 TFT_eSPI tft = TFT_eSPI(); // Invoke custom library

```

Define screen resolution.

```

15 #define SCREEN_WIDTH 240
16 #define SCREEN_HEIGHT 320

```

Draw color palette.

```

41 // Draw color palette
42 for (int i = 0; i < color_num; i++) {
43     tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
44 }

```

Detect whether the screen is pressed.

```
83  if (touch.Pressed())
```

Store the x and y coordinates of the touch point in variables.

```
88  x = touch.X();
```

```
89  y = touch.Y();
```

When the Clear button is pressed, fill the screen with the current color.

```
71  // Clicked Clear button
```

```
72  if(x >= ColorPalette * color_num)
```

```
73  {
```

```
74      tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
```

```
75  }
```

Calculate the selected color based on the coordinates of the touch point

```
77  // Clicked color palette
```

```
78  else{
```

```
79      // Calculate current color based on x-coordinate
```

```
80      color = colors[x / ColorPalette];
```

```
81
```

```
82      // Update display of current color
```

```
83      tft.fillRect(300, 5, 12, 12, color);
```

```
84  }
```

Draw a line based on coordinates

```
88  /* Drawing logic */
```

```
89  // Start of a new stroke
```

```
90  if(last_x == 0 && last_y == 0)
```

```
91  {
```

```
92      // Record current position
```

```
93      last_x = x;
```

```
94      last_y = y;
```

```
95  }
```

```
96
```

```
97  // Draw a line between the previous position and current position
```

```
98  tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
```

```
99
```

```
100 // Update coordinates
```

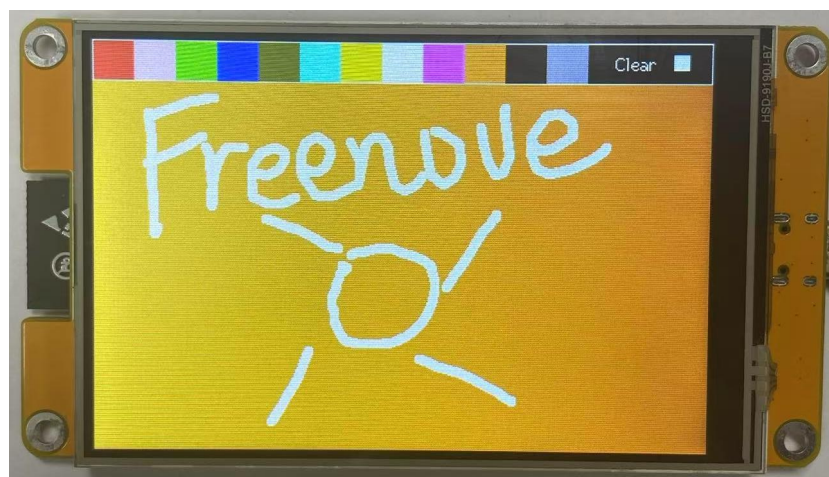
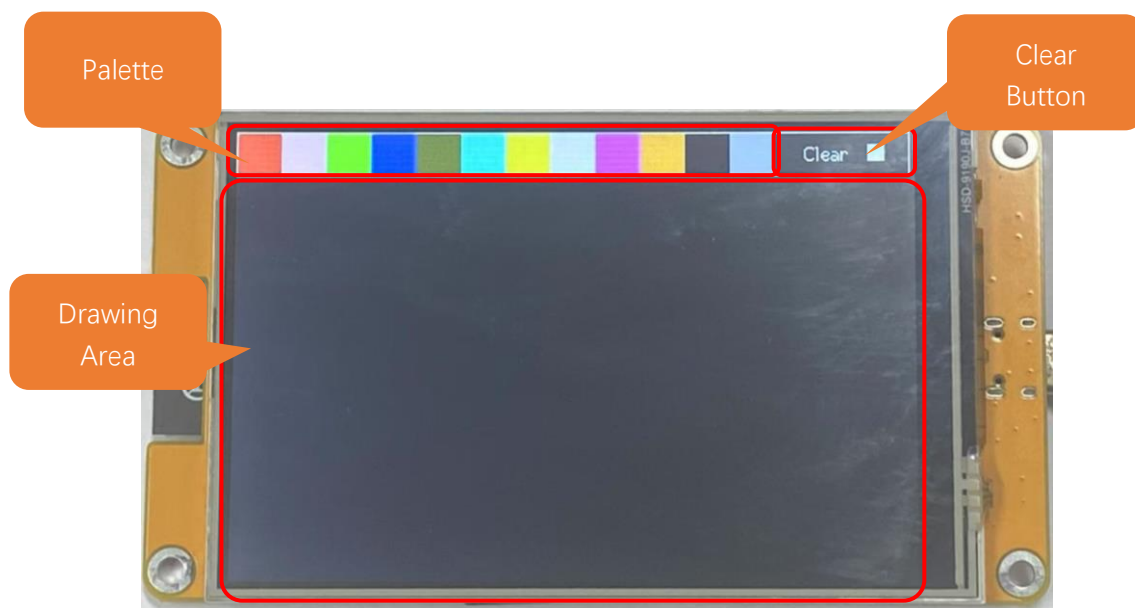
```
101 last_x = x;
```

```
102 last_y = y;
```

Add the calibration values obtained in the previous section to the code.

```
57 void setup()
58 {
59   Serial.begin(115200);
60
61   tft.init();
62
63   //This is the calibration line produced by the TFT_Touch_Calibrate_v2 sketch
64   touch.setCal(493, 3398, 706, 3446, 320, 240, 1);
65
66   // Set the TFT and touch screen to landscape orientation
```

Click **Upload** to upload the code to Freenove ESP32 Display. Set the baud rate to 115200



Sketch_12.2_TFT_Touch_Draw_3.2_Inch

The following is the program code:

```
1  #include <TFT_eSPI.h> // TFT display library
2
3  #define SCREEN_WIDTH 240
4  #define SCREEN_HEIGHT 320
5
6  TFT_eSPI tft = TFT_eSPI(SCREEN_WIDTH, SCREEN_HEIGHT);
7
8  // Define coordinate variables
9  uint16_t x, y;
10 uint16_t last_x, last_y;
11
12 const uint8_t ColorPalette = 21; // Height of the color palette area
13 const uint8_t BrushSize = 3; // Brush thickness (radius)
14 const uint8_t LineSize = 6; // Line thickness (radius)
15 const uint8_t color_num = 12;
16 int color = TFT_WHITE;
17
18 unsigned int colors[color_num] = { TFT_RED, TFT_PINK, TFT_GREEN, TFT_BLUE, TFT_OLIVE ,
19 TFT_CYAN, TFT_YELLOW, TFT_WHITE, TFT_MAGENTA, TFT_ORANGE, TFT_BLACK, TFT_SILVER};
20
21 void drawUI()
22 {
23     // Draw color palette
24     for (int i = 0; i < color_num; i++) {
25         tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
26     }
27
28     // Draw clear screen button
29     tft.setCursor(260 , 3, 2);
30     tft.setTextColor(TFT_WHITE);
31     tft.print("Clear");
32
33     // Display current color
34     tft.fillRect(300, 5, 12, 12, color);
35
36     // Add palette border/outline
37     tft.drawRect(0, 0, SCREEN_HEIGHT, ColorPalette, TFT_WHITE);
38 }
39
40 void setup() {
41     Serial.begin(115200);
42     tft.init();
```

```
43  uint16_t calData[5] = { 412, 3502, 262, 3596, 3 };
44  tft.setTouch(calData);
45  tft.setRotation(1);
46  tft.fillRect(TFT_BLACK);
47
48  drawUI();
49 }
50
51 void loop() {
52     bool pressed = tft.getTouch(&x, &y);
53
54     if (pressed) {
55         Serial.printf("x: %d, y: %d\r\n", x, y);
56
57         /* Color palette area logic */
58         if(y < ColorPalette + 3)
59         {
60             // Enter color selection area, stroke interrupted
61             last_x = 0;
62             last_y = 0;
63
64             /* Determine stylus click area */
65             // Clicked Clear button
66             if(x >= ColorPalette * color_num)
67             {
68                 tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
69             }
70             // Clicked color palette
71             else{
72                 // Calculate current color based on x-coordinate
73                 color = colors[x / ColorPalette];
74
75                 // Update display of current color
76                 tft.fillRect(300, 5, 12, 12, color);
77             }
78             delay(5);
79             return;
80         }
81
82         /* Drawing logic */
83         // Start of a new stroke
84         if(last_x == 0 && last_y == 0)
85         {
86             // Record current position
```

```

87     last_x = x;
88     last_y = y;
89 }
90
91 // Draw a line between the previous position and current position
92 tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
93
94 // Update coordinates
95 last_x = x;
96 last_y = y;
97 }
98 else{
99     // Reset coordinates when stylus leaves the screen
100    last_x = 0;
101    last_y = 0;
102 }
103 }

```

Code Explanation

Include the necessary library.

```
7 #include <TFT_eSPI.h> // TFT display library
```

Create TFT object instance.

```
12 // Invoke custom TFT driver library
13 TFT_eSPI tft = TFT_eSPI(); // Invoke custom library
```

Define screen resolution.

```
15 #define SCREEN_WIDTH 240
16 #define SCREEN_HEIGHT 320
```

Apply the calibration data.

```
34 uint16_t calData[5] = { 412, 3502, 262, 3596, 3 };
35 tft.setTouch(calData);
```

Draw color palette.

```
28 // Draw color palette
29 for (int i = 0; i < color_num; i++) {
30     tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
31 }
```

Detect whether the touch screen is pressed and store the x and y coordinates of the touch point in variables.

```
58 bool pressed = tft.getTouch(&x, &y);
```

When the Clear button is pressed, fill the screen with the current color.

```
71 // Clicked Clear button
72 if(x >= ColorPalette * color_num)
73 {
74     tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
75 }
```

Calculate the selected color based on the coordinates of the touch point

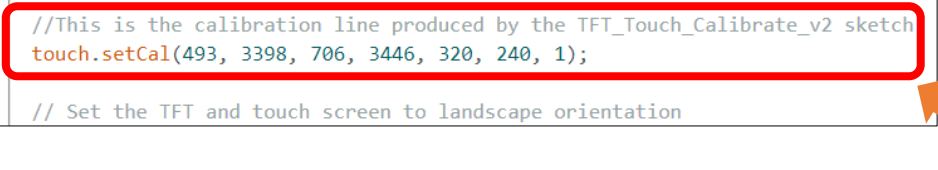
```
77 // Clicked color palette
78 else{
79     // Calculate current color based on x-coordinate
80     color = colors[x / ColorPalette];
81
82     // Update display of current color
83     tft.fillRect(300, 5, 12, 12, color);
84 }
```

Draw a line based on coordinates

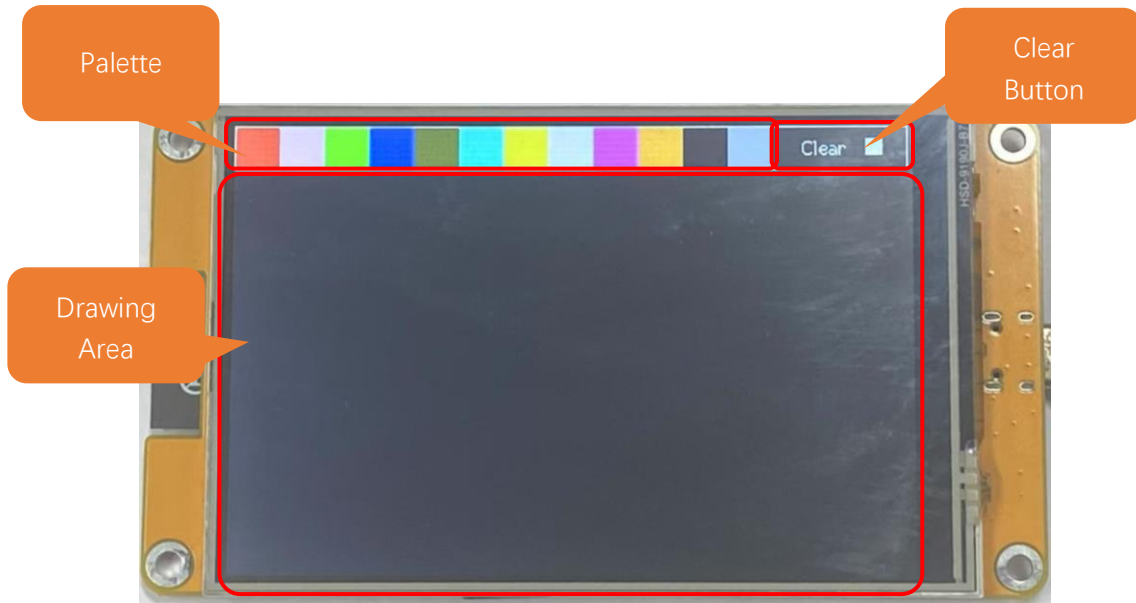
```
88 /* Drawing logic */
89 // Start of a new stroke
90 if(last_x == 0 && last_y == 0)
91 {
92     // Record current position
93     last_x = x;
94     last_y = y;
95 }
96
97 // Draw a line between the previous position and current position
98 tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
99
100 // Update coordinates
101 last_x = x;
102 last_y = y;
```

Add the calibration value obtained from the previous section to the code.

```
57 void setup()
58 {
59     Serial.begin(115200);
60
61     tft.init();
62
63     //This is the calibration line produced by the TFT_Touch_Calibrate_v2 sketch
64     touch.setCal(493, 3398, 706, 3446, 320, 240, 1);
65
66     // Set the TFT and touch screen to landscape orientation
```



Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



Sketch_12.3_TFT_Touch_Draw_3.5_4.0_Inch

The following is the program code:

```

1  #include <TFT_eSPI.h> // TFT display library
2
3  #define SCREEN_WIDTH 320
4  #define SCREEN_HEIGHT 480
5
6  TFT_eSPI tft = TFT_eSPI(SCREEN_WIDTH, SCREEN_HEIGHT);
7
8  // Define coordinate variables
9  uint16_t x, y;
10 uint16_t last_x, last_y;
11
12 const uint8_t ColorPalette = 32; // Height of the color palette area
13 const uint8_t LineSize = 7; // Line thickness (radius)
14 const uint8_t color_num = 12;
15 int color = TFT_WHITE;
16
17 unsigned int colors[color_num] = { TFT_RED, TFT_PINK, TFT_GREEN, TFT_BLUE, TFT_OLIVE ,
18 TFT_CYAN, TFT_YELLOW, TFT_WHITE, TFT_MAGENTA, TFT_BROWN, TFT_BLACK, TFT_SILVER};
19
20 void drawUI()
21 {
22     // Draw color palette
23     for (int i = 0; i < color_num; i++) {
24         tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
25     }
26     // Draw clear screen button
27     tft.setCursor(405 , 8, 2);
28     tft.setTextColor(TFT_WHITE);
29     tft.print("Clear");
30
31     // Display current color
32     tft.fillRect(450, 9, 12, 12, color);
33
34     // Add palette border/outline
35     tft.drawRect(0, 0, SCREEN_HEIGHT, ColorPalette, TFT_WHITE);
36 }
37
38 void setup() {
39     Serial.begin(115200);
40     tft.init();
41     uint16_t calData[5] = { 303, 3458, 350, 3304, 7 };
42     tft.setTouch(calData);

```

```
43     tft.setRotation(1);
44     tft.fillScreen(TFT_BLACK);
45
46     drawUI();
47 }
48
49 void loop() {
50     bool pressed = tft.getTouch(&x, &y);
51
52     if (pressed) {
53         Serial.printf("x: %d, y: %d\r\n", x, y);
54
55         /* Color palette area logic */
56         if(y < ColorPalette + 3)
57         {
58             // Enter color selection area, stroke interrupted
59             last_x = 0;
60             last_y = 0;
61
62             /* Determine stylus click area */
63             // Clicked Clear button
64             if(x >= ColorPalette * color_num)
65             {
66                 tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
67             }
68             // Clicked color palette
69             else{
70                 // Calculate current color based on x-coordinate
71                 color = colors[x / ColorPalette];
72
73                 // Update display of current color
74                 tft.fillRect(450, 9, 12, 12, color);
75             }
76             delay(5);
77             return;
78         }
79
80         /* Drawing logic */
81         // Start of a new stroke
82         if(last_x == 0 && last_y == 0)
83         {
84             // Record current position
85             last_x = x;
86             last_y = y;
```

```

87     }
88
89     // Draw a line between the previous position and current position
90     tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
91
92     // Update coordinates
93     last_x = x;
94     last_y = y;
95 }
96 else{
97     // Reset coordinates when stylus leaves the screen
98     last_x = 0;
99     last_y = 0;
100 }
101 }

```

Code Explanation

Include the necessary header file.

```
7 #include <TFT_eSPI.h> // TFT display library
```

Define screen resolution.

```
9 #define SCREEN_WIDTH 320
10 #define SCREEN_HEIGHT 480
```

Create TFT object instance.

```
12 TFT_eSPI tft = TFT_eSPI(); // Create TFT object instance
```

Apply the calibration data.

```
34 uint16_t calData[5] = { 303, 3458, 350, 3304, 7 };
35 tft.setTouch(calData);
```

Draw color palette.

```
28 // Draw color palette
29 for (int i = 0; i < color_num; i++) {
30     tft.fillRect(i * ColorPalette, 0, ColorPalette, ColorPalette, colors[i]);
31 }

```

Detect whether the touch screen is pressed and store the x and y coordinates of the touch point in variables.

```
58 bool pressed = tft.getTouch(&x, &y);
```

When the Clear button is pressed, fill the screen with the current color.

```
71 // Clicked Clear button
72 if(x >= ColorPalette * color_num)
73 {
74     tft.fillRect(0, ColorPalette, SCREEN_HEIGHT, SCREEN_WIDTH - ColorPalette, color);
75 }

```

Calculate the selected color based on the coordinates of the touch point

```
77 // Clicked color palette
78 else{

```



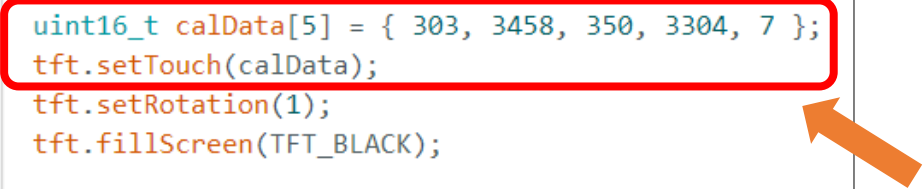
```
79 // Calculate current color based on x-coordinate
80 color = colors[x / ColorPalette];
81
82 // Update display of current color
83 tft.fillRect(450, 9, 12, 12, color);
84 }
```

Draw a line based on coordinates

```
88 /* Drawing logic */
89 // Start of a new stroke
90 if(last_x == 0 && last_y == 0)
91 {
92     // Record current position
93     last_x = x;
94     last_y = y;
95 }
96
97 // Draw a line between the previous position and current position
98 tft.drawWideLine(last_x, last_y, x, y, LineSize, color, color);
99
100 // Update coordinates
101 last_x = x;
102 last_y = y;
```

Add the calibration values obtained from the previous section to the code.

```
47 void setup() {
48     Serial.begin(115200);
49     tft.init();
50     uint16_t calData[5] = { 303, 3458, 350, 3304, 7 };
51     tft.setTouch(calData);
52     tft.setRotation(1);
53     tft.fillScreen(TFT_BLACK);
54
55     drawUI();
56 }
```



Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



Reference

```
void fillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color);
```

This function is used to draw a solid rectangle

Parameters:

x: x coordinate of the upper left corner of the solid rectangle

y: y coordinate of the upper left corner of the solid rectangle

w: width of the solid rectangle

h: height of the solid rectangle

color: fill color of the solid rectangle

```
void drawWideLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t width, uint16_t color1, uint16_t color2);
```

This function is used to draw lines

Parameters:

x0: x coordinate of the starting point of the line

y0: y coordinate of the starting point of the line

x1: x coordinate of the end point of the line

y1: y coordinate of the end point of the line

width: width of the line

color1: line color

color2: anti-aliasing transition color

Tips: Aliasing & Anti-aliasing

In computer graphics, **aliasing** refers to the **jagged or stair-step appearance** of lines and curves in digital images, particularly noticeable on diagonal lines and edges. This occurs due to the discrete nature of pixel grids - screens compose images from tiny square pixels that cannot perfectly represent continuous geometry.

Anti-aliasing mitigates these artifacts through technical means to **smooth edges**, achieving more natural-looking graphics. The core technique blends transitional colors at boundary pixels, simulating human visual perception of soft edges.

Chapter 13 LVGL

Project 13.1 LVGL

Component List

Freenove ESP32 Display x 1	USB cable x1
	

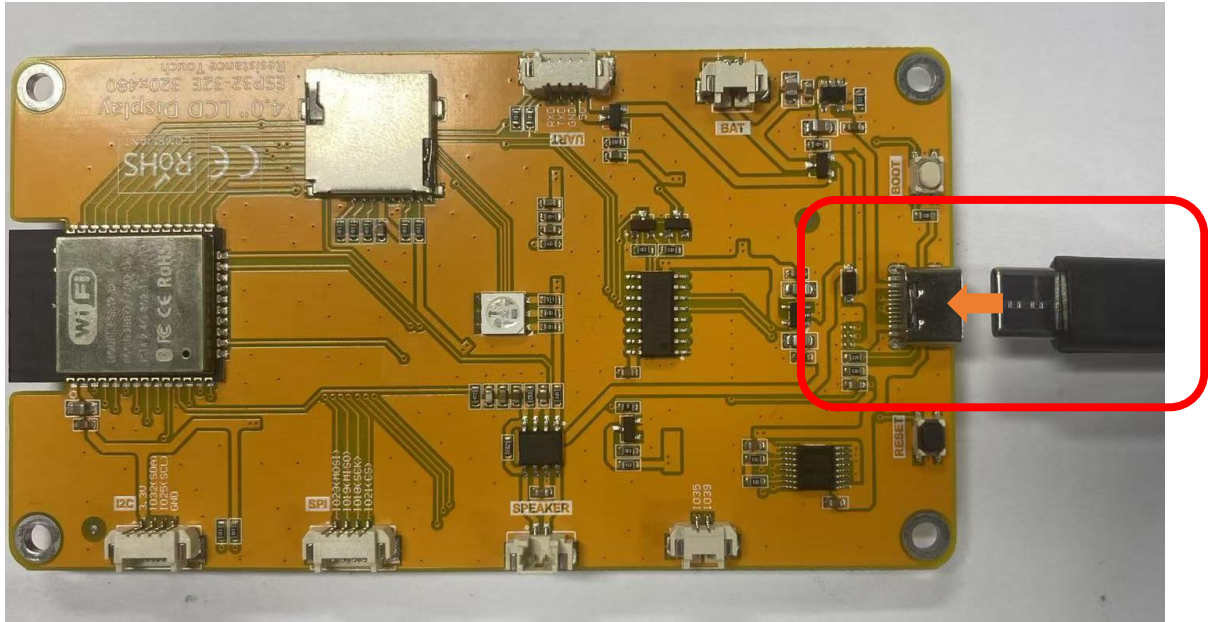
Component Knowledge

LVGL is a widely-used embedded GUI library that is implemented in pure C, making it highly portable and performant. It offers rich features and content, supporting both display and input devices such as touchscreens and keyboards.

	Features supported by LVGL
1	Powerful building blocks such as buttons, charts, lists, sliders, images, and more.
2	Advanced graphics with animation, anti-aliasing, opacity, and smooth scrolling.
3	Various input devices, such as touchpads, mice, keyboards, encoders, and more.
4	Multiple languages with UTF-8 encoding.
5	Multiple display types, including TFT and monochrome displays.
6	Fully customizable graphical elements.
7	LVGL can be used independently of any microcontroller or display hardware.
8	Highly extensible and can be configured to use very little memory (e.g. 64 kB of flash and 16 kB of RAM)
9	It can be used with or without an operating system, and supports external memory and GPUs as optional features.
10	Single-frame buffer operation, even with advanced graphics effects.
11	Written in C language to achieve maximum compatibility (compatible with C++ as well).
12	LVGL has a simulator that allows for embedded GUI design on a PC without any embedded hardware.
13	Resources to help developers quickly get started with the library, including tutorials, examples, and themes.
14	A wide range of resources.

Circuit

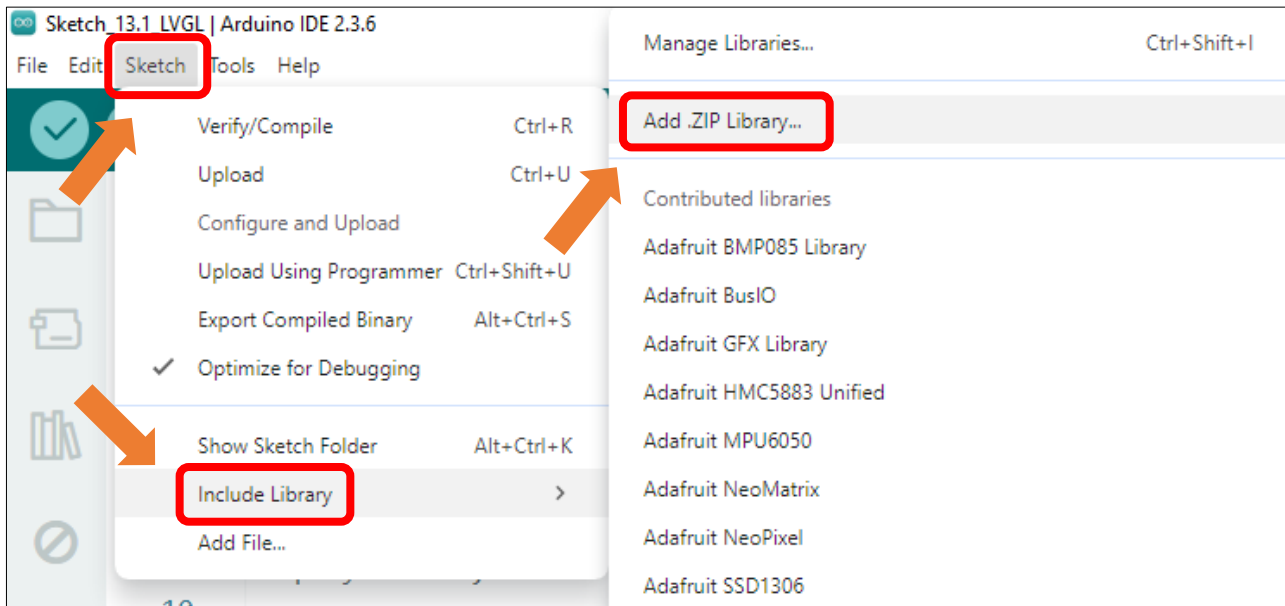
Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Install Libraries

Click **Sketch** -> **Include Library** -> **Add .ZIP Library...**



Install **lvgl_v8.4.0.zip**

Name	Date modified	Type	Size
ESP8266Audio_v2.0.0.zip	2025/5/30 14:06	WinRAR ZIP...	7,821 KB
lvgl_v8.4.0.zip	2025/5/30 13:52	WinRAR ZIP...	26,083 KB
TFT_eSPI_Setups_v1.0.0.zip	2025/6/11 15:45	WinRAR ZIP...	45 KB
TFT_eSPI_v2.5.43.zip	2025/6/11 15:45	WinRAR ZIP...	5,975 KB
TFT_Touch_v0.3.zip	2025/5/30 13:51	WinRAR ZIP...	14 KB
Tpg_Decoder_v1.1.0.zip	2025/5/30 14:08	WinRAR ZIP...	313 KB

Open “Sketch_13.1_LVGL” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_13.1_LVGL.ino”.

Sketch_13.1_LVGL

The following is the program code:

```

1  #include "display.h"
2
3  Display screen;
4
5  void setup()
6  {
7      /* prepare for possible serial debug */
8      Serial.begin( 115200 );
9
10     /**** Init screen ****/
11     screen.init();
12
13     String LVGL_Arduino = "Hello Arduino! ";
14     LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
15     lv_version_patch();
16
17     Serial.println( LVGL_Arduino );
18     Serial.println( "I am LVGL_Arduino" );
19
20     /* Create simple label */
21     lv_obj_t *label = lv_label_create( lv_scr_act() );
22     lv_label_set_text( label, LVGL_Arduino.c_str() );
23     lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );
24
25     Serial.println( "Setup done" );
26 }
27
28 void loop()
29 {
30     screen.routine(); /* let the GUI do its work */
31     delay( 5 );

```

```
32 }
```

Code Explanation

Include the header file.

```
7 #include "display.h"
```

Set the baud rate to 115200

```
19 Serial.begin( 115200 );
```

Initialize the screen.

```
11 screen.init();
```

Configure the screen interface.

```
15 String LVGL_Arduino = "Hello Arduino! ";
16 LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
17 lv_version_patch();
18
19 Serial.println( LVGL_Arduino );
20 Serial.println( "I am LVGL_Arduino" );
21
22 /* Create simple label */
23 lv_obj_t *label = lv_label_create( lv_scr_act() );
24 lv_label_set_text( label, LVGL_Arduino.c_str() );
25 lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );
```

Let the LVGL handle the tasks.

```
46 screen.routine(); /* let the GUI do its work */
```

Click **"Upload"** to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



Data will be printed on the serial monitor.

Message (Enter to send message to 'ES...	New Line	115200 baud
15:52:44.640 ->		19200 baud
15:52:44.640 -> rst:0x1 (POWERON_RESET),boot:0x1		31250 baud
15:52:44.640 -> configsip: 0, SPIWP:0xee		38400 baud
15:52:44.640 -> clk_drv:0x00,q_drv:0x00,d_drv:0x		57600 baud
15:52:44.640 -> mode:DIO, clock div:1		74880 baud
15:52:44.640 -> load:0x3fff0030,len:4832		115200 baud
15:52:44.640 -> load:0x40078000,len:16460		230400 baud
15:52:44.640 -> load:0x40080400,len:4		250000 baud
15:52:44.640 -> load:0x40080404,len:3504		460800 baud
15:52:44.640 -> entry:0x400805cc		
15:52:45.692 - Hello Arduino! V8.4.0		
15:52:45.692 - I am LVGL_Arduino		
15:52:45.692 - Setup done		

The text **"Hello Arduino! V8.4.0"** will be displayed on the screen.



Chapter 14 LVGL Picture

Project 14.1 LVGL Picture

Component List

Freenove ESP32 Display x 1



USB cable x1

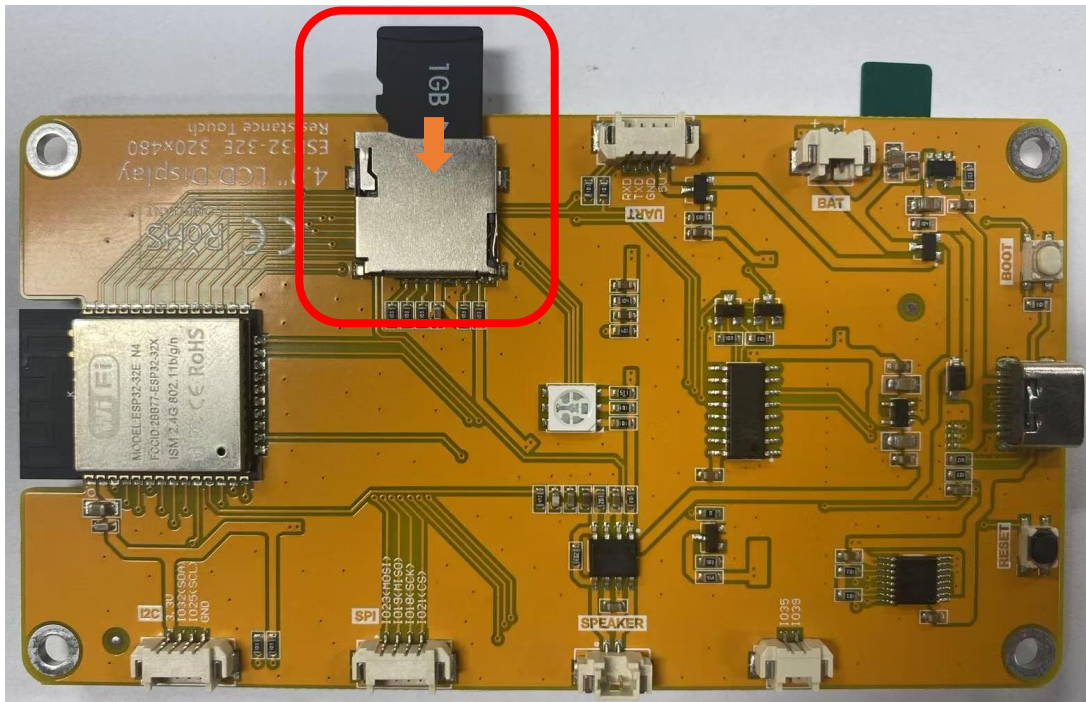


Stylus x 1

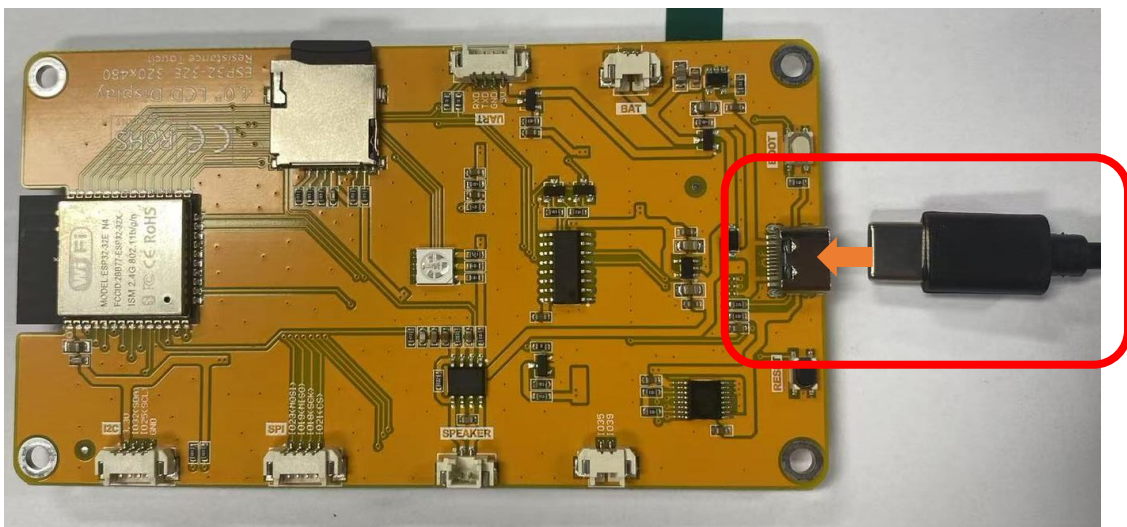


Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32 Display.



Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open “**Sketch_14.1_Lvgl_Picture**” folder under “**Freenove_ESP32_Display\Sketches**” and double-click “**Sketch_14.1_Lvgl_Picture.ino**”.

Sketch_14.1_Lvgl_Picture

The following is the program code:

```
1 #include "display.h"
2 #include "driver_sdspi.h"
```

```

3  #include "picture_ui.h"
4
5  Display screen;
6
7  #define SD_SCK    18
8  #define SD_MISO   19
9  #define SD_MOSI   23
10 #define SD_CS     5
11
12 void setup() {
13     /* prepare for possible serial debug */
14     Serial.begin( 115200 );
15
16     /*** Init drivers ***/
17     sdspi_init(SD_SCK, SD_MISO, SD_MOSI, SD_CS);    //Initialize the SD module
18     screen.init();
19
20     String LVGL_Arduino = "Hello Arduino! ";
21     LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
22     lv_version_patch();
23     Serial.println( LVGL_Arduino );
24     Serial.println( "I am LVGL_Arduino" );
25
26     setup_scr_picture(&guider_picture_ui);
27     lv_scr_load(guider_picture_ui.picture);
28
29     Serial.println( "Setup done" );
30 }
31
32 void loop() {
33     screen.routine(); /* let the GUI do its work */
34     delay( 5 );
35 }

```

Code Explanation

Include the header files.

```

7  #include "display.h"
8  #include "driver_sdspi.h"
9  #include "picture_ui.h"

```

Define the pins.

```

13 #define SD_SCK    18
14 #define SD_MISO   19
15 #define SD_MOSI   23
16 #define SD_CS     5

```

Set the baud rate to 115200

```
20 Serial.begin( 115200 );
```

Initialize configuration.

```
23 sdspi_init(SD_SCK, SD_MISO, SD_MOSI, SD_CS); //Initialize the SD module
```

```
24 screen.init();
```

Create and load the interface.

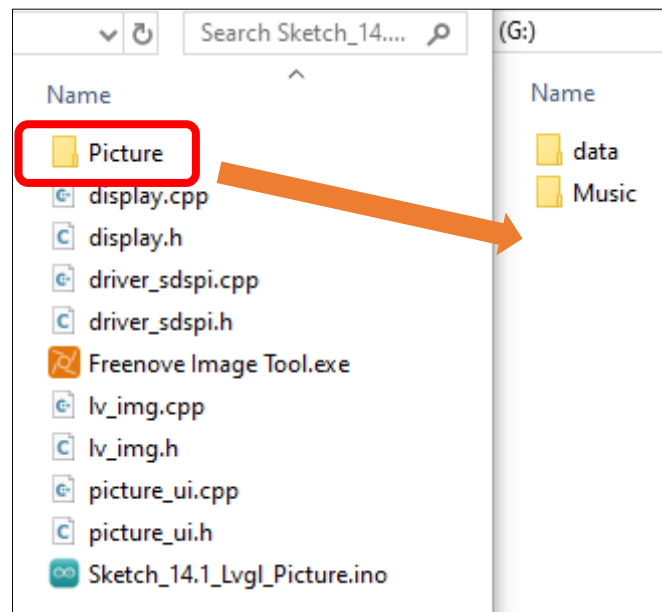
```
32 setup_scr_picture(&guider_picture_ui);
```

```
33 lv_scr_load(guider_picture_ui.picture);
```

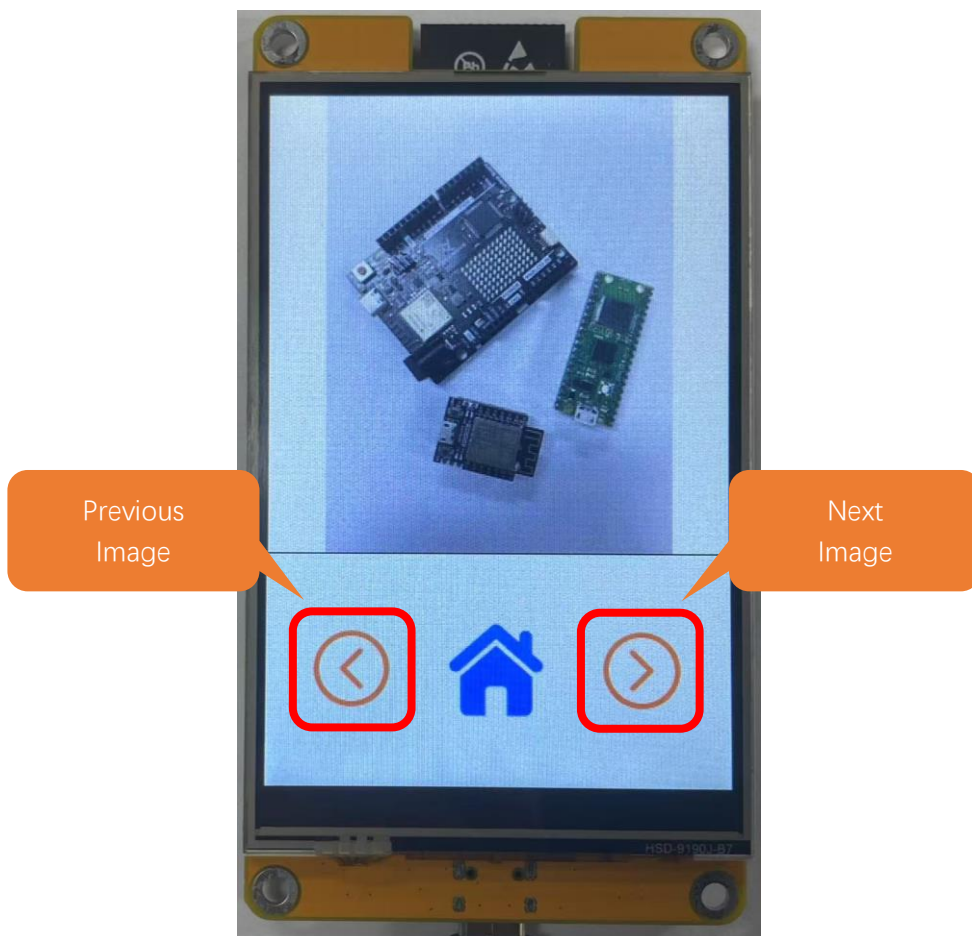
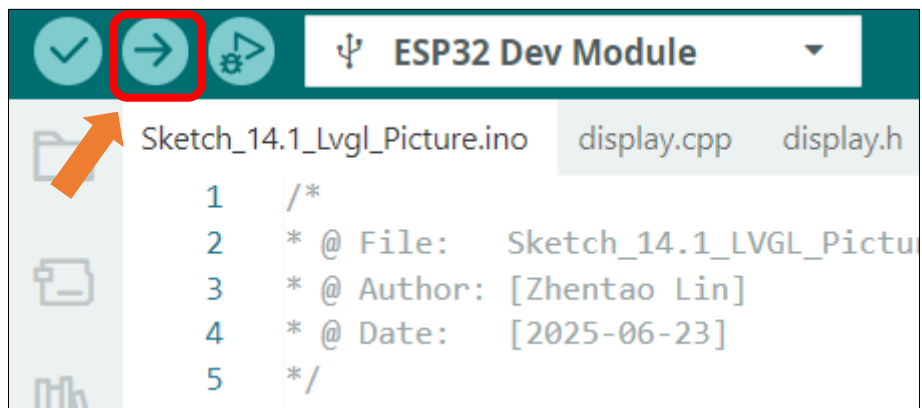
LVGL Task Processor

```
39 screen.routine(); /* let the GUI do its work */
```

Copy the Picture folder under the **Freenove_ESP32_Display\Sketches\Sketch_14.1_Lvgl_Picture** directory to the SD card root directory.



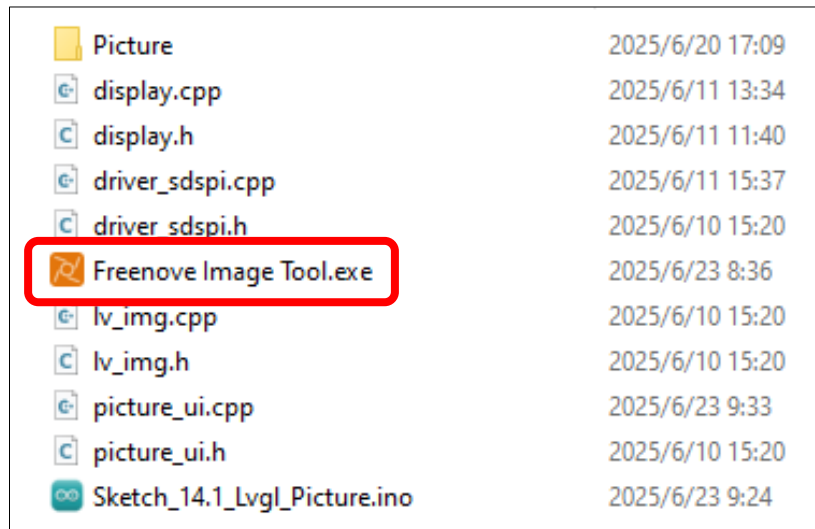
Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200



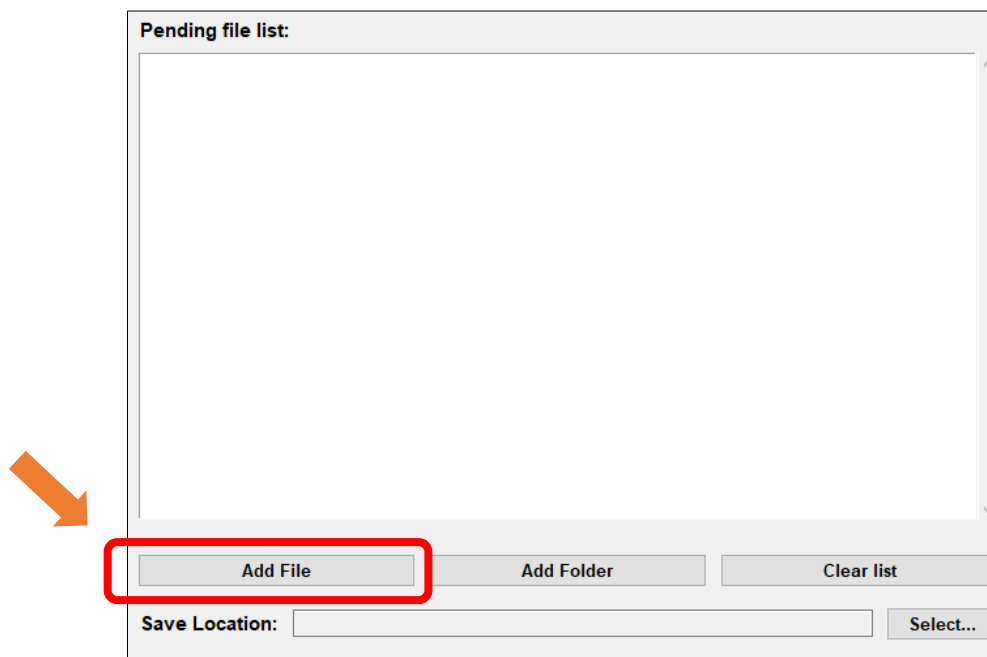
Custom image display

You can customize the image displayed on the display according to your personal preferences.

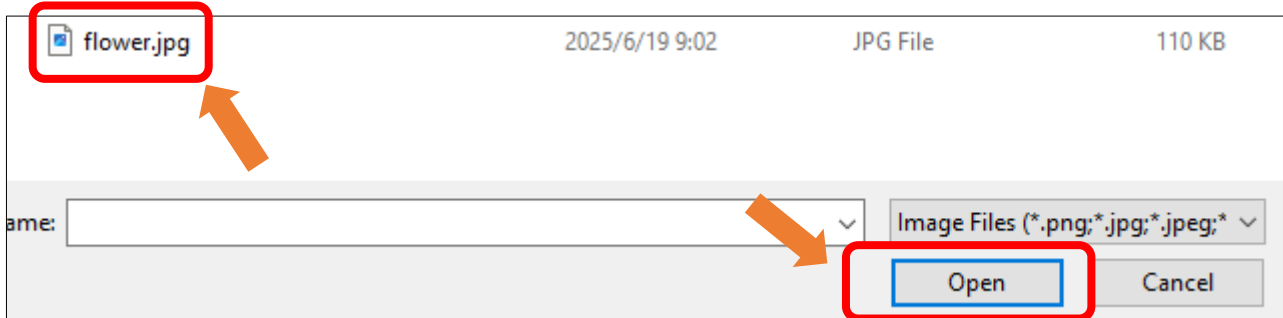
First, open **Freenove_ESP32_Display\Sketches\Sketch_14.1_Lvgl_Picture\Freenove Image Tool.exe**



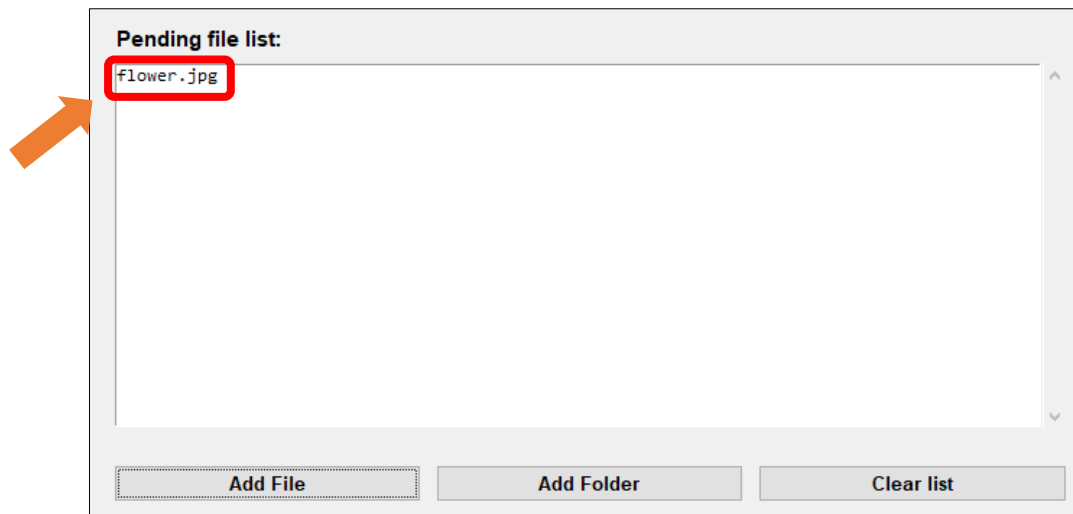
Click “Add File”



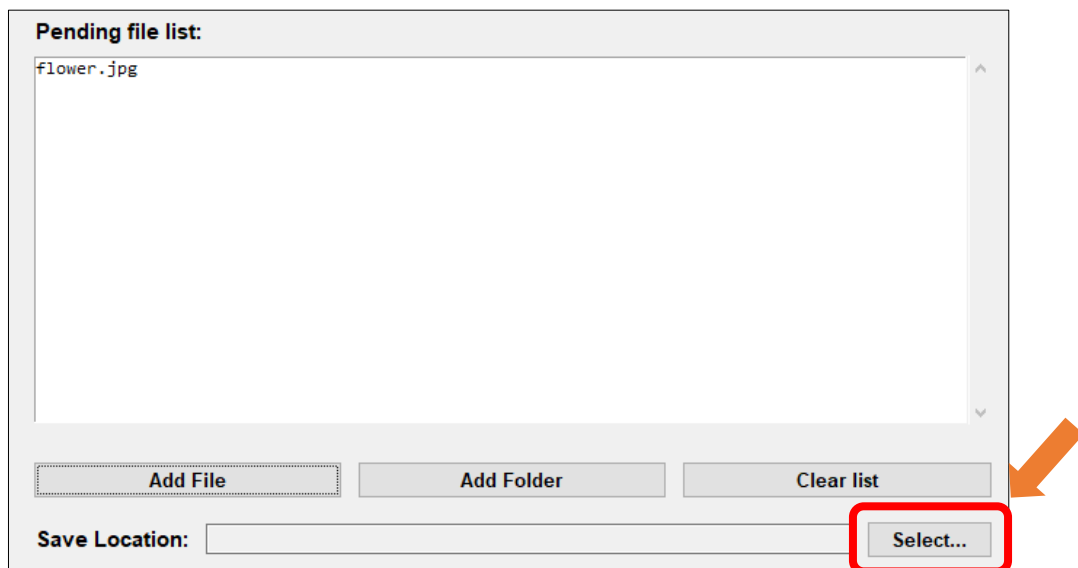
Select any image you like.



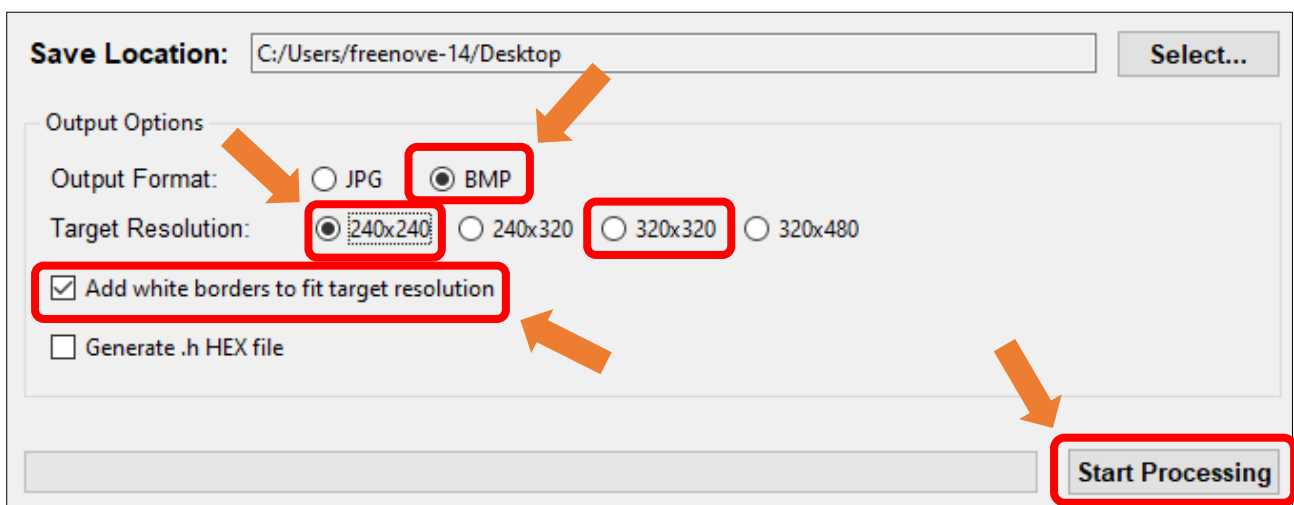
The image files from your folder will now appear in the **Pending File List**.



Click "Select..." to choose the image saving location.



Select **BMP** for the format, set the resolution to **240x240** or **320x320** based on your screen, and then click **Start Processing**.

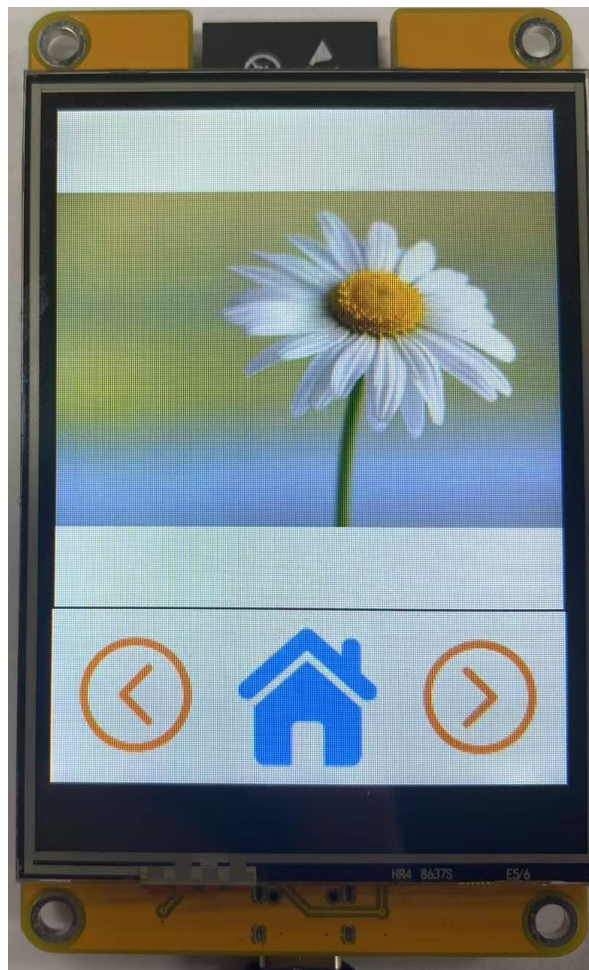
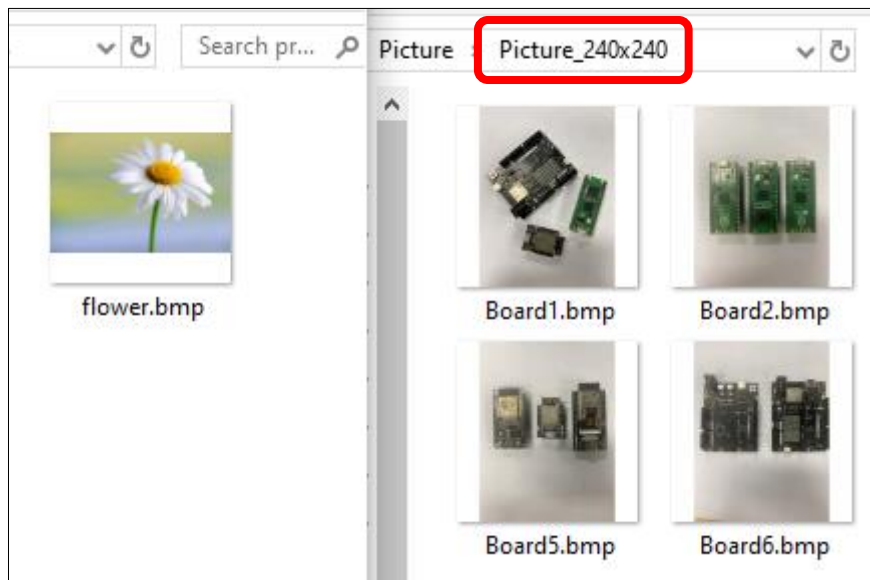


Wait for the progress bar to complete.

Copy or replace the generated image to the folder corresponding to the Picture folder in the root directory of the SD card.

If the screen resolution is **240x320**, put the image in **Picture_240x240**

If the screen resolution is **320x480**, put the image in **Picture_320x320**



Chapter 15 LVGL Timer

Project 15.1 LVGL Timer

Component List

Freenove ESP32 Display x 1



USB cable x1

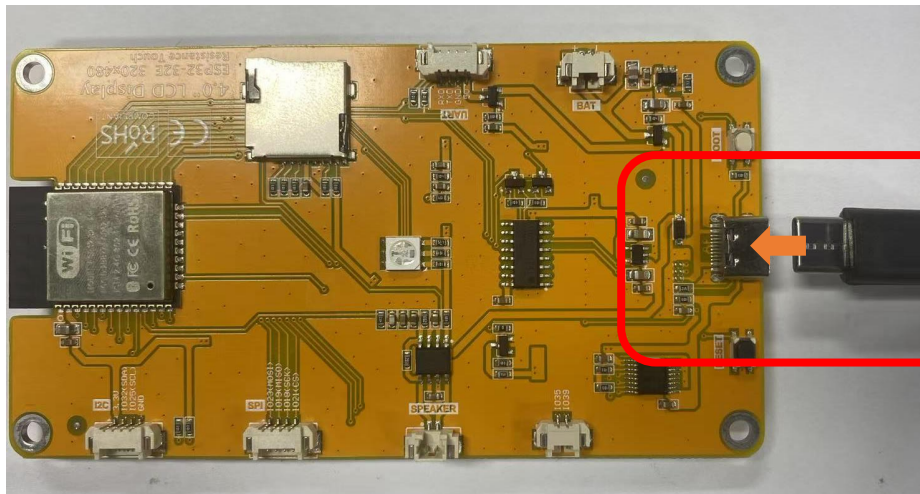


Stylus x 1



Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open “Sketch_15.1_Lvgl_Timer” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_15.1_Lvgl_Timer.ino”.

Sketch_15.1_Lvgl_Timer

The following is the program code:

```

1  #include "display.h"
2  #include "chronograph_ui.h"
3
4  Display screen;
5  void setup() {
6      Serial.begin(115200);
7
8      /**/ Init screen ***/
9      screen.init();
10
11     /**/ Print lvgl version ***/
12     String LVGL_Arduino = "Hello Arduino! ";
13     LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
14     lv_version_patch();
15     Serial.println(LVGL_Arduino);
16     Serial.println("I am LVGL_Arduino");
17     Serial.println("Setup done");
18
19     /**/ The custom code ***/
20     setup_scr_chronograph(&guider_chronograph_ui);
21     lv_scr_load(guider_chronograph_ui.chronograph);
22 }
23
24 void loop() {
25     screen.routine();
26     delay(5);
27 }
```

Code Explanation

Include the header files.

```

7  #include "display.h"
8  #include "chronograph_ui.h"
```

Set the baud rate to 115200.

```
20  Serial.begin( 115200 );
```

Initialize configuration.

```
23  screen.init();
```

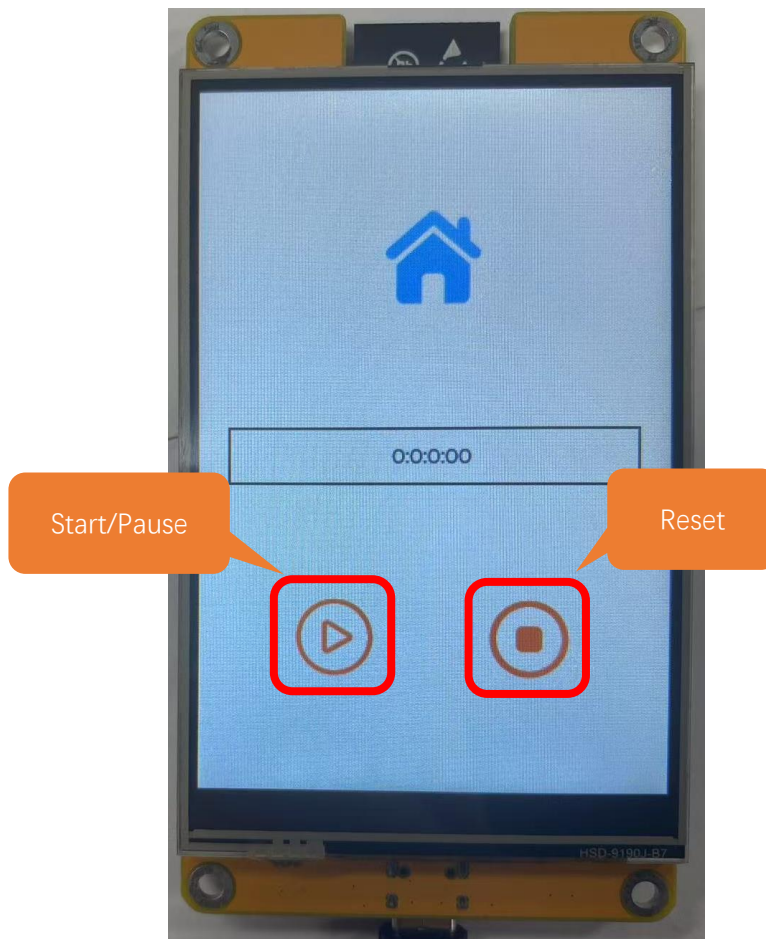
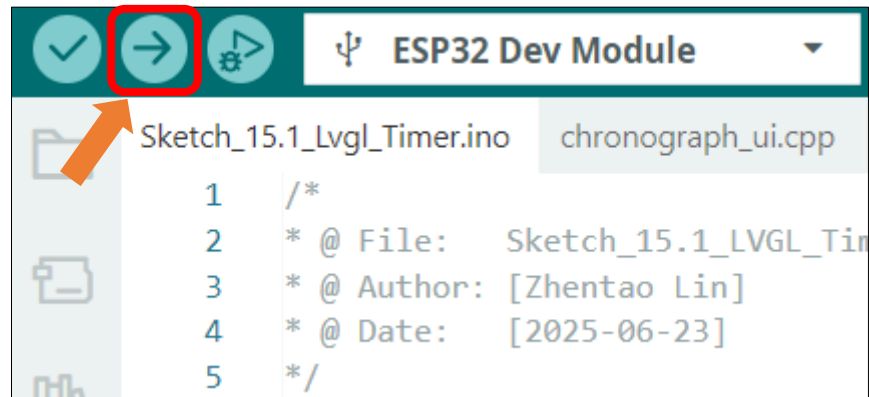
Create and load the interface.

```
32  setup_scr_chronograph(&guider_chronograph_ui);  
33  lv_scr_load(guider_chronograph_ui.chronograph);
```

LVGL task processor.

```
39  screen.routine(); /* let the GUI do its work */
```

Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.



Chapter 16 Lvgl RGB

Project 16.1 LVGL RGB

Component List

Freenove ESP32 Display x 1



USB cable x1

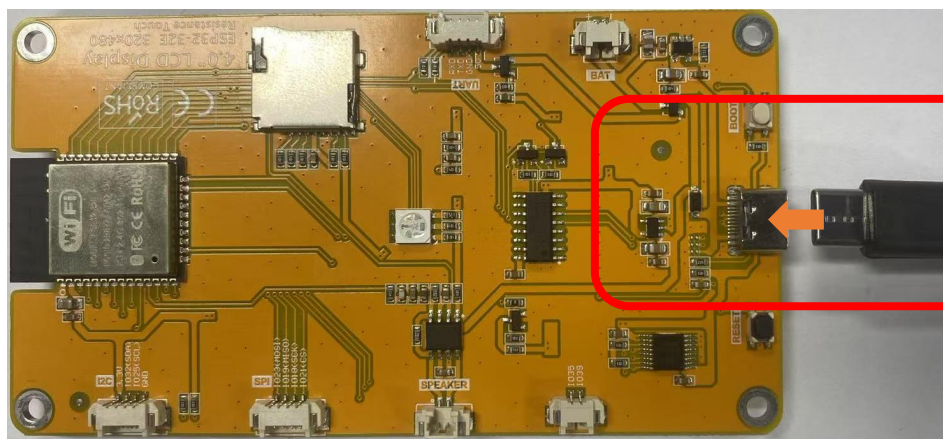


Stylus x 1



Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open “Sketch_16.1_Lvgl_RGB” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_16.1_Lvgl_RGB.ino”.

Sketch_16.1_Lvgl_RGB

The following is the program code:

```
1  #include "display.h"
2  #include "rgb_ui.h"
3
4  #define RED_PIN 22
5  #define GREEN_PIN 16
6  #define BLUE_PIN 17
7
8  Display screen;
9  void setup() {
10     Serial.begin(115200);
11
12     /**/ Init screen ***/
13     screen.init();
14
15     /**/ Print lvgl version ***/
16     String LVGL_Arduino = "Hello Arduino! ";
17     LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
18     lv_version_patch();
19     Serial.println(LVGL_Arduino);
20     Serial.println("I am LVGL_Arduino");
21     Serial.println("Setup done");
22
23     /**/ The custom code ***/
24     rgb_init(RED_PIN, GREEN_PIN, BLUE_PIN);
25     rgb_set_color(0, 0, 0);
26     setup_scr_rgb(&guider_rgb_ui);
27     lv_scr_load(guider_rgb_ui.rgb);
28 }
29
30 void loop() {
31     screen.routine();
32     delay(5);
33 }
```

Code Explanation

Include the header files.

```
7  #include "display.h"
8  #include "rgb_ui.h"
```

Define the pins.

```
13 #define RED_PIN 22
14 #define GREEN_PIN 16
15 #define BLUE_PIN 17
```

Set the baud rate to 115200

```
20 Serial.begin( 115200 );
```

Initialize the RGB LED.

```
23 rgb_init(RED_PIN, GREEN_PIN, BLUE_PIN);
24 rgb_set_color(0, 0, 0);
```

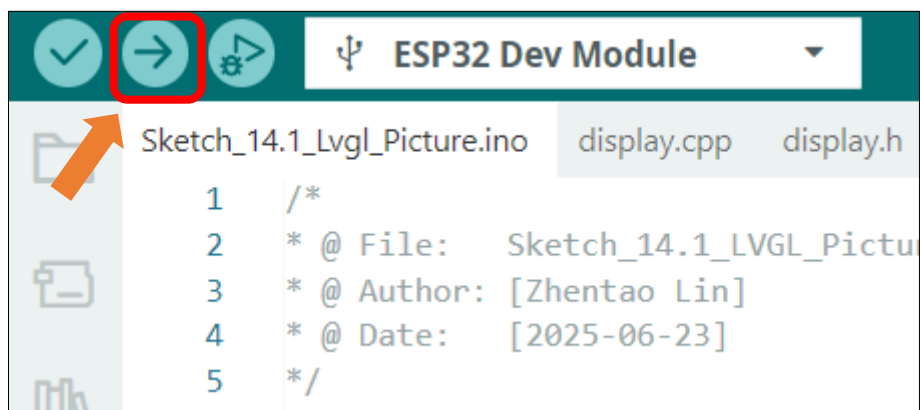
Create and load the interface.

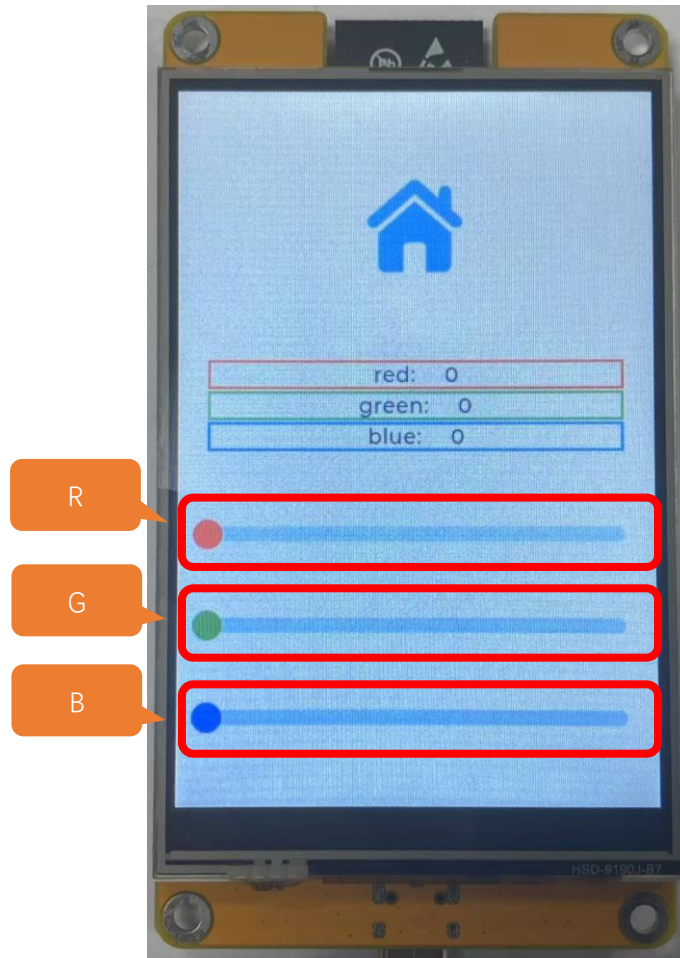
```
32 setup_scr_rgb(&guider_rgb_ui);
33 lv_scr_load(guider_rgb_ui.rgb);
```

LVGL task processor.

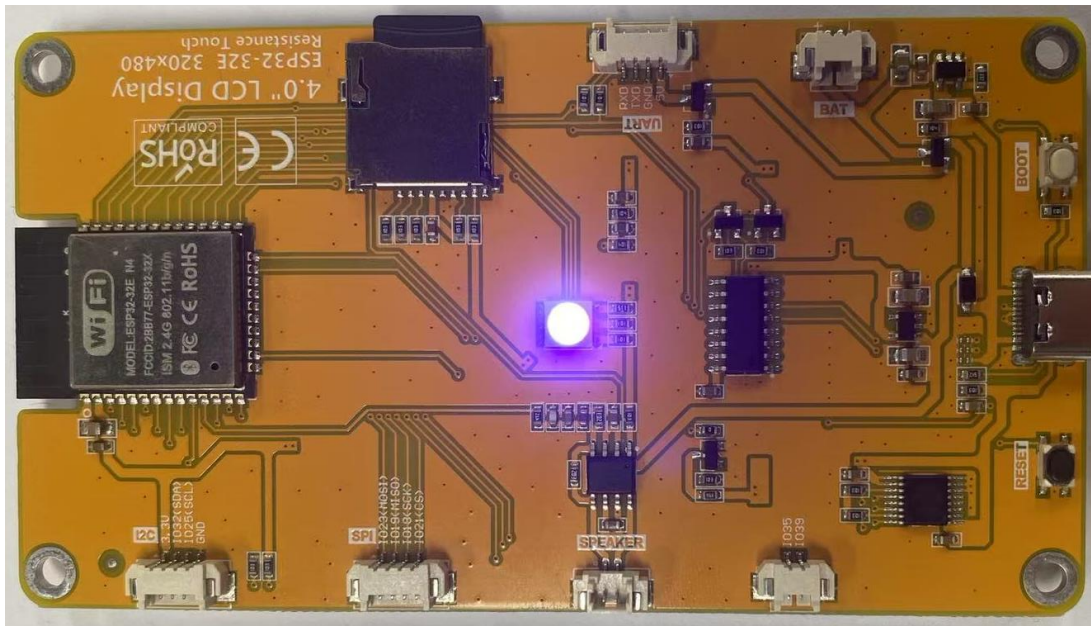
```
39 screen.routine(); /* let the GUI do its work */
```

Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.





Drag to adjust the value of the red (R), green (G), and blue (B) color, and you will see the color of the LED change.



Chapter 17 LVGL Music

Project 17.1 LVGL Music

Component List

Freenove ESP32 Display x 1



USB cable x1



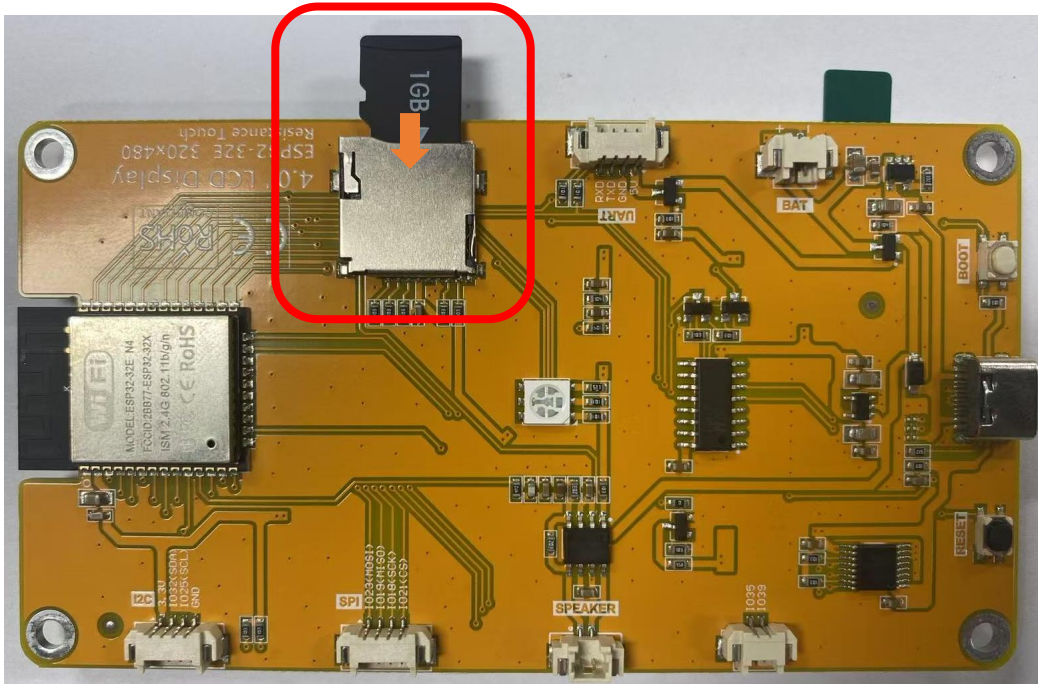
Stylus x 1



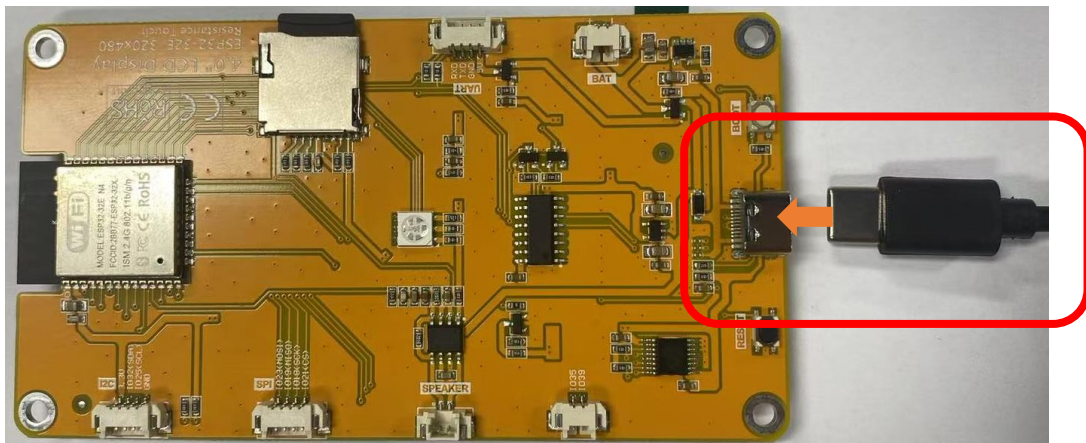
Note: This kit does not include speaker, SD card, or SD card reader. Please buy them yourself!

Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32 Display.



Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open "Sketch_17.1_Lvgl_Music" folder under "Freenove_ESP32_Display\Sketches" and double-click "Sketch_17.1_Lvgl_Music.ino".

Sketch_17.1_Lvgl_Music

The following is the program code:

```
1 #include "display.h"
2 #include "driver_sdspi.h"
3 #include "music_ui.h"
```

```

4
5   Display screen;
6
7   #define SD_SCK    18
8   #define SD_MISO   19
9   #define SD_MOSI   23
10  #define SD_CS      5
11  #define AUDIO_EN   4
12
13  void setup() {
14      /* prepare for possible serial debug */
15      Serial.begin( 115200 );
16
17      pinMode(AUDIO_EN, OUTPUT);
18      digitalWrite(AUDIO_EN, LOW);
19
20      /*** Init drivers ***/
21      sdspi_init(SD_SCK, SD_MISO, SD_MOSI, SD_CS);      //Initialize the SD module
22      screen.init();
23
24      String LVGL_Arduino = "Hello Arduino! ";
25      LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
26      lv_version_patch();
27      Serial.println( LVGL_Arduino );
28      Serial.println( "I am LVGL_Arduino" );
29
30      setup_scr_music(&guider_music_ui);
31      lv_scr_load(guider_music_ui.music);
32
33      Serial.println( "Setup done" );
34  }
35
36  void loop() {
37      screen.routine(); /* let the GUI do its work */
38      delay( 5 );
39  }

```

Code Explanation

Include the header files.

```

7   #include "display.h"
8   #include "driver_sdspi.h"
9   #include "music_ui.h"

```

Define the pins.

```

13  #define SD_SCK    18
14  #define SD_MISO   19

```

```

15 #define SD_MOSI 23
16 #define SD_CS   5
17 #define AUDIO_EN 4

```

Set the baud rate to 115200

```

21 Serial.begin( 115200 );

```

Initialize configuration.

```

23 screen.init();

```

Create and load the interface.

```

32 setup_scr_music(&guider_music_ui);
33 lv_scr_load(guider_music_ui.music);

```

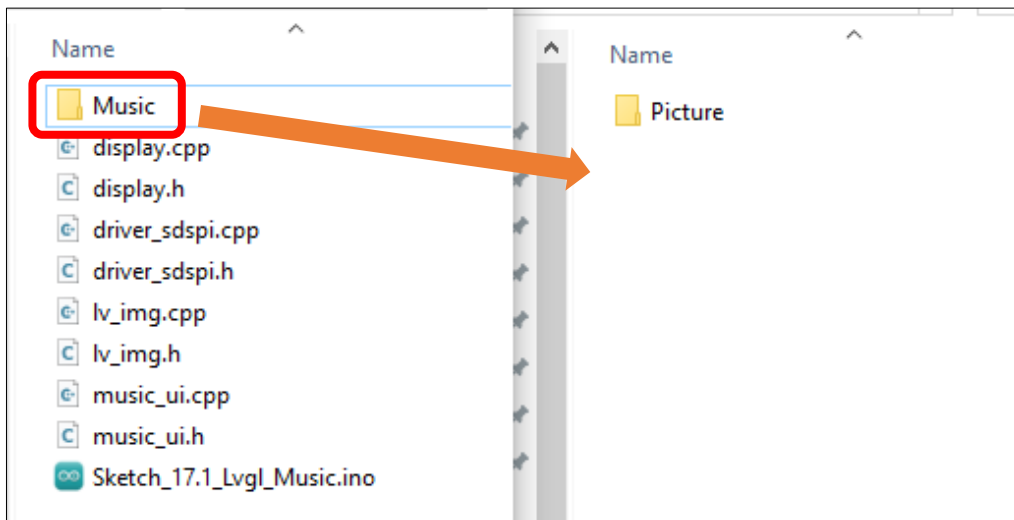
LVGL task processor.

```

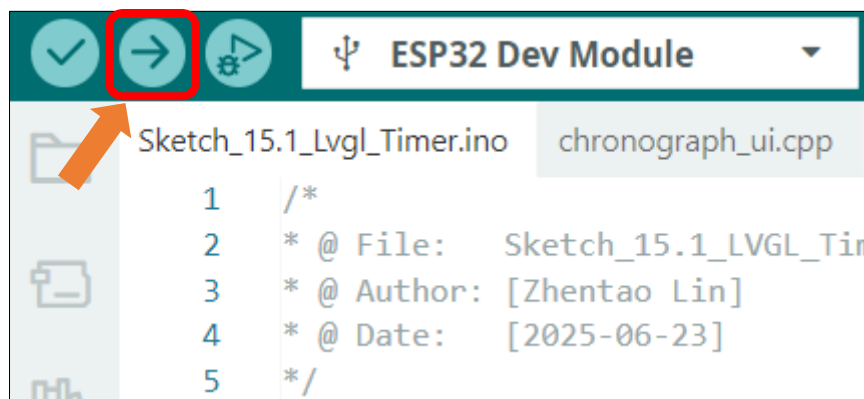
39 screen.routine(); /* let the GUI do its work */

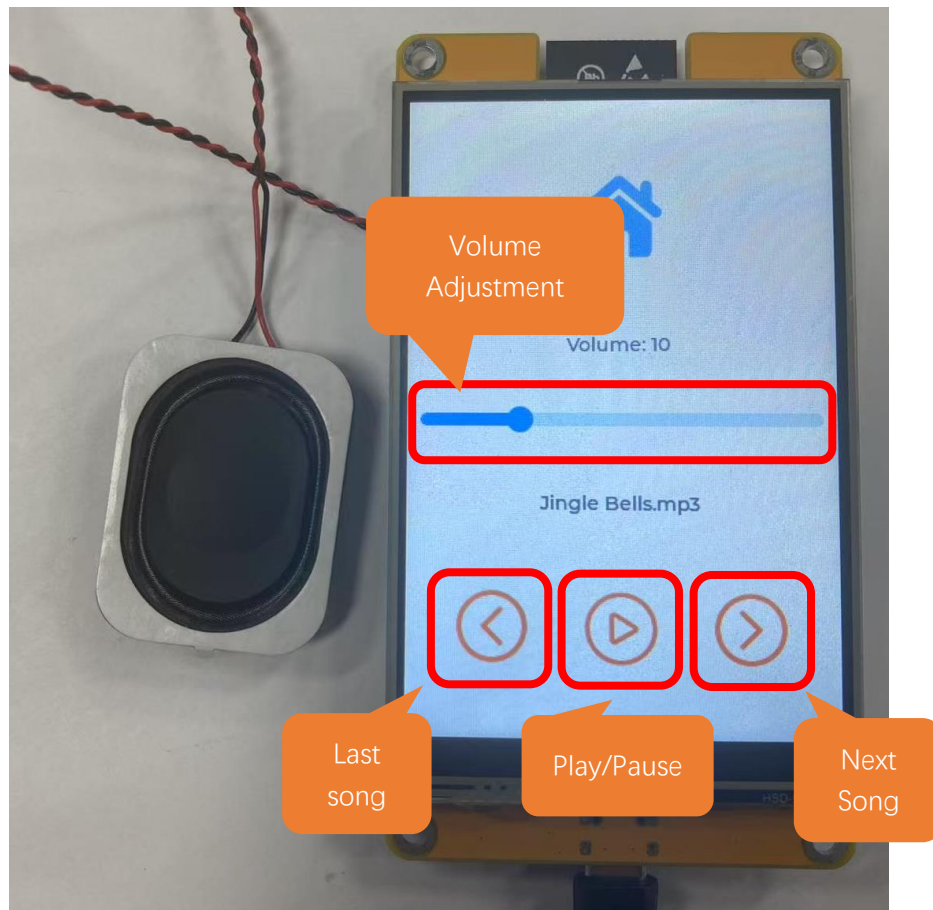
```

Insert the SD card to the card reader and plug them to the computer. Copy the **Music** folder under the **Freenove_ESP32_Display\Sketches\Sketch_17.1_Lvgl_Music** directory to the root directory of the SD card.



Click “**Upload**” to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.





Note: If the screen flickers during playback, it may be due to insufficient power supply. You can try powering with a battery.

Chapter 18 LVGL Multifunctionality

Project 18.1 LVGL Multifunctionality

Component List

Freenove ESP32 Display x 1



USB cable x1

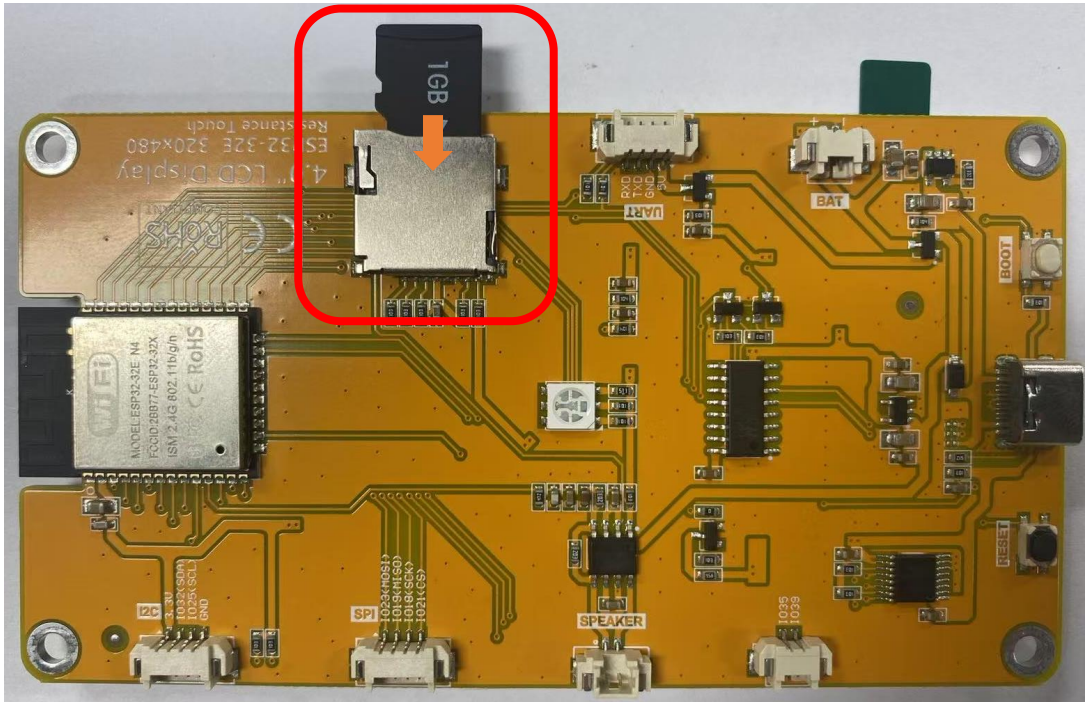


Stylus x 1

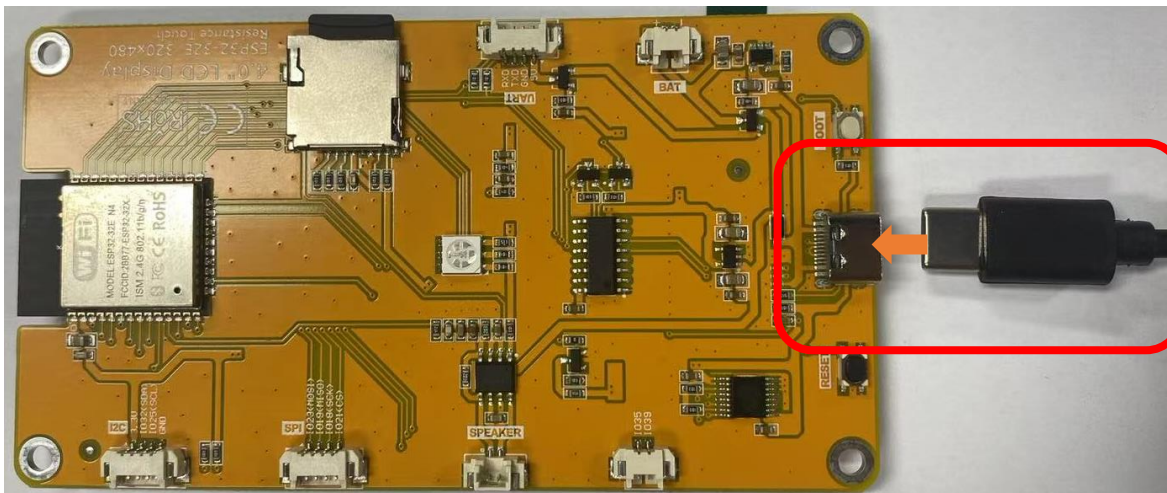


Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32 Display.



Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open “**Sketch_18.1_Lvgl_Multifunctionality**” folder under “**Freenove_ESP32_Display\Sketches**” and double-click “**Sketch_18.1_Lvgl_Multifunctionality.ino**”.

[Sketch_18.1_Lvgl_Multifunctionality](#)

The following is the program code:

1	<code>#include "display.h"</code>
2	<code>#include "driver_sdspi.h"</code>

```
3  #include "main_ui.h"
4
5  #define SD_SCK 18
6  #define SD_MISO 19
7  #define SD_MOSI 23
8  #define SD_CS 5
9  #define AUDIO_EN 4
10 #define RED_PIN 22
11 #define GREEN_PIN 16
12 #define BLUE_PIN 17
13
14 Display screen; // Create an instance of the Display class
15
16 void setup() {
17     /* Prepare for possible serial debug */
18     Serial.begin(115200);
19     while (!Serial) {
20         delay(10);
21     }
22     /*** Init drivers ***/
23     pinMode(AUDIO_EN, OUTPUT);
24     digitalWrite(AUDIO_EN, LOW);
25     rgb_init(RED_PIN, GREEN_PIN, BLUE_PIN);
26     rgb_set_color(0, 0, 0);
27     sdsdspi_init(SD_SCK, SD_MISO, SD_MOSI, SD_CS); //Initialize the SD module
28     screen.init();
29
30     String LVGL_Arduino = "Hello Arduino! ";
31     LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
32     lv_version_patch();
33     Serial.println( LVGL_Arduino );
34     Serial.println( "I am LVGL_Arduino" );
35
36     setup_scr_main(&guider_main_ui);
37     lv_scr_load(guider_main_ui.main);
38
39     // Print setup completion message to the serial monitor
40     Serial.println("Setup done");
41 }
42
43 void loop() {
44     screen.routine(); /* Let the GUI do its work */ // Handle routine display tasks
45     delay(5); // Add a small delay to prevent the loop from running too fast
46 }
```

Code Explanation

Include the header files.

```
7  #include "display.h"
8  #include "driver_sdspi.h"
9  #include "main_ui.h"
```

Define the pins.

```
11 #define SD_SCK 18
12 #define SD_MISO 19
13 #define SD_MOSI 23
14 #define SD_CS 5
15 #define AUDIO_EN 4
16 #define RED_PIN 22
17 #define GREEN_PIN 16
18 #define BLUE_PIN 17
```

Set the baud rate to 115200

```
24 Serial.begin( 115200 );
```

Initialize RGB LED configuration

```
31 rgb_init(RED_PIN, GREEN_PIN, BLUE_PIN);
32 rgb_set_color(0, 0, 0);
```

Initialize SD card.

```
33 sdspi_init(SD_SCK, SD_MISO, SD_MOSI, SD_CS); //Initialize the SD module
```

Initialize the screen.

```
34 screen.init();
```

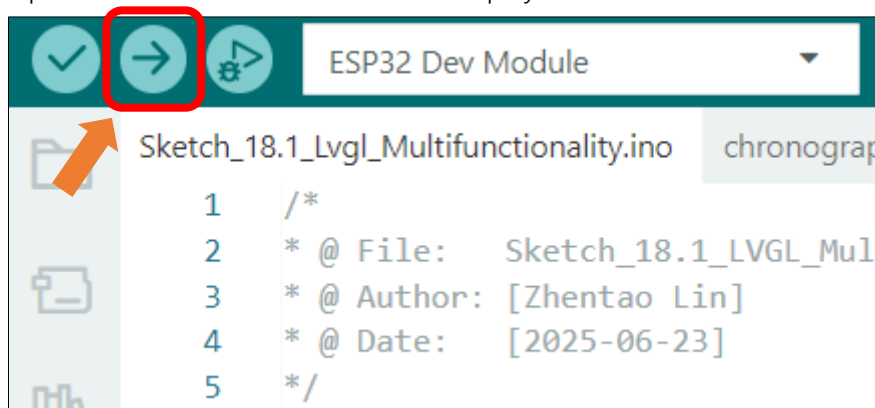
Create and load the interface.

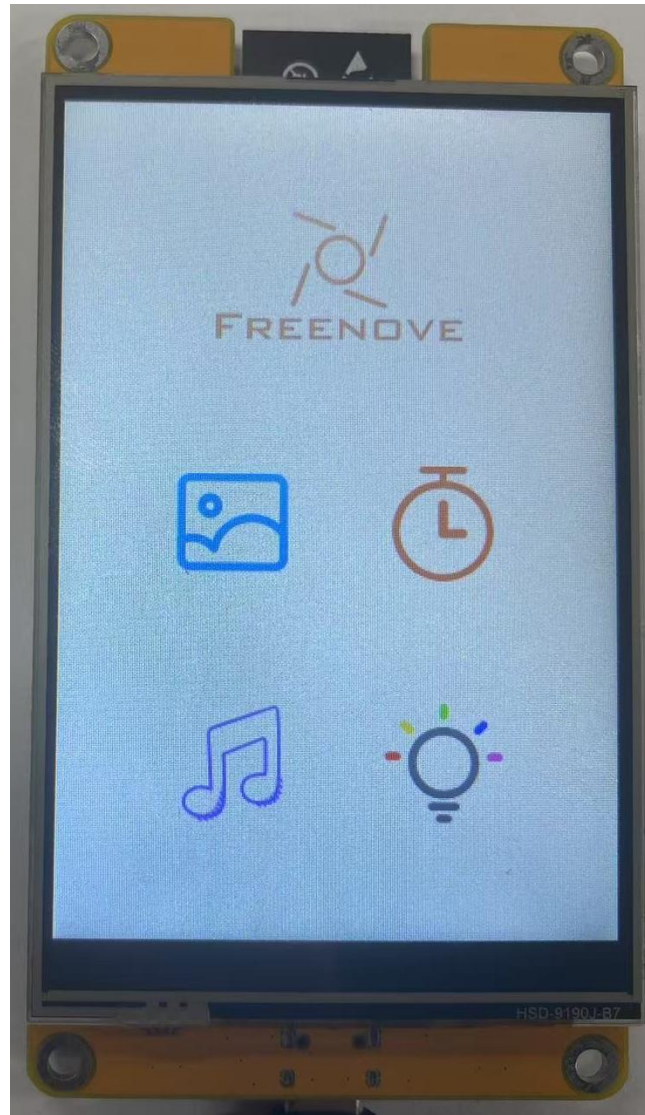
```
43 setup_scr_main(&guider_main_ui);
44 lv_scr_load(guider_main_ui.main);
```

LVGL task processor.

```
39 screen.routine(); /* let the GUI do its work */
```

Click **Upload** to upload the code to Freenove ESP32 Display. Set the baud rate to 115200.





Chapter 19 LVGL Arduino

Project 19.1 LVGL Arduino

Component List

Freenove ESP32 Display x 1



USB cable x1

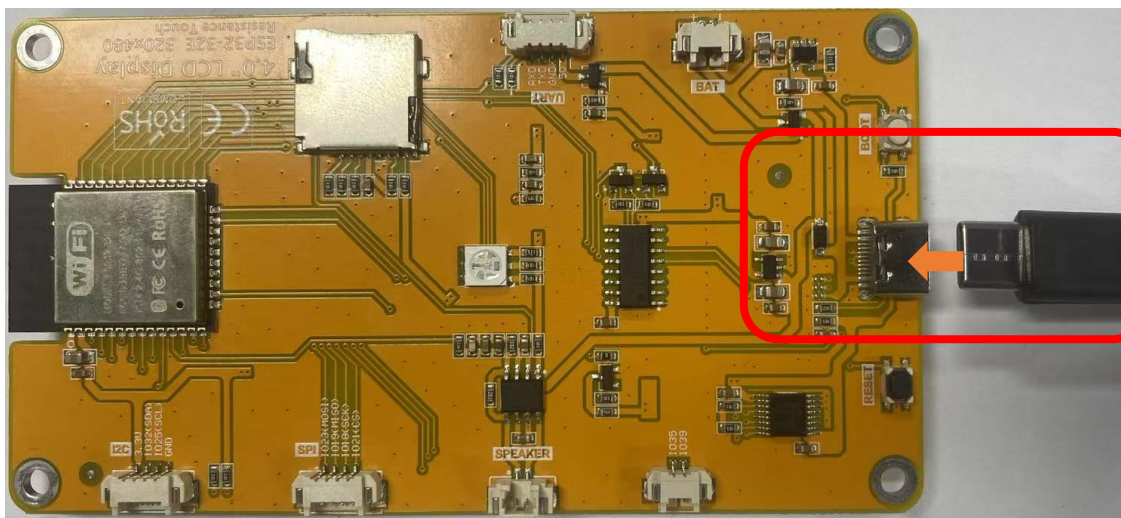


Stylus x 1



Circuit

Connect Freenove ESP32 Display to the computer using the USB cable.



Sketch

Open “Sketch_19.1_Lvgl_Arduino” folder under “Freenove_ESP32_Display\Sketches” and double-click “Sketch_19.1_Lvgl_Arduino.ino”.

Sketch_19.1_Lvgl_Arduino

The following is the program code:

```

1  /*Using LVGL with Arduino requires some extra steps:
2      *Be sure to read the docs here: https://docs.lvgl.io/master/get-
3  started/platforms/arduino.html */
4
5  #include <lvgl.h>
6  #include <TFT_eSPI.h>
7
8  /*To use the built-in examples and demos of LVGL uncomment the includes below respectively.
9      *You also need to copy `lvgl/examples` to `lvgl/src/examples`. Similarly for the demos
10 `lvgl/demos` to `lvgl/src/demos`.
11      Note that the `lv_examples` library is for LVGL v7 and you shouldn't install it for this
12 version (since LVGL v8)
13      as the examples and demos are now part of the main LVGL library. */
14
15 /*Change to your screen resolution*/
16 #define TFT_DIRECTION 0 //Select TFT Direction (0 - 3)
17
18 #if defined(FNK0103B_2P8_240x320_ST7789) || defined(FNK0103F_2P8_240x320_ILI9341)
19     #ifndef _TFT_Touch_H
20         #include "TFT_Touch.h"
21         #if !defined(_TFT_Touch_H)
22             #error "Please install TFT_Touch library before using this library!"
23         #endif
24     #endif
25 #define DOUT 39 /* Data out pin (T_D0) of touch screen */
26 #define DIN 32 /* Data in pin (T_DIN) of touch screen */
27 #define DCS 33 /* Chip select pin (T_CS) of touch screen */
28 #define DCLK 25 /* Clock pin (T_CLK) of touch screen */
29 #define TFT_SCREEN_WIDTH 240
30 #define TFT_SCREEN_HEIGHT 320
31 TFT_Touch touch = TFT_Touch(DCS, DCLK, DIN, DOUT);
32 #elif defined(FNK0103L_3P2_240x320_ST7789)
33 #define TFT_SCREEN_WIDTH 240
34 #define TFT_SCREEN_HEIGHT 320
35 #elif defined(FNK0103N_3P5_320x480_ST7796) || defined(FNK0103S_4P0_320x480_ST7796)
36 #define TFT_SCREEN_WIDTH 320

```

```

37 #define TFT_SCREEN_HEIGHT 480
38 #endif
39
40 static const uint16_t screenWidth = TFT_SCREEN_WIDTH;
41 static const uint16_t screenHeight = TFT_SCREEN_HEIGHT;
42 static const uint16_t screenHeightBuf = TFT_SCREEN_HEIGHT / 10;
43 static lv_disp_draw_buf_t draw_buf;
44 static lv_color_t draw_buf1[screenWidth * screenHeightBuf];
45
46 TFT_eSPI tft = TFT_eSPI(screenWidth, screenHeight); /* TFT instance */
47
48 #if LV_USE_LOG != 0
49 /* Serial debugging */
50 void my_print(const char * buf)
51 {
52     Serial.printf(buf);
53     Serial.flush();
54 }
55 #endif
56
57 /* Display flushing */
58 void my_disp_flush( lv_disp_drv_t *disp_drv, const lv_area_t *area, lv_color_t *color_p )
59 {
60     uint32_t w = ( area->x2 - area->x1 + 1 );
61     uint32_t h = ( area->y2 - area->y1 + 1 );
62
63     tft.startWrite();
64     tft.setAddrWindow( area->x1, area->y1, w, h );
65     tft.pushColors( ( uint16_t * )&color_p->full, w * h, true );
66     tft.endWrite();
67
68     lv_disp_flush_ready( disp_drv );
69 }
70
71 /*Read the touchpad*/
72 void my_touchpad_read(lv_indev_drv_t *indev_driver, lv_indev_data_t *data) {
73     uint16_t touchX, touchY;
74     bool touched;
75     #if defined(FNK0103B_2P8_240x320_ST7789) || defined(FNK0103F_2P8_240x320_ILI9341)
76     touched = touch.Pressed();
77     touchX = touch.X();
78     touchY = touch.Y();
79     if (!touched) {
80         data->state = LV_INDEV_STATE_REL;

```

```
81     } else {
82         data->state = LV_INDEV_STATE_PR;
83     #if (TFT_DIRECTION == 0)
84         data->point.x = tft.width() - touchY;
85         data->point.y = touchX;
86     #elif (TFT_DIRECTION == 1)
87         data->point.x = touchX;
88         data->point.y = touchY;
89     #elif (TFT_DIRECTION == 2)
90         data->point.x = touchY;
91         data->point.y = tft.height() - touchX;
92     #elif (TFT_DIRECTION == 3)
93         data->point.x = tft.width() - touchX;
94         data->point.y = tft.height() - touchY;
95     #endif
96     // Serial.print("Data x ");
97     // Serial.print(data->point.x);
98     // Serial.print("\tData y ");
99     // Serial.println(data->point.y);
100 }
101 #elif defined(FNK0103L_3P2_240x320_ST7789)
102     touched = tft.getTouch(&touchX, &touchY, 600);
103     if (!touched) {
104         data->state = LV_INDEV_STATE_REL;
105     } else {
106         data->state = LV_INDEV_STATE_PR;
107     #if (TFT_DIRECTION == 0)
108         data->point.x = touchX;
109         data->point.y = touchY;
110     #elif (TFT_DIRECTION == 1)
111         data->point.x = map(touchY, 0, tft.height(), 0, tft.width());
112         data->point.y = map((tft.width() - touchX), 0, tft.width(), 0, tft.height());
113     #elif (TFT_DIRECTION == 2)
114         data->point.x = tft.width() - touchX;
115         data->point.y = tft.height() - touchY;
116     #elif (TFT_DIRECTION == 3)
117         data->point.x = map((tft.height() - touchY), 0, tft.height(), 0, tft.width());
118         data->point.y = map(touchX, 0, tft.width(), 0, tft.height());
119     #endif
120     // Serial.print("Data x ");
121     // Serial.print(data->point.x);
122     // Serial.print("\tData y ");
123     // Serial.println(data->point.y);
124 }
```

```

125 #elif defined(FNK0103N_3P5_320x480_ST7796) || defined(FNK0103S_4P0_320x480_ST7796)
126     touched = tft.getTouch(&touchX, &touchY, 600);
127     if (!touched) {
128         data->state = LV_INDEV_STATE_REL;
129     } else {
130         data->state = LV_INDEV_STATE_PR;
131     #if (TFT_DIRECTION == 0)
132         data->point.x = tft.width() - touchX;
133         data->point.y = touchY;
134     #elif (TFT_DIRECTION == 1)
135         data->point.x = map(touchY, 0, tft.height(), 0, tft.width());
136         data->point.y = map(touchX, 0, tft.width(), 0, tft.height());
137     #elif (TFT_DIRECTION == 2)
138         data->point.x = touchX;
139         data->point.y = tft.height() - touchY;
140     #elif (TFT_DIRECTION == 3)
141         data->point.x = map((tft.height() - touchY), 0, tft.height(), 0, tft.width());
142         data->point.y = map(tft.width() - touchX, 0, tft.width(), 0, tft.height());
143     #endif
144     // Serial.print("Data x ");
145     // Serial.print(data->point.x);
146     // Serial.print("\tData y ");
147     // Serial.println(data->point.y);
148     }
149 #endif
150 }
151
152 void setup()
153 {
154     Serial.begin( 115200 ); /* prepare for possible serial debug */
155
156     String LVGL_Arduino = "Hello Arduino! ";
157     LVGL_Arduino += String('V') + lv_version_major() + "." + lv_version_minor() + "." +
158     lv_version_patch();
159
160     Serial.println( LVGL_Arduino );
161     Serial.println( "I am LVGL_Arduino" );
162
163     lv_init();
164
165     #if LV_USE_LOG != 0
166         lv_log_register_print_cb( my_print ); /* register print function for debugging */
167     #endif
168

```

```

169     tft.begin();           /* TFT init */
170     tft.setRotation(TFT_DIRECTION); /* Landscape orientation, flipped
171
172     /*Set the touchscreen calibration data,
173     the actual data for your display can be acquired using
174     the Generic -> Touch_calibrate example from the TFT_eSPI library*/
175     #if defined(FNK0103B_2P8_240x320_ST7789) || defined(FNK0103F_2P8_240x320_ILI9341)
176         touch.setCal(526, 3443, 750, 3377, screenHeight, screenWidth, 1);
177     #elif defined(FNK0103L_3P2_240x320_ST7789)
178         uint16_t calData[5] = { 286, 3534, 283, 3600, 6 };
179         tft.setTouch(calData);
180     #elif defined(FNK0103N_3P5_320x480_ST7796) || defined(FNK0103S_4P0_320x480_ST7796)
181         uint16_t calData[5] = { 286, 3534, 283, 3600, 6 };
182         tft.setTouch(calData);
183     #endif
184
185     lv_disp_draw_buf_init(&draw_buf, draw_buf1, NULL, screenWidth * screenHeightBuf);
186
187     /*Initialize the display*/
188     static lv_disp_drv_t disp_drv;
189     lv_disp_drv_init( &disp_drv );
190     /*Change the following line to your display resolution*/
191     #if (TFT_DIRECTION % 2 == 0)
192         disp_drv.hor_res = screenWidth;
193         disp_drv.ver_res = screenHeight;
194     #else
195         disp_drv.hor_res = screenHeight;
196         disp_drv.ver_res = screenWidth;
197     #endif
198     disp_drv.flush_cb = my_disp_flush;
199     disp_drv.draw_buf = &draw_buf;
200     lv_disp_drv_register( &disp_drv );
201
202     /*Initialize the (dummy) input device driver*/
203     static lv_indev_drv_t indev_drv;
204     lv_indev_drv_init( &indev_drv );
205     indev_drv.type = LV_INDEV_TYPE_POINTER;
206     indev_drv.read_cb = my_touchpad_read;
207     lv_indev_drv_register( &indev_drv );
208
209     /* Create simple label */
210     lv_obj_t *label = lv_label_create( lv_scr_act() );
211     lv_label_set_text( label, "Hello Ardino and LVGL!");
212     lv_obj_align( label, LV_ALIGN_CENTER, 0, 0 );

```

```

213
214     /* Try an example. See all the examples
215     * online: https://docs.lvgl.io/master/examples.html
216     * source codes:
217     https://github.com/lvgl/lvgl/tree/e7f88efa5853128bf871dde335c0ca8da9eb7731/examples */
218     //lv_example_btn_1();
219
220     /*Or try out a demo. Don't forget to enable the demos in lv_conf.h. E.g.
221     LV_USE_DEMOS_WIDGETS*/
222     lv_demo_widgets();
223     // lv_demo_benchmark();
224     // lv_demo_keypad_encoder();
225     // lv_demo_music();
226     // lv_demo_printer();
227     // lv_demo_stress();
228
229     Serial.println( "Setup done" );
230 }
231
232 void loop()
233 {
234     lv_timer_handler(); /* let the GUI do its work */
235     delay( 5 );
236 }

```

Click **Upload** to upload the code to Freenove ESP32 Display

